

Unitree SDK2 Python 完整 API 参考

Unitree SDK2 Python Bindings

目录

如何阅读本参考	6
覆盖统计	6
分类索引	6
系统辅助 API	7
类索引	7
unitree_sdk2_cpp.OsHelper	7
Typed DDS Channel API	12
模块函数导入	12
模块函数索引	12
类索引	12
unitree_sdk2_cpp.channel.ChannelPublisher	15
unitree_sdk2_cpp.channel.ChannelSubscriber	19
unitree_sdk2_cpp.idl.gl	23
模块函数导入	23
模块函数索引	23
Go2 IDL 消息	26
类索引	26
unitree_sdk2_cpp.idl.go2.AudioData	27
unitree_sdk2_cpp.idl.go2.BmsCmd	31
unitree_sdk2_cpp.idl.go2.BmsState	34
unitree_sdk2_cpp.idl.go2.ConfigChangeStatus	43
unitree_sdk2_cpp.idl.go2.Error	46
unitree_sdk2_cpp.idl.go2.Go2FrontVideoData	50
unitree_sdk2_cpp.idl.go2.HeightMap	55
unitree_sdk2_cpp.idl.go2.IMUState	62
unitree_sdk2_cpp.idl.go2.InterfaceConfig	68
unitree_sdk2_cpp.idl.go2.LidarState	72
unitree_sdk2_cpp.idl.go2.MotorCmd	87
unitree_sdk2_cpp.idl.go2.LowCmd	93
unitree_sdk2_cpp.idl.go2.MotorState	105
unitree_sdk2_cpp.idl.go2.LowState	115
unitree_sdk2_cpp.idl.go2.MotorCmds	132
unitree_sdk2_cpp.idl.go2.MotorStates	135
unitree_sdk2_cpp.idl.go2.PathPoint	138
unitree_sdk2_cpp.idl.go2.Reg	145
unitree_sdk2_cpp.idl.go2.Res	149

unitree_sdk2_cpp.idl.go2.SportModeCmd	153
unitree_sdk2_cpp.idl.go2.TimeSpec	163
unitree_sdk2_cpp.idl.go2.SportModeState	167
unitree_sdk2_cpp.idl.go2.UwbState	180
unitree_sdk2_cpp.idl.go2.UwbSwitch	194
unitree_sdk2_cpp.idl.go2.VoxelMapCompressed	196
unitree_sdk2_cpp.idl.go2.WirelessController	204
HG IDL 消息	209
类索引	209
unitree_sdk2_cpp.idl.hg.AgvBmsState	210
unitree_sdk2_cpp.idl.hg.BmsCmd	217
unitree_sdk2_cpp.idl.hg.BmsState	220
unitree_sdk2_cpp.idl.hg.MotorCmd	231
unitree_sdk2_cpp.idl.hg.HandCmd	238
unitree_sdk2_cpp.idl.hg.IMUState	242
unitree_sdk2_cpp.idl.hg.MotorState	248
unitree_sdk2_cpp.idl.hg.PressSensorState	257
unitree_sdk2_cpp.idl.hg.HandState	262
unitree_sdk2_cpp.idl.hg.LowCmd	270
unitree_sdk2_cpp.idl.hg.LowState	276
unitree_sdk2_cpp.idl.hg.MainBoardState	284
unitree_sdk2_cpp.idl.hg.SportModeState	289
HG Double-IMU IDL 消息	294
类索引	294
unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState	294
ROS2 兼容 IDL 消息	301
类索引	301
unitree_sdk2_cpp.idl.ros2.Time	302
unitree_sdk2_cpp.idl.ros2.Header	305
unitree_sdk2_cpp.idl.ros2.Quaternion	309
unitree_sdk2_cpp.idl.ros2.Vector3	314
unitree_sdk2_cpp.idl.ros2.Imu	318
unitree_sdk2_cpp.idl.ros2.Point	325
unitree_sdk2_cpp.idl.ros2.Pose	329
unitree_sdk2_cpp.idl.ros2.MapMetaData	333
unitree_sdk2_cpp.idl.ros2.OccupancyGrid	338
unitree_sdk2_cpp.idl.ros2.PoseWithCovariance	343
unitree_sdk2_cpp.idl.ros2.Twist	346
unitree_sdk2_cpp.idl.ros2.TwistWithCovariance	350
unitree_sdk2_cpp.idl.ros2.Odometry	353
unitree_sdk2_cpp.idl.ros2.Point32	358
unitree_sdk2_cpp.idl.ros2.PointField	363
unitree_sdk2_cpp.idl.ros2.PointCloud2	367
unitree_sdk2_cpp.idl.ros2.PointStamped	376
unitree_sdk2_cpp.idl.ros2.Pose2D	379
unitree_sdk2_cpp.idl.ros2.PoseStamped	384

unitree_sdk2_cpp.idl.ros2.PoseWithCovarianceStamped	387
unitree_sdk2_cpp.idl.ros2.QuaternionStamped	391
unitree_sdk2_cpp.idl.ros2.String	394
unitree_sdk2_cpp.idl.ros2.TwistStamped	397
unitree_sdk2_cpp.idl.ros2.TwistWithCovarianceStamped	401
Robot 公共基础 API	404
类索引	404
unitree_sdk2_cpp.robot.ApplyLeaseData	405
unitree_sdk2_cpp.robot.ApplyLeaseParameter	407
unitree_sdk2_cpp.robot.ChannelFactory	409
unitree_sdk2_cpp.robot.ChannelNamer	414
unitree_sdk2_cpp.robot.Client	416
unitree_sdk2_cpp.robot.ClientBase	419
unitree_sdk2_cpp.robot.ClientChannelNamer	422
unitree_sdk2_cpp.robot.ClientStub	423
unitree_sdk2_cpp.robot.LeaseCache	427
unitree_sdk2_cpp.robot.LeaseClient	432
unitree_sdk2_cpp.robot.LeaseContext	436
unitree_sdk2_cpp.robot.LeaseServer	440
unitree_sdk2_cpp.robot.RequestFuture	442
unitree_sdk2_cpp.robot.RequestFutureQueue	448
unitree_sdk2_cpp.robot.Server	453
unitree_sdk2_cpp.robot.ServerBase	458
unitree_sdk2_cpp.robot.ServerChannelNamer	462
unitree_sdk2_cpp.robot.ServerStub	463
A2 Robot API	466
类索引	466
unitree_sdk2_cpp.robot.a2.AudioClient	466
unitree_sdk2_cpp.robot.a2.LedControlParameter	473
unitree_sdk2_cpp.robot.a2.PathPoint	475
unitree_sdk2_cpp.robot.a2.PlayStopParameter	476
unitree_sdk2_cpp.robot.a2.PlayStreamParameter	479
unitree_sdk2_cpp.robot.a2.PoseVec4	481
unitree_sdk2_cpp.robot.a2.SportClient	483
unitree_sdk2_cpp.robot.a2.TtsMakerParameter	502
AS2 Robot API	505
类索引	505
unitree_sdk2_cpp.robot.as2.PoseVec4	505
unitree_sdk2_cpp.robot.as2.SportClient	507
B2 Robot API	533
类索引	533
unitree_sdk2_cpp.robot.b2.BackVideoClient	533
unitree_sdk2_cpp.robot.b2.ConfigClient	535
unitree_sdk2_cpp.robot.b2.ConfigDelParameter	542
unitree_sdk2_cpp.robot.b2.ConfigGetData	544
unitree_sdk2_cpp.robot.b2.ConfigGetParameter	546

unitree_sdk2_cpp.robot.b2.ConfigMeta	549
unitree_sdk2_cpp.robot.b2.ConfigMetaData	550
unitree_sdk2_cpp.robot.b2.ConfigMetaParameter	552
unitree_sdk2_cpp.robot.b2.ConfigSetParameter	555
unitree_sdk2_cpp.robot.b2.FrontVideoClient	557
unitree_sdk2_cpp.robot.b2.JsonizeConfigMeta	559
unitree_sdk2_cpp.robot.b2.JsonizeModeName	562
unitree_sdk2_cpp.robot.b2.JsonizeSilent	564
unitree_sdk2_cpp.robot.b2.LowPowerStatusData	566
unitree_sdk2_cpp.robot.b2.LowPowerSwitchParameter	569
unitree_sdk2_cpp.robot.b2.MotionSwitcherClient	571
unitree_sdk2_cpp.robot.b2.PkgVersionData	576
unitree_sdk2_cpp.robot.b2.RobotStateClient	579
unitree_sdk2_cpp.robot.b2.ServiceState	585
unitree_sdk2_cpp.robot.b2.ServiceStateData	586
unitree_sdk2_cpp.robot.b2.ServiceSwitchData	589
unitree_sdk2_cpp.robot.b2.ServiceSwitchParameter	591
unitree_sdk2_cpp.robot.b2.SetReportFreqParameter	593
unitree_sdk2_cpp.robot.b2.SportClient	596
unitree_sdk2_cpp.robot.b2.stPathPoint	615
G1 Robot API	616
模块函数导入	616
模块函数索引	616
类索引	617
unitree_sdk2_cpp.robot.g1.InternalFsmMode	623
unitree_sdk2_cpp.robot.g1.AgvClient	623
unitree_sdk2_cpp.robot.g1.AudioClient	626
unitree_sdk2_cpp.robot.g1.G1ArmActionClient	632
unitree_sdk2_cpp.robot.g1.JsonizeDataVecFloat	637
unitree_sdk2_cpp.robot.g1.JsonizeVelocityCommand	639
unitree_sdk2_cpp.robot.g1.LedControlParameter	642
unitree_sdk2_cpp.robot.g1.LocoClient	644
unitree_sdk2_cpp.robot.g1.MoveParameter	671
unitree_sdk2_cpp.robot.g1.PlayStopParameter	673
unitree_sdk2_cpp.robot.g1.PlayStreamParameter	676
unitree_sdk2_cpp.robot.g1.TtsMakerParameter	678
Go2 Robot API	680
类索引	680
unitree_sdk2_cpp.robot.go2.ConfigClient	681
unitree_sdk2_cpp.robot.go2.ConfigDelParameter	688
unitree_sdk2_cpp.robot.go2.ConfigGetData	690
unitree_sdk2_cpp.robot.go2.ConfigGetParameter	692
unitree_sdk2_cpp.robot.go2.ConfigMeta	695
unitree_sdk2_cpp.robot.go2.ConfigMetaData	696
unitree_sdk2_cpp.robot.go2.ConfigMetaParameter	698
unitree_sdk2_cpp.robot.go2.ConfigSetParameter	701

unitree_sdk2_cpp.robot.go2.JsonizeCommObjInt	703
unitree_sdk2_cpp.robot.go2.JsonizeConfigMeta	705
unitree_sdk2_cpp.robot.go2.JsonizeDataBool	708
unitree_sdk2_cpp.robot.go2.JsonizeDataDouble	710
unitree_sdk2_cpp.robot.go2.JsonizeDataFloat	712
unitree_sdk2_cpp.robot.go2.JsonizeDataInt	715
unitree_sdk2_cpp.robot.go2.JsonizeDataString	717
unitree_sdk2_cpp.robot.go2.JsonizeFlagBool	719
unitree_sdk2_cpp.robot.go2.JsonizePathPoint	722
unitree_sdk2_cpp.robot.go2.JsonizeQuat	724
unitree_sdk2_cpp.robot.go2.JsonizeVec3	727
unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient	729
unitree_sdk2_cpp.robot.go2.ObstaclesAvoidMoveParameter	736
unitree_sdk2_cpp.robot.go2.ObstaclesAvoidRemoteCommandSource	739
unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchGetData	741
unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchSetParameter	743
unitree_sdk2_cpp.robot.go2.RobotStateClient	746
unitree_sdk2_cpp.robot.go2.ServiceState	750
unitree_sdk2_cpp.robot.go2.ServiceStateData	751
unitree_sdk2_cpp.robot.go2.ServiceSwitchData	753
unitree_sdk2_cpp.robot.go2.ServiceSwitchParameter	756
unitree_sdk2_cpp.robot.go2.SetReportFreqParameter	758
unitree_sdk2_cpp.robot.go2.SportClient	760
unitree_sdk2_cpp.robot.go2.UtrackClient	791
unitree_sdk2_cpp.robot.go2.UtrackSwitchGetData	794
unitree_sdk2_cpp.robot.go2.UtrackSwitchSetParameter	797
unitree_sdk2_cpp.robot.go2.VideoClient	799
unitree_sdk2_cpp.robot.go2.VuiClient	801
unitree_sdk2_cpp.robot.go2.stPathPoint	807
H1 Robot API	808
类索引	808
unitree_sdk2_cpp.robot.h1.JsonizeDataVecFloat	809
unitree_sdk2_cpp.robot.h1.JsonizeTargetPos	811
unitree_sdk2_cpp.robot.h1.JsonizeVelocityCommand	813
unitree_sdk2_cpp.robot.h1.LocoClient	816
H2 Robot API	842
类索引	842
unitree_sdk2_cpp.robot.h2.FsmIdInfo	842
unitree_sdk2_cpp.robot.h2.H2ArmActionClient	844
unitree_sdk2_cpp.robot.h2.JsonizeArmActionCommand	849
unitree_sdk2_cpp.robot.h2.JsonizeArmActionName	851
unitree_sdk2_cpp.robot.h2.JsonizeDataVecFloat	853
unitree_sdk2_cpp.robot.h2.JsonizeFsmIdList	856
unitree_sdk2_cpp.robot.h2.JsonizeVelocityCommand	858
unitree_sdk2_cpp.robot.h2.LocoClient	861
R1 Robot API	888

类索引	889
<code>unitree_sdk2_cpp.robot.r1.AudioClient</code>	889
<code>unitree_sdk2_cpp.robot.r1.JsonizeDataVecFloat</code>	895
<code>unitree_sdk2_cpp.robot.r1.JsonizeVelocityCommand</code>	898
<code>unitree_sdk2_cpp.robot.r1.LedControlParameter</code>	900
<code>unitree_sdk2_cpp.robot.r1.LocoClient</code>	902
<code>unitree_sdk2_cpp.robot.r1.PlayStopParameter</code>	914
<code>unitree_sdk2_cpp.robot.r1.PlayStreamParameter</code>	916
<code>unitree_sdk2_cpp.robot.r1.TtsMakerParameter</code>	919

本参考采用常见科学计算库的 API Reference 组织方式：先按模块分类索引，再为每个类、函数和属性提供签名、参数、返回值、可用性、C++ 对应项和用法。学习路径和完整教程请先阅读 [从零开始指南](#)。

[!WARNING] 本文同时包含 `AVAILABLE` 和 `SIGNATURE_ONLY`。后者只是设计期类型签名。运动及硬件副作用方法已进入运行时绑定，但只表示接口可调用，不表示当前环境可安全执行；默认测试绝不会构造客户端、初始化 DDS 或发送机器人指令。

[!NOTE] 本文件由 `generator/generate_api_docs.py` 从 `.pyi`、`api_manifest.json` 和 Clang AST 清单生成。不要手工维护 API 条目；修改签名后应重新生成并运行测试。

如何阅读本参考

每个 API 条目包含以下部分：

- **签名**：Python 类型检查器看到的准确调用形态；
- **可用性与安全性**：区分当前实现和仅签名预览；
- **参数**：Python 类型、默认值和能够从 SDK 定义可靠确认的含义；
- **返回值**：包括 C++ 输出引用转换后的 Python 元组顺序；
- **对应 C++**：原类、原签名、绑定策略和头文件位置；
- **用法**：`AVAILABLE` 给出调用形态，`SIGNATURE_ONLY` 只给出 `TYPE_CHECKING` 计划代码。

各条目中的代码是局部调用片段，不是完整脚本。类章节会先给出导入语句；`obj`、`helper`、`client`、`publisher`、`subscriber` 和参数变量表示调用者已经按业务规则创建、初始化并校验好的对象。

参数名称无法表达单位、坐标系、范围或枚举值时，本文会明确要求查目标型号协议。这比根据名字猜测更可靠。

覆盖统计

项目	数量
有内容的 Python 模块	16
类和枚举	189
普通/重载函数签名	867
Python 属性	346
公开数据属性	173
Manifest 条目	1213
<code>AVAILABLE</code>	1168
<code>SIGNATURE_ONLY</code>	45

一个 Python 属性在 manifest 中占一个条目，但下文会同时写出 `getter` 和 `setter` 签名。重载方法按不同 C++ 签名分别展开。

分类索引

分类	Python 模块	函数	类
系统辅助 API	unitree_sdk2_cpp	0	1
Typed DDS Channel API	unitree_sdk2_cpp.channel	4	2
unitree_sdk2_cpp.idl.g1	unitree_sdk2_cpp.idl.g1	5	0
Go2 IDL 消息	unitree_sdk2_cpp.idl.go2	0	26
HG IDL 消息	unitree_sdk2_cpp.idl.hg	0	13
HG Double-IMU IDL 消息	unitree_sdk2_cpp.idl.hg_doubleimu	0	1
ROS2 兼容 IDL 消息	unitree_sdk2_cpp.idl.ros2	0	24
Robot 公共基础 API	unitree_sdk2_cpp.robot	0	18
A2 Robot API	unitree_sdk2_cpp.robot.a2	0	8
AS2 Robot API	unitree_sdk2_cpp.robot.as2	0	2
B2 Robot API	unitree_sdk2_cpp.robot.b2	0	25
G1 Robot API	unitree_sdk2_cpp.robot.g1	7	12
Go2 Robot API	unitree_sdk2_cpp.robot.go2	0	37
H1 Robot API	unitree_sdk2_cpp.robot.h1	0	4
H2 Robot API	unitree_sdk2_cpp.robot.h2	0	8
R1 Robot API	unitree_sdk2_cpp.robot.r1	0	8

系统辅助 API

模块: [unitree_sdk2_cpp](#)

类索引

类	公开函数签名	属性
OsHelper	7	0

unitree_sdk2_cpp.OsHelper

读取当前主机操作系统信息的单例辅助类。

导入

```
from unitree_sdk2_cpp import OsHelper
```

方法索引

方法	Python 签名	状态	安全分类
instance	<code>def instance() -> OsHelper</code>	AVAILABLE	READ_ONLY
get_uid	<code>def get_uid(self) -> int</code>	AVAILABLE	READ_ONLY
get_gid	<code>def get_gid(self) -> int</code>	AVAILABLE	READ_ONLY
get_user	<code>def get_user(self) -> str</code>	AVAILABLE	READ_ONLY
get_processor_number	<code>def get_processor_number(self) -> int</code>	AVAILABLE	READ_ONLY
get_page_size	<code>def get_page_size(self) -> int</code>	AVAILABLE	READ_ONLY
get_hostname	<code>def get_hostname(self) -> str</code>	AVAILABLE	READ_ONLY

unitree_sdk2_cpp.OsHelper.instance

返回进程内的 `OsHelper` 单例。

签名

```
@staticmethod
def instance() -> OsHelper
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现本机信息读取。该方法不初始化 DDS，也不访问机器人。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>OsHelper</code>	进程内的 <code>OsHelper</code> 单例。

对应 C++

- 类: `unitree_sdk2_cpp.OsHelper`
- 签名: `OsHelper::Instance()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
helper = OsHelper.instance()
```

unitree_sdk2_cpp.OsHelper.get_uid

返回当前进程用户的 Unix UID。

签名

```
def get_uid(self) -> int
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现本机信息读取。该方法不初始化 DDS，也不访问机器人。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	当前进程用户的 Unix UID。

对应 C++

- 类: `unitree_sdk2_cpp.OsHelper`
- 签名: `OsHelper::GetUID()`
- 绑定策略: 未单独标注

- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
uid = helper.get_uid()
```

unitree_sdk2_cpp.OsHelper.get_gid

返回当前进程用户的 Unix GID。

签名

```
def get_gid(self) -> int
```

可用性与安全性

AVAILABLE / READ_ONLY：当前绑定源码已实现本机信息读取。该方法不初始化 DDS，也不访问机器人。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	当前进程用户的 Unix GID。

对应 C++

- 类：unitree_sdk2_cpp.OsHelper
- 签名：OsHelper::GetGID()
- 绑定策略：未单独标注
- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
gid = helper.get_gid()
```

unitree_sdk2_cpp.OsHelper.get_user

返回当前进程对应的用户名。

签名

```
def get_user(self) -> str
```

可用性与安全性

AVAILABLE / READ_ONLY：当前绑定源码已实现本机信息读取。该方法不初始化 DDS，也不访问机器人。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	str	当前进程对应的用户名。

对应 C++

- 类: `unitree_sdk2_cpp.0sHelper`
- 签名: `0sHelper::GetUser()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
user = helper.get_user()
```

`unitree_sdk2_cpp.0sHelper.get_processor_number`

返回当前主机可见的处理器数量。

签名

```
def get_processor_number(self) -> int
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现本机信息读取。该方法不初始化 DDS, 也不访问机器人。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	当前主机可见的处理器数量。

对应 C++

- 类: `unitree_sdk2_cpp.0sHelper`
- 签名: `0sHelper::GetProcessorNumber()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
processor_count = helper.get_processor_number()
```

`unitree_sdk2_cpp.0sHelper.get_page_size`

返回当前主机的操作系统内存页大小。

签名

```
def get_page_size(self) -> int
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现本机信息读取。该方法不初始化 DDS，也不访问机器人。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	操作系统内存页大小，单位为字节。

对应 C++

- 类: `unitree_sdk2_cpp.OsHelper`
- 签名: `OsHelper::GetPageSize()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
page_size = helper.get_page_size()
```

`unitree_sdk2_cpp.OsHelper.get_hostname`

返回当前主机名。

签名

```
def get_hostname(self) -> str
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现本机信息读取。该方法不初始化 DDS，也不访问机器人。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>str</code>	当前主机名。

对应 C++

- 类: `unitree_sdk2_cpp.OsHelper`
- 签名: `OsHelper::GetHostname()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
hostname = helper.get_hostname()
```

Typed DDS Channel API

模块: `unitree_sdk2_cpp.channel`

模块函数导入

```
from unitree_sdk2_cpp.channel import registered_message_types, initialize,
    initialize_from_config, release
```

下面的函数用法片段假设已经完成上述导入。

模块函数索引

函数	Python 签名	状态	安全分类
<code>registered_message_types()</code>	<code>def registered_message_types() -> list[str]</code>	AVAILABLE	DDS_LIFECYCLE
<code>initialize()</code>	<code>def initialize(domain_id: int = 0, network_interface: str = '') -> None</code>	AVAILABLE	DDS_LIFECYCLE
<code>initialize_from_config()</code>	<code>def initialize_from_config(config_file: str = '') -> None</code>	AVAILABLE	DDS_LIFECYCLE
<code>release()</code>	<code>def release() -> None</code>	AVAILABLE	DDS_LIFECYCLE

类索引

类	公开函数签名	属性
<code>ChannelPublisher</code>	4	2
<code>ChannelSubscriber</code>	3	3

`unitree_sdk2_cpp.channel.registered_message_types`

返回当前二进制扩展已经注册的 typed DDS 消息类型名。

签名

```
def registered_message_types() -> list[str]
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

无参数。

返回值

位置	类型	含义
返回值	<code>list[str]</code>	当前扩展已注册的 C++ DDS 消息类型名列表。

对应 C++

- 类: `unitree_sdk2_cpp.channel`
- 签名: `registered_message_types()`

- 绑定策略：未单独标注
- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
message_type_names = registered_message_types()
```

unitree_sdk2_cpp.channel.initialize

使用 Domain ID 和网络接口初始化全局 DDS ChannelFactory。

签名

```
def initialize(domain_id: int = 0, network_interface: str = '') -> None
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE：当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

名称	Python 类型	默认值	含义
domain_id	int	0	DDS Domain ID。只有使用兼容 domain 的参与者才能按预期互相发现。
network_interface	str	''	DDS 使用的网络接口名称，例如 Linux 上的 eth0 或 enp3s0；必须按目标机实际网卡填写。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类：unitree_sdk2_cpp.channel
- 签名：ChannelFactory::Init(int32_t, const std::string &)
- 绑定策略：未单独标注
- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
initialize(domain_id=0, network_interface="eth0")
```

unitree_sdk2_cpp.channel.initialize_from_config

使用 CycloneDDS 配置文件初始化全局 DDS ChannelFactory。

签名

```
def initialize_from_config(config_file: str = '') -> None
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE：当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

名称	Python 类型	默认值	含义
config_file	str	''	CycloneDDS 配置文件路径；空字符串表示使用底层默认配置。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree_sdk2_cpp.channel`
- 签名: `ChannelFactory::Init(const std::string &)`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
initialize_from_config(config_file="cyclonedds.xml")
```

`unitree_sdk2_cpp.channel.release`

释放全局 DDS ChannelFactory 及其持有的底层资源。

签名

```
def release() -> None
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

无参数。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree_sdk2_cpp.channel`
- 签名: `ChannelFactory::Release()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
release()
```

unitree_sdk2_cpp.channel.ChannelPublisher

typed DDS 通道包装类。必须遵守初始化、启动、关闭和全局释放顺序。

导入

```
from unitree_sdk2_cpp.channel import ChannelPublisher
```

基类: Generic[MessageT]

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, topic: AVAILABLE str, message_type: type[MessageT]) -> None</code>		DDS_LIFECYCLE
<code>init_channel</code>	<code>def init_channel(self) -> AVAILABLE None</code>		DDS_LIFECYCLE
<code>close_channel</code>	<code>def close_channel(self) -> AVAILABLE None</code>		DDS_LIFECYCLE
<code>write</code>	<code>def write(self, message: AVAILABLE MessageT, wait_microsec: int = 0) -> bool</code>		DDS_LIFECYCLE

属性索引

属性	读取类型	写入类型	状态
<code>topic</code>	str	只读	AVAILABLE
<code>message_type_name</code>	str	只读	AVAILABLE

unitree_sdk2_cpp.channel.ChannelPublisher.__init__

初始化 ChannelPublisher 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, topic: str, message_type: type[MessageT]) -> None
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

名称	Python 类型	默认值	含义
<code>topic</code>	str	必填	DDS topic 名称。发布端和订阅端必须同时使用兼容的 topic、消息类型和 QoS。
<code>message_type</code>	type[MessageT]	必填	IDL 消息类本身，例如 LowState，不是 LowState() 实例。该类型必须存在于运行时注册表中。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelPublisher`
- 签名: `ChannelPublisher(const std::string &, type)`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ChannelPublisher(topic=topic, message_type=message_type)
```

`unitree_sdk2_cpp.channel.ChannelPublisher.init_channel`

创建并启动 Publisher 的底层 DDS 写通道。

签名

```
def init_channel(self) -> None
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelPublisher`
- 签名: `ChannelPublisher::init_channel()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init_channel()
```

`unitree_sdk2_cpp.channel.ChannelPublisher.close_channel`

关闭 Publisher 的底层 DDS 写通道。

签名

```
def close_channel(self) -> None
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelPublisher`
- 签名: `ChannelPublisher::close_channel()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.close_channel()
```

`unitree_sdk2_cpp.channel.ChannelPublisher.write`

把一个类型匹配的消息样本写入 Publisher 对应的 DDS topic。

签名

```
def write(self, message: MessageT, wait_microsec: int = 0) -> bool
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

名称	Python 类型	默认值	含义
<code>message</code>	<code>MessageT</code>	必填	要写入 DDS 的消息实例; 类型必须与 Publisher 构造时声明的消息类完全一致。
<code>wait_microsec</code>	<code>int</code>	0	写操作允许等待的时间, 单位为微秒; 0 使用当前绑定的默认非额外等待行为。

返回值

位置	类型	含义
返回值	<code>bool</code>	底层写操作是否被接受; 不表示所有订阅者已处理, 更不表示设备已经执行动作。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelPublisher`
- 签名: `ChannelPublisher::write(message, int64_t)`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.write(message=message, wait_microsec=wait_microsec)
```

unitree_sdk2_cpp.channel.ChannelPublisher.topic

Publisher 构造时保存的 DDS topic 名称，只读。

签名

```
@property
def topic(self) -> str
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

该属性只读，没有 setter 参数。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelPublisher`
- 字段类型: `ChannelPublisher::topic() const`
- 声明位置: 由 pybind11 手工绑定提供; manifest 未记录独立头文件行号。

用法

```
current_value = obj.topic
```

unitree_sdk2_cpp.channel.ChannelPublisher.message_type_name

Publisher 使用的已注册 C++ 消息类型名，只读。

签名

```
@property
def message_type_name(self) -> str
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

该属性只读，没有 setter 参数。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelPublisher`
- 字段类型: `ChannelPublisher::message_type_name() const`
- 声明位置: 由 pybind11 手工绑定提供; manifest 未记录独立头文件行号。

用法

```
current_value = obj.message_type_name
```

unitree_sdk2_cpp.channel.ChannelSubscriber

typed DDS 通道包装类。必须遵守初始化、启动、关闭和全局释放顺序。

导入

```
from unitree_sdk2_cpp.channel import ChannelSubscriber
```

基类: Generic[MessageT]

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, topic: AVAILABLE str, message_type: type[MessageT], callback: Callable[[MessageT], None], queue_length: int = 0) -> None</code>		DDS_LIFECYCLE
<code>init_channel</code>	<code>def init_channel(self) -> None</code>	AVAILABLE	DDS_LIFECYCLE
<code>close_channel</code>	<code>def close_channel(self) -> None</code>	AVAILABLE	DDS_LIFECYCLE

属性索引

属性	读取类型	写入类型	状态
<code>topic</code>	str	只读	AVAILABLE
<code>message_type_name</code>	str	只读	AVAILABLE
<code>last_data_</code>	int	只读	AVAILABLE
<code>available_time</code>			

unitree_sdk2_cpp.channel.ChannelSubscriber.__init__

初始化 ChannelSubscriber 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, topic: str, message_type: type[MessageT], callback: Callable[[MessageT], None], queue_length: int = 0) -> None
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

名称	Python 类型	默认值	含义
<code>topic</code>	str	必填	DDS topic 名称。发布端和订阅端必须同时使用兼容的 topic、消息类型和 QoS。
<code>message_type</code>	type[MessageT]	必填	IDL 消息类本身，例如 LowState，不是 LowState() 实例。该类型必须存在于运行时注册表中。
<code>callback</code>	Callable[[MessageT], None]	必填	收到数据或状态变化时调用的 Python 回调。回调签名必须与类型提示一致，并应快速返回。
<code>queue_length</code>	int	0	订阅队列长度。0 使用底层默认行为；更大值会允许更多积压，但也可能增加延迟和内存占用。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelSubscriber`
- 签名: `ChannelSubscriber(const std::string &, type, callback, int64_t)`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 回调可能由 SDK 工作线程触发；回调应快速返回、捕获异常，并通过线程安全队列移交耗时工作。

用法

```
value = ChannelSubscriber(topic=topic, message_type=message_type,
    callback=callback, queue_length=queue_length)
```

`unitree_sdk2_cpp.channel.ChannelSubscriber.init_channel`

创建并启动 Subscriber 的底层 DDS 读通道。

签名

```
def init_channel(self) -> None
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelSubscriber`
- 签名: `ChannelSubscriber::init_channel()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init_channel()
```

`unitree_sdk2_cpp.channel.ChannelSubscriber.close_channel`

关闭 Subscriber 的底层 DDS 读通道，并停止后续回调。

签名

```
def close_channel(self) -> None
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelSubscriber`
- 签名: `ChannelSubscriber::close_channel()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.close_channel()
```

`unitree_sdk2_cpp.channel.ChannelSubscriber.topic`

Subscriber 构造时保存的 DDS topic 名称，只读。

签名

```
@property
def topic(self) -> str
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

该属性只读，没有 `setter` 参数。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelSubscriber`
- 字段类型: `ChannelSubscriber::topic() const`
- 声明位置: 由 `pybind11` 手工绑定提供；manifest 未记录独立头文件行号。

用法

```
current_value = obj.topic
```

unitree_sdk2_cpp.channel.ChannelSubscriber.message_type_name

Subscriber 使用的已注册 C++ 消息类型名，只读。

签名

```
@property
def message_type_name(self) -> str
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

该属性只读，没有 setter 参数。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelSubscriber`
- 字段类型: `ChannelSubscriber::message_type_name() const`
- 声明位置: 由 pybind11 手工绑定提供; manifest 未记录独立头文件行号。

用法

```
current_value = obj.message_type_name
```

unitree_sdk2_cpp.channel.ChannelSubscriber.last_data_available_time

底层记录的最近一次数据可用时间。其单位和时间基准以 SDK 实现为准。

签名

```
@property
def last_data_available_time(self) -> int
```

可用性与安全性

AVAILABLE / DDS_LIFECYCLE: 当前绑定源码已实现。该操作可能创建、使用或释放 DDS 资源。

参数

该属性只读，没有 setter 参数。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_sdk2_cpp.channel.ChannelSubscriber`
- 字段类型: `ChannelSubscriber::last_data_available_time() const`
- 声明位置: 由 pybind11 手工绑定提供; manifest 未记录独立头文件行号。

用法

```
current_value = obj.last_data_available_time
```

unitree_sdk2_cpp.idl.g1

模块: unitree_sdk2_cpp.idl.g1

模块函数导入

```
from unitree_sdk2_cpp.idl.g1 import compute_crc, compute_crc, update_crc,
    validate_crc, validate_crc
```

下面的函数用法片段假设已经完成上述导入。

模块函数索引

函数	Python 签名	状态	安全分类
<code>compute_crc()</code>	<code>def compute_crc(message: LowCmd) -> int</code>	AVAILABLE	VALUE_TYPE
<code>compute_crc()</code>	<code>def compute_crc(message: LowState) -> int</code>	AVAILABLE	VALUE_TYPE
<code>update_crc()</code>	<code>def update_crc(message: LowCmd) -> int</code>	AVAILABLE	VALUE_TYPE
<code>validate_crc()</code>	<code>def validate_crc(message: LowCmd) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>validate_crc()</code>	<code>def validate_crc(message: LowState) -> bool</code>	AVAILABLE	VALUE_TYPE

unitree_sdk2_cpp.idl.g1.compute_crc (重载 1/2)

对应 C++ SDK 操作 `crc32_core(LowCmd_)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def compute_crc(message: LowCmd) -> int
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>message</code>	<code>LowCmd</code>	必填	要写入 DDS 的消息实例；类型必须与 Publisher 构造时声明的消息类完全一致。

返回值

位置	类型	含义
返回值	<code>int</code>	返回 <code>int</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 签名: `crc32_core(LowCmd_)`
- 绑定策略: `CRC_WRAPPER`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = compute_crc(message=message)
```

unitree_sdk2_cpp.idl.g1.compute_crc (重载 2/2)

对应 C++ SDK 操作 `crc32_core(LowState_)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def compute_crc(message: LowState) -> int
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
message	LowState	必填	要写入 DDS 的消息实例；类型必须与 Publisher 构造时声明的消息类完全一致。

返回值

位置	类型	含义
返回值	int	返回 int。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 签名: `crc32_core(LowState_)`
- 绑定策略: `CRC_WRAPPER`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = compute_crc(message=message)
```

unitree_sdk2_cpp.idl.g1.update_crc

对应 C++ SDK 操作 `crc32_core(LowCmd_)`；`LowCmd_::crc(uint32_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def update_crc(message: LowCmd) -> int
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
message	LowCmd	必填	要写入 DDS 的消息实例；类型必须与 Publisher 构造时声明的消息类完全一致。

返回值

位置	类型	含义
返回值	int	返回 int。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 签名: `crc32_core(LowCmd_); LowCmd_::crc(uint32_t)`
- 绑定策略: `CRC_WRAPPER`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = update_crc(message=message)
```

unitree_sdk2_cpp.idl.g1.validate_crc (重载 1/2)

对应 C++ SDK 操作 `LowCmd_::crc() == crc32_core(LowCmd_)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
@overload
def validate_crc(message: LowCmd) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
message	LowCmd	必填	要写入 DDS 的消息实例；类型必须与 Publisher 构造时声明的消息类完全一致。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 签名: `LowCmd_::crc() == crc32_core(LowCmd_)`
- 绑定策略: `CRC_WRAPPER`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = validate_crc(message=message)
```

unitree_sdk2_cpp.idl.g1.validate_crc (重载 2/2)

对应 C++ SDK 操作 `LowState_::crc() == crc32_core(LowState_)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def validate_crc(message: LowState) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
message	LowState	必填	要写入 DDS 的消息实例；类型必须与 Publisher 构造时声明的消息类完全一致。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 签名: `LowState_::crc() == crc32_core(LowState_)`
- 绑定策略: `CRC_WRAPPER`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = validate_crc(message=message)
```

Go2 IDL 消息

模块: `unitree_sdk2_cpp.idl.go2`

类索引

类	公开函数签名	属性
AudioData	3	2
BmsCmd	3	2

类	公开函数签名	属性
BmsState	3	9
ConfigChangeStatus	3	2
Error	3	2
Go2FrontVideoData	3	4
HeightMap	3	7
IMUState	3	5
InterfaceConfig	3	3
LidarState	3	18
MotorCmd	3	7
LowCmd	3	14
MotorState	3	11
LowState	3	22
MotorCmds	3	1
MotorStates	3	1
PathPoint	3	7
Req	3	2
Res	3	3
SportModeCmd	3	11
TimeSpec	3	2
SportModeState	3	16
UwbState	3	17
UwbSwitch	3	1
VoxelMapCompressed	3	7
WirelessController	3	5

unitree_sdk2_cpp.idl.go2.AudioData

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import AudioData
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>time_frame</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>data</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE

unitree_sdk2_cpp.idl.go2.AudioData.__init__

初始化 `AudioData` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::AudioData_`
- 签名: `AudioData_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/AudioData_.hpp:28`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = AudioData()
```

`unitree_sdk2_cpp.idl.go2.AudioData.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::AudioData_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.AudioData.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree_go::msg::dds_::AudioData_
- 签名: operator!=(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.AudioData.time_frame

底层 unitree_go::msg::dds_::AudioData_ 的 time_frame 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 18446744073709551615。

签名

```
@property
def time_frame(self) -> int

@time_frame.setter
def time_frame(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::AudioData_`
- 字段类型: `uint64_t`
- 声明位置: `include/unitree/idl/go2/AudioData_.hpp:24`

用法

```
current_value = obj.time_frame
obj.time_frame = new_value
```

unitree_sdk2_cpp.idl.go2.AudioData.data

底层 `unitree_go::msg::dds_::AudioData_` 的 `data` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 `vector`；元素约束：取值范围为 0 到 255。

签名

```
@property
def data(self) -> list[int]

@data.setter
def data(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、 <code>vector</code> 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::AudioData_`
- 字段类型: `std::vector<uint8_t>`
- 声明位置: `include/unitree/idl/go2/AudioData_.hpp:25`

用法

```
current_value = obj.data
obj.data = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.BmsCmd

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import BmsCmd
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>off</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>reserve</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE

unitree_sdk2_cpp.idl.go2.BmsCmd.__init__

初始化 `BmsCmd` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::BmsCmd_`
- 签名: `BmsCmd_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/BmsCmd_.hpp:28`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法


```
value = BmsCmd()
```

unitree_sdk2_cpp.idl.go2.BmsCmd.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::BmsCmd_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.BmsCmd.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::BmsCmd_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.go2.BmsCmd.off`

底层 `unitree_go::msg::dds_::BmsCmd_` 的 `off` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def off(self) -> int

@off.setter
def off(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::BmsCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/BmsCmd_.hpp:24`

用法

```
current_value = obj.off
obj.off = new_value
```

`unitree_sdk2_cpp.idl.go2.BmsCmd.reserve`

底层 `unitree_go::msg::dds_::BmsCmd_` 的 `reserve` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组, 写入序列必须正好包含 3 个

元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类：unitree_go::msg::dds_::BmsCmd_
- 字段类型：std::array<uint8_t, 3>
- 声明位置：include/unitree/idl/go2/BmsCmd_.hpp:25

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.BmsState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import BmsState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	def __eq__(self, other: object) -> bool	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	def __ne__(self, other: object) -> bool	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>version_high</code>	int	int	AVAILABLE
<code>version_low</code>	int	int	AVAILABLE
<code>status</code>	int	int	AVAILABLE
<code>soc</code>	int	int	AVAILABLE
<code>current</code>	int	int	AVAILABLE
<code>cycle</code>	int	int	AVAILABLE
<code>bq_ntc</code>	list[int]	Sequence[int]	AVAILABLE
<code>mcu_ntc</code>	list[int]	Sequence[int]	AVAILABLE
<code>cell_vol</code>	list[int]	Sequence[int]	AVAILABLE

`unitree_sdk2_cpp.idl.go2.BmsState.__init__`

初始化 `BmsState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 签名: `BmsState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:35`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = BmsState()
```

`unitree_sdk2_cpp.idl.go2.BmsState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.BmsState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.BmsState.version_high

底层 `unitree_go::msg::dds_::BmsState_` 的 `version_high` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def version_high(self) -> int

@version_high.setter
def version_high(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:24`

用法

```
current_value = obj.version_high
obj.version_high = new_value
```

unitree_sdk2_cpp.idl.go2.BmsState.version_low

底层 `unitree_go::msg::dds_::BmsState_` 的 `version_low` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def version_low(self) -> int

@version_low.setter
def version_low(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:25`

用法

```
current_value = obj.version_low
obj.version_low = new_value
```

unitree_sdk2_cpp.idl.go2.BmsState.status

底层 `unitree_go::msg::dds_::BmsState_` 的 `status` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def status(self) -> int

@status.setter
def status(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:26`

用法

```
current_value = obj.status
obj.status = new_value
```

unitree_sdk2_cpp.idl.go2.BmsState.soc

底层 `unitree_go::msg::dds_::BmsState_` 的 `soc` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def soc(self) -> int

@soc.setter
def soc(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:27`

用法

```
current_value = obj.soc
obj.soc = new_value
```

unitree_sdk2_cpp.idl.go2.BmsState.current

底层 `unitree_go::msg::dds_::BmsState_` 的 `current` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 -2147483648 到 2147483647。

签名

```
@property
def current(self) -> int

@current.setter
def current(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 字段类型: `int32_t`
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:28`

用法

```
current_value = obj.current
obj.current = new_value
```

`unitree_sdk2_cpp.idl.go2.BmsState.cycle`

底层 `unitree_go::msg::dds_::BmsState_` 的 `cycle` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 65535。

签名

```
@property
def cycle(self) -> int

@cycle.setter
def cycle(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 字段类型: `uint16_t`
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:29`

用法

```
current_value = obj.cycle
obj.cycle = new_value
```

`unitree_sdk2_cpp.idl.go2.BmsState.bq_ntc`

底层 `unitree_go::msg::dds_::BmsState_` 的 `bq_ntc` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def bq_ntc(self) -> list[int]

@bq_ntc.setter
def bq_ntc(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 字段类型: `std::array<uint8_t, 2>`
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:30`

用法

```
current_value = obj.bq_ntc
obj.bq_ntc = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.BmsState.mcu_ntc

底层 `unitree_go::msg::dds_::BmsState_` 的 `mcu_ntc` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def mcu_ntc(self) -> list[int]

@mcu_ntc.setter
def mcu_ntc(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 字段类型: `std::array<uint8_t, 2>`
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:31`

用法

```
current_value = obj.mcu_ntc
obj.mcu_ntc = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.BmsState.cell_vol

底层 `unitree_go::msg::dds_::BmsState_` 的 `cell_vol` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 15 个元素；元素约束：取值范围为 0 到 65535。

签名

```
@property
def cell_vol(self) -> list[int]

@cell_vol.setter
def cell_vol(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::BmsState_`
- 字段类型: `std::array<uint16_t, 15>`
- 声明位置: `include/unitree/idl/go2/BmsState_.hpp:32`

用法

```
current_value = obj.cell_vol
obj.cell_vol = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.ConfigChangeStatus

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import ConfigChangeStatus
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>name</code>	str	str	AVAILABLE
<code>content</code>	str	str	AVAILABLE

unitree_sdk2_cpp.idl.go2.ConfigChangeStatus.__init__

初始化 ConfigChangeStatus 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::ConfigChangeStatus_`
- 签名: `ConfigChangeStatus_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/ConfigChangeStatus_.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigChangeStatus()
```

unitree_sdk2_cpp.idl.go2.ConfigChangeStatus.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::ConfigChangeStatus_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.ConfigChangeStatus.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::ConfigChangeStatus_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.go2.ConfigChangeStatus.name`

底层 `unitree_go::msg::dds_::ConfigChangeStatus_` 的 `name` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串; 长度、编码和允许值由具体协议决定。

签名

```
@property
def name(self) -> str

@name.setter
def name(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::ConfigChangeStatus_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/ConfigChangeStatus_.hpp:23`

用法

```
current_value = obj.name
obj.name = new_value
```

unitree_sdk2_cpp.idl.go2.ConfigChangeStatus.content

底层 `unitree_go::msg::dds_::ConfigChangeStatus_` 的 `content` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def content(self) -> str

@content.setter
def content(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::ConfigChangeStatus_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/ConfigChangeStatus_.hpp:24`

用法

```
current_value = obj.content
obj.content = new_value
```

unitree_sdk2_cpp.idl.go2.Error

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import Error
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>source</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>state</code>	<code>int</code>	<code>int</code>	AVAILABLE

unitree_sdk2_cpp.idl.go2.Error.__init__

初始化 `Error` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / **VALUE_TYPE**: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::Error_`
- 签名: `Error_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/Error_.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Error()
```

unitree_sdk2_cpp.idl.go2.Error.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / **VALUE_TYPE**: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::Error_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.Error.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::Error_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.Error.source

底层 `unitree_go::msg::dds_::Error_` 的 `source` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def source(self) -> int

@source.setter
def source(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::Error_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/Error_.hpp:23`

用法

```
current_value = obj.source
obj.source = new_value
```

unitree_sdk2_cpp.idl.go2.Error.state

底层 `unitree_go::msg::dds_::Error_` 的 `state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def state(self) -> int

@state.setter
def state(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::Error_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/Error_.hpp:24`

用法

```
current_value = obj.state
obj.state = new_value
```

unitree_sdk2_cpp.idl.go2.Go2FrontVideoData

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import Go2FrontVideoData
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>time_frame</code>	int	int	AVAILABLE
<code>video720p</code>	list[int]	Sequence[int]	AVAILABLE
<code>video360p</code>	list[int]	Sequence[int]	AVAILABLE
<code>video180p</code>	list[int]	Sequence[int]	AVAILABLE

unitree_sdk2_cpp.idl.go2.Go2FrontVideoData.__init__

初始化 Go2FrontVideoData 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::Go2FrontVideoData_`
- 签名: `Go2FrontVideoData_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/Go2FrontVideoData_.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Go2FrontVideoData()
```

`unitree_sdk2_cpp.idl.go2.Go2FrontVideoData.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::Go2FrontVideoData_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.Go2FrontVideoData.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::Go2FrontVideoData_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.go2.Go2FrontVideoData.time_frame`

底层 `unitree_go::msg::dds_::Go2FrontVideoData_` 的 `time_frame` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 18446744073709551615。

签名

```
@property
def time_frame(self) -> int

@time_frame.setter
def time_frame(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::Go2FrontVideoData_`
- 字段类型: `uint64_t`
- 声明位置: `include/unitree/idl/go2/Go2FrontVideoData_.hpp:24`

用法

```
current_value = obj.time_frame
obj.time_frame = new_value
```

`unitree_sdk2_cpp.idl.go2.Go2FrontVideoData.video720p`

底层 `unitree_go::msg::dds_::Go2FrontVideoData_` 的 `video720p` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

签名

```
@property
def video720p(self) -> list[int]

@video720p.setter
def video720p(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::Go2FrontVideoData_`
- 字段类型: `std::vector<uint8_t>`
- 声明位置: `include/unitree/idl/go2/Go2FrontVideoData_.hpp:25`

用法

```
current_value = obj.video720p
obj.video720p = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.Go2FrontVideoData.video360p

底层 `unitree_go::msg::dds_::Go2FrontVideoData_` 的 `video360p` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

签名

```
@property
def video360p(self) -> list[int]

@video360p.setter
def video360p(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::Go2FrontVideoData_`
- 字段类型: `std::vector<uint8_t>`
- 声明位置: `include/unitree/idl/go2/Go2FrontVideoData_.hpp:26`

用法

```
current_value = obj.video360p
obj.video360p = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.Go2FrontVideoData.video180p

底层 `unitree_go::msg::dds_::Go2FrontVideoData_` 的 `video180p` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

签名

```
@property
def video180p(self) -> list[int]

@video180p.setter
def video180p(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::Go2FrontVideoData_`
- 字段类型: `std::vector<uint8_t>`
- 声明位置: `include/unitree/idl/go2/Go2FrontVideoData_.hpp:27`

用法

```
current_value = obj.video180p
obj.video180p = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.HeightMap

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import HeightMap
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>stamp</code>	float	float	AVAILABLE
<code>frame_id</code>	str	str	AVAILABLE
<code>resolution</code>	float	float	AVAILABLE
<code>width</code>	int	int	AVAILABLE
<code>height</code>	int	int	AVAILABLE
<code>origin</code>	list[float]	Sequence[float]	AVAILABLE
<code>data</code>	list[float]	Sequence[float]	AVAILABLE

unitree_sdk2_cpp.idl.go2.HeightMap.__init__

初始化 `HeightMap` 实例。是否能实际构造取决于下方可用性状态。

签名


```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::HeightMap_`
- 签名: `HeightMap_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/HeightMap_.hpp:35`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = HeightMap()
```

`unitree_sdk2_cpp.idl.go2.HeightMap.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::HeightMap_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.HeightMap.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree_go::msg::dds_::HeightMap_
- 签名: operator!=(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.HeightMap.stamp

底层 unitree_go::msg::dds_::HeightMap_ 的 stamp 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def stamp(self) -> float

@stamp.setter
def stamp(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::HeightMap_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/go2/HeightMap_.hpp:26`

用法

```
current_value = obj.stamp
obj.stamp = new_value
```

`unitree_sdk2_cpp.idl.go2.HeightMap.frame_id`

底层 `unitree_go::msg::dds_::HeightMap_` 的 `frame_id` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def frame_id(self) -> str

@frame_id.setter
def frame_id(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::HeightMap_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/HeightMap_.hpp:27`

用法

```
current_value = obj.frame_id
obj.frame_id = new_value
```

unitree_sdk2_cpp.idl.go2.HeightMap.resolution

底层 `unitree_go::msg::dds_::HeightMap_` 的 `resolution` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def resolution(self) -> float

@resolution.setter
def resolution(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::HeightMap_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/HeightMap_.hpp:28`

用法

```
current_value = obj.resolution
obj.resolution = new_value
```

unitree_sdk2_cpp.idl.go2.HeightMap.width

底层 `unitree_go::msg::dds_::HeightMap_` 的 `width` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def width(self) -> int

@width.setter
def width(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::HeightMap_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/HeightMap_.hpp:29`

用法

```
current_value = obj.width
obj.width = new_value
```

unitree_sdk2_cpp.idl.go2.HeightMap.height

底层 `unitree_go::msg::dds_::HeightMap_` 的 `height` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def height(self) -> int

@height.setter
def height(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::HeightMap_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/HeightMap_.hpp:30`

用法

```
current_value = obj.height
obj.height = new_value
```

unitree_sdk2_cpp.idl.go2.HeightMap.origin

底层 `unitree_go::msg::dds_::HeightMap_` 的 `origin` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def origin(self) -> list[float]

@origin.setter
def origin(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::HeightMap_`
- 字段类型: `std::array<float, 2>`
- 声明位置: `include/unitree/idl/go2/HeightMap_.hpp:31`

用法

```
current_value = obj.origin
obj.origin = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.HeightMap.data

底层 `unitree_go::msg::dds_::HeightMap_` 的 `data` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def data(self) -> list[float]

@data.setter
def data(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::HeightMap_`
- 字段类型: `std::vector<float>`
- 声明位置: `include/unitree/idl/go2/HeightMap_.hpp:32`

用法

```
current_value = obj.data
obj.data = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.IMUState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import IMUState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>quaternion</code>	list[float]	Sequence[float]	AVAILABLE
<code>gyroscope</code>	list[float]	Sequence[float]	AVAILABLE
<code>accelerometer</code>	list[float]	Sequence[float]	AVAILABLE
<code>rpy</code>	list[float]	Sequence[float]	AVAILABLE
<code>temperature</code>	int	int	AVAILABLE

unitree_sdk2_cpp.idl.go2.IMUState.__init__

初始化 `IMUState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::IMUState_`
- 签名: `IMUState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/IMUState_.hpp:31`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = IMUState()
```

`unitree_sdk2_cpp.idl.go2.IMUState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::IMUState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.IMUState.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree_go::msg::dds_::IMUState_
- 签名: operator!=(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.IMUState.quaternion

底层 unitree_go::msg::dds_::IMUState_ 的 quaternion 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组, 写入序列必须正好包含 4 个元素; 元素约束: 底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def quaternion(self) -> list[float]

@quaternion.setter
def quaternion(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::IMUState_`
- 字段类型: `std::array<float, 4>`
- 声明位置: `include/unitree/idl/go2/IMUState_.hpp:24`

用法

```
current_value = obj.quaternion
obj.quaternion = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.IMUState.gyroscope

底层 `unitree_go::msg::dds_::IMUState_` 的 `gyroscope` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def gyroscope(self) -> list[float]

@gyroscope.setter
def gyroscope(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::IMUState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/go2/IMUState_.hpp:25`

用法

```
current_value = obj.gyroscope
obj.gyroscope = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.IMUState.accelerometer

底层 `unitree_go::msg::dds_::IMUState_` 的 `accelerometer` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def accelerometer(self) -> list[float]

@accelerometer.setter
def accelerometer(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::IMUState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/go2/IMUState_.hpp:26`

用法

```
current_value = obj.accelerometer
obj.accelerometer = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.IMUState.rpy

底层 `unitree_go::msg::dds_::IMUState_` 的 `rpy` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def rpy(self) -> list[float]

@rpy.setter
def rpy(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::IMUState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/go2/IMUState_.hpp:27`

用法

```
current_value = obj.rpy
obj.rpy = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.IMUState.temperature

底层 `unitree_go::msg::dds_::IMUState_` 的 `temperature` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def temperature(self) -> int

@temperature.setter
def temperature(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::IMUState_`

- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/IMUState_.hpp:28`

用法

```
current_value = obj.temperature
obj.temperature = new_value
```

unitree_sdk2_cpp.idl.go2.InterfaceConfig

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import InterfaceConfig
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>mode</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>value</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>reserve</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE

unitree_sdk2_cpp.idl.go2.InterfaceConfig.__init__

初始化 `InterfaceConfig` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::InterfaceConfig_`
- 签名: `InterfaceConfig_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/InterfaceConfig_.hpp:29`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = InterfaceConfig()
```

unitree_sdk2_cpp.idl.go2.InterfaceConfig.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree_go::msg::dds_::InterfaceConfig_
- 签名: operator==(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.InterfaceConfig.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::InterfaceConfig_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.InterfaceConfig.mode

底层 `unitree_go::msg::dds_::InterfaceConfig_` 的 `mode` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode(self) -> int

@mode.setter
def mode(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::InterfaceConfig_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/InterfaceConfig_.hpp:24`

用法

```
current_value = obj.mode
obj.mode = new_value
```

unitree_sdk2_cpp.idl.go2.InterfaceConfig.value

底层 `unitree_go::msg::dds_::InterfaceConfig_` 的 `value` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def value(self) -> int

@value.setter
def value(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::InterfaceConfig_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/InterfaceConfig_.hpp:25`

用法

```
current_value = obj.value
obj.value = new_value
```

unitree_sdk2_cpp.idl.go2.InterfaceConfig.reserve

底层 `unitree_go::msg::dds_::InterfaceConfig_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::InterfaceConfig_`
- 字段类型: `std::array<uint8_t, 2>`
- 声明位置: `include/unitree/idl/go2/InterfaceConfig_.hpp:26`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LidarState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import LidarState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>stamp</code>	float	float	AVAILABLE
<code>firmware_version</code>	str	str	AVAILABLE
<code>software_version</code>	str	str	AVAILABLE
<code>sdk_version</code>	str	str	AVAILABLE
<code>sys_rotation_speed</code>	float	float	AVAILABLE
<code>com_rotation_speed</code>	float	float	AVAILABLE
<code>error_state</code>	int	int	AVAILABLE
<code>dirty_percentage</code>	int	int	AVAILABLE
<code>cloud_frequency</code>	float	float	AVAILABLE
<code>cloud_packet_loss_rate</code>	float	float	AVAILABLE
<code>cloud_size</code>	int	int	AVAILABLE
<code>cloud_scan_num</code>	int	int	AVAILABLE

属性	读取类型	写入类型	状态
<code>imu_frequency</code>	float	float	AVAILABLE
<code>imu_packet_loss_rate</code>	float	float	AVAILABLE
<code>imu_rpy</code>	list[float]	Sequence[float]	AVAILABLE
<code>serial_recv_stamp</code>	float	float	AVAILABLE
<code>serial_buffer_size</code>	int	int	AVAILABLE
<code>serial_buffer_read</code>	int	int	AVAILABLE

unitree_sdk2_cpp.idl.go2.LidarState.__init__

初始化 LidarState 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / **VALUE_TYPE**: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 签名: `LidarState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:45`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LidarState()
```

unitree_sdk2_cpp.idl.go2.LidarState.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / **VALUE_TYPE**: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.LidarState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.LidarState.stamp

底层 `unitree_go::msg::dds_::LidarState_` 的 `stamp` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def stamp(self) -> float

@stamp.setter
def stamp(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:25`

用法

```
current_value = obj.stamp
obj.stamp = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.firmware_version

底层 `unitree_go::msg::dds_::LidarState_` 的 `firmware_version` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def firmware_version(self) -> str

@firmware_version.setter
def firmware_version(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:26`

用法

```
current_value = obj.firmware_version
obj.firmware_version = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.software_version

底层 `unitree_go::msg::dds_::LidarState_` 的 `software_version` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def software_version(self) -> str

@software_version.setter
def software_version(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:27`

用法

```
current_value = obj.software_version
obj.software_version = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.sdk_version

底层 `unitree_go::msg::dds_::LidarState_` 的 `sdk_version` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def sdk_version(self) -> str

@sdk_version.setter
def sdk_version(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:28`

用法

```
current_value = obj.sdk_version
obj.sdk_version = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.sys_rotation_speed

底层 `unitree_go::msg::dds_::LidarState_` 的 `sys_rotation_speed` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def sys_rotation_speed(self) -> float

@sys_rotation_speed.setter
def sys_rotation_speed(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:29`

用法

```
current_value = obj.sys_rotation_speed
obj.sys_rotation_speed = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.com_rotation_speed

底层 `unitree_go::msg::dds_::LidarState_` 的 `com_rotation_speed` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def com_rotation_speed(self) -> float

@com_rotation_speed.setter
def com_rotation_speed(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:30`

用法

```
current_value = obj.com_rotation_speed
obj.com_rotation_speed = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.error_state

底层 `unitree_go::msg::dds_::LidarState_` 的 `error_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def error_state(self) -> int

@error_state.setter
def error_state(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:31`

用法

```
current_value = obj.error_state
obj.error_state = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.dirty_percentage

底层 `unitree_go::msg::dds_::LidarState_` 的 `dirty_percentage` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def dirty_percentage(self) -> int

@dirty_percentage.setter
def dirty_percentage(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:32`

用法

```
current_value = obj.dirty_percentage
obj.dirty_percentage = new_value
```

`unitree_sdk2_cpp.idl.go2.LidarState.cloud_frequency`

底层 `unitree_go::msg::dds_::LidarState_` 的 `cloud_frequency` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def cloud_frequency(self) -> float

@cloud_frequency.setter
def cloud_frequency(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:33`

用法

```
current_value = obj.cloud_frequency
obj.cloud_frequency = new_value
```

`unitree_sdk2_cpp.idl.go2.LidarState.cloud_packet_loss_rate`

底层 `unitree_go::msg::dds_::LidarState_` 的 `cloud_packet_loss_rate` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def cloud_packet_loss_rate(self) -> float

@cloud_packet_loss_rate.setter
def cloud_packet_loss_rate(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:34`

用法

```
current_value = obj.cloud_packet_loss_rate
obj.cloud_packet_loss_rate = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.cloud_size

底层 `unitree_go::msg::dds_::LidarState_` 的 `cloud_size` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def cloud_size(self) -> int

@cloud_size.setter
def cloud_size(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:35`

用法

```
current_value = obj.cloud_size
obj.cloud_size = new_value
```

`unitree_sdk2_cpp.idl.go2.LidarState.cloud_scan_num`

底层 `unitree_go::msg::dds_::LidarState_` 的 `cloud_scan_num` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def cloud_scan_num(self) -> int

@cloud_scan_num.setter
def cloud_scan_num(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:36`

用法

```
current_value = obj.cloud_scan_num
obj.cloud_scan_num = new_value
```

`unitree_sdk2_cpp.idl.go2.LidarState.imu_frequency`

底层 `unitree_go::msg::dds_::LidarState_` 的 `imu_frequency` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def imu_frequency(self) -> float

@imu_frequency.setter
def imu_frequency(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:37`

用法

```
current_value = obj.imu_frequency
obj.imu_frequency = new_value
```

`unitree_sdk2_cpp.idl.go2.LidarState.imu_packet_loss_rate`

底层 `unitree_go::msg::dds_::LidarState_` 的 `imu_packet_loss_rate` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def imu_packet_loss_rate(self) -> float

@imu_packet_loss_rate.setter
def imu_packet_loss_rate(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:38`

用法

```
current_value = obj.imu_packet_loss_rate
obj.imu_packet_loss_rate = new_value
```

`unitree_sdk2_cpp.idl.go2.LidarState.imu_rpy`

底层 `unitree_go::msg::dds_::LidarState_` 的 `imu_rpy` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def imu_rpy(self) -> list[float]

@imu_rpy.setter
def imu_rpy(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[float]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[float]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:39`

用法

```
current_value = obj.imu_rpy
obj.imu_rpy = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LidarState.serial_recv_stamp

底层 `unitree_go::msg::dds_::LidarState_` 的 `serial_recv_stamp` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def serial_recv_stamp(self) -> float

@serial_recv_stamp.setter
def serial_recv_stamp(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:40`

用法

```
current_value = obj.serial_recv_stamp
obj.serial_recv_stamp = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.serial_buffer_size

底层 `unitree_go::msg::dds_::LidarState_` 的 `serial_buffer_size` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def serial_buffer_size(self) -> int

@serial_buffer_size.setter
def serial_buffer_size(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:41`

用法

```
current_value = obj.serial_buffer_size
obj.serial_buffer_size = new_value
```

unitree_sdk2_cpp.idl.go2.LidarState.serial_buffer_read

底层 `unitree_go::msg::dds_::LidarState_` 的 `serial_buffer_read` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def serial_buffer_read(self) -> int

@serial_buffer_read.setter
def serial_buffer_read(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LidarState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/LidarState_.hpp:42`

用法

```
current_value = obj.serial_buffer_read
obj.serial_buffer_read = new_value
```

unitree_sdk2_cpp.idl.go2.MotorCmd

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import MotorCmd
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>mode</code>	int	int	AVAILABLE
<code>q</code>	float	float	AVAILABLE
<code>dq</code>	float	float	AVAILABLE
<code>tau</code>	float	float	AVAILABLE
<code>kp</code>	float	float	AVAILABLE
<code>kd</code>	float	float	AVAILABLE
<code>reserve</code>	list[int]	Sequence[int]	AVAILABLE

unitree_sdk2_cpp.idl.go2.MotorCmd.__init__

初始化 `MotorCmd` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 签名: `MotorCmd_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/MotorCmd_.hpp:33`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MotorCmd()
```

unitree_sdk2_cpp.idl.go2.MotorCmd.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.MotorCmd.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.go2.MotorCmd.mode`

底层 `unitree_go::msg::dds_::MotorCmd_` 的 `mode` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode(self) -> int

@mode.setter
def mode(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/MotorCmd_.hpp:24`

用法

```
current_value = obj.mode
obj.mode = new_value
```

`unitree_sdk2_cpp.idl.go2.MotorCmd.q`

底层 `unitree_go::msg::dds_::MotorCmd_` 的 `q` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数; 类型本身不说明单位、坐标系或业务安

全范围。

签名

```
@property
def q(self) -> float

@q.setter
def q(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorCmd_.hpp:25`

用法

```
current_value = obj.q
obj.q = new_value
```

`unitree_sdk2_cpp.idl.go2.MotorCmd.dq`

底层 `unitree_go::msg::dds_::MotorCmd_` 的 `dq` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def dq(self) -> float

@dq.setter
def dq(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorCmd_.hpp:26`

用法

```
current_value = obj.dq
obj.dq = new_value
```

unitree_sdk2_cpp.idl.go2.MotorCmd.tau

底层 `unitree_go::msg::dds_::MotorCmd_` 的 `tau` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def tau(self) -> float

@tau.setter
def tau(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorCmd_.hpp:27`

用法

```
current_value = obj.tau
obj.tau = new_value
```

unitree_sdk2_cpp.idl.go2.MotorCmd.kp

底层 `unitree_go::msg::dds_::MotorCmd_` 的 `kp` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def kp(self) -> float

@kp.setter
def kp(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorCmd_.hpp:28`

用法

```
current_value = obj.kp
obj.kp = new_value
```

unitree_sdk2_cpp.idl.go2.MotorCmd.kd

底层 `unitree_go::msg::dds_::MotorCmd_` 的 `kd` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def kd(self) -> float

@kd.setter
def kd(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorCmd_.hpp:29`

用法

```
current_value = obj.kd
obj.kd = new_value
```

`unitree_sdk2_cpp.idl.go2.MotorCmd.reserve`

底层 `unitree_go::msg::dds_::MotorCmd_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmd_`
- 字段类型: `std::array<uint32_t, 3>`
- 声明位置: `include/unitree/idl/go2/MotorCmd_.hpp:30`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

`unitree_sdk2_cpp.idl.go2.LowCmd`

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import LowCmd
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>head</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>level_flag</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>frame_reserve</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>sn</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>version</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>bandwidth</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>motor_cmd</code>	<code>list[MotorCmd]</code>	<code>Sequence[MotorCmd]</code>	AVAILABLE
<code>bms_cmd</code>	<code>BmsCmd</code>	<code>BmsCmd</code>	AVAILABLE
<code>wireless_remote</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>led</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>fan</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>gpio</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>reserve</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>crc</code>	<code>int</code>	<code>int</code>	AVAILABLE

unitree_sdk2_cpp.idl.go2.LowCmd.__init__

初始化 `LowCmd` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 签名: `LowCmd_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:44`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LowCmd()
```

unitree_sdk2_cpp.idl.go2.LowCmd.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree_go::msg::dds_::LowCmd_
- 签名: operator==(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.LowCmd.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.LowCmd.head

底层 `unitree_go::msg::dds_::LowCmd_` 的 `head` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组, 写入序列必须正好包含 2 个元素; 元素约束: 取值范围为 0 到 255。

签名

```
@property
def head(self) -> list[int]

@head.setter
def head(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `std::array<uint8_t, 2>`

- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:28`

用法

```
current_value = obj.head
obj.head = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 `getter` 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowCmd.level_flag

底层 `unitree_go::msg::dds_::LowCmd_` 的 `level_flag` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def level_flag(self) -> int

@level_flag.setter
def level_flag(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:29`

用法

```
current_value = obj.level_flag
obj.level_flag = new_value
```

unitree_sdk2_cpp.idl.go2.LowCmd.frame_reserve

底层 `unitree_go::msg::dds_::LowCmd_` 的 `frame_reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def frame_reserve(self) -> int

@frame_reserve.setter
def frame_reserve(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:30`

用法

```
current_value = obj.frame_reserve
obj.frame_reserve = new_value
```

unitree_sdk2_cpp.idl.go2.LowCmd.sn

底层 `unitree_go::msg::dds_::LowCmd_` 的 `sn` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def sn(self) -> list[int]

@sn.setter
def sn(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `std::array<uint32_t, 2>`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:31`

用法

```
current_value = obj.sn
obj.sn = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowCmd.version

底层 `unitree_go::msg::dds_::LowCmd_` 的 `version` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def version(self) -> list[int]

@version.setter
def version(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `std::array<uint32_t, 2>`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:32`

用法

```
current_value = obj.version
obj.version = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowCmd.bandwidth

底层 `unitree_go::msg::dds_::LowCmd_` 的 `bandwidth` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 65535。

签名

```
@property
def bandwidth(self) -> int

@bandwidth.setter
def bandwidth(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `uint16_t`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:33`

用法

```
current_value = obj.bandwidth
obj.bandwidth = new_value
```

unitree_sdk2_cpp.idl.go2.LowCmd.motor_cmd

底层 `unitree_go::msg::dds_::LowCmd_` 的 `motor_cmd` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 20 个元素

签名

```
@property
def motor_cmd(self) -> list[MotorCmd]

@motor_cmd.setter
def motor_cmd(self, value: Sequence[MotorCmd]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[MotorCmd]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[MotorCmd]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `std::array< ::unitree_go::msg::dds_::MotorCmd_, 20>`

- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:34`

用法

```
current_value = obj.motor_cmd
obj.motor_cmd = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 `getter` 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowCmd.bms_cmd

底层 `unitree_go::msg::dds_::LowCmd_` 的 `bms_cmd` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def bms_cmd(self) -> BmsCmd

@bms_cmd.setter
def bms_cmd(self, value: BmsCmd) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	BmsCmd	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	BmsCmd	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `::unitree_go::msg::dds_::BmsCmd_`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:35`

用法

```
current_value = obj.bms_cmd
obj.bms_cmd = new_value
```

unitree_sdk2_cpp.idl.go2.LowCmd.wireless_remote

底层 `unitree_go::msg::dds_::LowCmd_` 的 `wireless_remote` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 40 个元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def wireless_remote(self) -> list[int]

@wireless_remote.setter
def wireless_remote(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `std::array<uint8_t, 40>`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:36`

用法

```
current_value = obj.wireless_remote
obj.wireless_remote = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowCmd.led

底层 `unitree_go::msg::dds_::LowCmd_` 的 `led` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 12 个元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def led(self) -> list[int]

@led.setter
def led(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `std::array<uint8_t, 12>`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:37`

用法

```
current_value = obj.led
obj.led = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowCmd.fan

底层 `unitree_go::msg::dds_::LowCmd_` 的 `fan` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def fan(self) -> list[int]

@fan.setter
def fan(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `std::array<uint8_t, 2>`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:38`

用法

```
current_value = obj.fan
obj.fan = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowCmd.gpio

底层 `unitree_go::msg::dds_::LowCmd_` 的 `gpio` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名


```
@property
def gpio(self) -> int

@gpio.setter
def gpio(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:39`

用法

```
current_value = obj.gpio
obj.gpio = new_value
```

unitree_sdk2_cpp.idl.go2.LowCmd.reserve

底层 `unitree_go::msg::dds_::LowCmd_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> int

@reserve.setter
def reserve(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:40`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

unitree_sdk2_cpp.idl.go2.LowCmd.crc

底层 `unitree_go::msg::dds_::LowCmd_` 的 `crc` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def crc(self) -> int

@crc.setter
def crc(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowCmd_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/LowCmd_.hpp:41`

用法

```
current_value = obj.crc
obj.crc = new_value
```

unitree_sdk2_cpp.idl.go2.MotorState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import MotorState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE

方法	Python 签名	状态	安全分类
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>mode</code>	int	int	AVAILABLE
<code>q</code>	float	float	AVAILABLE
<code>dq</code>	float	float	AVAILABLE
<code>ddq</code>	float	float	AVAILABLE
<code>tau_est</code>	float	float	AVAILABLE
<code>q_raw</code>	float	float	AVAILABLE
<code>dq_raw</code>	float	float	AVAILABLE
<code>ddq_raw</code>	float	float	AVAILABLE
<code>temperature</code>	int	int	AVAILABLE
<code>lost</code>	int	int	AVAILABLE
<code>reserve</code>	list[int]	Sequence[int]	AVAILABLE

unitree_sdk2_cpp.idl.go2.MotorState.__init__

初始化 MotorState 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 签名: `MotorState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:37`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MotorState()
```

unitree_sdk2_cpp.idl.go2.MotorState.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.MotorState.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.go2.MotorState.mode`

底层 `unitree_go::msg::dds_::MotorState_` 的 `mode` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode(self) -> int

@mode.setter
def mode(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
<code>value</code>	<code>int</code>	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>int</code>	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:24`

用法

```
current_value = obj.mode
obj.mode = new_value
```

`unitree_sdk2_cpp.idl.go2.MotorState.q`

底层 `unitree_go::msg::dds_::MotorState_` 的 `q` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def q(self) -> float
```

```
@q.setter
def q(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:25`

用法

```
current_value = obj.q
obj.q = new_value
```

`unitree_sdk2_cpp.idl.go2.MotorState.dq`

底层 `unitree_go::msg::dds_::MotorState_` 的 `dq` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def dq(self) -> float

@dq.setter
def dq(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:26`

用法

```
current_value = obj.dq
obj.dq = new_value
```

`unitree_sdk2_cpp.idl.go2.MotorState.ddq`

底层 `unitree_go::msg::dds_::MotorState_` 的 `ddq` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def ddq(self) -> float

@ddq.setter
def ddq(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
<code>value</code>	<code>float</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>float</code>	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:27`

用法

```
current_value = obj.ddq
obj.ddq = new_value
```

`unitree_sdk2_cpp.idl.go2.MotorState.tau_est`

底层 `unitree_go::msg::dds_::MotorState_` 的 `tau_est` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def tau_est(self) -> float

@tau_est.setter
def tau_est(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:28`

用法

```
current_value = obj.tau_est
obj.tau_est = new_value
```

unitree_sdk2_cpp.idl.go2.MotorState.q_raw

底层 `unitree_go::msg::dds_::MotorState_` 的 `q_raw` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def q_raw(self) -> float

@q_raw.setter
def q_raw(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `float`

- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:29`

用法

```
current_value = obj.q_raw
obj.q_raw = new_value
```

unitree_sdk2_cpp.idl.go2.MotorState.dq_raw

底层 `unitree_go::msg::dds_::MotorState_` 的 `dq_raw` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def dq_raw(self) -> float

@dq_raw.setter
def dq_raw(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:30`

用法

```
current_value = obj.dq_raw
obj.dq_raw = new_value
```

unitree_sdk2_cpp.idl.go2.MotorState.ddq_raw

底层 `unitree_go::msg::dds_::MotorState_` 的 `ddq_raw` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def ddq_raw(self) -> float

@ddq_raw.setter
def ddq_raw(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:31`

用法

```
current_value = obj.ddq_raw
obj.ddq_raw = new_value
```

`unitree_sdk2_cpp.idl.go2.MotorState.temperature`

底层 `unitree_go::msg::dds_::MotorState_` 的 `temperature` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def temperature(self) -> int

@temperature.setter
def temperature(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:32`

用法

```
current_value = obj.temperature
obj.temperature = new_value
```

unitree_sdk2_cpp.idl.go2.MotorState.lost

底层 `unitree_go::msg::dds_::MotorState_` 的 `lost` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def lost(self) -> int

@lost.setter
def lost(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:33`

用法

```
current_value = obj.lost
obj.lost = new_value
```

unitree_sdk2_cpp.idl.go2.MotorState.reserve

底层 `unitree_go::msg::dds_::MotorState_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::MotorState_`
- 字段类型: `std::array<uint32_t, 2>`
- 声明位置: `include/unitree/idl/go2/MotorState_.hpp:34`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import LowState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>head</code>	list[int]	Sequence[int]	AVAILABLE
<code>level_flag</code>	int	int	AVAILABLE
<code>frame_reserve</code>	int	int	AVAILABLE
<code>sn</code>	list[int]	Sequence[int]	AVAILABLE
<code>version</code>	list[int]	Sequence[int]	AVAILABLE
<code>bandwidth</code>	int	int	AVAILABLE
<code>imu_state</code>	IMUState	IMUState	AVAILABLE
<code>motor_state</code>	list[MotorState]	Sequence[MotorState]	AVAILABLE
<code>bms_state</code>	BmsState	BmsState	AVAILABLE
<code>foot_force</code>	list[int]	Sequence[int]	AVAILABLE
<code>foot_force_est</code>	list[int]	Sequence[int]	AVAILABLE
<code>tick</code>	int	int	AVAILABLE

属性	读取类型	写入类型	状态
wireless_remote	list[int]	Sequence[int]	AVAILABLE
bit_flag	int	int	AVAILABLE
adc_reel	float	float	AVAILABLE
temperature_ntc1	int	int	AVAILABLE
temperature_ntc2	int	int	AVAILABLE
power_v	float	float	AVAILABLE
power_a	float	float	AVAILABLE
fan_frequency	list[int]	Sequence[int]	AVAILABLE
reserve	int	int	AVAILABLE
crc	int	int	AVAILABLE

unitree_sdk2_cpp.idl.go2.LowState.__init__

初始化 `LowState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 签名: `LowState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:54`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LowState()
```

unitree_sdk2_cpp.idl.go2.LowState.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.LowState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.LowState.head

底层 `unitree_go::msg::dds_::LowState_` 的 `head` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def head(self) -> list[int]

@head.setter
def head(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `std::array<uint8_t, 2>`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:30`

用法

```
current_value = obj.head
obj.head = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowState.level_flag

底层 `unitree_go::msg::dds_::LowState_` 的 `level_flag` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def level_flag(self) -> int

@level_flag.setter
def level_flag(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:31`

用法

```
current_value = obj.level_flag
obj.level_flag = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.frame_reserve

底层 `unitree_go::msg::dds_::LowState_` 的 `frame_reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def frame_reserve(self) -> int

@frame_reserve.setter
def frame_reserve(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:32`

用法

```
current_value = obj.frame_reserve
obj.frame_reserve = new_value
```


unitree_sdk2_cpp.idl.go2.LowState.sn

底层 `unitree_go::msg::dds_::LowState_` 的 `sn` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def sn(self) -> list[int]

@sn.setter
def sn(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `std::array<uint32_t, 2>`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:33`

用法

```
current_value = obj.sn
obj.sn = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowState.version

底层 `unitree_go::msg::dds_::LowState_` 的 `version` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def version(self) -> list[int]

@version.setter
def version(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `std::array<uint32_t, 2>`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:34`

用法

```
current_value = obj.version
obj.version = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowState.bandwidth

底层 `unitree_go::msg::dds_::LowState_` 的 `bandwidth` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 65535。

签名

```
@property
def bandwidth(self) -> int

@bandwidth.setter
def bandwidth(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `uint16_t`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:35`

用法

```
current_value = obj.bandwidth
obj.bandwidth = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.imu_state

底层 `unitree_go::msg::dds_::LowState_` 的 `imu_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def imu_state(self) -> IMUState

@imu_state.setter
def imu_state(self, value: IMUState) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	IMUState	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	IMUState	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `::unitree_go::msg::dds_::IMUState_`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:36`

用法

```
current_value = obj.imu_state
obj.imu_state = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.motor_state

底层 `unitree_go::msg::dds_::LowState_` 的 `motor_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 20 个元素

签名

```
@property
def motor_state(self) -> list[MotorState]

@motor_state.setter
def motor_state(self, value: Sequence[MotorState]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[MotorState]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[MotorState]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `std::array< ::unitree_go::msg::dds_::MotorState_, 20>`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:37`

用法

```
current_value = obj.motor_state
obj.motor_state = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowState.bms_state

底层 `unitree_go::msg::dds_::LowState_` 的 `bms_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def bms_state(self) -> BmsState

@bms_state.setter
def bms_state(self, value: BmsState) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	BmsState	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	BmsState	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `::unitree_go::msg::dds_::BmsState_`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:38`

用法

```
current_value = obj.bms_state
obj.bms_state = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.foot_force

底层 `unitree_go::msg::dds_::LowState_` 的 `foot_force` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 4 个元素；元素约束：取值范围为 -32768 到 32767。

签名

```
@property
def foot_force(self) -> list[int]

@foot_force.setter
def foot_force(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `std::array<int16_t, 4>`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:39`

用法

```
current_value = obj.foot_force
obj.foot_force = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowState.foot_force_est

底层 `unitree_go::msg::dds_::LowState_` 的 `foot_force_est` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 4 个元素；元素约束：取值范围为 -32768 到 32767。

签名

```
@property
def foot_force_est(self) -> list[int]

@foot_force_est.setter
def foot_force_est(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `std::array<int16_t, 4>`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:40`

用法

```
current_value = obj.foot_force_est
obj.foot_force_est = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowState.tick

底层 `unitree_go::msg::dds_::LowState_` 的 `tick` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def tick(self) -> int

@tick.setter
def tick(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `uint32_t`

- 声明位置: `include/unitree/idl/go2/LowState_.hpp:41`

用法

```
current_value = obj.tick
obj.tick = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.wireless_remote

底层 `unitree_go::msg::dds_::LowState_` 的 `wireless_remote` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 40 个元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def wireless_remote(self) -> list[int]

@wireless_remote.setter
def wireless_remote(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `std::array<uint8_t, 40>`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:42`

用法

```
current_value = obj.wireless_remote
obj.wireless_remote = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowState.bit_flag

底层 `unitree_go::msg::dds_::LowState_` 的 `bit_flag` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def bit_flag(self) -> int

@bit_flag.setter
def bit_flag(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:43`

用法

```
current_value = obj.bit_flag
obj.bit_flag = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.adc_reel

底层 `unitree_go::msg::dds_::LowState_` 的 `adc_reel` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def adc_reel(self) -> float

@adc_reel.setter
def adc_reel(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `float`

- 声明位置: `include/unitree/idl/go2/LowState_.hpp:44`

用法

```
current_value = obj.adc_reel
obj.adc_reel = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.temperature_ntc1

底层 `unitree_go::msg::dds_::LowState_` 的 `temperature_ntc1` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def temperature_ntc1(self) -> int

@temperature_ntc1.setter
def temperature_ntc1(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:45`

用法

```
current_value = obj.temperature_ntc1
obj.temperature_ntc1 = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.temperature_ntc2

底层 `unitree_go::msg::dds_::LowState_` 的 `temperature_ntc2` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def temperature_ntc2(self) -> int

@temperature_ntc2.setter
def temperature_ntc2(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:46`

用法

```
current_value = obj.temperature_ntc2
obj.temperature_ntc2 = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.power_v

底层 `unitree_go::msg::dds_::LowState_` 的 `power_v` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def power_v(self) -> float

@power_v.setter
def power_v(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:47`

用法

```
current_value = obj.power_v
obj.power_v = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.power_a

底层 `unitree_go::msg::dds_::LowState_` 的 `power_a` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def power_a(self) -> float

@power_a.setter
def power_a(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:48`

用法

```
current_value = obj.power_a
obj.power_a = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.fan_frequency

底层 `unitree_go::msg::dds_::LowState_` 的 `fan_frequency` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 4 个元素；元素约束：取值范围为 0 到 65535。

签名

```
@property
def fan_frequency(self) -> list[int]

@fan_frequency.setter
def fan_frequency(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `std::array<uint16_t, 4>`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:49`

用法

```
current_value = obj.fan_frequency
obj.fan_frequency = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.LowState.reserve

底层 `unitree_go::msg::dds_::LowState_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> int

@reserve.setter
def reserve(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:50`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

unitree_sdk2_cpp.idl.go2.LowState.crc

底层 `unitree_go::msg::dds_::LowState_` 的 `crc` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def crc(self) -> int

@crc.setter
def crc(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::LowState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/LowState_.hpp:51`

用法

```
current_value = obj.crc
obj.crc = new_value
```

unitree_sdk2_cpp.idl.go2.MotorCmds

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import MotorCmds
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
cmds	list [MotorCmd]	Sequence [MotorCmd]	AVAILABLE

unitree_sdk2_cpp.idl.go2.MotorCmds.__init__

初始化 `MotorCmds` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmds_`
- 签名: `MotorCmds_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/MotorCmds_.hpp:28`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MotorCmds()
```

unitree_sdk2_cpp.idl.go2.MotorCmds.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmds_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.MotorCmds.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmds_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.MotorCmds.cmds

底层 `unitree_go::msg::dds_::MotorCmds_` 的 `cmds` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector

签名

```
@property
def cmds(self) -> list[MotorCmd]

@cmds.setter
def cmds(self, value: Sequence[MotorCmd]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[MotorCmd]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[MotorCmd]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::MotorCmds_`
- 字段类型: `std::vector< ::unitree_go::msg::dds_::MotorCmd_>`
- 声明位置: `include/unitree/idl/go2/MotorCmds_.hpp:25`

用法

```
current_value = obj.cmds
obj.cmds = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.MotorStates

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import MotorStates
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>states</code>	<code>list[MotorState]</code>	<code>Sequence[MotorState]</code>	AVAILABLE

`unitree_sdk2_cpp.idl.go2.MotorStates.__init__`

初始化 `MotorStates` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::MotorStates_`
- 签名: `MotorStates_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/MotorStates_.hpp:28`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MotorStates()
```

`unitree_sdk2_cpp.idl.go2.MotorStates.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::MotorStates_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.MotorStates.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::MotorStates_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.MotorStates.states

底层 `unitree_go::msg::dds_::MotorStates_` 的 `states` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector

签名

```
@property
def states(self) -> list[MotorState]

@states.setter
def states(self, value: Sequence[MotorState]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[MotorState]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[MotorState]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::MotorStates_`
- 字段类型: `std::vector< ::unitree_go::msg::dds_::MotorState_>`
- 声明位置: `include/unitree/idl/go2/MotorStates_.hpp:25`

用法

```
current_value = obj.states
obj.states = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.PathPoint

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import PathPoint
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>t_from_start</code>	float	float	AVAILABLE
<code>x</code>	float	float	AVAILABLE
<code>y</code>	float	float	AVAILABLE
<code>yaw</code>	float	float	AVAILABLE
<code>vx</code>	float	float	AVAILABLE
<code>vy</code>	float	float	AVAILABLE
<code>vyaw</code>	float	float	AVAILABLE

`unitree_sdk2_cpp.idl.go2.PathPoint.__init__`

初始化 `PathPoint` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 签名: `PathPoint_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/PathPoint_.hpp:31`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PathPoint()
```

`unitree_sdk2_cpp.idl.go2.PathPoint.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.PathPoint.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.PathPoint.t_from_start

底层 `unitree_go::msg::dds_::PathPoint_` 的 `t_from_start` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def t_from_start(self) -> float

@t_from_start.setter
def t_from_start(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/PathPoint_.hpp:22`

用法

```
current_value = obj.t_from_start
obj.t_from_start = new_value
```

unitree_sdk2_cpp.idl.go2.PathPoint.x

底层 `unitree_go::msg::dds_::PathPoint_` 的 `x` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def x(self) -> float

@x.setter
def x(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/PathPoint_.hpp:23`

用法

```
current_value = obj.x
obj.x = new_value
```

unitree_sdk2_cpp.idl.go2.PathPoint.y

底层 `unitree_go::msg::dds_::PathPoint_` 的 `y` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def y(self) -> float

@y.setter
def y(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/PathPoint_.hpp:24`

用法

```
current_value = obj.y
obj.y = new_value
```

unitree_sdk2_cpp.idl.go2.PathPoint.yaw

底层 `unitree_go::msg::dds_::PathPoint_` 的 `yaw` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def yaw(self) -> float

@yaw.setter
def yaw(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/PathPoint_.hpp:25`

用法

```
current_value = obj.yaw
obj.yaw = new_value
```

unitree_sdk2_cpp.idl.go2.PathPoint.vx

底层 `unitree_go::msg::dds_::PathPoint_` 的 `vx` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def vx(self) -> float

@vx.setter
def vx(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/PathPoint_.hpp:26`

用法

```
current_value = obj.vx
obj.vx = new_value
```

unitree_sdk2_cpp.idl.go2.PathPoint.vy

底层 `unitree_go::msg::dds_::PathPoint_` 的 `vy` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def vy(self) -> float

@vy.setter
def vy(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/PathPoint_.hpp:27`

用法

```
current_value = obj.vy
obj.vy = new_value
```

unitree_sdk2_cpp.idl.go2.PathPoint.vyaw

底层 `unitree_go::msg::dds_::PathPoint_` 的 `vyaw` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def vyaw(self) -> float

@vyaw.setter
def vyaw(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::PathPoint_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/PathPoint_.hpp:28`

用法

```
current_value = obj.vyaw
obj.vyaw = new_value
```

unitree_sdk2_cpp.idl.go2.Req

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import Req
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>uuid</code>	<code>str</code>	<code>str</code>	AVAILABLE
<code>body</code>	<code>str</code>	<code>str</code>	AVAILABLE

`unitree_sdk2_cpp.idl.go2.Req.__init__`

初始化 `Req` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::Req_`
- 签名: `Req_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/Req_.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Req()
```

`unitree_sdk2_cpp.idl.go2.Req.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::Req_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.Req.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::Req_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.Req.uuid

底层 `unitree_go::msg::dds_::Req_` 的 `uuid` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def uuid(self) -> str

@uuid.setter
def uuid(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::Req_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/Req_.hpp:23`

用法

```
current_value = obj.uuid
obj.uuid = new_value
```

unitree_sdk2_cpp.idl.go2.Req.body

底层 `unitree_go::msg::dds_::Req_` 的 `body` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def body(self) -> str

@body.setter
def body(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::Req_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/Req_.hpp:24`

用法

```
current_value = obj.body
obj.body = new_value
```

unitree_sdk2_cpp.idl.go2.Res

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import Res
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>uuid</code>	str	str	AVAILABLE
<code>data</code>	list[int]	Sequence[int]	AVAILABLE
<code>body</code>	str	str	AVAILABLE

unitree_sdk2_cpp.idl.go2.Res.__init__

初始化 `Res` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::Res_`
- 签名: `Res_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/Res_.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Res()
```

`unitree_sdk2_cpp.idl.go2.Res.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::Res_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.Res.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::Res_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.Res.uuid

底层 `unitree_go::msg::dds_::Res_` 的 `uuid` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串; 长度、编码和允许值由具体协议决定。

签名

```
@property
def uuid(self) -> str

@uuid.setter
def uuid(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::Res_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/Res_.hpp:25`

用法

```
current_value = obj.uuid
obj.uuid = new_value
```

`unitree_sdk2_cpp.idl.go2.Res.data`

底层 `unitree_go::msg::dds_::Res_` 的 `data` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

签名

```
@property
def data(self) -> list[int]

@data.setter
def data(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::Res_`
- 字段类型: `std::vector<uint8_t>`
- 声明位置: `include/unitree/idl/go2/Res_.hpp:26`

用法

```
current_value = obj.data
obj.data = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

`unitree_sdk2_cpp.idl.go2.Res.body`

底层 `unitree_go::msg::dds_::Res_` 的 `body` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def body(self) -> str

@body.setter
def body(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::Res_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/Res_.hpp:27`

用法

```
current_value = obj.body
obj.body = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeCmd

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import SportModeCmd
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>mode</code>	int	int	AVAILABLE
<code>gait_type</code>	int	int	AVAILABLE
<code>speed_level</code>	int	int	AVAILABLE
<code>foot_raise_height</code>	float	float	AVAILABLE

属性	读取类型	写入类型	状态
<code>body_height</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>position</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>euler</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>velocity</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>yaw_speed</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>bms_cmd</code>	<code>BmsCmd</code>	<code>BmsCmd</code>	AVAILABLE
<code>path_point</code>	<code>list[PathPoint]</code>	<code>Sequence[PathPoint]</code>	AVAILABLE

unitree_sdk2_cpp.idl.go2.SportModeCmd.__init__

初始化 SportModeCmd 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 签名: `SportModeCmd_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:41`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = SportModeCmd()
```

unitree_sdk2_cpp.idl.go2.SportModeCmd.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.SportModeCmd.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.SportModeCmd.mode

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `mode` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode(self) -> int

@mode.setter
def mode(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:28`

用法

```
current_value = obj.mode
obj.mode = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeCmd.gait_type

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `gait_type` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def gait_type(self) -> int

@gait_type.setter
def gait_type(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:29`

用法

```
current_value = obj.gait_type
obj.gait_type = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeCmd.speed_level

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `speed_level` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def speed_level(self) -> int

@speed_level.setter
def speed_level(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:30`

用法

```
current_value = obj.speed_level
obj.speed_level = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeCmd.foot_raise_height

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `foot_raise_height` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def foot_raise_height(self) -> float

@foot_raise_height.setter
def foot_raise_height(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:31`

用法

```
current_value = obj.foot_raise_height
obj.foot_raise_height = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeCmd.body_height

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `body_height` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def body_height(self) -> float

@body_height.setter
def body_height(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:32`

用法

```
current_value = obj.body_height
obj.body_height = new_value
```

`unitree_sdk2_cpp.idl.go2.SportModeCmd.position`

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `position` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def position(self) -> list[float]

@position.setter
def position(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[float]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[float]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `std::array<float, 2>`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:33`

用法

```
current_value = obj.position
obj.position = new_value
```


[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.SportModeCmd.euler

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `euler` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def euler(self) -> list[float]

@euler.setter
def euler(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:34`

用法

```
current_value = obj.euler
obj.euler = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.SportModeCmd.velocity

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `velocity` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def velocity(self) -> list[float]

@velocity.setter
def velocity(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `std::array<float, 2>`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:35`

用法

```
current_value = obj.velocity
obj.velocity = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.SportModeCmd.yaw_speed

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `yaw_speed` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def yaw_speed(self) -> float

@yaw_speed.setter
def yaw_speed(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `float`

- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:36`

用法

```
current_value = obj.yaw_speed
obj.yaw_speed = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeCmd.bms_cmd

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `bms_cmd` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def bms_cmd(self) -> BmsCmd

@bms_cmd.setter
def bms_cmd(self, value: BmsCmd) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	BmsCmd	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	BmsCmd	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `::unitree_go::msg::dds_::BmsCmd_`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:37`

用法

```
current_value = obj.bms_cmd
obj.bms_cmd = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeCmd.path_point

底层 `unitree_go::msg::dds_::SportModeCmd_` 的 `path_point` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 30 个元素

签名

```
@property
def path_point(self) -> list[PathPoint]

@path_point.setter
def path_point(self, value: Sequence[PathPoint]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[PathPoint]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[PathPoint]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeCmd_`
- 字段类型: `std::array< ::unitree_go::msg::dds_::PathPoint_, 30>`
- 声明位置: `include/unitree/idl/go2/SportModeCmd_.hpp:38`

用法

```
current_value = obj.path_point
obj.path_point = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.TimeSpec

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import TimeSpec
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>sec</code>	int	int	AVAILABLE
<code>nanosec</code>	int	int	AVAILABLE

unitree_sdk2_cpp.idl.go2.TimeSpec.__init__

初始化 `TimeSpec` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::TimeSpec_`
- 签名: `TimeSpec_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/TimeSpec_.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = TimeSpec()
```

`unitree_sdk2_cpp.idl.go2.TimeSpec.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::TimeSpec_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.TimeSpec.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree_go::msg::dds_::TimeSpec_
- 签名: operator!=(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.TimeSpec.sec

底层 unitree_go::msg::dds_::TimeSpec_ 的 sec 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 -2147483648 到 2147483647。

签名

```
@property
def sec(self) -> int

@sec.setter
def sec(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::TimeSpec_`
- 字段类型: `int32_t`
- 声明位置: `include/unitree/idl/go2/TimeSpec_.hpp:23`

用法

```
current_value = obj.sec
obj.sec = new_value
```

`unitree_sdk2_cpp.idl.go2.TimeSpec.nanosec`

底层 `unitree_go::msg::dds_::TimeSpec_` 的 `nanosec` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def nanosec(self) -> int

@nanosec.setter
def nanosec(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::TimeSpec_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/TimeSpec_.hpp:24`

用法

```
current_value = obj.nanosec
obj.nanosec = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import SportModeState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>stamp</code>	TimeSpec	TimeSpec	AVAILABLE
<code>error_code</code>	int	int	AVAILABLE
<code>imu_state</code>	IMUState	IMUState	AVAILABLE
<code>mode</code>	int	int	AVAILABLE
<code>progress</code>	float	float	AVAILABLE
<code>gait_type</code>	int	int	AVAILABLE
<code>foot_raise_height</code>	float	float	AVAILABLE
<code>position</code>	list[float]	Sequence[float]	AVAILABLE
<code>body_height</code>	float	float	AVAILABLE
<code>velocity</code>	list[float]	Sequence[float]	AVAILABLE
<code>yaw_speed</code>	float	float	AVAILABLE
<code>range_obstacle</code>	list[float]	Sequence[float]	AVAILABLE
<code>foot_force</code>	list[int]	Sequence[int]	AVAILABLE
<code>foot_position_body</code>	list[float]	Sequence[float]	AVAILABLE
<code>foot_speed_body</code>	list[float]	Sequence[float]	AVAILABLE
<code>path_point</code>	list[PathPoint]	Sequence[PathPoint]	AVAILABLE

unitree_sdk2_cpp.idl.go2.SportModeState.__init__

初始化 SportModeState 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 签名: `SportModeState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:48`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = SportModeState()
```

`unitree_sdk2_cpp.idl.go2.SportModeState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.SportModeState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.SportModeState.stamp

底层 `unitree_go::msg::dds_::SportModeState_` 的 `stamp` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def stamp(self) -> TimeSpec

@stamp.setter
def stamp(self, value: TimeSpec) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	TimeSpec	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	TimeSpec	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `::unitree_go::msg::dds_::TimeSpec_`

- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:30`

用法

```
current_value = obj.stamp
obj.stamp = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeState.error_code

底层 `unitree_go::msg::dds_::SportModeState_` 的 `error_code` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def error_code(self) -> int

@error_code.setter
def error_code(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:31`

用法

```
current_value = obj.error_code
obj.error_code = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeState.imu_state

底层 `unitree_go::msg::dds_::SportModeState_` 的 `imu_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def imu_state(self) -> IMUState

@imu_state.setter
def imu_state(self, value: IMUState) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	IMUState	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	IMUState	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `::unitree_go::msg::dds_::IMUState_`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:32`

用法

```
current_value = obj.imu_state
obj.imu_state = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeState.mode

底层 `unitree_go::msg::dds_::SportModeState_` 的 `mode` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode(self) -> int

@mode.setter
def mode(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:33`

用法

```
current_value = obj.mode
obj.mode = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeState.progress

底层 `unitree_go::msg::dds_::SportModeState_` 的 `progress` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def progress(self) -> float

@progress.setter
def progress(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:34`

用法

```
current_value = obj.progress
obj.progress = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeState.gait_type

底层 `unitree_go::msg::dds_::SportModeState_` 的 `gait_type` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def gait_type(self) -> int

@gait_type.setter
def gait_type(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:35`

用法

```
current_value = obj.gait_type
obj.gait_type = new_value
```

`unitree_sdk2_cpp.idl.go2.SportModeState.foot_raise_height`

底层 `unitree_go::msg::dds_::SportModeState_` 的 `foot_raise_height` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def foot_raise_height(self) -> float

@foot_raise_height.setter
def foot_raise_height(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:36`

用法

```
current_value = obj.foot_raise_height
obj.foot_raise_height = new_value
```

`unitree_sdk2_cpp.idl.go2.SportModeState.position`

底层 `unitree_go::msg::dds_::SportModeState_` 的 `position` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好

包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def position(self) -> list[float]

@position.setter
def position(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:37`

用法

```
current_value = obj.position
obj.position = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

`unitree_sdk2_cpp.idl.go2.SportModeState.body_height`

底层 `unitree_go::msg::dds_::SportModeState_` 的 `body_height` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def body_height(self) -> float

@body_height.setter
def body_height(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:38`

用法

```
current_value = obj.body_height
obj.body_height = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeState.velocity

底层 `unitree_go::msg::dds_::SportModeState_` 的 `velocity` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def velocity(self) -> list[float]

@velocity.setter
def velocity(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[float]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[float]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:39`

用法

```
current_value = obj.velocity
obj.velocity = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.SportModeState.yaw_speed

底层 `unitree_go::msg::dds_::SportModeState_` 的 `yaw_speed` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def yaw_speed(self) -> float

@yaw_speed.setter
def yaw_speed(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:40`

用法

```
current_value = obj.yaw_speed
obj.yaw_speed = new_value
```

unitree_sdk2_cpp.idl.go2.SportModeState.range_obstacle

底层 `unitree_go::msg::dds_::SportModeState_` 的 `range_obstacle` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 4 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def range_obstacle(self) -> list[float]

@range_obstacle.setter
def range_obstacle(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `std::array<float, 4>`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:41`

用法

```
current_value = obj.range_obstacle
obj.range_obstacle = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.SportModeState.foot_force

底层 `unitree_go::msg::dds_::SportModeState_` 的 `foot_force` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 4 个元素；元素约束：取值范围为 -32768 到 32767。

签名

```
@property
def foot_force(self) -> list[int]

@foot_force.setter
def foot_force(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `std::array<int16_t, 4>`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:42`

用法

```
current_value = obj.foot_force
obj.foot_force = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.SportModeState.foot_position_body

底层 `unitree_go::msg::dds_::SportModeState_` 的 `foot_position_body` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 12 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def foot_position_body(self) -> list[float]

@foot_position_body.setter
def foot_position_body(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `std::array<float, 12>`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:43`

用法

```
current_value = obj.foot_position_body
obj.foot_position_body = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.SportModeState.foot_speed_body

底层 `unitree_go::msg::dds_::SportModeState_` 的 `foot_speed_body` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 12 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def foot_speed_body(self) -> list[float]
```

```
@foot_speed_body.setter
def foot_speed_body(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `std::array<float, 12>`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:44`

用法

```
current_value = obj.foot_speed_body
obj.foot_speed_body = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

`unitree_sdk2_cpp.idl.go2.SportModeState.path_point`

底层 `unitree_go::msg::dds_::SportModeState_` 的 `path_point` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 10 个元素

签名

```
@property
def path_point(self) -> list[PathPoint]

@path_point.setter
def path_point(self, value: Sequence[PathPoint]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[PathPoint]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[PathPoint]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::SportModeState_`
- 字段类型: `std::array< ::unitree_go::msg::dds_::PathPoint_, 10>`
- 声明位置: `include/unitree/idl/go2/SportModeState_.hpp:45`

用法

```
current_value = obj.path_point
obj.path_point = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.UwbState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import UwbState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>version</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>channel</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>joy_mode</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>orientation_est</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>pitch_est</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>distance_est</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>yaw_est</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>tag_roll</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>tag_pitch</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>tag_yaw</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>base_roll</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>base_pitch</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>base_yaw</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>joystick</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>error_state</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>buttons</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>enabled_from_app</code>	<code>int</code>	<code>int</code>	AVAILABLE

unitree_sdk2_cpp.idl.go2.UwbState.__init__

初始化 `UwbState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 签名: `UwbState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:43`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = UwbState()
```

unitree_sdk2_cpp.idl.go2.UwbState.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`

- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.go2.UwbState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.go2.UwbState.version`

底层 `unitree_go::msg::dds_::UwbState_` 的 `version` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组, 写入序列必须正好包含 2 个元素; 元素约束: 取值范围为 0 到 255。

签名

```
@property
def version(self) -> list[int]
```

```
@version.setter
def version(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	List[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `std::array<uint8_t, 2>`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:24`

用法

```
current_value = obj.version
obj.version = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.UwbState.channel

底层 `unitree_go::msg::dds_::UwbState_` 的 `channel` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def channel(self) -> int

@channel.setter
def channel(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:25`

用法

```
current_value = obj.channel
obj.channel = new_value
```

`unitree_sdk2_cpp.idl.go2.UwbState.joy_mode`

底层 `unitree_go::msg::dds_::UwbState_` 的 `joy_mode` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def joy_mode(self) -> int

@joy_mode.setter
def joy_mode(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:26`

用法

```
current_value = obj.joy_mode
obj.joy_mode = new_value
```

`unitree_sdk2_cpp.idl.go2.UwbState.orientation_est`

底层 `unitree_go::msg::dds_::UwbState_` 的 `orientation_est` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def orientation_est(self) -> float

@orientation_est.setter
def orientation_est(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:27`

用法

```
current_value = obj.orientation_est
obj.orientation_est = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.pitch_est

底层 `unitree_go::msg::dds_::UwbState_` 的 `pitch_est` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def pitch_est(self) -> float

@pitch_est.setter
def pitch_est(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`

- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:28`

用法

```
current_value = obj.pitch_est
obj.pitch_est = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.distance_est

底层 `unitree_go::msg::dds_::UwbState_` 的 `distance_est` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def distance_est(self) -> float

@distance_est.setter
def distance_est(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:29`

用法

```
current_value = obj.distance_est
obj.distance_est = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.yaw_est

底层 `unitree_go::msg::dds_::UwbState_` 的 `yaw_est` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def yaw_est(self) -> float

@yaw_est.setter
def yaw_est(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:30`

用法

```
current_value = obj.yaw_est
obj.yaw_est = new_value
```

`unitree_sdk2_cpp.idl.go2.UwbState.tag_roll`

底层 `unitree_go::msg::dds_::UwbState_` 的 `tag_roll` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def tag_roll(self) -> float

@tag_roll.setter
def tag_roll(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:31`

用法

```
current_value = obj.tag_roll
obj.tag_roll = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.tag_pitch

底层 `unitree_go::msg::dds_::UwbState_` 的 `tag_pitch` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def tag_pitch(self) -> float

@tag_pitch.setter
def tag_pitch(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:32`

用法

```
current_value = obj.tag_pitch
obj.tag_pitch = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.tag_yaw

底层 `unitree_go::msg::dds_::UwbState_` 的 `tag_yaw` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def tag_yaw(self) -> float

@tag_yaw.setter
def tag_yaw(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:33`

用法

```
current_value = obj.tag_yaw
obj.tag_yaw = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.base_roll

底层 `unitree_go::msg::dds_::UwbState_` 的 `base_roll` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def base_roll(self) -> float

@base_roll.setter
def base_roll(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:34`

用法

```
current_value = obj.base_roll
obj.base_roll = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.base_pitch

底层 `unitree_go::msg::dds_::UwbState_` 的 `base_pitch` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def base_pitch(self) -> float

@base_pitch.setter
def base_pitch(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:35`

用法

```
current_value = obj.base_pitch
obj.base_pitch = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.base_yaw

底层 `unitree_go::msg::dds_::UwbState_` 的 `base_yaw` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def base_yaw(self) -> float

@base_yaw.setter
def base_yaw(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:36`

用法

```
current_value = obj.base_yaw
obj.base_yaw = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.joystick

底层 `unitree_go::msg::dds_::UwbState_` 的 `joystick` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def joystick(self) -> list[float]

@joystick.setter
def joystick(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[float]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[float]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `std::array<float, 2>`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:37`

用法

```
current_value = obj.joystick
obj.joystick = new_value
```


[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.UwbState.error_state

底层 `unitree_go::msg::dds_::UwbState_` 的 `error_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def error_state(self) -> int

@error_state.setter
def error_state(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:38`

用法

```
current_value = obj.error_state
obj.error_state = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.buttons

底层 `unitree_go::msg::dds_::UwbState_` 的 `buttons` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def buttons(self) -> int

@buttons.setter
def buttons(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:39`

用法

```
current_value = obj.buttons
obj.buttons = new_value
```

unitree_sdk2_cpp.idl.go2.UwbState.enabled_from_app

底层 `unitree_go::msg::dds_::UwbState_` 的 `enabled_from_app` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def enabled_from_app(self) -> int

@enabled_from_app.setter
def enabled_from_app(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/UwbState_.hpp:40`

用法

```
current_value = obj.enabled_from_app
obj.enabled_from_app = new_value
```

unitree_sdk2_cpp.idl.go2.UwbSwitch

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import UwbSwitch
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>enabled</code>	int	int	AVAILABLE

unitree_sdk2_cpp.idl.go2.UwbSwitch.__init__

初始化 `UwbSwitch` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::UwbSwitch_`
- 签名: `UwbSwitch_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/UwbSwitch_.hpp:26`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = UwbSwitch()
```

unitree_sdk2_cpp.idl.go2.UwbSwitch.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::UwbSwitch_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.UwbSwitch.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::UwbSwitch_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.go2.UwbSwitch.enabled`

底层 `unitree_go::msg::dds_::UwbSwitch_` 的 `enabled` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def enabled(self) -> int

@enabled.setter
def enabled(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::UwbSwitch_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/go2/UwbSwitch_.hpp:23`

用法

```
current_value = obj.enabled
obj.enabled = new_value
```

`unitree_sdk2_cpp.idl.go2.VoxelMapCompressed`

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import VoxelMapCompressed
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>stamp</code>	float	float	AVAILABLE
<code>frame_id</code>	str	str	AVAILABLE
<code>resolution</code>	float	float	AVAILABLE
<code>origin</code>	list[float]	Sequence[float]	AVAILABLE
<code>width</code>	list[int]	Sequence[int]	AVAILABLE
<code>src_size</code>	int	int	AVAILABLE
<code>data</code>	list[int]	Sequence[int]	AVAILABLE

unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.__init__

初始化 `VoxelMapCompressed` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 签名: `VoxelMapCompressed_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/go2/VoxelMapCompressed_.hpp:35`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = VoxelMapCompressed()
```

unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.stamp

底层 `unitree_go::msg::dds_::VoxelMapCompressed_` 的 `stamp` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def stamp(self) -> float

@stamp.setter
def stamp(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/go2/VoxelMapCompressed_.hpp:26`

用法

```
current_value = obj.stamp
obj.stamp = new_value
```


unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.frame_id

底层 `unitree_go::msg::dds_::VoxelMapCompressed_` 的 `frame_id` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def frame_id(self) -> str

@frame_id.setter
def frame_id(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/go2/VoxelMapCompressed_.hpp:27`

用法

```
current_value = obj.frame_id
obj.frame_id = new_value
```

unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.resolution

底层 `unitree_go::msg::dds_::VoxelMapCompressed_` 的 `resolution` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def resolution(self) -> float

@resolution.setter
def resolution(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/go2/VoxelMapCompressed_.hpp:28`

用法

```
current_value = obj.resolution
obj.resolution = new_value
```

unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.origin

底层 `unitree_go::msg::dds_::VoxelMapCompressed_` 的 `origin` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def origin(self) -> list[float]

@origin.setter
def origin(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[float]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[float]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 字段类型: `std::array<double, 3>`
- 声明位置: `include/unitree/idl/go2/VoxelMapCompressed_.hpp:29`

用法

```
current_value = obj.origin
obj.origin = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.width

底层 `unitree_go::msg::dds_::VoxelMapCompressed_` 的 `width` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：取值范围为 0 到 65535。

签名

```
@property
def width(self) -> list[int]

@width.setter
def width(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 字段类型: `std::array<uint16_t, 3>`
- 声明位置: `include/unitree/idl/go2/VoxelMapCompressed_.hpp:30`

用法

```
current_value = obj.width
obj.width = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.src_size

底层 `unitree_go::msg::dds_::VoxelMapCompressed_` 的 `src_size` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 18446744073709551615。

签名

```
@property
def src_size(self) -> int

@src_size.setter
def src_size(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 字段类型: `uint64_t`
- 声明位置: `include/unitree/idl/go2/VoxelMapCompressed_.hpp:31`

用法

```
current_value = obj.src_size
obj.src_size = new_value
```

`unitree_sdk2_cpp.idl.go2.VoxelMapCompressed.data`

底层 `unitree_go::msg::dds_::VoxelMapCompressed_` 的 `data` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 `vector`；元素约束：取值范围为 0 到 255。

签名

```
@property
def data(self) -> list[int]

@data.setter
def data(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、 <code>vector</code> 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_go::msg::dds_::VoxelMapCompressed_`
- 字段类型: `std::vector<uint8_t>`
- 声明位置: `include/unitree/idl/go2/VoxelMapCompressed_.hpp:32`

用法

```
current_value = obj.data
obj.data = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.go2.WirelessController

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.go2 import WirelessController
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>lx</code>	float	float	AVAILABLE
<code>ly</code>	float	float	AVAILABLE
<code>rx</code>	float	float	AVAILABLE
<code>ry</code>	float	float	AVAILABLE
<code>keys</code>	int	int	AVAILABLE

unitree_sdk2_cpp.idl.go2.WirelessController.__init__

初始化 WirelessController 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_go::msg::dds_::WirelessController_`
- 签名: `WirelessController_()`
- 绑定策略: 未单独标注

- 声明位置: `include/unitree/idl/go2/WirelessController.hpp:30`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = WirelessController()
```

unitree_sdk2_cpp.idl.go2.WirelessController.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::WirelessController_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.go2.WirelessController.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_go::msg::dds_::WirelessController_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.go2.WirelessController.lx

底层 `unitree_go::msg::dds_::WirelessController_` 的 `lx` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def lx(self) -> float

@lx.setter
def lx(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::WirelessController_`
- 字段类型: `float`

- 声明位置: `include/unitree/idl/go2/WirelessController_.hpp:23`

用法

```
current_value = obj.lx
obj.lx = new_value
```

unitree_sdk2_cpp.idl.go2.WirelessController.ly

底层 `unitree_go::msg::dds_::WirelessController_` 的 `ly` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def ly(self) -> float

@ly.setter
def ly(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::WirelessController_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/WirelessController_.hpp:24`

用法

```
current_value = obj.ly
obj.ly = new_value
```

unitree_sdk2_cpp.idl.go2.WirelessController.rx

底层 `unitree_go::msg::dds_::WirelessController_` 的 `rx` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def rx(self) -> float

@rx.setter
def rx(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::WirelessController_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/WirelessController_.hpp:25`

用法

```
current_value = obj.rx
obj.rx = new_value
```

`unitree_sdk2_cpp.idl.go2.WirelessController.ry`

底层 `unitree_go::msg::dds_::WirelessController_` 的 `ry` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def ry(self) -> float

@ry.setter
def ry(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::WirelessController_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/go2/WirelessController_.hpp:26`

用法

```
current_value = obj.ry
obj.ry = new_value
```

unitree_sdk2_cpp.idl.go2.WirelessController.keys

底层 `unitree_go::msg::dds_::WirelessController_` 的 `keys` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 65535。

签名

```
@property
def keys(self) -> int

@keys.setter
def keys(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_go::msg::dds_::WirelessController_`
- 字段类型: `uint16_t`
- 声明位置: `include/unitree/idl/go2/WirelessController_.hpp:27`

用法

```
current_value = obj.keys
obj.keys = new_value
```

HG IDL 消息

模块: `unitree_sdk2_cpp.idl.hg`

类索引

类	公开函数签名	属性
AgvBmsState	3	7
BmsCmd	3	2
BmsState	3	13
MotorCmd	3	7
HandCmd	3	2
IMUState	3	5
MotorState	3	10

类	公开函数签名	属性
PressSensorState	3	4
HandState	3	9
LowCmd	3	5
LowState	3	9
MainBoardState	3	4
SportModeState	3	4

unitree_sdk2_cpp.idl.hg.AgvBmsState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import AgvBmsState
```

方法索引

方法	Python 签名	状态	安全分类
__init__	def __init__(self) -> None	AVAILABLE	VALUE_TYPE
__eq__	def __eq__(self, other: object) -> bool	AVAILABLE	VALUE_TYPE
__ne__	def __ne__(self, other: object) -> bool	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
software_version	str	str	AVAILABLE
battery_percentage	int	int	AVAILABLE
current	int	int	AVAILABLE
temperature	list[int]	Sequence[int]	AVAILABLE
docking_status	str	str	AVAILABLE
is_charging	bool	bool	AVAILABLE
is_dc_connected	bool	bool	AVAILABLE

unitree_sdk2_cpp.idl.hg.AgvBmsState.__init__

初始化 AgvBmsState 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`
- 签名: `AgvBmsState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/AgvBmsState_.hpp:34`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = AgvBmsState()
```

`unitree_sdk2_cpp.idl.hg.AgvBmsState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.hg.AgvBmsState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.AgvBmsState.software_version

底层 `unitree_hg::msg::dds_::AgvBmsState_` 的 `software_version` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串; 长度、编码和允许值由具体协议决定。

签名

```
@property
def software_version(self) -> str

@software_version.setter
def software_version(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`

- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/hg/AgvBmsState_.hpp:25`

用法

```
current_value = obj.software_version
obj.software_version = new_value
```

`unitree_sdk2_cpp.idl.hg.AgvBmsState.battery_percentage`

底层 `unitree_hg::msg::dds_::AgvBmsState_` 的 `battery_percentage` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def battery_percentage(self) -> int

@battery_percentage.setter
def battery_percentage(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/AgvBmsState_.hpp:26`

用法

```
current_value = obj.battery_percentage
obj.battery_percentage = new_value
```

`unitree_sdk2_cpp.idl.hg.AgvBmsState.current`

底层 `unitree_hg::msg::dds_::AgvBmsState_` 的 `current` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 -2147483648 到 2147483647。

签名

```
@property
def current(self) -> int

@current.setter
def current(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`
- 字段类型: `int32_t`
- 声明位置: `include/unitree/idl/hg/AgvBmsState_.hpp:27`

用法

```
current_value = obj.current
obj.current = new_value
```

`unitree_sdk2_cpp.idl.hg.AgvBmsState.temperature`

底层 `unitree_hg::msg::dds_::AgvBmsState_` 的 `temperature` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：取值范围为 -32768 到 32767。

签名

```
@property
def temperature(self) -> list[int]

@temperature.setter
def temperature(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`
- 字段类型: `std::array<int16_t, 3>`
- 声明位置: `include/unitree/idl/hg/AgvBmsState_.hpp:28`

用法

```
current_value = obj.temperature
obj.temperature = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.AgvBmsState.docking_status

底层 `unitree_hg::msg::dds_::AgvBmsState_` 的 `docking_status` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def docking_status(self) -> str

@docking_status.setter
def docking_status(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/hg/AgvBmsState_.hpp:29`

用法

```
current_value = obj.docking_status
obj.docking_status = new_value
```

unitree_sdk2_cpp.idl.hg.AgvBmsState.is_charging

底层 `unitree_hg::msg::dds_::AgvBmsState_` 的 `is_charging` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。只接受布尔语义。

签名

```
@property
def is_charging(self) -> bool

@is_charging.setter
def is_charging(self, value: bool) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	bool	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	bool	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`
- 字段类型: `bool`
- 声明位置: `include/unitree/idl/hg/AgvBmsState_.hpp:30`

用法

```
current_value = obj.is_charging
obj.is_charging = new_value
```

`unitree_sdk2_cpp.idl.hg.AgvBmsState.is_dc_connected`

底层 `unitree_hg::msg::dds_::AgvBmsState_` 的 `is_dc_connected` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。只接受布尔语义。

签名

```
@property
def is_dc_connected(self) -> bool

@is_dc_connected.setter
def is_dc_connected(self, value: bool) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	bool	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	bool	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::AgvBmsState_`
- 字段类型: `bool`
- 声明位置: `include/unitree/idl/hg/AgvBmsState_.hpp:31`

用法

```
current_value = obj.is_dc_connected
obj.is_dc_connected = new_value
```

unitree_sdk2_cpp.idl.hg.BmsCmd

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import BmsCmd
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>cmd</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>reserve</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE

unitree_sdk2_cpp.idl.hg.BmsCmd.__init__

初始化 `BmsCmd` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsCmd_`
- 签名: `BmsCmd_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/BmsCmd_.hpp:28`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = BmsCmd()
```

unitree_sdk2_cpp.idl.hg.BmsCmd.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsCmd_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.hg.BmsCmd.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsCmd_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.hg.BmsCmd.cmd`

底层 `unitree_hg::msg::dds_::BmsCmd_` 的 `cmd` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def cmd(self) -> int

@cmd.setter
def cmd(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/BmsCmd_.hpp:24`

用法

```
current_value = obj.cmd
obj.cmd = new_value
```

`unitree_sdk2_cpp.idl.hg.BmsCmd.reserve`

底层 `unitree_hg::msg::dds_::BmsCmd_` 的 `reserve` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组, 写入序列必须正好包含 40 个

元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类：unitree_hg::msg::dds_::BmsCmd_
- 字段类型：std::array<uint8_t, 40>
- 声明位置：include/unitree/idl/hg/BmsCmd_.hpp:25

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.BmsState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import BmsState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>version_high</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>version_low</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>fn</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>cell_vol</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>bmsvoltage</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>current</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>soc</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>soh</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>temperature</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>cycle</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>manufacturer_date</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>bmsstate</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>reserve</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE

`unitree_sdk2_cpp.idl.hg.BmsState.__init__`

初始化 `BmsState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 签名: `BmsState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:39`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = BmsState()
```

`unitree_sdk2_cpp.idl.hg.BmsState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.hg.BmsState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.BmsState.version_high

底层 unitree_hg::msg::dds_::BmsState_ 的 version_high 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def version_high(self) -> int

@version_high.setter
def version_high(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类：unitree_hg::msg::dds_::BmsState_
- 字段类型：uint8_t
- 声明位置：include/unitree/idl/hg/BmsState_.hpp:24

用法

```
current_value = obj.version_high
obj.version_high = new_value
```

unitree_sdk2_cpp.idl.hg.BmsState.version_low

底层 unitree_hg::msg::dds_::BmsState_ 的 version_low 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def version_low(self) -> int

@version_low.setter
def version_low(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:25`

用法

```
current_value = obj.version_low
obj.version_low = new_value
```

unitree_sdk2_cpp.idl.hg.BmsState.fn

底层 `unitree_hg::msg::dds_::BmsState_` 的 `fn` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def fn(self) -> int

@fn.setter
def fn(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:26`

用法

```
current_value = obj.fn
obj.fn = new_value
```

unitree_sdk2_cpp.idl.hg.BmsState.cell_vol

底层 `unitree_hg::msg::dds_::BmsState_` 的 `cell_vol` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 40 个元素；元素约束：取值范围为 0 到 65535。

签名

```
@property
def cell_vol(self) -> list[int]

@cell_vol.setter
def cell_vol(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `std::array<uint16_t, 40>`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:27`

用法

```
current_value = obj.cell_vol
obj.cell_vol = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.BmsState.bmsvoltage

底层 `unitree_hg::msg::dds_::BmsState_` 的 `bmsvoltage` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def bmsvoltage(self) -> list[int]

@bmsvoltage.setter
def bmsvoltage(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `std::array<uint32_t, 3>`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:28`

用法

```
current_value = obj.bmsvoltage
obj.bmsvoltage = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.BmsState.current

底层 `unitree_hg::msg::dds_::BmsState_` 的 `current` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 -2147483648 到 2147483647。

签名

```
@property
def current(self) -> int

@current.setter
def current(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `int32_t`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:29`

用法

```
current_value = obj.current
obj.current = new_value
```

unitree_sdk2_cpp.idl.hg.BmsState.soc

底层 `unitree_hg::msg::dds_::BmsState_` 的 `soc` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def soc(self) -> int

@soc.setter
def soc(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:30`

用法

```
current_value = obj.soc
obj.soc = new_value
```

unitree_sdk2_cpp.idl.hg.BmsState.soh

底层 `unitree_hg::msg::dds_::BmsState_` 的 `soh` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def soh(self) -> int

@soh.setter
def soh(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:31`

用法

```
current_value = obj.soh
obj.soh = new_value
```

`unitree_sdk2_cpp.idl.hg.BmsState.temperature`

底层 `unitree_hg::msg::dds_::BmsState_` 的 `temperature` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 12 个元素；元素约束：取值范围为 -32768 到 32767。

签名

```
@property
def temperature(self) -> list[int]

@temperature.setter
def temperature(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `std::array<int16_t, 12>`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:32`

用法

```
current_value = obj.temperature
obj.temperature = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.BmsState.cycle

底层 `unitree_hg::msg::dds_::BmsState_` 的 `cycle` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 65535。

签名

```
@property
def cycle(self) -> int

@cycle.setter
def cycle(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `uint16_t`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:33`

用法

```
current_value = obj.cycle
obj.cycle = new_value
```

unitree_sdk2_cpp.idl.hg.BmsState.manufacturer_date

底层 `unitree_hg::msg::dds_::BmsState_` 的 `manufacturer_date` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 65535。

签名

```
@property
def manufacturer_date(self) -> int

@manufacturer_date.setter
def manufacturer_date(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `uint16_t`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:34`

用法

```
current_value = obj.manufacturer_date
obj.manufacturer_date = new_value
```

unitree_sdk2_cpp.idl.hg.BmsState.bmsstate

底层 `unitree_hg::msg::dds_::BmsState_` 的 `bmsstate` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 5 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def bmsstate(self) -> list[int]

@bmsstate.setter
def bmsstate(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `std::array<uint32_t, 5>`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:35`

用法

```
current_value = obj.bmsstate
obj.bmsstate = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.BmsState.reserve

底层 `unitree_hg::msg::dds_::BmsState_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::BmsState_`
- 字段类型: `std::array<uint32_t, 3>`
- 声明位置: `include/unitree/idl/hg/BmsState_.hpp:36`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.MotorCmd

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import MotorCmd
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

方法	Python 签名	状态	安全分类
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>mode</code>	int	int	AVAILABLE
<code>q</code>	float	float	AVAILABLE
<code>dq</code>	float	float	AVAILABLE
<code>tau</code>	float	float	AVAILABLE
<code>kp</code>	float	float	AVAILABLE
<code>kd</code>	float	float	AVAILABLE
<code>reserve</code>	int	int	AVAILABLE

unitree_sdk2_cpp.idl.hg.MotorCmd.__init__

初始化 `MotorCmd` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorCmd_`
- 签名: `MotorCmd_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/MotorCmd_.hpp:32`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MotorCmd()
```

unitree_sdk2_cpp.idl.hg.MotorCmd.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorCmd_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.hg.MotorCmd.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorCmd_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注

- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.MotorCmd.mode

底层 unitree_hg::msg::dds_::MotorCmd_ 的 mode 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode(self) -> int

@mode.setter
def mode(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类：unitree_hg::msg::dds_::MotorCmd_
- 字段类型：uint8_t
- 声明位置：include/unitree/idl/hg/MotorCmd_.hpp:23

用法

```
current_value = obj.mode
obj.mode = new_value
```

unitree_sdk2_cpp.idl.hg.MotorCmd.q

底层 unitree_hg::msg::dds_::MotorCmd_ 的 q 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def q(self) -> float

@q.setter
def q(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/MotorCmd_.hpp:24`

用法

```
current_value = obj.q
obj.q = new_value
```

unitree_sdk2_cpp.idl.hg.MotorCmd.dq

底层 `unitree_hg::msg::dds_::MotorCmd_` 的 `dq` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def dq(self) -> float

@dq.setter
def dq(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorCmd_`
- 字段类型: `float`

- 声明位置: `include/unitree/idl/hg/MotorCmd_.hpp:25`

用法

```
current_value = obj.dq
obj.dq = new_value
```

unitree_sdk2_cpp.idl.hg.MotorCmd.tau

底层 `unitree_hg::msg::dds_::MotorCmd_` 的 `tau` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def tau(self) -> float

@tau.setter
def tau(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/MotorCmd_.hpp:26`

用法

```
current_value = obj.tau
obj.tau = new_value
```

unitree_sdk2_cpp.idl.hg.MotorCmd.kp

底层 `unitree_hg::msg::dds_::MotorCmd_` 的 `kp` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def kp(self) -> float

@kp.setter
def kp(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/MotorCmd_.hpp:27`

用法

```
current_value = obj.kp
obj.kp = new_value
```

unitree_sdk2_cpp.idl.hg.MotorCmd.kd

底层 `unitree_hg::msg::dds_::MotorCmd_` 的 `kd` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def kd(self) -> float

@kd.setter
def kd(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorCmd_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/MotorCmd_.hpp:28`

用法

```
current_value = obj.kd
obj.kd = new_value
```

unitree_sdk2_cpp.idl.hg.MotorCmd.reserve

底层 `unitree_hg::msg::dds_::MotorCmd_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> int

@reserve.setter
def reserve(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorCmd_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg/MotorCmd_.hpp:29`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

unitree_sdk2_cpp.idl.hg.HandCmd

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import HandCmd
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>motor_cmd</code>	<code>list[MotorCmd]</code>	<code>Sequence[MotorCmd]</code>	AVAILABLE
<code>reserve</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE

`unitree_sdk2_cpp.idl.hg.HandCmd.__init__`

初始化 `HandCmd` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::HandCmd_`
- 签名: `HandCmd_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/HandCmd_.hpp:31`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = HandCmd()
```

`unitree_sdk2_cpp.idl.hg.HandCmd.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::HandCmd_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.hg.HandCmd.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::HandCmd_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.HandCmd.motor_cmd

底层 `unitree_hg::msg::dds_::HandCmd_` 的 `motor_cmd` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector

签名

```
@property
def motor_cmd(self) -> list[MotorCmd]

@motor_cmd.setter
def motor_cmd(self, value: Sequence[MotorCmd]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[MotorCmd]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[MotorCmd]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::HandCmd_`
- 字段类型: `std::vector< ::unitree_hg::msg::dds_::MotorCmd_>`
- 声明位置: `include/unitree/idl/hg/HandCmd_.hpp:27`

用法

```
current_value = obj.motor_cmd
obj.motor_cmd = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.HandCmd.reserve

底层 `unitree_hg::msg::dds_::HandCmd_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 4 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::HandCmd_`
- 字段类型: `std::array<uint32_t, 4>`
- 声明位置: `include/unitree/idl/hg/HandCmd_.hpp:28`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.IMUState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import IMUState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>quaternion</code>	list[float]	Sequence[float]	AVAILABLE
<code>gyroscope</code>	list[float]	Sequence[float]	AVAILABLE
<code>accelerometer</code>	list[float]	Sequence[float]	AVAILABLE
<code>rpy</code>	list[float]	Sequence[float]	AVAILABLE
<code>temperature</code>	int	int	AVAILABLE

unitree_sdk2_cpp.idl.hg.IMUState.__init__

初始化 `IMUState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::IMUState_`
- 签名: `IMUState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/IMUState_.hpp:31`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = IMUState()
```

`unitree_sdk2_cpp.idl.hg.IMUState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::IMUState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.hg.IMUState.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree_hg::msg::dds_::IMUState_
- 签名: operator!=(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.IMUState.quaternion

底层 unitree_hg::msg::dds_::IMUState_ 的 quaternion 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组, 写入序列必须正好包含 4 个元素; 元素约束: 底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def quaternion(self) -> list[float]

@quaternion.setter
def quaternion(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::IMUState_`
- 字段类型: `std::array<float, 4>`
- 声明位置: `include/unitree/idl/hg/IMUState_.hpp:24`

用法

```
current_value = obj.quaternion
obj.quaternion = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.IMUState.gyroscope

底层 `unitree_hg::msg::dds_::IMUState_` 的 `gyroscope` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def gyroscope(self) -> list[float]

@gyroscope.setter
def gyroscope(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::IMUState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/hg/IMUState_.hpp:25`

用法

```
current_value = obj.gyroscope
obj.gyroscope = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.IMUState.accelerometer

底层 `unitree_hg::msg::dds_::IMUState_` 的 `accelerometer` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def accelerometer(self) -> list[float]

@accelerometer.setter
def accelerometer(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::IMUState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/hg/IMUState_.hpp:26`

用法

```
current_value = obj.accelerometer
obj.accelerometer = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.IMUState.rpy

底层 `unitree_hg::msg::dds_::IMUState_` 的 `rpy` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def rpy(self) -> list[float]

@rpy.setter
def rpy(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::IMUState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/hg/IMUState_.hpp:27`

用法

```
current_value = obj.rpy
obj.rpy = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.IMUState.temperature

底层 `unitree_hg::msg::dds_::IMUState_` 的 `temperature` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 -32768 到 32767。

签名

```
@property
def temperature(self) -> int

@temperature.setter
def temperature(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::IMUState_`

- 字段类型: `int16_t`
- 声明位置: `include/unitree/idl/hg/IMUState_.hpp:28`

用法

```
current_value = obj.temperature
obj.temperature = new_value
```

unitree_sdk2_cpp.idl.hg.MotorState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import MotorState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>mode</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>q</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>dq</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>ddq</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>tau_est</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>temperature</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>vol</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>sensor</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>motorstate</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>reserve</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE

unitree_sdk2_cpp.idl.hg.MotorState.__init__

初始化 `MotorState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 签名: `MotorState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/MotorState_.hpp:36`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MotorState()
```

`unitree_sdk2_cpp.idl.hg.MotorState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.hg.MotorState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.hg.MotorState.mode`

底层 `unitree_hg::msg::dds_::MotorState_` 的 `mode` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode(self) -> int

@mode.setter
def mode(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`

- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/MotorState_.hpp:24`

用法

```
current_value = obj.mode
obj.mode = new_value
```

unitree_sdk2_cpp.idl.hg.MotorState.q

底层 `unitree_hg::msg::dds_::MotorState_` 的 `q` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def q(self) -> float

@q.setter
def q(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/MotorState_.hpp:25`

用法

```
current_value = obj.q
obj.q = new_value
```

unitree_sdk2_cpp.idl.hg.MotorState.dq

底层 `unitree_hg::msg::dds_::MotorState_` 的 `dq` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def dq(self) -> float

@dq.setter
def dq(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/MotorState_.hpp:26`

用法

```
current_value = obj.ddq
obj.ddq = new_value
```

unitree_sdk2_cpp.idl.hg.MotorState.ddq

底层 `unitree_hg::msg::dds_::MotorState_` 的 `ddq` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def ddq(self) -> float

@ddq.setter
def ddq(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 字段类型: `float`

- 声明位置: `include/unitree/idl/hg/MotorState_.hpp:27`

用法

```
current_value = obj.ddq
obj.ddq = new_value
```

unitree_sdk2_cpp.idl.hg.MotorState.tau_est

底层 `unitree_hg::msg::dds_::MotorState_` 的 `tau_est` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def tau_est(self) -> float

@tau_est.setter
def tau_est(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/MotorState_.hpp:28`

用法

```
current_value = obj.tau_est
obj.tau_est = new_value
```

unitree_sdk2_cpp.idl.hg.MotorState.temperature

底层 `unitree_hg::msg::dds_::MotorState_` 的 `temperature` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 -32768 到 32767。

签名

```
@property
def temperature(self) -> list[int]

@temperature.setter
def temperature(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 字段类型: `std::array<int16_t, 2>`
- 声明位置: `include/unitree/idl/hg/MotorState_.hpp:29`

用法

```
current_value = obj.temperature
obj.temperature = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.MotorState.vol

底层 `unitree_hg::msg::dds_::MotorState_` 的 `vol` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def vol(self) -> float

@vol.setter
def vol(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`

- 字段类型: float
- 声明位置: include/unitree/idl/hg/MotorState_.hpp:30

用法

```
current_value = obj.vol
obj.vol = new_value
```

unitree_sdk2_cpp.idl.hg.MotorState.sensor

底层 unitree_hg::msg::dds_::MotorState_ 的 sensor 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def sensor(self) -> list[int]

@sensor.setter
def sensor(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: unitree_hg::msg::dds_::MotorState_
- 字段类型: std::array<uint32_t, 2>
- 声明位置: include/unitree/idl/hg/MotorState_.hpp:31

用法

```
current_value = obj.sensor
obj.sensor = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.MotorState.motorstate

底层 unitree_hg::msg::dds_::MotorState_ 的 motorstate 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def motorstate(self) -> int

@motorstate.setter
def motorstate(self, value: int) -> None
```


可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg/MotorState_.hpp:32`

用法

```
current_value = obj.motorstate
obj.motorstate = new_value
```

unitree_sdk2_cpp.idl.hg.MotorState.reserve

底层 `unitree_hg::msg::dds_::MotorState_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 4 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::MotorState_`
- 字段类型: `std::array<uint32_t, 4>`

- 声明位置: `include/unitree/idl/hg/MotorState_.hpp:33`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 `getter` 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.PressSensorState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import PressSensorState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>pressure</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>temperature</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>lost</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>reserve</code>	<code>int</code>	<code>int</code>	AVAILABLE

unitree_sdk2_cpp.idl.hg.PressSensorState.__init__

初始化 `PressSensorState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::PressSensorState_`
- 签名: `PressSensorState_()`

- 绑定策略：未单独标注
- 声明位置：include/unitree/idl/hg/PressSensorState_.hpp:30

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PressSensorState()
```

unitree_sdk2_cpp.idl.hg.PressSensorState.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类：unitree_hg::msg::dds_::PressSensorState_
- 签名：operator==(const value &) const
- 绑定策略：未单独标注
- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.hg.PressSensorState.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::PressSensorState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.hg.PressSensorState.pressure`

底层 `unitree_hg::msg::dds_::PressSensorState_` 的 `pressure` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组, 写入序列必须正好包含 12 个元素; 元素约束: 底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def pressure(self) -> list[float]

@pressure.setter
def pressure(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[float]</code>	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[float]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::PressSensorState_`
- 字段类型: `std::array<float, 12>`

- 声明位置: `include/unitree/idl/hg/PressSensorState_.hpp:24`

用法

```
current_value = obj.pressure
obj.pressure = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.PressSensorState.temperature

底层 `unitree_hg::msg::dds_::PressSensorState_` 的 `temperature` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 12 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def temperature(self) -> list[float]

@temperature.setter
def temperature(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[float]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[float]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::PressSensorState_`
- 字段类型: `std::array<float, 12>`
- 声明位置: `include/unitree/idl/hg/PressSensorState_.hpp:25`

用法

```
current_value = obj.temperature
obj.temperature = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.PressSensorState.lost

底层 `unitree_hg::msg::dds_::PressSensorState_` 的 `lost` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def lost(self) -> int
```

```
@lost.setter
def lost(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::PressSensorState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg/PressSensorState_.hpp:26`

用法

```
current_value = obj.lost
obj.lost = new_value
```

`unitree_sdk2_cpp.idl.hg.PressSensorState.reserve`

底层 `unitree_hg::msg::dds_::PressSensorState_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> int

@reserve.setter
def reserve(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::PressSensorState_`

- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg/PressSensorState_.hpp:27`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

unitree_sdk2_cpp.idl.hg.HandState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import HandState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>motor_state</code>	<code>list[MotorState]</code>	<code>Sequence[MotorState]</code>	AVAILABLE
<code>press_sensor_state</code>	<code>list[PressSensorState]</code>	<code>Sequence[PressSensorState]</code>	AVAILABLE
<code>imu_state</code>	<code>IMUState</code>	<code>IMUState</code>	AVAILABLE
<code>power_v</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>power_a</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>system_v</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>device_v</code>	<code>float</code>	<code>float</code>	AVAILABLE
<code>error</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>reserve</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE

unitree_sdk2_cpp.idl.hg.HandState.__init__

初始化 `HandState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 签名: `HandState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:42`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = HandState()
```

`unitree_sdk2_cpp.idl.hg.HandState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.hg.HandState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.HandState.motor_state

底层 `unitree_hg::msg::dds_::HandState_` 的 `motor_state` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 `vector`

签名

```
@property
def motor_state(self) -> list[MotorState]

@motor_state.setter
def motor_state(self, value: Sequence[MotorState]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[MotorState]</code>	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[MotorState]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 字段类型: `std::vector< ::unitree_hg::msg::dds_::MotorState_>`
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:31`

用法

```
current_value = obj.motor_state
obj.motor_state = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

`unitree_sdk2_cpp.idl.hg.HandState.press_sensor_state`

底层 `unitree_hg::msg::dds_::HandState_` 的 `press_sensor_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector

签名

```
@property
def press_sensor_state(self) -> list[PressSensorState]

@press_sensor_state.setter
def press_sensor_state(self, value: Sequence[PressSensorState]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[PressSensorState]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[PressSensorState]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 字段类型: `std::vector< ::unitree_hg::msg::dds_::PressSensorState_>`
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:32`

用法

```
current_value = obj.press_sensor_state
obj.press_sensor_state = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.HandState.imu_state

底层 `unitree_hg::msg::dds_::HandState_` 的 `imu_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def imu_state(self) -> IMUState

@imu_state.setter
def imu_state(self, value: IMUState) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	IMUState	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	IMUState	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 字段类型: `::unitree_hg::msg::dds_::IMUState_`
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:33`

用法

```
current_value = obj.imu_state
obj.imu_state = new_value
```

unitree_sdk2_cpp.idl.hg.HandState.power_v

底层 `unitree_hg::msg::dds_::HandState_` 的 `power_v` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def power_v(self) -> float

@power_v.setter
def power_v(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:34`

用法

```
current_value = obj.power_v
obj.power_v = new_value
```

`unitree_sdk2_cpp.idl.hg.HandState.power_a`

底层 `unitree_hg::msg::dds_::HandState_` 的 `power_a` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def power_a(self) -> float

@power_a.setter
def power_a(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:35`

用法

```
current_value = obj.power_a
obj.power_a = new_value
```

`unitree_sdk2_cpp.idl.hg.HandState.system_v`

底层 `unitree_hg::msg::dds_::HandState_` 的 `system_v` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或

业务安全范围。

签名

```
@property
def system_v(self) -> float

@system_v.setter
def system_v(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:36`

用法

```
current_value = obj.system_v
obj.system_v = new_value
```

unitree_sdk2_cpp.idl.hg.HandState.device_v

底层 `unitree_hg::msg::dds_::HandState_` 的 `device_v` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def device_v(self) -> float

@device_v.setter
def device_v(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:37`

用法

```
current_value = obj.device_v
obj.device_v = new_value
```

unitree_sdk2_cpp.idl.hg.HandState.error

底层 `unitree_hg::msg::dds_::HandState_` 的 `error` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def error(self) -> list[int]

@error.setter
def error(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 字段类型: `std::array<uint32_t, 2>`
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:38`

用法

```
current_value = obj.error
obj.error = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.HandState.reserve

底层 `unitree_hg::msg::dds_::HandState_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::HandState_`
- 字段类型: `std::array<uint32_t, 2>`
- 声明位置: `include/unitree/idl/hg/HandState_.hpp:39`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.LowCmd

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import LowCmd
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>mode_pr</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>mode_machine</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>motor_cmd</code>	<code>list[MotorCmd]</code>	<code>Sequence[MotorCmd]</code>	AVAILABLE
<code>reserve</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>crc</code>	<code>int</code>	<code>int</code>	AVAILABLE

`unitree_sdk2_cpp.idl.hg.LowCmd.__init__`

初始化 `LowCmd` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / **VALUE_TYPE**: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 签名: `LowCmd_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/LowCmd_.hpp:33`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LowCmd()
```

`unitree_sdk2_cpp.idl.hg.LowCmd.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / **VALUE_TYPE**: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.hg.LowCmd.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.LowCmd.mode_pr

底层 `unitree_hg::msg::dds_::LowCmd_` 的 `mode_pr` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode_pr(self) -> int

@mode_pr.setter
def mode_pr(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/LowCmd_.hpp:26`

用法

```
current_value = obj.mode_pr
obj.mode_pr = new_value
```

unitree_sdk2_cpp.idl.hg.LowCmd.mode_machine

底层 `unitree_hg::msg::dds_::LowCmd_` 的 `mode_machine` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode_machine(self) -> int

@mode_machine.setter
def mode_machine(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/LowCmd_.hpp:27`

用法

```
current_value = obj.mode_machine
obj.mode_machine = new_value
```

unitree_sdk2_cpp.idl.hg.LowCmd.motor_cmd

底层 `unitree_hg::msg::dds_::LowCmd_` 的 `motor_cmd` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 35 个元素

签名

```
@property
def motor_cmd(self) -> list[MotorCmd]

@motor_cmd.setter
def motor_cmd(self, value: Sequence[MotorCmd]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[MotorCmd]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[MotorCmd]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 字段类型: `std::array< ::unitree_hg::msg::dds_::MotorCmd_, 35>`
- 声明位置: `include/unitree/idl/hg/LowCmd_.hpp:28`

用法

```
current_value = obj.motor_cmd
obj.motor_cmd = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.LowCmd.reserve

底层 `unitree_hg::msg::dds_::LowCmd_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 4 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 字段类型: `std::array<uint32_t, 4>`
- 声明位置: `include/unitree/idl/hg/LowCmd_.hpp:29`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.LowCmd.crc

底层 `unitree_hg::msg::dds_::LowCmd_` 的 `crc` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def crc(self) -> int

@crc.setter
def crc(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::LowCmd_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg/LowCmd_.hpp:30`

用法

```
current_value = obj.crc
obj.crc = new_value
```

unitree_sdk2_cpp.idl.hg.LowState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import LowState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>version</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>mode_pr</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>mode_machine</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>tick</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>imu_state</code>	<code>IMUState</code>	<code>IMUState</code>	AVAILABLE
<code>motor_state</code>	<code>list[MotorState]</code>	<code>Sequence[MotorState]</code>	AVAILABLE
<code>wireless_remote</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>reserve</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>crc</code>	<code>int</code>	<code>int</code>	AVAILABLE

unitree_sdk2_cpp.idl.hg.LowState.__init__

初始化 `LowState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 签名: `LowState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:39`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LowState()
```

`unitree_sdk2_cpp.idl.hg.LowState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.hg.LowState.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类：unitree_hg::msg::dds_::LowState_
- 签名：operator!=(const value &) const
- 绑定策略：未单独标注
- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.LowState.version

底层 unitree_hg::msg::dds_::LowState_ 的 version 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 2 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def version(self) -> list[int]

@version.setter
def version(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 字段类型: `std::array<uint32_t, 2>`
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:28`

用法

```
current_value = obj.version
obj.version = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.LowState.mode_pr

底层 `unitree_hg::msg::dds_::LowState_` 的 `mode_pr` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode_pr(self) -> int

@mode_pr.setter
def mode_pr(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:29`

用法

```
current_value = obj.mode_pr
obj.mode_pr = new_value
```

unitree_sdk2_cpp.idl.hg.LowState.mode_machine

底层 `unitree_hg::msg::dds_::LowState_` 的 `mode_machine` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def mode_machine(self) -> int

@mode_machine.setter
def mode_machine(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:30`

用法

```
current_value = obj.mode_machine
obj.mode_machine = new_value
```

unitree_sdk2_cpp.idl.hg.LowState.tick

底层 `unitree_hg::msg::dds_::LowState_` 的 `tick` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def tick(self) -> int

@tick.setter
def tick(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:31`

用法

```
current_value = obj.tick
obj.tick = new_value
```

unitree_sdk2_cpp.idl.hg.LowState.imu_state

底层 `unitree_hg::msg::dds_::LowState_` 的 `imu_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def imu_state(self) -> IMUState

@imu_state.setter
def imu_state(self, value: IMUState) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	IMUState	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	IMUState	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 字段类型: `::unitree_hg::msg::dds_::IMUState_`
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:32`

用法

```
current_value = obj.imu_state
obj.imu_state = new_value
```

unitree_sdk2_cpp.idl.hg.LowState.motor_state

底层 `unitree_hg::msg::dds_::LowState_` 的 `motor_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 35 个元素

签名

```
@property
def motor_state(self) -> list[MotorState]

@motor_state.setter
def motor_state(self, value: Sequence[MotorState]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[MotorState]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[MotorState]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 字段类型: `std::array< ::unitree_hg::msg::dds_::MotorState_, 35>`
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:33`

用法

```
current_value = obj.motor_state
obj.motor_state = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.LowState.wireless_remote

底层 `unitree_hg::msg::dds_::LowState_` 的 `wireless_remote` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 40 个元素；元素约束：取值范围为 0 到 255。

签名

```
@property
def wireless_remote(self) -> list[int]

@wireless_remote.setter
def wireless_remote(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 字段类型: `std::array<uint8_t, 40>`
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:34`

用法

```
current_value = obj.wireless_remote
obj.wireless_remote = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.LowState.reserve

底层 `unitree_hg::msg::dds_::LowState_` 的 `reserve` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 4 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def reserve(self) -> list[int]

@reserve.setter
def reserve(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 字段类型: `std::array<uint32_t, 4>`
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:35`

用法

```
current_value = obj.reserve
obj.reserve = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.LowState.crc

底层 `unitree_hg::msg::dds_::LowState_` 的 `crc` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def crc(self) -> int

@crc.setter
def crc(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::LowState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg/LowState_.hpp:36`

用法

```
current_value = obj.crc
obj.crc = new_value
```

unitree_sdk2_cpp.idl.hg.MainBoardState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import MainBoardState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

方法	Python 签名	状态	安全分类
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>fan_state</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>temperature</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE
<code>value</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>state</code>	<code>list[int]</code>	<code>Sequence[int]</code>	AVAILABLE

`unitree_sdk2_cpp.idl.hg.MainBoardState.__init__`

初始化 `MainBoardState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::MainBoardState_`
- 签名: `MainBoardState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/MainBoardState_.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MainBoardState()
```

`unitree_sdk2_cpp.idl.hg.MainBoardState.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::MainBoardState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.hg.MainBoardState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::MainBoardState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.MainBoardState.fan_state

底层 `unitree_hg::msg::dds_::MainBoardState_` 的 `fan_state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 6 个元素；元素约束：取值范围为 0 到 65535。

签名

```
@property
def fan_state(self) -> list[int]

@fan_state.setter
def fan_state(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::MainBoardState_`
- 字段类型: `std::array<uint16_t, 6>`
- 声明位置: `include/unitree/idl/hg/MainBoardState_.hpp:24`

用法

```
current_value = obj.fan_state
obj.fan_state = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.MainBoardState.temperature

底层 `unitree_hg::msg::dds_::MainBoardState_` 的 `temperature` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 6 个元素；元素约束：取值范围为 -32768 到 32767。

签名

```
@property
def temperature(self) -> list[int]

@temperature.setter
def temperature(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::MainBoardState_`
- 字段类型: `std::array<int16_t, 6>`
- 声明位置: `include/unitree/idl/hg/MainBoardState_.hpp:25`

用法

```
current_value = obj.temperature
obj.temperature = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.MainBoardState.value

底层 `unitree_hg::msg::dds_::MainBoardState_` 的 `value` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 6 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def value(self) -> list[float]

@value.setter
def value(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::MainBoardState_`

- 字段类型: `std::array<float, 6>`
- 声明位置: `include/unitree/idl/hg/MainBoardState_.hpp:26`

用法

```
current_value = obj.value
obj.value = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 `getter` 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.MainBoardState.state

底层 `unitree_hg::msg::dds_::MainBoardState_` 的 `state` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 6 个元素；元素约束：取值范围为 0 到 4294967295。

签名

```
@property
def state(self) -> list[int]

@state.setter
def state(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
<code>value</code>	<code>Sequence[int]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
<code>getter</code> 返回值	<code>list[int]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg::msg::dds_::MainBoardState_`
- 字段类型: `std::array<uint32_t, 6>`
- 声明位置: `include/unitree/idl/hg/MainBoardState_.hpp:27`

用法

```
current_value = obj.state
obj.state = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 `getter` 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg.SportModeState

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg import SportModeState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>fsm_id</code>	int	int	AVAILABLE
<code>fsm_mode</code>	int	int	AVAILABLE
<code>task_id</code>	int	int	AVAILABLE
<code>task_time</code>	float	float	AVAILABLE

unitree_sdk2_cpp.idl.hg.SportModeState.__init__

初始化 SportModeState 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg::msg::dds_::SportModeState_`
- 签名: `SportModeState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg/SportModeState_.hpp:29`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = SportModeState()
```

unitree_sdk2_cpp.idl.hg.SportModeState.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::SportModeState_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.hg.SportModeState.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg::msg::dds_::SportModeState_`
- 签名: `operator!=(const value &) const`

- 绑定策略：未单独标注
- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg.SportModeState.fsm_id

底层 `unitree_hg::msg::dds_::SportModeState_` 的 `fsm_id` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def fsm_id(self) -> int

@fsm_id.setter
def fsm_id(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类： `unitree_hg::msg::dds_::SportModeState_`
- 字段类型： `uint32_t`
- 声明位置： `include/unitree/idl/hg/SportModeState_.hpp:23`

用法

```
current_value = obj.fsm_id
obj.fsm_id = new_value
```

unitree_sdk2_cpp.idl.hg.SportModeState.fsm_mode

底层 `unitree_hg::msg::dds_::SportModeState_` 的 `fsm_mode` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def fsm_mode(self) -> int

@fsm_mode.setter
def fsm_mode(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::SportModeState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg/SportModeState_.hpp:24`

用法

```
current_value = obj.fsm_mode
obj.fsm_mode = new_value
```

unitree_sdk2_cpp.idl.hg.SportModeState.task_id

底层 `unitree_hg::msg::dds_::SportModeState_` 的 `task_id` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def task_id(self) -> int

@task_id.setter
def task_id(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::SportModeState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg/SportModeState_.hpp:25`

用法

```
current_value = obj.task_id
obj.task_id = new_value
```

`unitree_sdk2_cpp.idl.hg.SportModeState.task_time`

底层 `unitree_hg::msg::dds_::SportModeState_` 的 `task_time` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def task_time(self) -> float

@task_time.setter
def task_time(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `unitree_hg::msg::dds_::SportModeState_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/hg/SportModeState_.hpp:26`

用法

```
current_value = obj.task_time
obj.task_time = new_value
```

HG Double-IMU IDL 消息

模块: `unitree_sdk2_cpp.idl.hg_doubleimu`

类索引

类	公开函数签名	属性
doubleIMUState	3	6

`unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState`

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.hg_doubleimu import doubleIMUState
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>quaternion</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>gyroscope</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>accelerometer</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>rpy</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>temperature</code>	<code>int</code>	<code>int</code>	AVAILABLE
<code>tick</code>	<code>int</code>	<code>int</code>	AVAILABLE

`unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState.__init__`

初始化 `doubleIMUState` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree_hg_doubleimu::msg::dds_::doubleIMUState_`
- 签名: `doubleIMUState_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/hg_doubleimu/doubleIMUState_.hpp:32`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = doubleIMUState()
```


unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree_hg_doubleimu::msg::dds_::doubleIMUState_
- 签名: operator==(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree_hg_doubleimu::msg::dds_::doubleIMUState_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState.quaternion

底层 `unitree_hg_doubleimu::msg::dds_::doubleIMUState_` 的 `quaternion` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组, 写入序列必须正好包含 4 个元素; 元素约束: 底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def quaternion(self) -> list[float]

@quaternion.setter
def quaternion(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[float]</code>	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[float]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg_doubleimu::msg::dds_::doubleIMUState_`
- 字段类型: `std::array<float, 4>`
- 声明位置: `include/unitree/idl/hg_doubleimu/doubleIMUState_.hpp:24`

用法

```
current_value = obj.quaternion
obj.quaternion = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState.gyroscope

底层 `unitree_hg_doubleimu::msg::dds_::doubleIMUState_` 的 `gyroscope` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def gyroscope(self) -> list[float]

@gyroscope.setter
def gyroscope(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg_doubleimu::msg::dds_::doubleIMUState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/hg_doubleimu/doubleIMUState_.hpp:25`

用法

```
current_value = obj.gyroscope
obj.gyroscope = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState.accelerometer

底层 `unitree_hg_doubleimu::msg::dds_::doubleIMUState_` 的 `accelerometer` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def accelerometer(self) -> list[float]

@accelerometer.setter
def accelerometer(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg_doubleimu::msg::dds_::doubleIMUState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/hg_doubleimu/doubleIMUState_.hpp:26`

用法

```
current_value = obj.accelerometer
obj.accelerometer = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState.rpy

底层 `unitree_hg_doubleimu::msg::dds_::doubleIMUState_` 的 `rpy` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 3 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def rpy(self) -> list[float]

@rpy.setter
def rpy(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `unitree_hg_doubleimu::msg::dds_::doubleIMUState_`
- 字段类型: `std::array<float, 3>`
- 声明位置: `include/unitree/idl/hg_doubleimu/doubleIMUState_.hpp:27`

用法

```
current_value = obj.rpy
obj.rpy = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState.temperature

底层 `unitree_hg_doubleimu::msg::dds_::doubleIMUState_` 的 `temperature` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 -32768 到 32767。

签名

```
@property
def temperature(self) -> int

@temperature.setter
def temperature(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg_doubleimu::msg::dds_::doubleIMUState_`
- 字段类型: `int16_t`
- 声明位置: `include/unitree/idl/hg_doubleimu/doubleIMUState_.hpp:28`

用法

```
current_value = obj.temperature
obj.temperature = new_value
```

unitree_sdk2_cpp.idl.hg_doubleimu.doubleIMUState.tick

底层 `unitree_hg_doubleimu::msg::dds_::doubleIMUState_` 的 `tick` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def tick(self) -> int

@tick.setter
def tick(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `unitree_hg_doubleimu::msg::dds_::doubleIMUState_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/hg_doubleimu/doubleIMUState_.hpp:29`

用法

```
current_value = obj.tick
obj.tick = new_value
```

ROS2 兼容 IDL 消息

模块: `unitree_sdk2_cpp.idl.ros2`

类索引

类	公开函数签名	属性
Time	3	2
Header	3	2
Quaternion	3	4
Vector3	3	3
Imu	3	7
Point	3	3
Pose	3	2
MapMetaData	3	5
OccupancyGrid	3	3
PoseWithCovariance	3	2
Twist	3	2
TwistWithCovariance	3	2
Odometry	3	4
Point32	3	3
PointField	3	4
PointCloud2	3	9
PointStamped	3	2
Pose2D	3	3
PoseStamped	3	2
PoseWithCovariance	3	2
Stamped		
QuaternionStamped	3	2
String	3	1
TwistStamped	3	2

类	公开函数签名	属性
<code>TwistWithCovarianceStamped</code>	3	2

unitree_sdk2_cpp.idl.ros2.Time

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Time
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>sec</code>	int	int	AVAILABLE
<code>nanosec</code>	int	int	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Time.__init__

初始化 `Time` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `builtin_interfaces::msg::dds::Time_`
- 签名: `Time_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Time_.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Time()
```

unitree_sdk2_cpp.idl.ros2.Time.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: builtin_interfaces::msg::dds_::Time_
- 签名: operator==(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Time.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `builtin_interfaces::msg::dds_::Time_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.ros2.Time.sec`

底层 `builtin_interfaces::msg::dds_::Time_` 的 `sec` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 -2147483648 到 2147483647。

签名

```
@property
def sec(self) -> int

@sec.setter
def sec(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `builtin_interfaces::msg::dds_::Time_`
- 字段类型: `int32_t`
- 声明位置: `include/unitree/idl/ros2/Time_.hpp:23`

用法

```
current_value = obj.sec
obj.sec = new_value
```

`unitree_sdk2_cpp.idl.ros2.Time.nanosec`

底层 `builtin_interfaces::msg::dds_::Time_` 的 `nanosec` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def nanosec(self) -> int

@nanosec.setter
def nanosec(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: builtin_interfaces::msg::dds_::Time_
- 字段类型: uint32_t
- 声明位置: include/unitree/idl/ros2/Time_.hpp:24

用法

```
current_value = obj.nanosec
obj.nanosec = new_value
```

unitree_sdk2_cpp.idl.ros2.Header

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Header
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>stamp</code>	Any	Any	AVAILABLE
<code>frame_id</code>	str	str	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Header.__init__

初始化 `Header` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `std_msgs::msg::dds_::Header_`
- 签名: `Header_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Header_.hpp:29`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Header()
```

unitree_sdk2_cpp.idl.ros2.Header.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `std_msgs::msg::dds_::Header_`

- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Header.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `std_msgs::msg::dds_::Header_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.Header.stamp

底层 `std_msgs::msg::dds_::Header_` 的 `stamp` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def stamp(self) -> Any

@stamp.setter
def stamp(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `std_msgs::msg::dds_::Header_`
- 字段类型: `::builtin_interfaces::msg::dds_::Time_`
- 声明位置: `include/unitree/idl/ros2/Header_.hpp:25`

用法

```
current_value = obj.stamp
obj.stamp = new_value
```

unitree_sdk2_cpp.idl.ros2.Header.frame_id

底层 `std_msgs::msg::dds_::Header_` 的 `frame_id` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def frame_id(self) -> str

@frame_id.setter
def frame_id(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `std_msgs::msg::dds_::Header_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/ros2/Header_.hpp:26`

用法

```
current_value = obj.frame_id
obj.frame_id = new_value
```

unitree_sdk2_cpp.idl.ros2.Quaternion

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Quaternion
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>x</code>	float	float	AVAILABLE
<code>y</code>	float	float	AVAILABLE
<code>z</code>	float	float	AVAILABLE
<code>w</code>	float	float	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Quaternion.__init__

初始化 Quaternion 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::Quaternion_`
- 签名: `Quaternion_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Quaternion_.hpp:28`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Quaternion()
```

unitree_sdk2_cpp.idl.ros2.Quaternion.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: geometry_msgs::msg::dds_::Quaternion_
- 签名: operator==(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Quaternion.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Quaternion_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.Quaternion.x

底层 `geometry_msgs::msg::dds_::Quaternion_` 的 `x` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def x(self) -> float

@x.setter
def x(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Quaternion_`
- 字段类型: `double`

- 声明位置: `include/unitree/idl/ros2/Quaternion_.hpp:22`

用法

```
current_value = obj.x
obj.x = new_value
```

unitree_sdk2_cpp.idl ros2.Quaternion.y

底层 `geometry_msgs::msg::dds_::Quaternion_` 的 `y` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def y(self) -> float

@y.setter
def y(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Quaternion_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Quaternion_.hpp:23`

用法

```
current_value = obj.y
obj.y = new_value
```

unitree_sdk2_cpp.idl ros2.Quaternion.z

底层 `geometry_msgs::msg::dds_::Quaternion_` 的 `z` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def z(self) -> float

@z.setter
def z(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Quaternion_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Quaternion_.hpp:24`

用法

```
current_value = obj.z
obj.z = new_value
```

unitree_sdk2_cpp.idl.ros2.Quaternion.w

底层 `geometry_msgs::msg::dds_::Quaternion_` 的 `w` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def w(self) -> float

@w.setter
def w(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Quaternion_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Quaternion_.hpp:25`

用法

```
current_value = obj.w
obj.w = new_value
```

unitree_sdk2_cpp.idl.ros2.Vector3

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Vector3
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>x</code>	float	float	AVAILABLE
<code>y</code>	float	float	AVAILABLE
<code>z</code>	float	float	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Vector3.__init__

初始化 `Vector3` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::Vector3_`
- 签名: `Vector3_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Vector3_.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Vector3()
```

unitree_sdk2_cpp.idl.ros2.Vector3.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Vector3_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Vector3.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Vector3_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.Vector3.x

底层 `geometry_msgs::msg::dds_::Vector3_` 的 `x` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def x(self) -> float

@x.setter
def x(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Vector3_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Vector3_.hpp:22`

用法

```
current_value = obj.x
obj.x = new_value
```

unitree_sdk2_cpp.idl.ros2.Vector3.y

底层 `geometry_msgs::msg::dds_::Vector3_` 的 `y` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def y(self) -> float

@y.setter
def y(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Vector3_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Vector3_.hpp:23`

用法

```
current_value = obj.y
obj.y = new_value
```

unitree_sdk2_cpp.idl.ros2.Vector3.z

底层 `geometry_msgs::msg::dds_::Vector3_` 的 `z` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def z(self) -> float

@z.setter
def z(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Vector3_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Vector3_.hpp:24`

用法

```
current_value = obj.z
obj.z = new_value
```

unitree_sdk2_cpp.idl.ros2.Imu

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Imu
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>header</code>	Any	Any	AVAILABLE
<code>orientation</code>	Any	Any	AVAILABLE
<code>orientation_covariance</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>angular_velocity</code>	Any	Any	AVAILABLE
<code>angular_velocity_covariance</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE
<code>linear_acceleration</code>	Any	Any	AVAILABLE
<code>linear_acceleration_covariance</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Imu.__init__

初始化 `Imu` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `sensor_msgs::msg::dds_::Imu_`
- 签名: `Imu_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Imu_.hpp:38`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Imu()
```

`unitree_sdk2_cpp.idl.ros2.Imu.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `sensor_msgs::msg::dds_::Imu_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Imu.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: sensor_msgs::msg::dds_::Imu_
- 签名: operator!=(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.Imu.header

底层 sensor_msgs::msg::dds_::Imu_ 的 header 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::Imu_`
- 字段类型: `::std_msgs::msg::dds_::Header_`
- 声明位置: `include/unitree/idl/ros2/Imu_.hpp:29`

用法

```
current_value = obj.header
obj.header = new_value
```

unitree_sdk2_cpp.idl.ros2.Imu.orientation

底层 `sensor_msgs::msg::dds_::Imu_` 的 `orientation` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def orientation(self) -> Any

@orientation.setter
def orientation(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::Imu_`
- 字段类型: `::geometry_msgs::msg::dds_::Quaternion_`
- 声明位置: `include/unitree/idl/ros2/Imu_.hpp:30`

用法

```
current_value = obj.orientation
obj.orientation = new_value
```

unitree_sdk2_cpp.idl.ros2.Imu.orientation_covariance

底层 `sensor_msgs::msg::dds_::Imu_` 的 `orientation_covariance` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 9 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def orientation_covariance(self) -> list[float]

@orientation_covariance.setter
def orientation_covariance(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `sensor_msgs::msg::dds_::Imu_`
- 字段类型: `std::array<double, 9>`
- 声明位置: `include/unitree/idl/ros2/Imu_.hpp:31`

用法

```
current_value = obj.orientation_covariance
obj.orientation_covariance = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.ros2.Imu.angular_velocity

底层 `sensor_msgs::msg::dds_::Imu_` 的 `angular_velocity` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def angular_velocity(self) -> Any

@angular_velocity.setter
def angular_velocity(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::Imu_`
- 字段类型: `::geometry_msgs::msg::dds_::Vector3_`
- 声明位置: `include/unitree/idl/ros2/Imu_.hpp:32`

用法

```
current_value = obj.angular_velocity
obj.angular_velocity = new_value
```

unitree_sdk2_cpp.idl ros2.Imu.angular_velocity_covariance

底层 `sensor_msgs::msg::dds_::Imu_` 的 `angular_velocity_covariance` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 9 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def angular_velocity_covariance(self) -> list[float]

@angular_velocity_covariance.setter
def angular_velocity_covariance(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[float]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[float]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `sensor_msgs::msg::dds_::Imu_`
- 字段类型: `std::array<double, 9>`
- 声明位置: `include/unitree/idl/ros2/Imu_.hpp:33`

用法

```
current_value = obj.angular_velocity_covariance
obj.angular_velocity_covariance = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.ros2.Imu.linear_acceleration

底层 `sensor_msgs::msg::dds_::Imu_` 的 `linear_acceleration` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def linear_acceleration(self) -> Any

@linear_acceleration.setter
def linear_acceleration(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::Imu_`
- 字段类型: `::geometry_msgs::msg::dds_::Vector3_`
- 声明位置: `include/unitree/idl/ros2/Imu_.hpp:34`

用法

```
current_value = obj.linear_acceleration
obj.linear_acceleration = new_value
```

unitree_sdk2_cpp.idl.ros2.Imu.linear_acceleration_covariance

底层 `sensor_msgs::msg::dds_::Imu_` 的 `linear_acceleration_covariance` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 9 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def linear_acceleration_covariance(self) -> list[float]

@linear_acceleration_covariance.setter
def linear_acceleration_covariance(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `sensor_msgs::msg::dds_::Imu_`
- 字段类型: `std::array<double, 9>`
- 声明位置: `include/unitree/idl/ros2/Imu_.hpp:35`

用法

```
current_value = obj.linear_acceleration_covariance
obj.linear_acceleration_covariance = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.ros2.Point

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Point
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>x</code>	float	float	AVAILABLE
<code>y</code>	float	float	AVAILABLE
<code>z</code>	float	float	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Point.__init__

初始化 `Point` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point_`
- 签名: `Point_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Point_.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Point()
```

`unitree_sdk2_cpp.idl.ros2.Point.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Point.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: geometry_msgs::msg::dds_::Point_
- 签名: operator!=(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.Point.x

底层 geometry_msgs::msg::dds_::Point_ 的 x 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def x(self) -> float

@x.setter
def x(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Point_.hpp:22`

用法

```
current_value = obj.x
obj.x = new_value
```

unitree_sdk2_cpp.idl ros2.Point.y

底层 `geometry_msgs::msg::dds_::Point_` 的 `y` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def y(self) -> float

@y.setter
def y(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Point_.hpp:23`

用法

```
current_value = obj.y
obj.y = new_value
```

unitree_sdk2_cpp.idl.ros2.Point.z

底层 `geometry_msgs::msg::dds_::Point_` 的 `z` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def z(self) -> float

@z.setter
def z(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Point_.hpp:24`

用法

```
current_value = obj.z
obj.z = new_value
```

unitree_sdk2_cpp.idl.ros2.Pose

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Pose
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>position</code>	Point	Point	AVAILABLE
<code>orientation</code>	Quaternion	Quaternion	AVAILABLE

`unitree_sdk2_cpp.idl.ros2.Pose.__init__`

初始化 `Pose` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose_`
- 签名: `Pose_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Pose_.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Pose()
```

`unitree_sdk2_cpp.idl.ros2.Pose.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Pose.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.Pose.position

底层 `geometry_msgs::msg::dds_::Pose_` 的 `position` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def position(self) -> Point

@position.setter
def position(self, value: Point) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Point	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Point	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose_`
- 字段类型: `::geometry_msgs::msg::dds_::Point_`
- 声明位置: `include/unitree/idl/ros2/Pose_.hpp:26`

用法

```
current_value = obj.position
obj.position = new_value
```

unitree_sdk2_cpp.idl.ros2.Pose.orientation

底层 `geometry_msgs::msg::dds_::Pose_` 的 `orientation` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def orientation(self) -> Quaternion

@orientation.setter
def orientation(self, value: Quaternion) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Quaternion	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Quaternion	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose_`
- 字段类型: `::geometry_msgs::msg::dds_::Quaternion_`
- 声明位置: `include/unitree/idl/ros2/Pose_.hpp:27`

用法

```
current_value = obj.orientation
obj.orientation = new_value
```

unitree_sdk2_cpp.idl.ros2.MapMetaData

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import MapMetaData
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>map_load_time</code>	Any	Any	AVAILABLE
<code>resolution</code>	float	float	AVAILABLE
<code>width</code>	int	int	AVAILABLE
<code>height</code>	int	int	AVAILABLE
<code>origin</code>	Any	Any	AVAILABLE

unitree_sdk2_cpp.idl.ros2.MapMetaData.__init__

初始化 MapMetaData 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `nav_msgs::msg::dds_::MapMetaData_`
- 签名: `MapMetaData_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/MapMetaData_.hpp:34`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MapMetaData()
```

`unitree_sdk2_cpp.idl/ros2.MapMetaData.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `nav_msgs::msg::dds_::MapMetaData_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl/ros2.MapMetaData.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `nav_msgs::msg::dds_::MapMetaData_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.MapMetaData.map_load_time

底层 `nav_msgs::msg::dds_::MapMetaData_` 的 `map_load_time` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def map_load_time(self) -> Any

@map_load_time.setter
def map_load_time(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `nav_msgs::msg::dds_::MapMetaData_`
- 字段类型: `::builtin_interfaces::msg::dds_::Time_`
- 声明位置: `include/unitree/idl/ros2/MapMetaData_.hpp:27`

用法

```
current_value = obj.map_load_time
obj.map_load_time = new_value
```

unitree_sdk2_cpp.idl ros2.MapMetaData.resolution

底层 `nav_msgs::msg::dds_::MapMetaData_` 的 `resolution` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def resolution(self) -> float

@resolution.setter
def resolution(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `nav_msgs::msg::dds_::MapMetaData_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/ros2/MapMetaData_.hpp:28`

用法

```
current_value = obj.resolution
obj.resolution = new_value
```

unitree_sdk2_cpp.idl ros2.MapMetaData.width

底层 `nav_msgs::msg::dds_::MapMetaData_` 的 `width` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def width(self) -> int

@width.setter
def width(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `nav_msgs::msg::dds_::MapMetaData_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/ros2/MapMetaData_.hpp:29`

用法

```
current_value = obj.width
obj.width = new_value
```

unitree_sdk2_cpp.idl.ros2.MapMetaData.height

底层 `nav_msgs::msg::dds_::MapMetaData_` 的 `height` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def height(self) -> int

@height.setter
def height(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `nav_msgs::msg::dds_::MapMetaData_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/ros2/MapMetaData_.hpp:30`

用法

```
current_value = obj.height
obj.height = new_value
```

unitree_sdk2_cpp.idl ros2.MapMetaData.origin

底层 `nav_msgs::msg::dds_::MapMetaData_` 的 `origin` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def origin(self) -> Any

@origin.setter
def origin(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `nav_msgs::msg::dds_::MapMetaData_`
- 字段类型: `::geometry_msgs::msg::dds_::Pose_`
- 声明位置: `include/unitree/idl/ros2/MapMetaData_.hpp:31`

用法

```
current_value = obj.origin
obj.origin = new_value
```

unitree_sdk2_cpp.idl ros2.OccupancyGrid

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import OccupancyGrid
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE

方法	Python 签名	状态	安全分类
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>header</code>	Any	Any	AVAILABLE
<code>info</code>	MapMetaData	MapMetaData	AVAILABLE
<code>data</code>	list[int]	Sequence[int]	AVAILABLE

unitree_sdk2_cpp.idl.ros2.OccupancyGrid.__init__

初始化 OccupancyGrid 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `nav_msgs::msg::dds_::OccupancyGrid_`
- 签名: `OccupancyGrid_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/OccupancyGrid_.hpp:33`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = OccupancyGrid()
```

unitree_sdk2_cpp.idl.ros2.OccupancyGrid.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `nav_msgs::msg::dds_::OccupancyGrid_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.ros2.OccupancyGrid.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `nav_msgs::msg::dds_::OccupancyGrid_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.OccupancyGrid.header

底层 nav_msgs::msg::dds_::OccupancyGrid_ 的 header 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类：nav_msgs::msg::dds_::OccupancyGrid_
- 字段类型：::std_msgs::msg::dds_::Header_
- 声明位置：include/unitree/idl/ros2/OccupancyGrid_.hpp:28

用法

```
current_value = obj.header
obj.header = new_value
```

unitree_sdk2_cpp.idl.ros2.OccupancyGrid.info

底层 nav_msgs::msg::dds_::OccupancyGrid_ 的 info 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def info(self) -> MapMetaData

@info.setter
def info(self, value: MapMetaData) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	MapMetaData	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	MapMetaData	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `nav_msgs::msg::dds_::OccupancyGrid_`
- 字段类型: `::nav_msgs::msg::dds_::MapMetaData_`
- 声明位置: `include/unitree/idl/ros2/OccupancyGrid_.hpp:29`

用法

```
current_value = obj.info
obj.info = new_value
```

unitree_sdk2_cpp.idl.ros2.OccupancyGrid.data

底层 `nav_msgs::msg::dds_::OccupancyGrid_` 的 `data` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

签名

```
@property
def data(self) -> list[int]

@data.setter
def data(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `nav_msgs::msg::dds_::OccupancyGrid_`
- 字段类型: `std::vector<uint8_t>`
- 声明位置: `include/unitree/idl/ros2/OccupancyGrid_.hpp:30`

用法

```
current_value = obj.data
obj.data = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.ros2.PoseWithCovariance

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import PoseWithCovariance
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>pose</code>	Pose	Pose	AVAILABLE
<code>covariance</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE

unitree_sdk2_cpp.idl.ros2.PoseWithCovariance.__init__

初始化 PoseWithCovariance 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseWithCovariance_`
- 签名: `PoseWithCovariance_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/PoseWithCovariance_.hpp:29`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法


```
value = PoseWithCovariance()
```

unitree_sdk2_cpp.idl.ros2.PoseWithCovariance.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: geometry_msgs::msg::dds_::PoseWithCovariance_
- 签名: operator==(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.PoseWithCovariance.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseWithCovariance_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.ros2.PoseWithCovariance.pose`

底层 `geometry_msgs::msg::dds_::PoseWithCovariance_` 的 `pose` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def pose(self) -> Pose

@pose.setter
def pose(self, value: Pose) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Pose	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Pose	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseWithCovariance_`
- 字段类型: `::geometry_msgs::msg::dds_::Pose_`
- 声明位置: `include/unitree/idl/ros2/PoseWithCovariance_.hpp:25`

用法

```
current_value = obj.pose
obj.pose = new_value
```

`unitree_sdk2_cpp.idl.ros2.PoseWithCovariance.covariance`

底层 `geometry_msgs::msg::dds_::PoseWithCovariance_` 的 `covariance` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组, 写

入序列必须正好包含 36 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def covariance(self) -> list[float]

@covariance.setter
def covariance(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseWithCovariance_`
- 字段类型: `std::array<double, 36>`
- 声明位置: `include/unitree/idl/ros2/PoseWithCovariance_.hpp:26`

用法

```
current_value = obj.covariance
obj.covariance = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.ros2.Twist

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Twist
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>linear</code>	Vector3	Vector3	AVAILABLE
<code>angular</code>	Vector3	Vector3	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Twist.__init__

初始化 `Twist` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::Twist_`
- 签名: `Twist_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Twist_.hpp:28`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Twist()
```

unitree_sdk2_cpp.idl.ros2.Twist.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Twist_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Twist.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Twist_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.Twist.linear

底层 `geometry_msgs::msg::dds_::Twist_` 的 `linear` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def linear(self) -> Vector3

@linear.setter
def linear(self, value: Vector3) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Vector3	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Vector3	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::Twist_`
- 字段类型: `::geometry_msgs::msg::dds_::Vector3_`
- 声明位置: `include/unitree/idl/ros2/Twist_.hpp:24`

用法

```
current_value = obj.linear
obj.linear = new_value
```

unitree_sdk2_cpp.idl.ros2.Twist.angular

底层 `geometry_msgs::msg::dds_::Twist_` 的 `angular` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def angular(self) -> Vector3

@angular.setter
def angular(self, value: Vector3) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Vector3	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Vector3	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::Twist_`
- 字段类型: `::geometry_msgs::msg::dds_::Vector3_`
- 声明位置: `include/unitree/idl/ros2/Twist_.hpp:25`

用法

```
current_value = obj.angular
obj.angular = new_value
```

unitree_sdk2_cpp.idl ros2.TwistWithCovariance

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import TwistWithCovariance
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>twist</code>	Twist	Twist	AVAILABLE
<code>covariance</code>	<code>list[float]</code>	<code>Sequence[float]</code>	AVAILABLE

unitree_sdk2_cpp.idl.ros2.TwistWithCovariance.__init__

初始化 `TwistWithCovariance` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistWithCovariance_`
- 签名: `TwistWithCovariance_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/TwistWithCovariance_.hpp:29`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = TwistWithCovariance()
```

`unitree_sdk2_cpp.idl ros2.TwistWithCovariance.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistWithCovariance_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl ros2.TwistWithCovariance.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistWithCovariance_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.TwistWithCovariance.twist

底层 `geometry_msgs::msg::dds_::TwistWithCovariance_` 的 `twist` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def twist(self) -> Twist

@twist.setter
def twist(self, value: Twist) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Twist	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Twist	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistWithCovariance_`
- 字段类型: `::geometry_msgs::msg::dds_::Twist_`

- 声明位置: `include/unitree/idl/ros2/TwistWithCovariance_.hpp:25`

用法

```
current_value = obj.twist
obj.twist = new_value
```

unitree_sdk2_cpp.idl ros2.TwistWithCovariance.covariance

底层 `geometry_msgs::msg::dds_::TwistWithCovariance_` 的 `covariance` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是固定长度数组，写入序列必须正好包含 36 个元素；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def covariance(self) -> list[float]

@covariance.setter
def covariance(self, value: Sequence[float]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[float]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[float]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistWithCovariance_`
- 字段类型: `std::array<double, 36>`
- 声明位置: `include/unitree/idl/ros2/TwistWithCovariance_.hpp:26`

用法

```
current_value = obj.covariance
obj.covariance = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl ros2.Odometry

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Odometry
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>header</code>	Any	Any	AVAILABLE
<code>child_frame_id</code>	str	str	AVAILABLE
<code>pose</code>	Any	Any	AVAILABLE
<code>twist</code>	Any	Any	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Odometry.__init__

初始化 `Odometry` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `nav_msgs::msg::dds_::Odometry_`
- 签名: `Odometry_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Odometry_.hpp:35`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Odometry()
```

unitree_sdk2_cpp.idl.ros2.Odometry.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `nav_msgs::msg::dds_::Odometry_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Odometry.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `nav_msgs::msg::dds_::Odometry_`
- 签名: `operator!=(const value &) const`

- 绑定策略：未单独标注
- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.0dometry.header

底层 nav_msgs::msg::dds_::0dometry_ 的 header 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类：nav_msgs::msg::dds_::0dometry_
- 字段类型：::std_msgs::msg::dds_::Header_
- 声明位置：include/unitree/idl/ros2/0dometry_.hpp:29

用法

```
current_value = obj.header
obj.header = new_value
```

unitree_sdk2_cpp.idl.ros2.0dometry.child_frame_id

底层 nav_msgs::msg::dds_::0dometry_ 的 child_frame_id 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串；长度、编码和允许值由具体协议决定。

签名

```
@property
def child_frame_id(self) -> str

@child_frame_id.setter
def child_frame_id(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `nav_msgs::msg::dds_::Odometry_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/ros2/Odometry_.hpp:30`

用法

```
current_value = obj.child_frame_id
obj.child_frame_id = new_value
```

unitree_sdk2_cpp.idl.ros2.Odometry.pose

底层 `nav_msgs::msg::dds_::Odometry_` 的 `pose` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def pose(self) -> Any

@pose.setter
def pose(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `nav_msgs::msg::dds_::Odometry_`
- 字段类型: `::geometry_msgs::msg::dds_::PoseWithCovariance_`
- 声明位置: `include/unitree/idl/ros2/Odometry_.hpp:31`

用法

```
current_value = obj.pose
obj.pose = new_value
```

unitree_sdk2_cpp.idl.ros2.0dometry.twist

底层 `nav_msgs::msg::dds_::0dometry_` 的 `twist` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def twist(self) -> Any

@twist.setter
def twist(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `nav_msgs::msg::dds_::0dometry_`
- 字段类型: `::geometry_msgs::msg::dds_::TwistWithCovariance_`
- 声明位置: `include/unitree/idl/ros2/0dometry_.hpp:32`

用法

```
current_value = obj.twist
obj.twist = new_value
```

unitree_sdk2_cpp.idl.ros2.Point32

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Point32
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
x	float	float	AVAILABLE
y	float	float	AVAILABLE
z	float	float	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Point32.__init__

初始化 `Point32` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point32_`
- 签名: `Point32_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Point32_.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Point32()
```

unitree_sdk2_cpp.idl.ros2.Point32.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point32_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.Point32.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point32_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.Point32.x

底层 `geometry_msgs::msg::dds_::Point32_` 的 `x` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def x(self) -> float

@x.setter
def x(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point32_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/ros2/Point32_.hpp:22`

用法

```
current_value = obj.x
obj.x = new_value
```

unitree_sdk2_cpp.idl.ros2.Point32.y

底层 `geometry_msgs::msg::dds_::Point32_` 的 `y` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def y(self) -> float

@y.setter
def y(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point32_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/ros2/Point32_.hpp:23`

用法

```
current_value = obj.y
obj.y = new_value
```

unitree_sdk2_cpp.idl.ros2.Point32.z

底层 `geometry_msgs::msg::dds_::Point32_` 的 `z` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def z(self) -> float

@z.setter
def z(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Point32_`
- 字段类型: `float`
- 声明位置: `include/unitree/idl/ros2/Point32_.hpp:24`

用法

```
current_value = obj.z
obj.z = new_value
```

unitree_sdk2_cpp.idl.ros2.PointField

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import PointField
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>name</code>	str	str	AVAILABLE
<code>offset</code>	int	int	AVAILABLE
<code>datatype</code>	int	int	AVAILABLE
<code>count</code>	int	int	AVAILABLE

unitree_sdk2_cpp.idl.ros2.PointField.__init__

初始化 PointField 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointField_`
- 签名: `PointField_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/PointField_.hpp:50`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PointField()
```

unitree_sdk2_cpp.idl.ros2.PointField.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointField_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.PointField.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointField_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.PointField.name

底层 `sensor_msgs::msg::dds_::PointField_` 的 `name` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串; 长度、编码和允许值由具体协议决定。

签名

```
@property
def name(self) -> str

@name.setter
def name(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointField_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/ros2/PointField_.hpp:44`

用法

```
current_value = obj.name
obj.name = new_value
```

unitree_sdk2_cpp.idl.ros2.PointField.offset

底层 `sensor_msgs::msg::dds_::PointField_` 的 `offset` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def offset(self) -> int

@offset.setter
def offset(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointField_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/ros2/PointField_.hpp:45`

用法

```
current_value = obj.offset
obj.offset = new_value
```

unitree_sdk2_cpp.idl.ros2.PointField.datatype

底层 `sensor_msgs::msg::dds_::PointField_` 的 `datatype` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 255。

签名

```
@property
def datatype(self) -> int

@datatype.setter
def datatype(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointField_`
- 字段类型: `uint8_t`
- 声明位置: `include/unitree/idl/ros2/PointField_.hpp:46`

用法

```
current_value = obj.datatype
obj.datatype = new_value
```

unitree_sdk2_cpp.idl.ros2.PointField.count

底层 `sensor_msgs::msg::dds_::PointField_` 的 `count` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def count(self) -> int

@count.setter
def count(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointField_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/ros2/PointField_.hpp:47`

用法

```
current_value = obj.count
obj.count = new_value
```

unitree_sdk2_cpp.idl.ros2.PointCloud2

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import PointCloud2
```


方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>header</code>	Any	Any	AVAILABLE
<code>height</code>	int	int	AVAILABLE
<code>width</code>	int	int	AVAILABLE
<code>fields</code>	list[PointField]	Sequence[PointField]	AVAILABLE
<code>is_bigendian</code>	bool	bool	AVAILABLE
<code>point_step</code>	int	int	AVAILABLE
<code>row_step</code>	int	int	AVAILABLE
<code>data</code>	list[int]	Sequence[int]	AVAILABLE
<code>is_dense</code>	bool	bool	AVAILABLE

unitree_sdk2_cpp.idl.ros2.PointCloud2.__init__

初始化 PointCloud2 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 签名: `PointCloud2_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/PointCloud2_.hpp:39`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PointCloud2()
```

unitree_sdk2_cpp.idl.ros2.PointCloud2.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.PointCloud2.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.ros2.PointCloud2.header`

底层 `sensor_msgs::msg::dds_::PointCloud2_` 的 `header` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
<code>value</code>	Any	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 字段类型: `::std_msgs::msg::dds_::Header_`
- 声明位置: `include/unitree/idl/ros2/PointCloud2_.hpp:28`

用法

```
current_value = obj.header
obj.header = new_value
```

`unitree_sdk2_cpp.idl.ros2.PointCloud2.height`

底层 `sensor_msgs::msg::dds_::PointCloud2_` 的 `height` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def height(self) -> int

@height.setter
def height(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/ros2/PointCloud2_.hpp:29`

用法

```
current_value = obj.height
obj.height = new_value
```

unitree_sdk2_cpp.idl.ros2.PointCloud2.width

底层 `sensor_msgs::msg::dds_::PointCloud2_` 的 `width` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def width(self) -> int

@width.setter
def width(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/ros2/PointCloud2_.hpp:30`

用法

```
current_value = obj.width
obj.width = new_value
```

`unitree_sdk2_cpp.idl.ros2.PointCloud2.fields`

底层 `sensor_msgs::msg::dds_::PointCloud2_` 的 `fields` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector

签名

```
@property
def fields(self) -> list[PointField]

@fields.setter
def fields(self, value: Sequence[PointField]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	<code>Sequence[PointField]</code>	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	<code>list[PointField]</code>	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 字段类型: `std::vector< ::sensor_msgs::msg::dds_::PointField>`
- 声明位置: `include/unitree/idl/ros2/PointCloud2_.hpp:31`

用法

```
current_value = obj.fields
obj.fields = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

`unitree_sdk2_cpp.idl.ros2.PointCloud2.is_bigendian`

底层 `sensor_msgs::msg::dds_::PointCloud2_` 的 `is_bigendian` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。只接受布尔语义。

签名

```
@property
def is_bigendian(self) -> bool

@is_bigendian.setter
def is_bigendian(self, value: bool) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	bool	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	bool	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 字段类型: `bool`
- 声明位置: `include/unitree/idl/ros2/PointCloud2_.hpp:32`

用法

```
current_value = obj.is_bigendian
obj.is_bigendian = new_value
```

unitree_sdk2_cpp.idl.ros2.PointCloud2.point_step

底层 `sensor_msgs::msg::dds_::PointCloud2_` 的 `point_step` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def point_step(self) -> int

@point_step.setter
def point_step(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/ros2/PointCloud2_.hpp:33`

用法

```
current_value = obj.point_step
obj.point_step = new_value
```

`unitree_sdk2_cpp.idl.ros2.PointCloud2.row_step`

底层 `sensor_msgs::msg::dds_::PointCloud2_` 的 `row_step` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。取值范围为 0 到 4294967295。

签名

```
@property
def row_step(self) -> int

@row_step.setter
def row_step(self, value: int) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	int	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	int	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 字段类型: `uint32_t`
- 声明位置: `include/unitree/idl/ros2/PointCloud2_.hpp:34`

用法

```
current_value = obj.row_step
obj.row_step = new_value
```

`unitree_sdk2_cpp.idl.ros2.PointCloud2.data`

底层 `sensor_msgs::msg::dds_::PointCloud2_` 的 `data` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

签名

```
@property
def data(self) -> list[int]

@data.setter
def data(self, value: Sequence[int]) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Sequence[int]	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	list[int]	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: sensor_msgs::msg::dds_::PointCloud2_
- 字段类型: std::vector<uint8_t>
- 声明位置: include/unitree/idl/ros2/PointCloud2_.hpp:35

用法

```
current_value = obj.data
obj.data = new_value
```

[!TIP] 读取容器或嵌套值后应遵循“读取、修改、重新赋值”。只修改 getter 返回的副本不会可靠地更新原 C++ 消息。

unitree_sdk2_cpp.idl.ros2.PointCloud2.is_dense

底层 sensor_msgs::msg::dds_::PointCloud2_ 的 is_dense 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。只接受布尔语义。

签名

```
@property
def is_dense(self) -> bool

@is_dense.setter
def is_dense(self, value: bool) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	bool	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	bool	返回属性当前值。

对应 C++

- 类: `sensor_msgs::msg::dds_::PointCloud2_`
- 字段类型: `bool`
- 声明位置: `include/unitree/idl/ros2/PointCloud2_.hpp:36`

用法

```
current_value = obj.is_dense
obj.is_dense = new_value
```

`unitree_sdk2_cpp.idl ros2.PointStamped`

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import PointStamped
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>header</code>	Any	Any	AVAILABLE
<code>point</code>	Point	Point	AVAILABLE

`unitree_sdk2_cpp.idl.ros2.PointStamped.__init__`

初始化 `PointStamped` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::PointStamped_`
- 签名: `PointStamped_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/PointStamped_.hpp:30`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PointStamped()
```

`unitree_sdk2_cpp.idl ros2.PointStamped.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::PointStamped_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl ros2.PointStamped.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::PointStamped_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.PointStamped.header

底层 `geometry_msgs::msg::dds_::PointStamped_` 的 `header` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::PointStamped_`
- 字段类型: `::std_msgs::msg::dds_::Header_`

- 声明位置: `include/unitree/idl/ros2/PointStamped_.hpp:26`

用法

```
current_value = obj.header
obj.header = new_value
```

unitree_sdk2_cpp.idl.ros2.PointStamped.point

底层 `geometry_msgs::msg::dds_::PointStamped_` 的 `point` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def point(self) -> Point

@point.setter
def point(self, value: Point) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Point	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Point	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::PointStamped_`
- 字段类型: `::geometry_msgs::msg::dds_::Point_`
- 声明位置: `include/unitree/idl/ros2/PointStamped_.hpp:27`

用法

```
current_value = obj.point
obj.point = new_value
```

unitree_sdk2_cpp.idl.ros2.Pose2D

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import Pose2D
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

方法	Python 签名	状态	安全分类
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>x</code>	float	float	AVAILABLE
<code>y</code>	float	float	AVAILABLE
<code>theta</code>	float	float	AVAILABLE

unitree_sdk2_cpp.idl.ros2.Pose2D.__init__

初始化 Pose2D 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose2D_`
- 签名: `Pose2D_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/Pose2D_.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = Pose2D()
```

unitree_sdk2_cpp.idl.ros2.Pose2D.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose2D_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

`unitree_sdk2_cpp.idl.ros2.Pose2D.__ne__`

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose2D_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.Pose2D.x

底层 `geometry_msgs::msg::dds_::Pose2D_` 的 `x` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def x(self) -> float

@x.setter
def x(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose2D_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Pose2D_.hpp:22`

用法

```
current_value = obj.x
obj.x = new_value
```

unitree_sdk2_cpp.idl.ros2.Pose2D.y

底层 `geometry_msgs::msg::dds_::Pose2D_` 的 `y` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def y(self) -> float

@y.setter
def y(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose2D_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Pose2D_.hpp:23`

用法

```
current_value = obj.y
obj.y = new_value
```

unitree_sdk2_cpp.idl ros2.Pose2D.theta

底层 `geometry_msgs::msg::dds_::Pose2D_` 的 `theta` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

签名

```
@property
def theta(self) -> float

@theta.setter
def theta(self, value: float) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	float	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	float	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::Pose2D_`
- 字段类型: `double`
- 声明位置: `include/unitree/idl/ros2/Pose2D_.hpp:24`

用法

```
current_value = obj.theta
obj.theta = new_value
```


unitree_sdk2_cpp.idl.ros2.PoseStamped

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import PoseStamped
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>header</code>	Any	Any	AVAILABLE
<code>pose</code>	Pose	Pose	AVAILABLE

unitree_sdk2_cpp.idl.ros2.PoseStamped.__init__

初始化 PoseStamped 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseStamped_`
- 签名: `PoseStamped_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/PoseStamped_.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PoseStamped()
```

unitree_sdk2_cpp.idl.ros2.PoseStamped.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: geometry_msgs::msg::dds_::PoseStamped_
- 签名: operator==(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.PoseStamped.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseStamped_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.ros2.PoseStamped.header`

底层 `geometry_msgs::msg::dds_::PoseStamped_` 的 `header` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseStamped_`
- 字段类型: `::std_msgs::msg::dds_::Header_`
- 声明位置: `include/unitree/idl/ros2/PoseStamped_.hpp:26`

用法

```
current_value = obj.header
obj.header = new_value
```

`unitree_sdk2_cpp.idl.ros2.PoseStamped.pose`

底层 `geometry_msgs::msg::dds_::PoseStamped_` 的 `pose` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def pose(self) -> Pose

@pose.setter
def pose(self, value: Pose) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Pose	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Pose	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: geometry_msgs::msg::dds_::PoseStamped_
- 字段类型: ::geometry_msgs::msg::dds_::Pose_
- 声明位置: include/unitree/idl/ros2/PoseStamped_.hpp:27

用法

```
current_value = obj.pose
obj.pose = new_value
```

unitree_sdk2_cpp.idl.ros2.PoseWithCovarianceStamped

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import PoseWithCovarianceStamped
```

方法索引

方法	Python 签名	状态	安全分类
__init__	def __init__(self) -> None	AVAILABLE	VALUE_TYPE
__eq__	def __eq__(self, other: object) -> bool	AVAILABLE	VALUE_TYPE
__ne__	def __ne__(self, other: object) -> bool	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
header	Any	Any	AVAILABLE
pose	PoseWithCovariance	PoseWithCovariance	AVAILABLE

unitree_sdk2_cpp.idl.ros2.PoseWithCovarianceStamped.__init__

初始化 PoseWithCovarianceStamped 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseWithCovarianceStamped_`
- 签名: `PoseWithCovarianceStamped_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/PoseWithCovarianceStamped_.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PoseWithCovarianceStamped()
```

unitree_sdk2_cpp.idl.ros2.PoseWithCovarianceStamped.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseWithCovarianceStamped_`

- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.PoseWithCovarianceStamped.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseWithCovarianceStamped_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.PoseWithCovarianceStamped.header

底层 `geometry_msgs::msg::dds_::PoseWithCovarianceStamped_` 的 `header` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseWithCovarianceStamped_`
- 字段类型: `::std_msgs::msg::dds_::Header_`
- 声明位置: `include/unitree/idl/ros2/PoseWithCovarianceStamped_.hpp:26`

用法

```
current_value = obj.header
obj.header = new_value
```

unitree_sdk2_cpp.idl.ros2.PoseWithCovarianceStamped.pose

底层 `geometry_msgs::msg::dds_::PoseWithCovarianceStamped_` 的 `pose` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def pose(self) -> PoseWithCovariance

@pose.setter
def pose(self, value: PoseWithCovariance) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	PoseWithCovariance	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	PoseWithCovariance	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::PoseWithCovarianceStamped_`
- 字段类型: `::geometry_msgs::msg::dds_::PoseWithCovariance_`
- 声明位置: `include/unitree/idl/ros2/PoseWithCovarianceStamped_.hpp:27`

用法

```
current_value = obj.pose
obj.pose = new_value
```

unitree_sdk2_cpp.idl.ros2.QuaternionStamped

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import QuaternionStamped
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>header</code>	Any	Any	AVAILABLE
<code>quaternion</code>	Quaternion	Quaternion	AVAILABLE

unitree_sdk2_cpp.idl.ros2.QuaternionStamped.__init__

初始化 QuaternionStamped 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::QuaternionStamped_`
- 签名: `QuaternionStamped_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/QuaternionStamped.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = QuaternionStamped()
```

unitree_sdk2_cpp.idl.ros2.QuaternionStamped.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::QuaternionStamped_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.QuaternionStamped.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::QuaternionStamped_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.ros2.QuaternionStamped.header`

底层 `geometry_msgs::msg::dds_::QuaternionStamped_` 的 `header` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::QuaternionStamped_`
- 字段类型: `::std_msgs::msg::dds_::Header_`
- 声明位置: `include/unitree/idl/ros2/QuaternionStamped.hpp:26`

用法

```
current_value = obj.header
obj.header = new_value
```

`unitree_sdk2_cpp.idl.ros2.QuaternionStamped.quaternion`

底层 `geometry_msgs::msg::dds_::QuaternionStamped_` 的 `quaternion` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def quaternion(self) -> Quaternion

@quaternion.setter
def quaternion(self, value: Quaternion) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Quaternion	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Quaternion	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: geometry_msgs::msg::dds_::QuaternionStamped_
- 字段类型: ::geometry_msgs::msg::dds_::Quaternion_
- 声明位置: include/unitree/idl/ros2/QuaternionStamped_.hpp:27

用法

```
current_value = obj.quaternion
obj.quaternion = new_value
```

unitree_sdk2_cpp.idl.ros2.String

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import String
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	def __eq__(self, other: object) -> bool	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	def __ne__(self, other: object) -> bool	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>data</code>	str	str	AVAILABLE

unitree_sdk2_cpp.idl.ros2.String.__init__

初始化 `String` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `std_msgs::msg::dds_::String_`
- 签名: `String_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/String_.hpp:26`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = String()
```

unitree_sdk2_cpp.idl.ros2.String.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `std_msgs::msg::dds_::String_`

- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.String.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `std_msgs::msg::dds_::String_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.String.data

底层 `std_msgs::msg::dds_::String_` 的 `data` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。底层为字符串; 长度、编码和允许值由具体协议决定。

签名

```
@property
def data(self) -> str

@data.setter
def data(self, value: str) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	str	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	str	返回属性当前值。

对应 C++

- 类: `std_msgs::msg::dds_::String_`
- 字段类型: `std::string`
- 声明位置: `include/unitree/idl/ros2/String_.hpp:23`

用法

```
current_value = obj.data
obj.data = new_value
```

unitree_sdk2_cpp.idl.ros2.TwistStamped

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import TwistStamped
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>header</code>	Any	Any	AVAILABLE
<code>twist</code>	Twist	Twist	AVAILABLE

unitree_sdk2_cpp.idl.ros2.TwistStamped.__init__

初始化 TwistStamped 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistStamped_`
- 签名: `TwistStamped_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/TwistStamped_.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = TwistStamped()
```

`unitree_sdk2_cpp.idl ros2.TwistStamped.__eq__`

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
<code>other</code>	<code>object</code>	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 <code>False</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistStamped_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.TwistStamped.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: geometry_msgs::msg::dds_::TwistStamped_
- 签名: operator!=(const value &) const
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

unitree_sdk2_cpp.idl.ros2.TwistStamped.header

底层 geometry_msgs::msg::dds_::TwistStamped_ 的 header 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistStamped_`
- 字段类型: `::std_msgs::msg::dds_::Header_`
- 声明位置: `include/unitree/idl/ros2/TwistStamped_.hpp:26`

用法

```
current_value = obj.header
obj.header = new_value
```

unitree_sdk2_cpp.idl.ros2.TwistStamped.twist

底层 `geometry_msgs::msg::dds_::TwistStamped_` 的 `twist` 字段。类型签名只说明 Python 表示；单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def twist(self) -> Twist

@twist.setter
def twist(self, value: Twist) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Twist	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Twist	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistStamped_`
- 字段类型: `::geometry_msgs::msg::dds_::Twist_`
- 声明位置: `include/unitree/idl/ros2/TwistStamped_.hpp:27`

用法

```
current_value = obj.twist
obj.twist = new_value
```

unitree_sdk2_cpp.idl.ros2.TwistWithCovarianceStamped

可由 Python 读写的 DDS/IDL 消息值类型。字段采用复制语义。

导入

```
from unitree_sdk2_cpp.idl.ros2 import TwistWithCovarianceStamped
```

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	VALUE_TYPE
<code>__eq__</code>	<code>def __eq__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE
<code>__ne__</code>	<code>def __ne__(self, other: object) -> bool</code>	AVAILABLE	VALUE_TYPE

属性索引

属性	读取类型	写入类型	状态
<code>header</code>	Any	Any	AVAILABLE
<code>twist</code>	TwistWithCovariance	TwistWithCovariance	AVAILABLE

unitree_sdk2_cpp.idl.ros2.TwistWithCovarianceStamped.__init__

初始化 TwistWithCovarianceStamped 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE：当前绑定源码已实现该消息值操作。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistWithCovarianceStamped_`
- 签名: `TwistWithCovarianceStamped_()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/idl/ros2/TwistWithCovarianceStamped_.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = TwistWithCovarianceStamped()
```

unitree_sdk2_cpp.idl.ros2.TwistWithCovarianceStamped.__eq__

按底层 IDL 消息内容比较两个对象是否相等。

签名

```
def __eq__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistWithCovarianceStamped_`
- 签名: `operator==(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供; manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
same = left == right
```

unitree_sdk2_cpp.idl.ros2.TwistWithCovarianceStamped.__ne__

按底层 IDL 消息内容比较两个对象是否不相等。

签名

```
def __ne__(self, other: object) -> bool
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	默认值	含义
other	object	必填	用于比较的另一个 Python 对象。类型不兼容时比较结果通常为 False。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistWithCovarianceStamped_`
- 签名: `operator!=(const value &) const`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
different = left != right
```

`unitree_sdk2_cpp.idl.ros2.TwistWithCovarianceStamped.header`

底层 `geometry_msgs::msg::dds_::TwistWithCovarianceStamped_` 的 `header` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def header(self) -> Any

@header.setter
def header(self, value: Any) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	Any	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	Any	返回属性当前值。

对应 C++

- 类: `geometry_msgs::msg::dds_::TwistWithCovarianceStamped_`
- 字段类型: `::std_msgs::msg::dds_::Header_`
- 声明位置: `include/unitree/idl/ros2/TwistWithCovarianceStamped_.hpp:26`

用法

```
current_value = obj.header
obj.header = new_value
```

`unitree_sdk2_cpp.idl.ros2.TwistWithCovarianceStamped.twist`

底层 `geometry_msgs::msg::dds_::TwistWithCovarianceStamped_` 的 `twist` 字段。类型签名只说明 Python 表示; 单位、数值范围、数组固定长度和枚举含义需查对应 IDL 头文件。

签名

```
@property
def twist(self) -> TwistWithCovariance

@twist.setter
def twist(self, value: TwistWithCovariance) -> None
```

可用性与安全性

AVAILABLE / VALUE_TYPE: 当前绑定源码已实现该消息值操作。

参数

名称	Python 类型	含义
value	TwistWithCovariance	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。

返回值

位置	类型	含义
getter 返回值	TwistWithCovariance	返回属性当前值。数组、vector 和嵌套消息采用复制语义。

对应 C++

- 类: geometry_msgs::msg::dds_::TwistWithCovarianceStamped_
- 字段类型: ::geometry_msgs::msg::dds_::TwistWithCovariance_
- 声明位置: include/unitree/idl/ros2/TwistWithCovarianceStamped_.hpp:27

用法

```
current_value = obj.twist
obj.twist = new_value
```

Robot 公共基础 API

模块: unitree_sdk2_cpp.robot

类索引

类	公开函数签名	属性
ApplyLeaseData	3	0
ApplyLeaseParameter	3	0
ChannelFactory	5	0
ChannelNamer	2	0
Client	5	0
ClientBase	4	0
ClientChannelNamer	1	0
ClientStub	4	0
LeaseCache	7	0
LeaseClient	5	0
LeaseContext	6	0
LeaseServer	3	0
RequestFuture	7	0
RequestFutureQueue	5	0
Server	7	0
ServerBase	4	0

类	公开函数签名	属性
<code>ServerChannelNamer</code>	1	0
<code>ServerStub</code>	3	0

unitree_sdk2_cpp.robot.ApplyLeaseData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot import ApplyLeaseData
```

基类： object

公开属性

名称	Python 类型	含义	用法
id	int	传给该接口的 ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.id / obj.id = value</code>
term	int	传给该接口的 term 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.term / obj.term = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.ApplyLeaseData.__init__

初始化 `ApplyLeaseData` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::ApplyLeaseData`
- 签名: `ApplyLeaseData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/internal/internal_api.hpp:96`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ApplyLeaseData()
```

`unitree_sdk2_cpp.robot.ApplyLeaseData.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ApplyLeaseData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/internal/internal_api.hpp:102`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.ApplyLeaseData.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::ApplyLeaseData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/internal/internal_api.hpp:108`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.ApplyLeaseParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot import ApplyLeaseParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.ApplyLeaseParameter.__init__

初始化 `ApplyLeaseParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```


可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::ApplyLeaseParameter`
- 签名: `ApplyLeaseParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/internal/internal_api.hpp:69`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ApplyLeaseParameter()
```

`unitree_sdk2_cpp.robot.ApplyLeaseParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ApplyLeaseParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/internal/internal_api.hpp:75`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.ApplyLeaseParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::ApplyLeaseParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/internal/internal_api.hpp:80`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.ChannelFactory

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import ChannelFactory
```

基类: `object`

方法索引

方法	Python 签名	状态	安全分类
instance	<code>def instance(self) -> ChannelFactory</code>	SIGNATURE_ONLY	UNCLASSIFIED
init (重载 1/3)	<code>def init(self, domain_id: int, network_interface: str = ...) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED
init (重载 2/3)	<code>def init(self, config_file_name: str = ...) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED

方法	Python 签名	状态	安全分类
init (重载 3/3)	<code>def init(self, json_map: dict[str, Any]) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED
release	<code>def release(self) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED

unitree_sdk2_cpp.robot.ChannelFactory.instance

对应 C++ SDK 操作 `Instance()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def instance(self) -> ChannelFactory
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>ChannelFactory</code>	返回 <code>ChannelFactory</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::ChannelFactory`
- 签名: `Instance()`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/channel/channel_factory.hpp:19`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_instance(obj: ChannelFactory) -> ChannelFactory:
        return obj.instance()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ChannelFactory.init (重载 1/3)

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
@overload
def init(self, domain_id: int, network_interface: str = ...) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
domain_id	int	必填	DDS Domain ID。只有使用兼容 domain 的参与者才能按预期互相发现。对应 C++ 参数 domainId: int32_t。取值范围为 -2147483648 到 2147483647。
network_interface	str	...	DDS 使用的网络接口名称，例如 Linux 上的 eth0 或 enp3s0；必须按目标机实际网卡填写。对应 C++ 参数 networkInterface: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ChannelFactory`
- 签名: `Init(int32_t, const std::string &)`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/channel/channel_factory.hpp:25`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 默认值显示为 `...`，表示 C++ 声明存在默认参数，但当前 AST 清单没有保存其字面量；省略参数可使用上游默认行为。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_init(obj: ChannelFactory, domain_id: int, network_interface:
        str) -> None:
        obj.init(domain_id=domain_id, network_interface=network_interface)
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ChannelFactory.init (重载 2/3)

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
@overload
def init(self, config_file_name: str = ...) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
config_file_name	str	...	传给该接口的配置 file 名称参数, Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 configFileName: const std::string &。底层为字符串; 长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::ChannelFactory
- 签名: Init(const std::string &)
- 绑定策略: SIGNATURE_PREVIEW
- 声明位置: include/unitree/robot/channel/channel_factory.hpp:26

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 默认值显示为 ..., 表示 C++ 声明存在默认参数, 但当前 AST 清单没有保存其字面量; 省略参数可使用上游默认行为。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_init(obj: ChannelFactory, config_file_name: str) -> None:
        obj.init(config_file_name=config_file_name)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假, 不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ChannelFactory.init (重载 3/3)

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
@overload
def init(self, json_map: dict[str, Any]) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名, 不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
json_map	dict[str, Any]	必填	传给该接口的 json map 参数, Python 类型为 dict[str, Any]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 jsonMap: const common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ChannelFactory`
- 签名: `Init(const common::JsonMap &)`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/channel/channel_factory.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_init(obj: ChannelFactory, json_map: dict[str, Any]) -> None:
        obj.init(json_map=json_map)
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

`unitree_sdk2_cpp.robot.ChannelFactory.release`

对应 C++ SDK 操作 `Release()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def release(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ChannelFactory`
- 签名: `Release()`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/channel/channel_factory.hpp:29`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_release(obj: ChannelFactory) -> None:
        obj.release()
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ChannelNamer

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import ChannelNamer
```

基类： object

方法索引

方法	Python 签名	状态	安全分类
get_send_channel_name	def get_send_channel_name(self, name: str) -> str	SIGNATURE_ONLY	UNCLASSIFIED
get_rcv_channel_name	def get_rcv_channel_name(self, name: str) -> str	SIGNATURE_ONLY	UNCLASSIFIED

unitree_sdk2_cpp.robot.ChannelNamer.get_send_channel_name

查询或检查 send channel 名称。

签名

```
def get_send_channel_name(self, name: str) -> str
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	str	返回 str。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::ChannelNamer
- 签名: GetSendChannelName(const std::string &)
- 绑定策略: SIGNATURE_PREVIEW

- 声明位置: `include/unitree/robot/channel/channel_namer.hpp:21`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_get_send_channel_name(obj: ChannelNamer, name: str) -> str:
        return obj.get_send_channel_name(name=name)
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假, 不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ChannelNamer.get_rcv_channel_name

查询或检查 `rcv channel` 名称。

签名

```
def get_rcv_channel_name(self, name: str) -> str
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名, 不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
<code>name</code>	<code>str</code>	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串; 长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	<code>str</code>	返回 <code>str</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::ChannelNamer`
- 签名: `GetRcvChannelName(const std::string &)`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/channel/channel_namer.hpp:22`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_get_rcv_channel_name(obj: ChannelNamer, name: str) -> str:
        return obj.get_rcv_channel_name(name=name)
```


[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.Client

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot import Client
```

基类: ClientBase

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, name: str, enable_lease: bool = False) -> None</code>	SIGNATURE_ONLY	CONSTRUCTION
<code>wait_lease_applied</code>	<code>def wait_lease_applied(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>get_lease_id</code>	<code>def get_lease_id(self) -> int</code>	AVAILABLE	READ_ONLY
<code>get_api_version</code>	<code>def get_api_version(self) -> str</code>	AVAILABLE	READ_ONLY
<code>get_server_api_version</code>	<code>def get_server_api_version(self) -> str</code>	AVAILABLE	READ_ONLY

unitree_sdk2_cpp.robot.Client.__init__

初始化 Client 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, name: str, enable_lease: bool = False) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
<code>name</code>	<code>str</code>	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。
<code>enable_lease</code>	<code>bool</code>	<code>False</code>	是否启用 SDK lease 机制；lease 的获得、续期和释放规则以服务协议为准。对应 C++ 参数 <code>enableLease: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::Client`
- 签名: `Client(const std::string &, bool)`

- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/client/client.hpp:24

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct(name: str, enable_lease: bool) -> Client:
        return Client(name=name, enable_lease=enable_lease)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.Client.wait_lease_applied

对应 C++ SDK 操作 WaitLeaseApplied()。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def wait_lease_applied(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类：unitree::robot::Client
- 签名：WaitLeaseApplied()
- 绑定策略：DIRECT
- 声明位置：include/unitree/robot/client/client.hpp:27

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.wait_lease_applied()
```

unitree_sdk2_cpp.robot.Client.get_lease_id

查询或检查 lease ID。

签名

```
def get_lease_id(self) -> int
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	返回 <code>int</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::Client`
- 签名: `GetLeaseId()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/client.hpp:28`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.get_lease_id()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.Client.get_api_version

查询或检查 API version。

签名

```
def get_api_version(self) -> str
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>str</code>	返回 <code>str</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::Client`
- 签名: `GetApiVersion() const`
- 绑定策略: `REFERENCE_POLICY`
- 声明位置: `include/unitree/robot/client/client.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.get_api_version()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.Client.get_server_api_version

查询或检查 server API version。

签名

```
def get_server_api_version(self) -> str
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	str	返回 str。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::Client`
- 签名: `GetServerApiVersion()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/client.hpp:31`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.get_server_api_version()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.ClientBase

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import ClientBase
```

基类: `object`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, name: str) -> None</code>	SIGNATURE_ONLY	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	SIGNATURE_ONLY	INITIALIZATION
<code>set_timeout_microseconds</code>	<code>def set_timeout_microseconds(self, microseconds: int) -> None</code>	AVAILABLE	INITIALIZATION

方法	Python 签名	状态	安全分类
<code>set_timeout</code>	<code>def set_timeout(self, seconds: float) -> None</code>	AVAILABLE	INITIALIZATION

`unitree_sdk2_cpp.robot.ClientBase.__init__`

初始化 `ClientBase` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, name: str) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
<code>name</code>	<code>str</code>	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::ClientBase`
- 签名: `ClientBase(const std::string &)`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/client/client_base.hpp:23`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct(name: str) -> ClientBase:
        return ClientBase(name=name)
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

`unitree_sdk2_cpp.robot.ClientBase.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / INITIALIZATION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ClientBase`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/client_base.hpp:26`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_init(obj: ClientBase) -> None:
        obj.init()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

`unitree_sdk2_cpp.robot.ClientBase.set_timeout_microseconds`

设置超时 `microseconds`。具体副作用和安全边界见下方状态。

签名

```
def set_timeout_microseconds(self, microseconds: int) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>microseconds</code>	<code>int</code>	必填	客户端请求超时，单位为微秒。对应 C++ 参数 <code>timeout: int64_t</code> 。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ClientBase`
- 签名: `SetTimeout(int64_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/client_base.hpp:28`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.set_timeout_microseconds(microseconds=microseconds)
```

unitree_sdk2_cpp.robot.ClientBase.setTimeout

设置超时。具体副作用和安全边界见下方状态。

签名

```
def set_timeout(self, seconds: float) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
seconds	float	必填	客户端请求超时, 单位为秒。对应 C++ 参数 <code>timeout: float</code> 。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ClientBase`
- 签名: `SetTimeout(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/client_base.hpp:29`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.set_timeout(seconds=seconds)
```

unitree_sdk2_cpp.robot.ClientChannelNamer

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import ClientChannelNamer
```

基类: `ChannelNamer`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	SIGNATURE_ONLY	CONSTRUCTION

`unitree_sdk2_cpp.robot.ClientChannelNamer.__init__`

初始化 `ClientChannelNamer` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::ClientChannelNamer`
- 签名: `ClientChannelNamer()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/channel/channel_namer.hpp:34`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct() -> ClientChannelNamer:
        return ClientChannelNamer()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

`unitree_sdk2_cpp.robot.ClientStub`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import ClientStub
```

基类: `object`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	SIGNATURE_ONLY	CONSTRUCTION
<code>init</code>	<code>def init(self, name: str) -> None</code>	SIGNATURE_ONLY	INITIALIZATION
<code>send</code>	<code>def send(self, req: Any, wait_timeout: int) -> bool</code>	SIGNATURE_ONLY	HARDWARE_SIDE_EFFECT
<code>send_request</code>	<code>def send_request(self, req: Any, wait_timeout: int) -> RequestFuture</code>	SIGNATURE_ONLY	HARDWARE_SIDE_EFFECT

unitree_sdk2_cpp.robot.ClientStub.__init__

初始化 ClientStub 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::ClientStub`
- 签名: `ClientStub()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/client/client_stub.hpp:15`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct() -> ClientStub:
        return ClientStub()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ClientStub.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self, name: str) -> None
```

可用性与安全性

SIGNATURE_ONLY / INITIALIZATION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ClientStub`
- 签名: `Init(const std::string &)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/client_stub.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_init(obj: ClientStub, name: str) -> None:
        obj.init(name=name)
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ClientStub.send

对应 C++ SDK 操作 `Send(const unitree::robot::Request &, int64_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def send(self, req: Any, wait_timeout: int) -> bool
```

可用性与安全性

SIGNATURE_ONLY / HARDWARE_SIDE_EFFECT: 仅用于补全和静态检查。当前二进制没有该 Python 方法，未来实现还需单独评审硬件副作用。

参数

名称	Python 类型	默认值	含义
req	Any	必填	传给该接口的 req 参数，Python 类型为 Any。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 req: const unitree::robot::Request &。
wait_timeout	int	必填	请求等待超时参数；精确单位沿用对应 C++ 接口定义。对应 C++ 参数 waitTimeout: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::ClientStub
- 签名: Send(const unitree::robot::Request &, int64_t)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/client/client_stub.hpp:20

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_send(obj: ClientStub, req: Any, wait_timeout: int) -> bool:
        return obj.send(req=req, wait_timeout=wait_timeout)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ClientStub.send_request

对应 C++ SDK 操作 SendRequest(const unitree::robot::Request &, int64_t)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def send_request(self, req: Any, wait_timeout: int) -> RequestFuture
```

可用性与安全性

SIGNATURE_ONLY / HARDWARE_SIDE_EFFECT: 仅用于补全和静态检查。当前二进制没有该 Python 方法，未来实现还需单独评审硬件副作用。

参数

名称	Python 类型	默认值	含义
req	Any	必填	传给该接口的 req 参数，Python 类型为 Any。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 req: const unitree::robot::Request &。

名称	Python 类型	默认值	含义
wait_timeout	int	必填	请求等待超时参数；精确单位沿用对应 C++ 接口定义。对应 C++ 参数 waitTimeout: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
返回值	RequestFuture	返回 RequestFuture。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::ClientStub`
- 签名: `SendRequest(const unitree::robot::Request &, int64_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/client_stub.hpp:21`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_send_request(obj: ClientStub, req: Any, wait_timeout: int) -> RequestFuture:
        return obj.send_request(req=req, wait_timeout=wait_timeout)
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.LeaseCache

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import LeaseCache
```

基类: `object`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>set</code>	<code>def set(self, id: int, m_name: str, last_modified: int = 0) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>renewal</code>	<code>def renewal(self, last_modified: int = 0) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>clear</code>	<code>def clear(self) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>get_last_modified</code>	<code>def get_last_modified(self) -> int</code>	AVAILABLE	UNCLASSIFIED

方法	Python 签名	状态	安全分类
<code>get_id</code>	<code>def get_id(self) -> int</code>	AVAILABLE	UNCLASSIFIED
<code>get_name</code>	<code>def get_name(self) -> str</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.LeaseCache.__init__

初始化 LeaseCache 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::LeaseCache`
- 签名: `LeaseCache()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/server/lease_server.hpp:13`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LeaseCache()
```

unitree_sdk2_cpp.robot.LeaseCache.set

对应 C++ SDK 操作 `Set(int64_t, const std::string &, int64_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def set(self, id: int, m_name: str, last_modified: int = 0) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>id</code>	<code>int</code>	必填	传给该接口的 ID 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>id: int64_t</code> 。取值范围为 -9223372036854775808 到 9223372036854775807。

名称	Python 类型	默认值	含义
<code>m_name</code>	<code>str</code>	必填	传给该接口的 <code>m</code> 名称参数, Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>mName: const std::string &</code> 。底层为字符串; 长度、编码和允许值由具体协议决定。
<code>last_modified</code>	<code>int</code>	0	传给该接口的 <code>last modified</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>lastModified: int64_t</code> 。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::LeaseCache`
- 签名: `Set(int64_t, const std::string &, int64_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/server/lease_server.hpp:16`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.set(id=id, m_name=m_name, last_modified=last_modified)
```

`unitree_sdk2_cpp.robot.LeaseCache.renewal`

对应 C++ SDK 操作 `Renewal(int64_t)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def renewal(self, last_modified: int = 0) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>last_modified</code>	<code>int</code>	0	传给该接口的 <code>last modified</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>lastModified: int64_t</code> 。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::LeaseCache`
- 签名: `Renewal(int64_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/server/lease_server.hpp:17`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.renewal(last_modified=last_modified)
```

`unitree_sdk2_cpp.robot.LeaseCache.clear`

对应 C++ SDK 操作 `clear()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def clear(self) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::LeaseCache`
- 签名: `clear()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/server/lease_server.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.clear()
```

`unitree_sdk2_cpp.robot.LeaseCache.get_last_modified`

查询或检查 `last modified`。

签名

```
def get_last_modified(self) -> int
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::LeaseCache`
- 签名: `GetLastModified() const`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/server/lease_server.hpp:20`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.get_last_modified()
```

`unitree_sdk2_cpp.robot.LeaseCache.get_id`

查询或检查 ID。

签名

```
def get_id(self) -> int
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::LeaseCache`
- 签名: `GetId() const`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/server/lease_server.hpp:21`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.get_id()
```


unitree_sdk2_cpp.robot.LeaseCache.get_name

查询或检查名称。

签名

```
def get_name(self) -> str
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	str	返回 str。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::LeaseCache`
- 签名: `GetName() const`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/server/lease_server.hpp:22`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.get_name()
```

unitree_sdk2_cpp.robot.LeaseClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot import LeaseClient
```

基类: `ClientBase`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, name: str) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>wait_applied</code>	<code>def wait_applied(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>get_id</code>	<code>def get_id(self) -> int</code>	AVAILABLE	READ_ONLY
<code>applied</code>	<code>def applied(self) -> bool</code>	AVAILABLE	READ_ONLY

unitree_sdk2_cpp.robot.LeaseClient.__init__

初始化 `LeaseClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, name: str) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::LeaseClient`
- 签名: `LeaseClient(const std::string &)`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/client/lease_client.hpp:35`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LeaseClient(name=name)
```

unitree_sdk2_cpp.robot.LeaseClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::LeaseClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/lease_client.hpp:38`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

unitree_sdk2_cpp.robot.LeaseClient.wait_applied

对应 C++ SDK 操作 WaitApplied()。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def wait_applied(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类：unitree::robot::LeaseClient
- 签名：WaitApplied()
- 绑定策略：DIRECT
- 声明位置：include/unitree/robot/client/lease_client.hpp:40

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.wait_applied()
```

unitree_sdk2_cpp.robot.LeaseClient.get_id

查询或检查 ID。

签名

```
def get_id(self) -> int
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::LeaseClient`
- 签名: `GetId()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/lease_client.hpp:41`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.get_id()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

`unitree_sdk2_cpp.robot.LeaseClient.applied`

对应 C++ SDK 操作 `Applied()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def applied(self) -> bool
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::LeaseClient`
- 签名: `Applied()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/lease_client.hpp:42`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.applied()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.LeaseContext

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import LeaseContext
```

基类: object

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>update</code>	<code>def update(self, id: int, term: int) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>reset</code>	<code>def reset(self) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>valid</code>	<code>def valid(self) -> bool</code>	AVAILABLE	UNCLASSIFIED
<code>get_id</code>	<code>def get_id(self) -> int</code>	AVAILABLE	UNCLASSIFIED
<code>get_term</code>	<code>def get_term(self) -> int</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.LeaseContext.__init__

初始化 LeaseContext 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::LeaseContext`
- 签名: `LeaseContext()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/client/lease_client.hpp:14`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LeaseContext()
```

unitree_sdk2_cpp.robot.LeaseContext.update

对应 C++ SDK 操作 `Update(int64_t, int64_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def update(self, id: int, term: int) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
id	int	必填	传给该接口的 ID 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 id: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。
term	int	必填	传给该接口的 term 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 term: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::LeaseContext
- 签名: Update(int64_t, int64_t)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/client/lease_client.hpp:17

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.update(id=id, term=term)
```

unitree_sdk2_cpp.robot.LeaseContext.reset

对应 C++ SDK 操作 Reset()。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def reset(self) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::LeaseContext`
- 签名: `Reset()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/lease_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.reset()
```

`unitree_sdk2_cpp.robot.LeaseContext.valid`

对应 C++ SDK 操作 `Valid()` `const`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def valid(self) -> bool
```

可用性与安全性

AVAILABLE / UNCLASSIFIED：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::LeaseContext`
- 签名: `Valid() const`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/lease_client.hpp:20`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.valid()
```

`unitree_sdk2_cpp.robot.LeaseContext.get_id`

查询或检查 ID。

签名

```
def get_id(self) -> int
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::LeaseContext`
- 签名: `GetId() const`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/lease_client.hpp:22`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.get_id()
```

`unitree_sdk2_cpp.robot.LeaseContext.get_term`

查询或检查 `termo`

签名

```
def get_term(self) -> int
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::LeaseContext`
- 签名: `GetTerm() const`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/client/lease_client.hpp:23`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.get_term()
```


unitree_sdk2_cpp.robot.LeaseServer

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import LeaseServer
```

基类: ServerBase

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, name: str, term: int) -> None</code>	SIGNATURE_ONLY	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED
<code>check_request_lease_denied</code>	<code>def check_request_lease_denied(self, lease_id: int) -> bool</code>	SIGNATURE_ONLY	UNCLASSIFIED

unitree_sdk2_cpp.robot.LeaseServer.__init__

初始化 LeaseServer 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, name: str, term: int) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
<code>name</code>	<code>str</code>	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。
<code>term</code>	<code>int</code>	必填	传给该接口的 <code>term</code> 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>term: int64_t</code> 。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::LeaseServer`
- 签名: `LeaseServer(const std::string &, int64_t)`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/server/lease_server.hpp:33`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct(name: str, term: int) -> LeaseServer:
        return LeaseServer(name=name, term=term)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.LeaseServer.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::LeaseServer`
- 签名: `Init()`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/server/lease_server.hpp:36`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_init(obj: LeaseServer) -> None:
        obj.init()
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.LeaseServer.check_request_lease_denied

查询或检查请求 `lease denied`。

签名

```
def check_request_lease_denied(self, lease_id: int) -> bool
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
lease_id	int	必填	传给该接口的 lease ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 leaseId: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::LeaseServer
- 签名: CheckRequestLeaseDenied(int64_t)
- 绑定策略: SIGNATURE_PREVIEW
- 声明位置: include/unitree/robot/server/lease_server.hpp:38

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_check_request_lease_denied(obj: LeaseServer, lease_id: int)
    -> bool:
        return obj.check_request_lease_denied(lease_id=lease_id)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFuture

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import RequestFuture
```

基类: object

方法索引

方法	Python 签名	状态	安全分类
__init__ (重载 1/2)	def __init__(self) -> None	SIGNATURE_ONLY	CONSTRUCTION
__init__ (重载 2/2)	def __init__(self, request_id: int) -> None	SIGNATURE_ONLY	CONSTRUCTION
set_request_id	def set_request_id(self, request_id: int) -> None	SIGNATURE_ONLY	UNCLASSIFIED

方法	Python 签名	状态	安全分类
<code>get_request_id</code>	<code>def get_request_id(self) -> int</code>	SIGNATURE_ONLY	UNCLASSIFIED
<code>set_queue</code>	<code>def set_queue(self, future_queue_ptr: RequestFutureQueue) -> bool</code>	SIGNATURE_ONLY	UNCLASSIFIED
<code>get_response</code>	<code>def get_response(self, microsec: int) -> Any</code>	SIGNATURE_ONLY	UNCLASSIFIED
<code>ready</code>	<code>def ready(self, request_ptr: Any) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED

unitree_sdk2_cpp.robot.RequestFuture.__init__ (重载 1/2)

初始化 RequestFuture 实例。是否能实际构造取决于下方可用性状态。

签名

```
@overload
def __init__(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::RequestFuture`
- 签名: `RequestFuture()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/future/request_future.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct() -> RequestFuture:
        return RequestFuture()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFuture.__init__ (重载 2/2)

初始化 RequestFuture 实例。是否能实际构造取决于下方可用性状态。

签名

```
@overload
```

```
def __init__(self, request_id: int) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
request_id	int	必填	传给该接口的请求 ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 requestId: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: unitree::robot::RequestFuture
- 签名: RequestFuture(int64_t)
- 绑定策略: 未单独标注
- 声明位置: include/unitree/robot/future/request_future.hpp:28

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct(request_id: int) -> RequestFuture:
        return RequestFuture(request_id=request_id)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFuture.set_request_id

设置请求 ID。具体副作用和安全边界见下方状态。

签名

```
def set_request_id(self, request_id: int) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
request_id	int	必填	传给该接口的请求 ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 requestId: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::RequestFuture
- 签名: SetRequestId(int64_t)
- 绑定策略: SIGNATURE_PREVIEW
- 声明位置: include/unitree/robot/future/request_future.hpp:31

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_set_request_id(obj: RequestFuture, request_id: int) -> None:
        obj.set_request_id(request_id=request_id)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFuture.get_request_id

查询或检查请求 ID。

签名

```
def get_request_id(self) -> int
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::RequestFuture
- 签名: GetRequestId() const

- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/future/request_future.hpp:32`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_get_request_id(obj: RequestFuture) -> int:
        return obj.get_request_id()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假, 不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFuture.set_queue

设置 `queue`。具体副作用和安全边界见下方状态。

签名

```
def set_queue(self, future_queue_ptr: RequestFutureQueue) -> bool
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名, 不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
<code>future_queue_ptr</code>	<code>RequestFutureQueue</code>	必填	传给该接口的 <code>future queue ptr</code> 参数, Python 类型为 <code>RequestFutureQueue</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>future QueuePtr: const std::shared_ptr<Request FutureQueue> &</code> 。

返回值

位置	类型	含义
返回值	<code>bool</code>	返回 <code>bool</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::RequestFuture`
- 签名: `SetQueue(const std::shared_ptr<RequestFutureQueue> &)`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/future/request_future.hpp:34`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
```

```
def planned_set_queue(obj: RequestFuture, future_queue_ptr:
RequestFutureQueue) -> bool:
    return obj.set_queue(future_queue_ptr=future_queue_ptr)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFuture.get_response

查询或检查响应。

签名

```
def get_response(self, microsec: int) -> Any
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
microsec	int	必填	等待时间，单位为微秒。具体超时结果以 SDK 方法约定为准。对应 C++ 参数 microsec: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
返回值	Any	返回 Any。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::RequestFuture`
- 签名: `GetResponse(int64_t)`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/future/request_future.hpp:36`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_get_response(obj: RequestFuture, microsec: int) -> Any:
        return obj.get_response(microsec=microsec)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFuture.ready

对应 C++ SDK 操作 `Ready(const unitree::robot::ResponsePtr &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名


```
def ready(self, request_ptr: Any) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
request_ptr	Any	必填	传给该接口的请求 ptr 参数，Python 类型为 Any。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。 对应 C++ 参数 requestPtr: const unitree::robot::ResponsePtr &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::RequestFuture
- 签名: Ready(const unitree::robot::ResponsePtr &)
- 绑定策略: SIGNATURE_PREVIEW
- 声明位置: include/unitree/robot/future/request_future.hpp:39

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_ready(obj: RequestFuture, request_ptr: Any) -> None:
        obj.ready(request_ptr=request_ptr)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFutureQueue

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import RequestFutureQueue
```

基类: object

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	SIGNATURE_ONLY	CONSTRUCTION
<code>get</code>	def get(self, request_id: int) -> RequestFuture	SIGNATURE_ONLY	UNCLASSIFIED

方法	Python 签名	状态	安全分类
<code>put</code>	<code>def put(self, request_id: SIGNATURE_ONLY int, future_ptr: Request Future) -> bool</code>		UNCLASSIFIED
<code>remove</code>	<code>def remove(self, request_id: int) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED
<code>size</code>	<code>def size(self) -> int</code>	SIGNATURE_ONLY	UNCLASSIFIED

`unitree_sdk2_cpp.robot.RequestFutureQueue.__init__`

初始化 `RequestFutureQueue` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::RequestFutureQueue`
- 签名: `RequestFutureQueue()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/future/request_future.hpp:65`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct() -> RequestFutureQueue:
        return RequestFutureQueue()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

`unitree_sdk2_cpp.robot.RequestFutureQueue.get`

对应 C++ SDK 操作 `Get(int64_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def get(self, request_id: int) -> RequestFuture
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
request_id	int	必填	传给该接口的请求 ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 requestId: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
返回值	RequestFuture	返回 RequestFuture。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::RequestFutureQueue
- 签名: Get(int64_t)
- 绑定策略: SIGNATURE_PREVIEW
- 声明位置: include/unitree/robot/future/request_future.hpp:68

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_get(obj: RequestFutureQueue, request_id: int) -> RequestFuture:
        return obj.get(request_id=request_id)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFutureQueue.put

对应 C++ SDK 操作 Put(int64_t, const unitree::robot::RequestFuturePtr &)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def put(self, request_id: int, future_ptr: RequestFuture) -> bool
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
request_id	int	必填	传给该接口的请求 ID 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 requestId: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。
future_ptr	RequestFuture	必填	传给该接口的 future ptr 参数, Python 类型为 RequestFuture。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 futurePtr: const unitree::robot::RequestFuturePtr &。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::RequestFutureQueue
- 签名: Put(int64_t, const unitree::robot::RequestFuturePtr &)
- 绑定策略: SIGNATURE_PREVIEW
- 声明位置: include/unitree/robot/future/request_future.hpp:69

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_put(obj: RequestFutureQueue, request_id: int, future_ptr:
        RequestFuture) -> bool:
        return obj.put(request_id=request_id, future_ptr=future_ptr)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假, 不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.RequestFutureQueue.remove

对应 C++ SDK 操作 Remove(int64_t)。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def remove(self, request_id: int) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名, 不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
<code>request_id</code>	<code>int</code>	必填	传给该接口的请求 ID 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>requestId: int64_t</code> 。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::RequestFutureQueue`
- 签名: `Remove(int64_t)`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/future/request_future.hpp:70`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_remove(obj: RequestFutureQueue, request_id: int) -> None:
        obj.remove(request_id=request_id)
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

`unitree_sdk2_cpp.robot.RequestFutureQueue.size`

对应 C++ SDK 操作 `Size()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def size(self) -> int
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::RequestFutureQueue`

- 签名: `Size()`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/future/request_future.hpp:72`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_size(obj: RequestFutureQueue) -> int:
        return obj.size()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假, 不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.Server

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import Server
```

基类: ServerBase

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, name: str) -> None</code>	<code>SIGNATURE_ONLY</code>	<code>CONSTRUCTION</code>
<code>init</code>	<code>def init(self) -> None</code>	<code>SIGNATURE_ONLY</code>	<code>UNCLASSIFIED</code>
<code>start_lease</code> (重载 1/2)	<code>def start_lease(self, lease_term: int) -> None</code>	<code>SIGNATURE_ONLY</code>	<code>UNCLASSIFIED</code>
<code>start_lease</code> (重载 2/2)	<code>def start_lease(self, lease_term: float) -> None</code>	<code>SIGNATURE_ONLY</code>	<code>UNCLASSIFIED</code>
<code>get_name</code>	<code>def get_name(self) -> str</code>	<code>SIGNATURE_ONLY</code>	<code>UNCLASSIFIED</code>
<code>get_api_version</code>	<code>def get_api_version(self) -> str</code>	<code>SIGNATURE_ONLY</code>	<code>UNCLASSIFIED</code>
<code>get_current_api_id</code>	<code>def get_current_api_id(self) -> int</code>	<code>SIGNATURE_ONLY</code>	<code>UNCLASSIFIED</code>

unitree_sdk2_cpp.robot.Server.__init__

初始化 `Server` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, name: str) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名, 不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::Server`
- 签名: `Server(const std::string &)`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/server/server.hpp:29`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct(name: str) -> Server:
        return Server(name=name)
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.Server.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::Server`
- 签名: `Init()`
- 绑定策略: `SIGNATURE_PREVIEW`

- 声明位置: `include/unitree/robot/server/server.hpp:32`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_init(obj: Server) -> None:
        obj.init()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假, 不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.Server.start_lease (重载 1/2)

启动对应 SDK 操作。具体副作用和安全边界见下方状态。

签名

```
@overload
def start_lease(self, lease_term: int) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名, 不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
<code>lease_term</code>	<code>int</code>	必填	传给该接口的 lease term 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>leaseTerm: int64_t</code> 。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::Server`
- 签名: `StartLease(int64_t)`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/server/server.hpp:34`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_start_lease(obj: Server, lease_term: int) -> None:
```



```
obj.start_lease(lease_term=lease_term)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.Server.start_lease (重载 2/2)

启动对应 SDK 操作。具体副作用和安全边界见下方状态。

签名

```
@overload
def start_lease(self, lease_term: float) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
lease_term	float	必填	传给该接口的 lease term 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 leaseTerm: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::Server
- 签名: StartLease(float)
- 绑定策略: SIGNATURE_PREVIEW
- 声明位置: include/unitree/robot/server/server.hpp:35

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_start_lease(obj: Server, lease_term: float) -> None:
        obj.start_lease(lease_term=lease_term)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.Server.get_name

查询或检查名称。

签名

```
def get_name(self) -> str
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>str</code>	返回 <code>str</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::Server`
- 签名: `GetName()`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/server/server.hpp:37`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_get_name(obj: Server) -> str:
        return obj.get_name()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

`unitree_sdk2_cpp.robot.Server.get_api_version`

查询或检查 API `version`。

签名

```
def get_api_version(self) -> str
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>str</code>	返回 <code>str</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::Server`
- 签名: `GetApiVersion() const`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/server/server.hpp:38`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_get_api_version(obj: Server) -> str:
        return obj.get_api_version()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.Server.get_current_api_id

查询或检查 `current` API ID。

签名

```
def get_current_api_id(self) -> int
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 <code>0</code> 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::Server`
- 签名: `GetCurrentApiId() const`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/server/server.hpp:40`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_get_current_api_id(obj: Server) -> int:
        return obj.get_current_api_id()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ServerBase

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import ServerBase
```

基类: object

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, name: str) -> None</code>	SIGNATURE_ONLY	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED
<code>start</code>	<code>def start(self, enable_proi_queue: bool = ...) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED
<code>get_name</code>	<code>def get_name(self) -> str</code>	SIGNATURE_ONLY	UNCLASSIFIED

unitree_sdk2_cpp.robot.ServerBase.__init__

初始化 ServerBase 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, name: str) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::ServerBase`
- 签名: `ServerBase(const std::string &)`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/server/server_base.hpp:13`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct(name: str) -> ServerBase:
        return ServerBase(name=name)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ServerBase.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ServerBase`
- 签名: `Init()`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/server/server_base.hpp:16`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_init(obj: ServerBase) -> None:
        obj.init()
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ServerBase.start

启动对应 SDK 操作。具体副作用和安全边界见下方状态。

签名

```
def start(self, enable_proi_queue: bool = ...) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
enable_proi_queue	bool	...	是否启用 SDK 中名为 proi 的队列选项；该名称沿用上游接口，具体行为需查对应头文件。对应 C++ 参数 enableProiQueue: bool。只接受布尔语义。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::ServerBase`
- 签名: `Start(bool)`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/server/server_base.hpp:17`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 默认值显示为 `...`，表示 C++ 声明存在默认参数，但当前 AST 清单没有保存其字面量；省略参数可使用上游默认行为。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_start(obj: ServerBase, enable_proi_queue: bool) -> None:
        obj.start(enable_proi_queue=enable_proi_queue)
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

`unitree_sdk2_cpp.robot.ServerBase.get_name`

查询或检查名称。

签名

```
def get_name(self) -> str
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	str	返回 str。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::ServerBase`

- 签名: `GetName()` `const`
- 绑定策略: `SIGNATURE_PREVIEW`
- 声明位置: `include/unitree/robot/server/server_base.hpp:19`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_get_name(obj: ServerBase) -> str:
        return obj.get_name()
```

[!CAUTION] 上面的 `TYPE_CHECKING` 示例只表达计划调用形态。该分支在普通 Python 运行时为假, 不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ServerChannelNamer

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import ServerChannelNamer
```

基类: ChannelNamer

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	<code>SIGNATURE_ONLY</code>	<code>CONSTRUCTION</code>

unitree_sdk2_cpp.robot.ServerChannelNamer.__init__

初始化 `ServerChannelNamer` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名, 不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::ServerChannelNamer`
- 签名: `ServerChannelNamer()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/channel/channel_namer.hpp:51`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct() -> ServerChannelNamer:
        return ServerChannelNamer()
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ServerStub

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot import ServerStub
```

基类：object

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	SIGNATURE_ONLY	CONSTRUCTION
<code>init</code>	<code>def init(self, name: str, handler: Callable[..., Any], enable_proi_queue: bool) -> None</code>	SIGNATURE_ONLY	UNCLASSIFIED
<code>send</code>	<code>def send(self, response: Any, timeout: int = ...) -> bool</code>	SIGNATURE_ONLY	UNCLASSIFIED

unitree_sdk2_cpp.robot.ServerStub.__init__

初始化 ServerStub 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

SIGNATURE_ONLY / CONSTRUCTION: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::ServerStub`
- 签名: `ServerStub()`

- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/server/server_stub.hpp:18

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_construct() -> ServerStub:
        return ServerStub()
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ServerStub.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self, name: str, handler: Callable[..., Any], enable_proi_queue:
    bool) -> None
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。
handler	Callable[..., Any]	必填	传给该接口的 handler 参数，Python 类型为 Callable[..., Any]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 handler: const unitree::robot::ServerRequestHandler &。
enable_proi_queue	bool	必填	是否启用 SDK 中名为 proi 的队列选项；该名称沿用上游接口，具体行为需查对应头文件。对应 C++ 参数 enableProiQueue: bool。只接受布尔语义。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类：unitree::robot::ServerStub
- 签名：Init(const std::string &, const unitree::robot::ServerRequestHandler &, bool)
- 绑定策略：SIGNATURE_PREVIEW
- 声明位置：include/unitree/robot/server/server_stub.hpp:21

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该条目可以被编辑器和类型检查器识别，但当前运行时可能在属性查找阶段直接失败。
- 回调可能由 SDK 工作线程触发；回调应快速返回、捕获异常，并通过线程安全队列移交耗时工作。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_init(obj: ServerStub, name: str, handler: Callable[..., Any],
                     enable_proi_queue: bool) -> None:
        obj.init(name=name, handler=handler,
                 enable_proi_queue=enable_proi_queue)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假，不代表当前扩展能执行此接口。

unitree_sdk2_cpp.robot.ServerStub.send

对应 C++ SDK 操作 Send(const unitree::robot::Response &, int64_t)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def send(self, response: Any, timeout: int = ...) -> bool
```

可用性与安全性

SIGNATURE_ONLY / UNCLASSIFIED: 当前只有设计期签名，不能假设已存在于运行时扩展。

参数

名称	Python 类型	默认值	含义
response	Any	必填	传给该接口的响应参数，Python 类型为 Any。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 response: const unitree::robot::Response &。
timeout	int	...	传给该接口的超时参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 timeout: int64_t。取值范围为 -9223372036854775808 到 9223372036854775807。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::ServerStub
- 签名: Send(const unitree::robot::Response &, int64_t)
- 绑定策略: SIGNATURE_PREVIEW
- 声明位置: include/unitree/robot/server/server_stub.hpp:22

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

- 默认值显示为 ..., 表示 C++ 声明存在默认参数, 但当前 AST 清单没有保存其字面量; 省略参数可使用上游默认行为。
- 该条目可以被编辑器和类型检查器识别, 但当前运行时可能在属性查找阶段直接失败。

用法

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    def planned_send(obj: ServerStub, response: Any, timeout: int) -> bool:
        return obj.send(response=response, timeout=timeout)
```

[!CAUTION] 上面的 TYPE_CHECKING 示例只表达计划调用形态。该分支在普通 Python 运行时为假, 不代表当前扩展能执行此接口。

A2 Robot API

模块: unitree_sdk2_cpp.robot.a2

类索引

类	公开函数签名	属性
AudioClient	8	0
LedControlParameter	3	0
PathPoint	1	0
PlayStopParameter	3	0
PlayStreamParameter	3	0
PoseVec4	3	0
SportClient	24	0
TtsMakerParameter	3	0

unitree_sdk2_cpp.robot.a2.AudioClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.a2 import AudioClient
```

基类: Client

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>tts_maker</code>	<code>def tts_maker(self, text: str, speaker_id: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>get_volume</code>	<code>def get_volume(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>set_volume</code>	<code>def set_volume(self, volume: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>play_stream</code>	<code>def play_stream(self, app_name: str, stream_id: str, pcm_data: Sequence[int]) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT

方法	Python 签名	状态	安全分类
<code>play_stop</code>	<code>def play_stop(self, app_name: str) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>led_control</code>	<code>def led_control(self, r: int, g: int, b: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT

`unitree_sdk2_cpp.robot.a2.AudioClient.__init__`

初始化 `AudioClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::a2::AudioClient`
- 签名: `AudioClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/a2/audio/audio_client.hpp:15`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = AudioClient()
```

`unitree_sdk2_cpp.robot.a2.AudioClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::a2::AudioClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/audio/audio_client.hpp:19`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.a2.AudioClient.tts_maker`

对应 C++ SDK 操作 `TtsMaker(const std::string &, int32_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def tts_maker(self, text: str, speaker_id: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>text</code>	<code>str</code>	必填	要处理的文本内容；编码、长度和语言支持由目标服务决定。对应 C++ 参数 <code>text: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。
<code>speaker_id</code>	<code>int</code>	必填	语音合成说话人 ID。有效编号由机器人音频服务和固件决定。对应 C++ 参数 <code>speaker_id: int32_t</code> 。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::AudioClient`
- 签名: `TtsMaker(const std::string &, int32_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/audio/audio_client.hpp:31`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.tts_maker(text=text, speaker_id=speaker_id)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.AudioClient.get_volume

查询或检查音量。

签名

```
def get_volume(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] volume	int	传给该接口的音量参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>volume: uint8_t &</code> 。取值范围为 0 到 255。

对应 C++

- 类: `unitree::robot::a2::AudioClient`
- 签名: `GetVolume(uint8_t &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/a2/audio/audio_client.hpp:43`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, volume = obj.get_volume()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.a2.AudioClient.set_volume

设置音量。具体副作用和安全边界见下方状态。

签名

```
def set_volume(self, volume: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
volume	int	必填	传给该接口的音量参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 volume: uint8_t。取值范围为 0 到 255。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::AudioClient`
- 签名: `SetVolume(uint8_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/audio/audio_client.hpp:57`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set_volume(volume=volume)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.AudioClient.play_stream

对应 C++ SDK 操作 `PlayStream(std::string, std::string, std::vector<uint8_t>)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def play_stream(self, app_name: str, stream_id: str, pcm_data: Sequence[int])
    -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
app_name	str	必填	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。对应 C++ 参数 app_name: std::string。底层为字符串；长度、编码和允许值由具体协议决定。
stream_id	str	必填	音频流标识符，用于区分同一应用下的播放流。对应 C++ 参数 stream_id: std::string。底层为字符串；长度、编码和允许值由具体协议决定。
pcm_data	Sequence[int]	必填	传给该接口的 pcm 数据参数，Python 类型为 Sequence[int]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 pcm_data: std::vector<uint8_t>。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::AudioClient`
- 签名: `PlayStream(std::string, std::string, std::vector<uint8_t>)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/audio/audio_client.hpp:68`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.play_stream(app_name=app_name, stream_id=stream_id,
                        pcm_data=pcm_data)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.a2.AudioClient.play_stop`

对应 C++ SDK 操作 `PlayStop(std::string)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def play_stop(self, app_name: str) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>app_name</code>	<code>str</code>	必填	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。对应 C++ 参数 <code>app_name: std::string</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::AudioClient`
- 签名: `PlayStop(std::string)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/audio/audio_client.hpp:80`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.play_stop(app_name=app_name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.AudioClient.led_control

对应 C++ SDK 操作 LedControl(uint8_t, uint8_t, uint8_t)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def led_control(self, r: int, g: int, b: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
r	int	必填	传给该接口的 r 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 R: uint8_t。取值范围为 0 到 255。
g	int	必填	传给该接口的 g 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 G: uint8_t。取值范围为 0 到 255。
b	int	必填	传给该接口的 b 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 B: uint8_t。取值范围为 0 到 255。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::a2::AudioClient
- 签名: LedControl(uint8_t, uint8_t, uint8_t)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/a2/audio/audio_client.hpp:91

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.led_control(r=r, g=g, b=b)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.LedControlParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.a2 import LedControlParameter
```

基类：object

公开属性

名称	Python 类型	含义	用法
r	int	传给该接口的 r 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.r / obj.r = value
g	int	传给该接口的 g 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.g / obj.g = value
b	int	传给该接口的 b 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.b / obj.b = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.a2.LedControlParameter.__init__

初始化 LedControlParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::a2::LedControlParameter`
- 签名: `LedControlParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:75`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LedControlParameter()
```

`unitree_sdk2_cpp.robot.a2.LedControlParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::a2::LedControlParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:78`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.a2.LedControlParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::a2::LedControlParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:80`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.a2.PathPoint`

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.a2 import PathPoint
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>t_from_start</code>	<code>float</code>	传给该接口的 <code>t from start</code> 参数, Python 类型为 <code>float</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.t_from_start / obj.t_from_start</code>
<code>x</code>	<code>float</code>	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.x / obj.x = value</code>
<code>y</code>	<code>float</code>	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.y / obj.y = value</code>
<code>yaw</code>	<code>float</code>	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。	<code>value = obj.yaw / obj.yaw = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

unitree_sdk2_cpp.robot.a2.PathPoint.__init__

初始化 `PathPoint` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::a2::PathPoint`
- 签名: `PathPoint()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PathPoint()
```

unitree_sdk2_cpp.robot.a2.PlayStopParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.a2 import PlayStopParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>app_name</code>	<code>str</code>	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。	<code>value = obj.app_name / obj.app_name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.a2.PlayStopParameter.__init__

初始化 PlayStopParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::a2::PlayStopParameter`
- 签名: `PlayStopParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:61`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PlayStopParameter()
```

unitree_sdk2_cpp.robot.a2.PlayStopParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::a2::PlayStopParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:64`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.a2.PlayStopParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::a2::PlayStopParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:66`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.a2.PlayStreamParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.a2 import PlayStreamParameter
```

基类：object

公开属性

名称	Python 类型	含义	用法
app_name	str	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。	value = obj.app_name / obj.app_name = value
stream_id	str	音频流标识符，用于区分同一应用下的播放流。	value = obj.stream_id / obj.stream_id = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.a2.PlayStreamParameter.__init__

初始化 PlayStreamParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类：unitree::robot::a2::PlayStreamParameter
- 签名：PlayStreamParameter()
- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/a2/audio/audio_api.hpp:45

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PlayStreamParameter()
```

`unitree_sdk2_cpp.robot.a2.PlayStreamParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::a2::PlayStreamParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:48`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.a2.PlayStreamParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::a2::PlayStreamParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:50`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.a2.PoseVec4

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.a2 import PoseVec4
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>x</code>	<code>float</code>	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.x / obj.x = value</code>
<code>y</code>	<code>float</code>	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.y / obj.y = value</code>
<code>z</code>	<code>float</code>	Z 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.z / obj.z = value</code>
<code>yaw</code>	<code>float</code>	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。	<code>value = obj.yaw / obj.yaw = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.a2.PoseVec4.__init__

初始化 PoseVec4 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::a2::PoseVec4`
- 签名: `PoseVec4()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:35`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PoseVec4()
```

unitree_sdk2_cpp.robot.a2.PoseVec4.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common: JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::a2::PoseVec4`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:43`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.fromJson(value=value)
```

`unitree_sdk2_cpp.robot.a2.PoseVec4.toJson`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def toJson(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::a2::PoseVec4`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:51`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

`unitree_sdk2_cpp.robot.a2.SportClient`

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.a2 import SportClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>damp</code>	<code>def damp(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>balance_stand</code>	<code>def balance_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stop_move</code>	<code>def stop_move(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stand_up</code>	<code>def stand_up(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stand_down</code>	<code>def stand_down(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>recovery_stand</code>	<code>def recovery_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>euler</code>	<code>def euler(self, roll: float, pitch: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move</code>	<code>def move(self, vx: float, vy: float, vyaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_gait</code>	<code>def switch_gait(self, gait_type: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>body_height</code>	<code>def body_height(self, height: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>speed_level</code>	<code>def speed_level(self, level: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>body_position</code>	<code>def body_position(self, x: float, y: float, z: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>left_side_gait</code>	<code>def left_side_gait(self, enter: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>right_side_gait</code>	<code>def right_side_gait(self, enter: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>hand_stand</code>	<code>def hand_stand(self, enter: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>biped_stand</code>	<code>def biped_stand(self, enter: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>front_flip</code>	<code>def front_flip(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>back_flip</code>	<code>def back_flip(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>reset_estimator</code>	<code>def reset_estimator(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>trajectory</code>	<code>def trajectory(self, path: Sequence[Path Point], feedback_mode: int = 0, external_x: float = 0.0, external_y: float = 0.0, external_yaw: float = 0.0) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_auto_recovery</code>	<code>def set_auto_recovery(self, switch_on: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>get_state</code>	<code>def get_state(self) -> tuple[int, dict[str, str]]</code>	AVAILABLE	READ_ONLY

`unitree_sdk2_cpp.robot.a2.SportClient.__init__`

初始化 `SportClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `SportClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:77`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = SportClient()
```

`unitree_sdk2_cpp.robot.a2.SportClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:81`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

unitree_sdk2_cpp.robot.a2.SportClient.damp

对应 C++ SDK 操作 `Damp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def damp(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `Damp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:112`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.damp()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.balance_stand

对应 C++ SDK 操作 `BalanceStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def balance_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `BalanceStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:118`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.balance_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.a2.SportClient.stop_move`

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_move(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `StopMove()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:123`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stop_move()
```


[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.stand_up

对应 C++ SDK 操作 `StandUp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_up(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `StandUp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:129`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_up()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.stand_down

对应 C++ SDK 操作 `StandDown()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_down(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `StandDown()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:135`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_down()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.a2.SportClient.recovery_stand`

对应 C++ SDK 操作 `RecoveryStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def recovery_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `RecoveryStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:141`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.recovery_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.euler

对应 C++ SDK 操作 Euler(float, float, float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def euler(self, roll: float, pitch: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND：当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
roll	float	必填	传给该接口的 roll 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 roll: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
pitch	float	必填	传给该接口的 pitch 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 pitch: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 yaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::a2::SportClient
- 签名: Euler(float, float, float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/a2/sport/sport_client.hpp:147

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.euler(roll=roll, pitch=pitch, yaw=yaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.move

对应 C++ SDK 操作 Move(float, float, float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def move(self, vx: float, vy: float, vyaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vx: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vy: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vyaw	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vyaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `Move(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:158`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.switch_gait

对应 C++ SDK 操作 `SwitchGait(int)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_gait(self, gait_type: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>gait_type</code>	<code>int</code>	必填	传给该接口的 <code>gait type</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>gait_type</code> : <code>int</code> 。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `SwitchGait(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:169`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_gait(gait_type=gait_type)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.a2.SportClient.body_height`

对应 C++ SDK 操作 `BodyHeight(float)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def body_height(self, height: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>height</code>	<code>float</code>	必填	传给该接口的高度参数, Python 类型为 <code>float</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>height</code> : <code>float</code> 。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `BodyHeight(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:178`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.body_height(height=height)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.speed_level

对应 C++ SDK 操作 `SpeedLevel(int)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def speed_level(self, level: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
level	int	必填	传给该接口的 level 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 level: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `SpeedLevel(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:187`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.speed_level(level=level)
```


[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.body_position

对应 C++ SDK 操作 BodyPosition(float, float, float, float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def body_position(self, x: float, y: float, z: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND：当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
x	float	必填	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 x: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
y	float	必填	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 y: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
z	float	必填	Z 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 z: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 yaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::a2::SportClient
- 签名: BodyPosition(float, float, float, float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/a2/sport/sport_client.hpp:196

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.body_position(x=x, y=y, z=z, yaw=yaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.left_side_gait

对应 C++ SDK 操作 LeftSideGait(int)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def left_side_gait(self, enter: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
enter	int	必填	传给该接口的 enter 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 enter: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::a2::SportClient
- 签名: LeftSideGait(int)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/a2/sport/sport_client.hpp:208

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.left_side_gait(enter=enter)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.right_side_gait

对应 C++ SDK 操作 RightSideGait(int)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def right_side_gait(self, enter: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
enter	int	必填	传给该接口的 enter 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。 对应 C++ 参数 enter: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `RightSideGait(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:217`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.right_side_gait(enter=enter)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.hand_stand

对应 C++ SDK 操作 `HandStand(int)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def hand_stand(self, enter: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
enter	int	必填	传给该接口的 enter 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。 对应 C++ 参数 enter: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `HandStand(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:226`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.hand_stand(enter=enter)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.biped_stand

对应 C++ SDK 操作 `BipedStand(int)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def biped_stand(self, enter: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
enter	int	必填	传给该接口的 enter 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 enter: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `BipedStand(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:235`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.biped_stand(enter=enter)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.front_flip

对应 C++ SDK 操作 `FrontFlip()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def front_flip(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `FrontFlip()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:244`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.front_flip()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.back_flip

对应 C++ SDK 操作 `BackFlip()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def back_flip(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `BackFlip()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:250`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.back_flip()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.a2.SportClient.reset_estimator`

对应 C++ SDK 操作 `ResetEstimator()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def reset_estimator(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `ResetEstimator()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:256`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.reset_estimator()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.trajectory

对应 C++ SDK 操作 Trajectory(const std::vector<PathPoint> &, int, float, float, float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def trajectory(self, path: Sequence[PathPoint], feedback_mode: int = 0,
               external_x: float = 0.0, external_y: float = 0.0, external_yaw: float =
               0.0) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
path	Sequence[PathPoint]	必填	传给该接口的 path 参数，Python 类型为 Sequence[PathPoint]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 path: const std::vector<PathPoint> &。底层是可变长度 vector
feedback_mode	int	0	传给该接口的 feedback 模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 feedback_mode: int。
external_x	float	0.0	传给该接口的 external x 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 external_x: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
external_y	float	0.0	传给该接口的 external y 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 external_y: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
external_yaw	float	0.0	传给该接口的 external yaw 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 external_yaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::a2::SportClient
- 签名: Trajectory(const std::vector<PathPoint> &, int, float, float, float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/a2/sport/sport_client.hpp:262

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.trajectory(path=path, feedback_mode=feedback_mode,
    external_x=external_x, external_y=external_y, external_yaw=external_yaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.set_auto_recovery

设置 auto recovery。具体副作用和安全边界见下方状态。

签名

```
def set_auto_recovery(self, switch_on: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
switch_on	int	必填	传给该接口的开关 on 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 switch_on: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `SetAutoRecovery(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:286`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_auto_recovery(switch_on=switch_on)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.a2.SportClient.get_state

查询或检查状态。

签名

```
def get_state(self) -> tuple[int, dict[str, str]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] state_map	dict[str, str]	传给该接口的状态 map 参数，Python 类型为 dict[str, str]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 state_map: std::map<std::string, std::string> &。

对应 C++

- 类: `unitree::robot::a2::SportClient`
- 签名: `GetState(std::map<std::string, std::string> &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/a2/sport/sport_client.hpp:295`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, state_map = obj.get_state()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.a2.TtsMakerParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.a2 import TtsMakerParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
index	int	传给该接口的 index 参数， <code>value = obj.index / obj.index = value</code> Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	
speaker_id	int	语音合成说话人 ID。有效编号由机器人音频服务和固件决定。	<code>value = obj.speaker_id / obj.speaker_id = value</code>
text	str	要处理的文本内容；编码、长度和语言支持由目标服务决定。	<code>value = obj.text / obj.text = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.a2.TtsMakerParameter.__init__

初始化 TtsMakerParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::a2::TtsMakerParameter`
- 签名: `TtsMakerParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = TtsMakerParameter()
```

unitree_sdk2_cpp.robot.a2.TtsMakerParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::a2::TtsMakerParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.fromJson(value=value)
```

`unitree_sdk2_cpp.robot.a2.TtsMakerParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::a2::TtsMakerParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/a2/audio/audio_api.hpp:32`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

AS2 Robot API

模块: `unitree_sdk2_cpp.robot.as2`

类索引

类	公开函数签名	属性
<code>PoseVec4</code>	3	0
<code>SportClient</code>	34	0

`unitree_sdk2_cpp.robot.as2.PoseVec4`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.as2 import PoseVec4
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>x</code>	<code>float</code>	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.x / obj.x = value</code>
<code>y</code>	<code>float</code>	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.y / obj.y = value</code>
<code>z</code>	<code>float</code>	Z 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.z / obj.z = value</code>
<code>yaw</code>	<code>float</code>	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。	<code>value = obj.yaw / obj.yaw = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.as2.PoseVec4.__init__`

初始化 `PoseVec4` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::as2::PoseVec4`
- 签名: `PoseVec4()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PoseVec4()
```

`unitree_sdk2_cpp.robot.as2.PoseVec4.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::as2::PoseVec4`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:26`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.as2.PoseVec4.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::as2::PoseVec4`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:34`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.as2.SportClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.as2 import SportClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>damp</code>	<code>def damp(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>balance_stand</code>	<code>def balance_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stop_move</code>	<code>def stop_move(self) -> int</code>	AVAILABLE	MOTION_COMMAND

方法	Python 签名	状态	安全分类
<code>stand_up</code>	<code>def stand_up(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stand_down</code>	<code>def stand_down(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>recovery_stand</code>	<code>def recovery_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>euler</code>	<code>def euler(self, roll: float, pitch: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move</code>	<code>def move(self, vx: float, vy: float, vyaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_gait</code>	<code>def switch_gait(self, gait_type: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>body_height</code>	<code>def body_height(self, height: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>speed_level</code>	<code>def speed_level(self, level: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>body_position</code>	<code>def body_position(self, x: float, y: float, z: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>left_side_gait</code>	<code>def left_side_gait(self, enter: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>right_side_gait</code>	<code>def right_side_gait(self, enter: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>hand_stand</code>	<code>def hand_stand(self, enter: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>biped_stand</code>	<code>def biped_stand(self, enter: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>front_flip</code>	<code>def front_flip(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>back_flip</code>	<code>def back_flip(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>greeting</code>	<code>def greeting(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>heart</code>	<code>def heart(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>content</code>	<code>def content(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>dance1</code>	<code>def dance1(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>dance2</code>	<code>def dance2(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>handshake</code>	<code>def handshake(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stretch</code>	<code>def stretch(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>sit</code>	<code>def sit(self, enter: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>front_jump</code>	<code>def front_jump(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>push_up</code>	<code>def push_up(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>up_jump</code>	<code>def up_jump(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_auto_recovery</code>	<code>def set_auto_recovery(self, switch_on: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_joystick</code>	<code>def switch_joystick(self, switch_on: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>get_state</code>	<code>def get_state(self) -> tuple[int, dict[str, str]]</code>	AVAILABLE	READ_ONLY

`unitree_sdk2_cpp.robot.as2.SportClient.__init__`

初始化 `SportClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `SportClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:52`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = SportClient()
```

`unitree_sdk2_cpp.robot.as2.SportClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:56`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

unitree_sdk2_cpp.robot.as2.SportClient.damp

对应 C++ SDK 操作 `Damp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def damp(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Damp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:97`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.damp()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.balance_stand

对应 C++ SDK 操作 `BalanceStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def balance_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `BalanceStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:103`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.balance_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.stop_move`

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_move(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `StopMove()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:108`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stop_move()
```


[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.stand_up

对应 C++ SDK 操作 `StandUp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_up(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `StandUp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:114`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_up()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.stand_down

对应 C++ SDK 操作 `StandDown()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_down(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `StandDown()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:120`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_down()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.recovery_stand`

对应 C++ SDK 操作 `RecoveryStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def recovery_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `RecoveryStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:126`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.recovery_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.euler

对应 C++ SDK 操作 Euler(float, float, float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def euler(self, roll: float, pitch: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
roll	float	必填	传给该接口的 roll 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 roll: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
pitch	float	必填	传给该接口的 pitch 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 pitch: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 yaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::as2::SportClient
- 签名: Euler(float, float, float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/as2/sport/sport_client.hpp:132

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.euler(roll=roll, pitch=pitch, yaw=yaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.move

对应 C++ SDK 操作 Move(float, float, float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def move(self, vx: float, vy: float, vyaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vx: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vy: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vyaw	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vyaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Move(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:143`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.switch_gait`

对应 C++ SDK 操作 `SwitchGait(int)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_gait(self, gait_type: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>gait_type</code>	<code>int</code>	必填	传给该接口的 <code>gait type</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>gait_type</code> : <code>int</code> 。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `SwitchGait(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:154`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_gait(gait_type=gait_type)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.body_height`

对应 C++ SDK 操作 `BodyHeight(float)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def body_height(self, height: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>height</code>	<code>float</code>	必填	传给该接口的高度参数, Python 类型为 <code>float</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>height</code> : <code>float</code> 。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `BodyHeight(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:163`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.body_height(height=height)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.speed_level`

对应 C++ SDK 操作 `SpeedLevel(int)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def speed_level(self, level: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>level</code>	<code>int</code>	必填	传给该接口的 <code>level</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>level: int</code> 。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `SpeedLevel(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:172`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.speed_level(level=level)
```


[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.body_position

对应 C++ SDK 操作 `BodyPosition(float, float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def body_position(self, x: float, y: float, z: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND：当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
x	float	必填	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 <code>x: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
y	float	必填	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 <code>y: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
z	float	必填	Z 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 <code>z: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 <code>yaw: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类：unitree::robot::as2::SportClient
- 签名：BodyPosition(float, float, float, float)
- 绑定策略：DIRECT
- 声明位置：include/unitree/robot/as2/sport/sport_client.hpp:181

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.body_position(x=x, y=y, z=z, yaw=yaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.left_side_gait

对应 C++ SDK 操作 LeftSideGait(int)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def left_side_gait(self, enter: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
enter	int	必填	传给该接口的 enter 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 enter: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::as2::SportClient
- 签名: LeftSideGait(int)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/as2/sport/sport_client.hpp:193

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.left_side_gait(enter=enter)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.right_side_gait

对应 C++ SDK 操作 RightSideGait(int)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def right_side_gait(self, enter: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
enter	int	必填	传给该接口的 enter 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。 对应 C++ 参数 enter: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `RightSideGait(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:202`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.right_side_gait(enter=enter)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.hand_stand`

对应 C++ SDK 操作 `HandStand(int)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def hand_stand(self, enter: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
enter	int	必填	传给该接口的 enter 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。 对应 C++ 参数 enter: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `HandStand(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:211`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.hand_stand(enter=enter)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.biped_stand`

对应 C++ SDK 操作 `BipedStand(int)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def biped_stand(self, enter: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>enter</code>	<code>int</code>	必填	传给该接口的 <code>enter</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>enter: int</code> 。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `BipedStand(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:220`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.biped_stand(enter=enter)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.front_flip

对应 C++ SDK 操作 `FrontFlip()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def front_flip(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `FrontFlip()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:229`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.front_flip()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.back_flip

对应 C++ SDK 操作 `BackFlip()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def back_flip(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `BackFlip()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:235`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.back_flip()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.greeting`

对应 C++ SDK 操作 `Greeting()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def greeting(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Greeting()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:241`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.greeting()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.heart

对应 C++ SDK 操作 `Heart()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def heart(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Heart()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:247`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.heart()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.content

对应 C++ SDK 操作 `Content()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def content(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Content()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:253`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.content()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.dance1`

对应 C++ SDK 操作 `Dance1()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def dance1(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Dance1()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:259`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.dance1()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.dance2

对应 C++ SDK 操作 `Dance2()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def dance2(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Dance2()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:265`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.dance2()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.handshake

对应 C++ SDK 操作 `Handshake()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def handshake(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Handshake()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:271`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.handshake()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.stretch`

对应 C++ SDK 操作 `Stretch()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stretch(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Stretch()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:277`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stretch()
```


[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.sit

对应 C++ SDK 操作 `Sit(int)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def sit(self, enter: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
enter	int	必填	传给该接口的 enter 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 enter: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `Sit(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:283`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.sit(enter=enter)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.front_jump

对应 C++ SDK 操作 `FrontJump()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def front_jump(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `FrontJump()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:292`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.front_jump()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.push_up`

对应 C++ SDK 操作 `PushUp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def push_up(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `PushUp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:298`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.push_up()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.up_jump

对应 C++ SDK 操作 `UpJump()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def up_jump(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `UpJump()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:304`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.up_jump()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.set_auto_recovery

设置 `auto recovery`。具体副作用和安全边界见下方状态。

签名

```
def set_auto_recovery(self, switch_on: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
switch_on	int	必填	传给该接口的开关 on 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。 对应 C++ 参数 switch_on: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `SetAutoRecovery(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:310`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_auto_recovery(switch_on=switch_on)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.as2.SportClient.switch_joystick

对应 C++ SDK 操作 `SwitchJoystick(int)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def switch_joystick(self, switch_on: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
switch_on	int	必填	传给该接口的开关 on 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。 对应 C++ 参数 switch_on: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `SwitchJoystick(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:319`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_joystick(switch_on=switch_on)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.as2.SportClient.get_state`

查询或检查状态。

签名

```
def get_state(self) -> tuple[int, dict[str, str]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务; 纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] <code>status</code>	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务错误码解释。
[1] <code>state_map</code>	<code>dict[str, str]</code>	传给该接口的状态 map 参数, Python 类型为 <code>dict[str, str]</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>state_map: std::map<std::string, std::string> &</code> 。

对应 C++

- 类: `unitree::robot::as2::SportClient`
- 签名: `GetState(std::map<std::string, std::string> &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/as2/sport/sport_client.hpp:328`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, state_map = obj.get_state()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码, 并处理超时和网络中断。

B2 Robot API

模块: `unitree_sdk2_cpp.robot.b2`

类索引

类	公开函数签名	属性
BackVideoClient	3	0
ConfigClient	8	0
ConfigDelParameter	3	0
ConfigGetData	3	0
ConfigGetParameter	3	0
ConfigMeta	1	0
ConfigMetaData	3	0
ConfigMetaParameter	3	0
ConfigSetParameter	3	0
FrontVideoClient	3	0
JsonizeConfigMeta	3	0
JsonizeModeName	3	0
JsonizeSilent	3	0
LowPowerStatusData	3	0
LowPowerSwitch	3	0
Parameter		
MotionSwitcher	7	0
Client		
PkgVersionData	3	0
RobotStateClient	8	0
ServiceState	1	0
ServiceStateData	3	0
ServiceSwitchData	3	0
ServiceSwitch	3	0
Parameter		
SetReportFreq	3	0
Parameter		
SportClient	25	0
stPathPoint	1	0

`unitree_sdk2_cpp.robot.b2.BackVideoClient`

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.b2 import BackVideoClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
__init__	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
init	def init(self) -> None	AVAILABLE	INITIALIZATION
get_image_sample	def get_image_sample(self) -> tuple[int, list[int]]	AVAILABLE	READ_ONLY

`unitree_sdk2_cpp.robot.b2.BackVideoClient.__init__`

初始化 `BackVideoClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::BackVideoClient`
- 签名: `BackVideoClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/back_video/back_video_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = BackVideoClient()
```

`unitree_sdk2_cpp.robot.b2.BackVideoClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::BackVideoClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/back_video/back_video_client.hpp:21`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

unitree_sdk2_cpp.robot.b2.BackVideoClient.get_image_sample

查询或检查图像 `sample`。

签名

```
def get_image_sample(self) -> tuple[int, list[int]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] output_1	list[int]	传给该接口的 output 1 参数，Python 类型为 list[int]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 output_1: std::vector<uint8_t> &。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

对应 C++

- 类: `unitree::robot::b2::BackVideoClient`
- 签名: `GetImageSample(std::vector<uint8_t> &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/b2/back_video/back_video_client.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, output_1 = obj.get_image_sample()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.b2.ConfigClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.b2 import ConfigClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION

方法	Python 签名	状态	安全分类
<code>set</code>	<code>def set(self, name: str, content: str) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>get</code>	<code>def get(self, name: str) -> tuple[int, str]</code>	AVAILABLE	READ_ONLY
<code>del_</code>	<code>def del_(self, name: str) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>meta_config_meta</code>	<code>def meta_config_meta(self, name: str) -> tuple[int, ConfigMeta]</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>meta_string</code>	<code>def meta_string(self, name: str) -> tuple[int, str]</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>subscribe_change_status</code>	<code>def subscribe_change_status(self, name: str, callback: Callable[[str], None]) -> None</code>	AVAILABLE	HARDWARE_SIDE_EFFECT

unitree_sdk2_cpp.robot.b2.ConfigClient.__init__

初始化 ConfigClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ConfigClient`
- 签名: `ConfigClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/config/config_client.hpp:41`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigClient()
```

unitree_sdk2_cpp.robot.b2.ConfigClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ConfigClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/config/config_client.hpp:44`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.b2.ConfigClient.set`

对应 C++ SDK 操作 `Set(const std::string &, const std::string &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def set(self, name: str, content: str) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>name</code>	<code>str</code>	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。
<code>content</code>	<code>str</code>	必填	配置、请求或序列化内容。具体格式由对应服务协议定义。对应 C++ 参数 <code>content: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::ConfigClient`
- 签名: `Set(const std::string &, const std::string &)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/config/config_client.hpp:46`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set(name=name, content=content)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.ConfigClient.get`

对应 C++ SDK 操作 `Get(const std::string &, std::string &)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def get(self, name: str) -> tuple[int, str]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务; 纯本地 getter 则不一定需要全部条件。

参数

名称	Python 类型	默认值	含义
<code>name</code>	<code>str</code>	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串; 长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
<code>[0] status</code>	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务错误码解释。
<code>[1] content</code>	<code>str</code>	配置、请求或序列化内容。具体格式由对应服务协议定义。对应 C++ 参数 <code>content: std::string &</code> 。底层为字符串; 长度、编码和允许值由具体协议决定。

对应 C++

- 类: `unitree::robot::b2::ConfigClient`
- 签名: `Get(const std::string &, std::string &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/b2/config/config_client.hpp:47`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, content = obj.get(name=name)
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.b2.ConfigClient.del_

对应 C++ SDK 操作 `Del(const std::string &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def del_(self, name: str) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::ConfigClient`
- 签名: `Del(const std::string &)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/config/config_client.hpp:48`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.del_(name=name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.ConfigClient.meta_config_meta

对应 C++ SDK 操作 `Meta(const std::string &, unitree::robot::b2::ConfigMeta &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def meta_config_meta(self, name: str) -> tuple[int, ConfigMeta]
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] meta	ConfigMeta	传给该接口的 meta 参数，Python 类型为 ConfigMeta。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 meta: unitree::robot::b2::ConfigMeta &。

对应 C++

- 类: unitree::robot::b2::ConfigClient
- 签名: Meta(const std::string &, unitree::robot::b2::ConfigMeta &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/b2/config/config_client.hpp:49

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, meta = obj.meta_config_meta(name=name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.ConfigClient.meta_string

对应 C++ SDK 操作 Meta(const std::string &, std::string &)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def meta_string(self, name: str) -> tuple[int, str]
```

可用性与安全性

AVAILABLE / **HARDWARE_SIDE_EFFECT**: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] meta	str	传给该接口的 meta 参数，Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 meta: std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

对应 C++

- 类: unitree::robot::b2::ConfigClient
- 签名: Meta(const std::string &, std::string &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/b2/config/config_client.hpp:50

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, meta = obj.meta_string(name=name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.ConfigClient.subscribe_change_status

对应 C++ SDK 操作 SubscribeChangeStatus(const std::string &, const unitree::robot::b2::ConfigChangeStatusCallback &)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def subscribe_change_status(self, name: str, callback: Callable[[str, str], None]) -> None
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。
callback	Callable[[str, str], None]	必填	收到数据或状态变化时调用的 Python 回调。回调签名必须与类型提示一致，并应快速返回。对应 C++ 参数 callback: const unitree::robot::b2::ConfigChangeStatusCallback &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ConfigClient`
- 签名: `SubscribeChangeStatus(const std::string &, const unitree::robot::b2::ConfigChangeStatusCallback &)`
- 绑定策略: `CALLBACK_MANUAL`
- 声明位置: `include/unitree/robot/b2/config/config_client.hpp:52`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 回调可能由 SDK 工作线程触发；回调应快速返回、捕获异常，并通过线程安全队列移交耗时工作。

用法

```
obj.subscribe_change_status(name=name, callback=callback)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.ConfigDelParameter`

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.b2 import ConfigDelParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.b2.ConfigDelParameter.__init__`

初始化 `ConfigDelParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ConfigDelParameter`
- 签名: `ConfigDelParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:140`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigDelParameter()
```

`unitree_sdk2_cpp.robot.b2.ConfigDelParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ConfigDelParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:146`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```


unitree_sdk2_cpp.robot.b2.ConfigDelParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::ConfigDelParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:151`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.ConfigGetData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import ConfigGetData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>content</code>	<code>str</code>	配置、请求或序列化内容。具体格式由对应服务协议定义。	<code>value = obj.content / obj.content = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED

方法	Python 签名	状态	安全分类
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.b2.ConfigGetData.__init__`

初始化 `ConfigGetData` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ConfigGetData`
- 签名: `ConfigGetData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:117`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigGetData()
```

`unitree_sdk2_cpp.robot.b2.ConfigGetData.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ConfigGetData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:123`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.b2.ConfigGetData.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::ConfigGetData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:128`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.ConfigGetParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.b2 import ConfigGetParameter
```

基类: object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.ConfigGetParameter.__init__

初始化 ConfigGetParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ConfigGetParameter`
- 签名: `ConfigGetParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:94`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigGetParameter()
```

unitree_sdk2_cpp.robot.b2.ConfigGetParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::b2::ConfigGetParameter
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/b2/config/config_api.hpp:100

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.b2.ConfigGetParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::b2::ConfigGetParameter
- 签名: toJson(common::JsonMap &) const
- 绑定策略: JSON_DICT_OUTPUT

- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:105`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.b2.ConfigMeta`

SDK 数据值类型; 公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import ConfigMeta
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>
<code>last_modified</code>	<code>str</code>	传给该接口的 <code>last modified</code> 参数, Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.last_modified / obj.last_modified = value</code>
<code>size</code>	<code>int</code>	传给该接口的 <code>size</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.size / obj.size = value</code>
<code>epoch</code>	<code>int</code>	传给该接口的 <code>epoch</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.epoch / obj.epoch = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

`unitree_sdk2_cpp.robot.b2.ConfigMeta.__init__`

初始化 `ConfigMeta` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ConfigMeta`
- 签名: `ConfigMeta()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/config/config_client.hpp:20`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigMeta()
```

`unitree_sdk2_cpp.robot.b2.ConfigMetaData`

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import ConfigMetaData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>meta</code>	<code>JsonizeConfigMeta</code>	传给该接口的 <code>meta</code> 参数， <code>value = obj.meta / obj.meta = value</code> Python 类型为 <code>JsonizeConfigMeta</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.b2.ConfigMetaData.__init__`

初始化 `ConfigMetaData` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ConfigMetaData`
- 签名: `ConfigMetaData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:186`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigMetaData()
```

`unitree_sdk2_cpp.robot.b2.ConfigMetaData.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ConfigMetaData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:192`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.b2.ConfigMetaData.toJson

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::ConfigMetaData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:197`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.ConfigMetaParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.b2 import ConfigMetaParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.b2.ConfigMetaParameter.__init__`

初始化 `ConfigMetaParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ConfigMetaParameter`
- 签名: `ConfigMetaParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:163`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigMetaParameter()
```

`unitree_sdk2_cpp.robot.b2.ConfigMetaParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ConfigMetaParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:169`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.b2.ConfigMetaParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::ConfigMetaParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:174`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.ConfigSetParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.b2 import ConfigSetParameter
```

基类：object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>
content	str	配置、请求或序列化内容。具体格式由对应服务协议定义。	<code>value = obj.content / obj.content = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.ConfigSetParameter.__init__

初始化 ConfigSetParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类：unitree::robot::b2::ConfigSetParameter
- 签名：ConfigSetParameter()
- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/b2/config/config_api.hpp:68

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigSetParameter()
```

`unitree_sdk2_cpp.robot.b2.ConfigSetParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ConfigSetParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:74`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.b2.ConfigSetParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::ConfigSetParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:80`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.FrontVideoClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.b2 import FrontVideoClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>get_image_sample</code>	<code>def get_image_sample(self) -> tuple[int, list[int]]</code>	AVAILABLE	READ_ONLY

unitree_sdk2_cpp.robot.b2.FrontVideoClient.__init__

初始化 `FrontVideoClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::FrontVideoClient`
- 签名: `FrontVideoClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/front_video/front_video_client.hpp:18`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = FrontVideoClient()
```

`unitree_sdk2_cpp.robot.b2.FrontVideoClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::FrontVideoClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/front_video/front_video_client.hpp:21`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.b2.FrontVideoClient.get_image_sample`

查询或检查图像 `sample`。

签名

```
def get_image_sample(self) -> tuple[int, list[int]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务; 纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] output_1	list[int]	传给该接口的 output 1 参数，Python 类型为 list[int]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 output_1: std::vector<uint8_t> &。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

对应 C++

- 类: `unitree::robot::b2::FrontVideoClient`
- 签名: `GetImageSample(std::vector<uint8_t> &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/b2/front_video/front_video_client.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, output_1 = obj.get_image_sample()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

`unitree_sdk2_cpp.robot.b2.JsonizeConfigMeta`

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import JsonizeConfigMeta
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>
last_modified	str	传给该接口的 last modified 参数，Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.last_modified / obj.last_modified = value</code>
size	int	传给该接口的 size 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.size / obj.size = value</code>
epoch	int	传给该接口的 epoch 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.epoch / obj.epoch = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.JsonizeConfigMeta.__init__

初始化 `JsonizeConfigMeta` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::JsonizeConfigMeta`
- 签名: `JsonizeConfigMeta()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:36`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeConfigMeta()
```

unitree_sdk2_cpp.robot.b2.JsonizeConfigMeta.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::JsonizeConfigMeta`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:42`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.b2.JsonizeConfigMeta.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::JsonizeConfigMeta`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/config/config_api.hpp:50`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.JsonizeModeName

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.b2 import JsonizeModeName
```

基类: object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>
form	str	传给该接口的 form 参数, Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.form / obj.form = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.JsonizeModeName.__init__

初始化 JsonizeModeName 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::JsonizeModeName`
- 签名: `JsonizeModeName()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_api.hpp:37`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeModeName()
```

unitree_sdk2_cpp.robot.b2.JsonizeModeName.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::JsonizeModeName`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_api.hpp:44`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.b2.JsonizeModeName.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::JsonizeModeName`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_api.hpp:55`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.JsonizeSilent

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.b2 import JsonizeSilent
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>silent</code>	<code>bool</code>	传给该接口的静音状态参数, Python 类型为 <code>bool</code> 。 精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.silent / obj.silent = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.JsonizeSilent.__init__

初始化 `JsonizeSilent` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::JsonizeSilent`
- 签名: `JsonizeSilent()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_api.hpp:73`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeSilent()
```

`unitree_sdk2_cpp.robot.b2.JsonizeSilent.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::JsonizeSilent`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_api.hpp:80`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.b2.JsonizeSilent.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::JsonizeSilent`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_api.hpp:86`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.LowPowerStatusData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import LowPowerStatusData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
status	int	传给该接口的状态参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.status / obj.status = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.LowPowerStatusData.__init__

初始化 LowPowerStatusData 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::LowPowerStatusData`
- 签名: `LowPowerStatusData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:183`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LowPowerStatusData()
```

unitree_sdk2_cpp.robot.b2.LowPowerStatusData.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::LowPowerStatusData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:189`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.b2.LowPowerStatusData.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::LowPowerStatusData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:194`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.LowPowerSwitchParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.b2 import LowPowerSwitchParameter
```

基类：object

公开属性

名称	Python 类型	含义	用法
swit	int	传给该接口的 swit 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.swit / obj.swit = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.LowPowerSwitchParameter.__init__

初始化 LowPowerSwitchParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类：unitree::robot::b2::LowPowerSwitchParameter
- 签名：LowPowerSwitchParameter()
- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/b2/robot_state/robot_state_api.hpp:157

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LowPowerSwitchParameter()
```

unitree_sdk2_cpp.robot.b2.LowPowerSwitchParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::b2::LowPowerSwitchParameter
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/b2/robot_state/robot_state_api.hpp:163

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.b2.LowPowerSwitchParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::LowPowerSwitchParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:168`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.b2.MotionSwitcherClient`

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.b2 import MotionSwitcherClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>check_mode</code>	<code>def check_mode(self) -> tuple[int, str, str]</code>	AVAILABLE	READ_ONLY
<code>select_mode</code>	<code>def select_mode(self, name_or_alias: str) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>release_mode</code>	<code>def release_mode(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_silent</code>	<code>def set_silent(self, silent: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>get_silent</code>	<code>def get_silent(self) -> tuple[int, bool]</code>	AVAILABLE	READ_ONLY

`unitree_sdk2_cpp.robot.b2.MotionSwitcherClient.__init__`

初始化 `MotionSwitcherClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::MotionSwitcherClient`
- 签名: `MotionSwitcherClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MotionSwitcherClient()
```

`unitree_sdk2_cpp.robot.b2.MotionSwitcherClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::MotionSwitcherClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_client.hpp:21`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.b2.MotionSwitcherClient.check_mode`

查询或检查模式。

签名

```
def check_mode(self) -> tuple[int, str, str]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] form	str	传给该接口的 form 参数，Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 form: std::string &。底层为字符串；长度、编码和允许值由具体协议决定。
[2] name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

对应 C++

- 类: `unitree::robot::b2::MotionSwitcherClient`
- 签名: `CheckMode(std::string &, std::string &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_client.hpp`: 23

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, form, name = obj.check_mode()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

`unitree_sdk2_cpp.robot.b2.MotionSwitcherClient.select_mode`

对应 C++ SDK 操作 `SelectMode(const std::string &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def select_mode(self, name_or_alias: str) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
name_or_alias	str	必填	传给该接口的名称 or alias 参数, Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 nameOrAlias: const std::string &。底层为字符串; 长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::MotionSwitcherClient`
- 签名: `SelectMode(const std::string &)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_client.hpp:24`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.select_mode(name_or_alias=name_or_alias)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.MotionSwitcherClient.release_mode`

对应 C++ SDK 操作 `ReleaseMode()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def release_mode(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::MotionSwitcherClient`

- 签名: `ReleaseMode()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_client.hpp: 25`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.release_mode()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.MotionSwitcherClient.set_silent

设置静音状态。具体副作用和安全边界见下方状态。

签名

```
def set_silent(self, silent: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>silent</code>	<code>bool</code>	必填	传给该接口的静音状态参数, Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>silent: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::MotionSwitcherClient`
- 签名: `SetSilent(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_client.hpp: 26`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_silent(silent=silent)
```


[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.MotionSwitcherClient.get_silent

查询或检查静音状态。

签名

```
def get_silent(self) -> tuple[int, bool]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] silent	bool	传给该接口的静音状态参数，Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>silent: bool &</code> 。只接受布尔语义。

对应 C++

- 类: `unitree::robot::b2::MotionSwitcherClient`
- 签名: `GetSilent(bool &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/b2/motion_switcher/motion_switcher_client.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, silent = obj.get_silent()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.b2.PkgVersionData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import PkgVersionData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
package_version	str	传给该接口的 package version 参数, Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.package_version / obj.package_version = value
module_version_map	dict[str, str]	传给该接口的 module version map 参数, Python 类型为 dict[str, str]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.module_version_map / obj.module_version_map = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.PkgVersionData.__init__

初始化 PkgVersionData 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::PkgVersionData`
- 签名: `PkgVersionData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:209`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PkgVersionData()
```

unitree_sdk2_cpp.robot.b2.PkgVersionData.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::b2::PkgVersionData
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/b2/robot_state/robot_state_api.hpp:215

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.b2.PkgVersionData.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::PkgVersionData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:221`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

unitree_sdk2_cpp.robot.b2.RobotStateClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.b2 import RobotStateClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>service_list</code>	<code>def service_list(self) -> tuple[int, list[Service State]]</code>	AVAILABLE	READ_ONLY
<code>service_switch</code>	<code>def service_switch(self, name: str, swit: int) -> tuple[int, int]</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>set_report_freq</code>	<code>def set_report_freq(self, interval: int, duration: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>low_power_switch</code>	<code>def low_power_switch(self, swit: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>low_power_status</code>	<code>def low_power_status(self) -> tuple[int, int]</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>get_pkg_version</code>	<code>def get_pkg_version(self) -> tuple[int, str, dict[str, str]]</code>	AVAILABLE	READ_ONLY

unitree_sdk2_cpp.robot.b2.RobotStateClient.__init__

初始化 `RobotStateClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::RobotStateClient`
- 签名: `RobotStateClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_client.hpp:32`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = RobotStateClient()
```

`unitree_sdk2_cpp.robot.b2.RobotStateClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::RobotStateClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_client.hpp:35`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.b2.RobotStateClient.service_list`

对应 C++ SDK 操作 `ServiceList(std::vector<ServiceState> &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def service_list(self) -> tuple[int, list[ServiceState]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] serviceStateList	list[ServiceState]	传给该接口的 serviceStateList 参数，Python 类型为 list[ServiceState]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 serviceStateList: std::vector<ServiceState> &。底层是可变长度 vector

对应 C++

- 类: `unitree::robot::b2::RobotStateClient`
- 签名: `ServiceList(std::vector<ServiceState> &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_client.hpp:37`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, serviceStateList = obj.service_list()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.b2.RobotStateClient.service_switch

对应 C++ SDK 操作 `ServiceSwitch(const std::string &, int32_t, int32_t &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def service_switch(self, name: str, swit: int) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。
swit	int	必填	传给该接口的 swit 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>swit: int32_t</code> 。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
[0] status	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务错误码解释。
[1] status	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务错误码解释。

对应 C++

- 类: `unitree::robot::b2::RobotStateClient`
- 签名: `ServiceSwitch(const std::string &, int32_t, int32_t &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_client.hpp:38`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, status = obj.service_switch(name=name, swit=swit)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.RobotStateClient.set_report_freq`

设置 `report_freq`。具体副作用和安全边界见下方状态。

签名

```
def set_report_freq(self, interval: int, duration: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现, 调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>interval</code>	<code>int</code>	必填	传给该接口的 <code>interval</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>interval: int32_t</code> 。取值范围为 -2147483648 到 2147483647。
<code>duration</code>	<code>int</code>	必填	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。对应 C++ 参数 <code>duration: int32_t</code> 。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::RobotStateClient`
- 签名: `SetReportFreq(int32_t, int32_t)`
- 绑定策略: `DIRECT`

- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_client.hpp:39`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set_report_freq(interval=interval, duration=duration)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.RobotStateClient.low_power_switch`

对应 C++ SDK 操作 `LowPowerSwitch(int32_t)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def low_power_switch(self, swit: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现, 调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>swit</code>	<code>int</code>	必填	传给该接口的 <code>swit</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>swit: int32_t</code> 。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::RobotStateClient`
- 签名: `LowPowerSwitch(int32_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_client.hpp:40`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.low_power_switch(swit=swit)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.RobotStateClient.low_power_status`

对应 C++ SDK 操作 `LowPowerStatus(int32_t &)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def low_power_status(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。

对应 C++

- 类: `unitree::robot::b2::RobotStateClient`
- 签名: `LowPowerStatus(int32_t &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_client.hpp:41`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, status = obj.low_power_status()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.RobotStateClient.get_pkg_version

查询或检查 `pkg version`。

签名

```
def get_pkg_version(self) -> tuple[int, str, dict[str, str]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] packageVersion	str	传给该接口的 <code>packageVersion</code> 参数，Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>packageVersion: std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

位置	类型	含义
[2] moduleVersionMap	dict[str, str]	传给该接口的 moduleVersionMap 参数, Python 类型为 dict[str, str]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 moduleVersionMap: std::map<std::string, std::string> &。

对应 C++

- 类: unitree::robot::b2::RobotStateClient
- 签名: GetPkgVersion(std::string &, std::map<std::string, std::string> &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/b2/robot_state/robot_state_client.hpp:42

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, packageVersion, moduleVersionMap = obj.get_pkg_version()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码, 并处理超时和网络中断。

unitree_sdk2_cpp.robot.b2.ServiceState

SDK 数据值类型; 公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import ServiceState
```

基类: object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	value = obj.name / obj.name = value
status	int	传给该接口的状态参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.status / obj.status = value
protect	int	传给该接口的 protect 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.protect / obj.protect = value

方法索引

方法	Python 签名	状态	安全分类
__init__	def __init__(self) -> None	AVAILABLE	CONSTRUCTION

unitree_sdk2_cpp.robot.b2.ServiceState.__init__

初始化 ServiceState 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ServiceState`
- 签名: `ServiceState()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ServiceState()
```

`unitree_sdk2_cpp.robot.b2.ServiceStateData`

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import ServiceStateData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>
<code>status</code>	<code>int</code>	传给该接口的状态参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.status / obj.status = value</code>
<code>protect</code>	<code>int</code>	传给该接口的 <code>protect</code> 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.protect / obj.protect = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.ServiceStateData.__init__

初始化 ServiceStateData 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ServiceStateData`
- 签名: `ServiceStateData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:125`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ServiceStateData()
```

unitree_sdk2_cpp.robot.b2.ServiceStateData.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ServiceStateData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:131`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.b2.ServiceStateData.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::ServiceStateData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:138`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.ServiceSwitchData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import ServiceSwitchData
```

基类：object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	value = obj.name / obj.name = value
status	int	传给该接口的状态参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.status / obj.status = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.ServiceSwitchData.__init__

初始化 ServiceSwitchData 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类：unitree::robot::b2::ServiceSwitchData
- 签名：ServiceSwitchData()
- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/b2/robot_state/robot_state_api.hpp:67

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ServiceSwitchData()
```

`unitree_sdk2_cpp.robot.b2.ServiceSwitchData.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ServiceSwitchData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:73`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.b2.ServiceSwitchData.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::ServiceSwitchData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:79`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.ServiceSwitchParameter

SDK 请求参数值类型; 当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.b2 import ServiceSwitchParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>
<code>swit</code>	<code>int</code>	传给该接口的 <code>swit</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.swit / obj.swit = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.b2.ServiceSwitchParameter.__init__

初始化 `ServiceSwitchParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```


可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::ServiceSwitchParameter`
- 签名: `ServiceSwitchParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:38`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ServiceSwitchParameter()
```

`unitree_sdk2_cpp.robot.b2.ServiceSwitchParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::ServiceSwitchParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:44`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.b2.ServiceSwitchParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::ServiceSwitchParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:50`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.SetReportFreqParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.b2 import SetReportFreqParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>interval</code>	<code>int</code>	传给该接口的 <code>interval</code> 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.interval / obj.interval = value</code>
<code>duration</code>	<code>int</code>	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。	<code>value = obj.duration / obj.duration = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.b2.SetReportFreqParameter.__init__`

初始化 `SetReportFreqParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::SetReportFreqParameter`
- 签名: `SetReportFreqParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:96`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = SetReportFreqParameter()
```

`unitree_sdk2_cpp.robot.b2.SetReportFreqParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::SetReportFreqParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:102`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.b2.SetReportFreqParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::b2::SetReportFreqParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/b2/robot_state/robot_state_api.hpp:108`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.b2.SportClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.b2 import SportClient
```

基类: Client

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, enable_lease: bool = False) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>damp</code>	<code>def damp(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>balance_stand</code>	<code>def balance_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stop_move</code>	<code>def stop_move(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stand_up</code>	<code>def stand_up(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stand_down</code>	<code>def stand_down(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>recovery_stand</code>	<code>def recovery_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move</code>	<code>def move(self, vx: float, vy: float, vyaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_gait</code>	<code>def switch_gait(self, d: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>body_height</code>	<code>def body_height(self, height: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>speed_level</code>	<code>def speed_level(self, level: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>trajectory_follow</code>	<code>def trajectory_follow(self, path: Sequence[PathPoint]) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>continuous_gait</code>	<code>def continuous_gait(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move_to_pos</code>	<code>def move_to_pos(self, x: float, y: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_move_mode</code>	<code>def switch_move_mode(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>vision_walk</code>	<code>def vision_walk(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>hand_stand</code>	<code>def hand_stand(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>auto_recovery_set</code>	<code>def auto_recovery_set(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>free_walk</code>	<code>def free_walk(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>classic_walk</code>	<code>def classic_walk(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>fast_walk</code>	<code>def fast_walk(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND

方法	Python 签名	状态	安全分类
<code>euler</code>	<code>def euler(self, roll: float, pitch: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>free_height</code>	<code>def free_height(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>gait_height</code>	<code>def gait_height(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND

`unitree_sdk2_cpp.robot.b2.SportClient.__init__`

初始化 `SportClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, enable_lease: bool = False) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>enable_lease</code>	<code>bool</code>	<code>False</code>	是否启用 SDK lease 机制；lease 的获得、续期和释放规则以服务协议为准。对应 C++ 参数 <code>enableLease: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `SportClient(bool)`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:34`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = SportClient(enable_lease=enable_lease)
```

`unitree_sdk2_cpp.robot.b2.SportClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:37`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.b2.SportClient.damp`

对应 C++ SDK 操作 `Damp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def damp(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `Damp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:39`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.damp()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.balance_stand

对应 C++ SDK 操作 `BalanceStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def balance_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `BalanceStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:41`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.balance_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.stop_move

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_move(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `StopMove()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:43`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stop_move()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.SportClient.stand_up`

对应 C++ SDK 操作 `StandUp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_up(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `StandUp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:45`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_up()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.stand_down

对应 C++ SDK 操作 `StandDown()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_down(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `StandDown()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:47`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_down()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.recovery_stand

对应 C++ SDK 操作 `RecoveryStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def recovery_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `RecoveryStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:49`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.recovery_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.SportClient.move`

对应 C++ SDK 操作 `Move(float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def move(self, vx: float, vy: float, vyaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>vx</code>	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vx: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>vy</code>	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vy: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>vyaw</code>	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vyaw: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `Move(float, float, float)`

- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:51`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.switch_gait

对应 C++ SDK 操作 `SwitchGait(int)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def switch_gait(self, d: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>d</code>	<code>int</code>	必填	传给该接口的 <code>d</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>d: int</code> 。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `SwitchGait(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:53`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_gait(d=d)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.body_height

对应 C++ SDK 操作 `BodyHeight(float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def body_height(self, height: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
height	float	必填	传给该接口的高度参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>height: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `BodyHeight(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:55`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.body_height(height=height)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.speed_level

对应 C++ SDK 操作 `SpeedLevel(int)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def speed_level(self, level: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
level	int	必填	传给该接口的 level 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 level: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `SpeedLevel(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:57`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.speed_level(level=level)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.trajectory_follow

对应 C++ SDK 操作 `TrajectoryFollow(std::vector<unitree::robot::b2::PathPoint> &)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def trajectory_follow(self, path: Sequence[PathPoint]) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
path	<code>Sequence[PathPoint]</code>	必填	传给该接口的 path 参数, Python 类型为 <code>Sequence[PathPoint]</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>path: std::vector<unitree::robot::b2::PathPoint> &</code> 。底层是可变长度 vector

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `TrajectoryFollow(std::vector<unitree::robot::b2::PathPoint> &)`
- 绑定策略: `MUTABLE_INPUT_COPY`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:59`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.trajectory_follow(path=path)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.continuous_gait

对应 C++ SDK 操作 `ContinuousGait(bool)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def continuous_gait(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>flag</code>	<code>bool</code>	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `ContinuousGait(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:61`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.continuous_gait(flag=flag)
```


[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.move_to_pos

对应 C++ SDK 操作 MoveToPos(float, float, float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def move_to_pos(self, x: float, y: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
x	float	必填	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 x: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
y	float	必填	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 y: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 yaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::b2::SportClient
- 签名: MoveToPos(float, float, float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/b2/sport/sport_client.hpp:63

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.move_to_pos(x=x, y=y, yaw=yaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.switch_move_mode

对应 C++ SDK 操作 SwitchMoveMode(bool)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名


```
def switch_move_mode(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::b2::SportClient
- 签名: SwitchMoveMode(bool)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/b2/sport/sport_client.hpp:65

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.switch_move_mode(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.vision_walk

对应 C++ SDK 操作 VisionWalk(bool)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def vision_walk(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `VisionWalk(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:67`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.vision_walk(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.SportClient.hand_stand`

对应 C++ SDK 操作 `HandStand(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def hand_stand(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `HandStand(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:69`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.hand_stand(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.auto_recovery_set

对应 C++ SDK 操作 `AutoRecoverySet(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def auto_recovery_set(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `AutoRecoverySet(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:71`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.auto_recovery_set(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.free_walk

对应 C++ SDK 操作 `FreeWalk()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def free_walk(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `FreeWalk()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:73`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.free_walk()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.classic_walk

对应 C++ SDK 操作 `ClassicWalk(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def classic_walk(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>flag</code>	<code>bool</code>	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `ClassicWalk(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:75`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.classic_walk(flag=flag)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.SportClient.fast_walk`

对应 C++ SDK 操作 `FastWalk(bool)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def fast_walk(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `FastWalk(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:77`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.fast_walk(flag=flag)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.euler

对应 C++ SDK 操作 Euler(float, float, float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def euler(self, roll: float, pitch: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
roll	float	必填	传给该接口的 roll 参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 roll: float。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。
pitch	float	必填	传给该接口的 pitch 参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 pitch: float。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 yaw: float。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::b2::SportClient
- 签名: Euler(float, float, float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/b2/sport/sport_client.hpp:79

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.euler(roll=roll, pitch=pitch, yaw=yaw)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.free_height

对应 C++ SDK 操作 FreeHeight(bool)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def free_height(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `FreeHeight(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:81`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.free_height(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.b2.SportClient.gait_height

对应 C++ SDK 操作 `GaitHeight(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def gait_height(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::b2::SportClient`
- 签名: `GaitHeight(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/b2/sport/sport_client.hpp:83`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.gait_height(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.b2.stPathPoint`

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.b2 import stPathPoint
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>time_from_start</code>	<code>float</code>	传给该接口的 <code>time from start</code> 参数, Python 类型为 <code>float</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.time_from_start / obj.time_from_start = value</code>
<code>x</code>	<code>float</code>	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.x / obj.x = value</code>
<code>y</code>	<code>float</code>	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.y / obj.y = value</code>
<code>yaw</code>	<code>float</code>	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。	<code>value = obj.yaw / obj.yaw = value</code>
<code>vx</code>	<code>float</code>	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vx / obj.vx = value</code>
<code>vy</code>	<code>float</code>	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vy / obj.vy = value</code>
<code>vyaw</code>	<code>float</code>	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vyaw / obj.vyaw = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

`unitree_sdk2_cpp.robot.b2.stPathPoint.__init__`

初始化 `stPathPoint` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::b2::stPathPoint`
- 签名: `stPathPoint()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = stPathPoint()
```

G1 Robot API

模块: `unitree_sdk2_cpp.robot.g1`

模块函数导入

```
from unitree_sdk2_cpp.robot.g1 import bad_orientation, joint_vel_out_of_limit,
    ang_vel_out_of_limit, motor_winding_overheat, motor_casing_overheat,
    low_battery, lost_connection
```

下面的函数用法片段假设已经完成上述导入。

模块函数索引

函数	Python 签名	状态	安全分类
<code>bad_orientation()</code>	<code>def bad_orientation(low_state: LowState, limit_angle: float = 1.0) -> bool</code>	AVAILABLE	SAFETY_CHECK
<code>joint_vel_out_of_limit()</code>	<code>def joint_vel_out_of_limit(low_state: LowState, limit_vel: float = 10.0) -> bool</code>	AVAILABLE	SAFETY_CHECK
<code>ang_vel_out_of_limit()</code>	<code>def ang_vel_out_of_limit(low_state: LowState, limit_vel: float = 6.0) -> bool</code>	AVAILABLE	SAFETY_CHECK
<code>motor_winding_overheat()</code>	<code>def motor_winding_overheat(low_state: LowState, limit_temp: float = 120.0) -> bool</code>	AVAILABLE	SAFETY_CHECK
<code>motor_casing_overheat()</code>	<code>def motor_casing_overheat(low_state: LowState, limit_temp: float = 85.0) -> bool</code>	AVAILABLE	SAFETY_CHECK
<code>low_battery()</code>	<code>def low_battery(bms_state: BmsState, limit_soc: float = 20.0) -> bool</code>	AVAILABLE	SAFETY_CHECK
<code>lost_connection()</code>	<code>def lost_connection(subscriber: ChannelSubscriber[LowState], timeout_ms: int = 1000) -> bool</code>	AVAILABLE	SAFETY_CHECK

类索引

类	公开函数签名	属性
<code>InternalFsmMode</code>	0	0
<code>AgvClient</code>	4	0
<code>AudioClient</code>	8	0
<code>G1ArmActionClient</code>	6	0
<code>JsonizeDataVecFloat</code>	3	0
<code>JsonizeVelocity</code>	3	0
<code>Command</code>		
<code>LedControlParameter</code>	3	0
<code>LocoClient</code>	35	0
<code>MoveParameter</code>	3	0
<code>PlayStopParameter</code>	3	0
<code>PlayStreamParameter</code>	3	0
<code>TtsMakerParameter</code>	3	0

unitree_sdk2_cpp.robot.g1.bad_orientation

对应 C++ SDK 操作 `bad_orientation(const unitree_hg::msg::dds_::LowState_ &, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def bad_orientation(low_state: LowState, limit_angle: float = 1.0) -> bool
```

可用性与安全性

AVAILABLE / SAFETY_CHECK: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
low_state	LowState	必填	传给该接口的 low 状态参数, Python 类型为 LowState。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。
limit_angle	float	1.0	传给该接口的 limit angle 参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1`
- 签名: `bad_orientation(const unitree_hg::msg::dds_::LowState_ &, float)`
- 绑定策略: `DIRECT`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = bad_orientation(low_state=low_state, limit_angle=limit_angle)
```

unitree_sdk2_cpp.robot.g1.joint_vel_out_of_limit

对应 C++ SDK 操作 `joint_vel_out_of_limit(const unitree_hg::msg::dds_::LowState_ &, float)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def joint_vel_out_of_limit(low_state: LowState, limit_vel: float = 10.0) ->
    bool
```

可用性与安全性

AVAILABLE / SAFETY_CHECK: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
low_state	LowState	必填	传给该接口的 low 状态参数, Python 类型为 LowState。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。
limit_vel	float	10.0	传给该接口的 limit vel 参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1`
- 签名: `joint_vel_out_of_limit(const unitree_hg::msg::dds_::LowState_ &, float)`
- 绑定策略: `DIRECT`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = joint_vel_out_of_limit(low_state=low_state, limit_vel=limit_vel)
```

`unitree_sdk2_cpp.robot.g1.ang_vel_out_of_limit`

对应 C++ SDK 操作 `ang_vel_out_of_limit(const unitree_hg::msg::dds_::LowState_ &, float)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def ang_vel_out_of_limit(low_state: LowState, limit_vel: float = 6.0) -> bool
```

可用性与安全性

AVAILABLE / SAFETY_CHECK: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>low_state</code>	<code>LowState</code>	必填	传给该接口的 <code>low</code> 状态参数, Python 类型为 <code>LowState</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。
<code>limit_vel</code>	<code>float</code>	<code>6.0</code>	传给该接口的 <code>limit vel</code> 参数, Python 类型为 <code>float</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1`
- 签名: `ang_vel_out_of_limit(const unitree_hg::msg::dds_::LowState_ &, float)`
- 绑定策略: `DIRECT`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = ang_vel_out_of_limit(low_state=low_state, limit_vel=limit_vel)
```

unitree_sdk2_cpp.robot.g1.motor_winding_overheat

对应 C++ SDK 操作 `motor_winding_overheat(const unitree_hg::msg::dds_::LowState_ &, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def motor_winding_overheat(low_state: LowState, limit_temp: float = 120.0) ->
    bool
```

可用性与安全性

AVAILABLE / SAFETY_CHECK: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
low_state	LowState	必填	传给该接口的 low 状态参数，Python 类型为 LowState。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。
limit_temp	float	120.0	传给该接口的 limit temp 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1`
- 签名: `motor_winding_overheat(const unitree_hg::msg::dds_::LowState_ &, float)`
- 绑定策略: `DIRECT`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = motor_winding_overheat(low_state=low_state, limit_temp=limit_temp)
```

unitree_sdk2_cpp.robot.g1.motor_casing_overheat

对应 C++ SDK 操作 `motor_casing_overheat(const unitree_hg::msg::dds_::LowState_ &, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def motor_casing_overheat(low_state: LowState, limit_temp: float = 85.0) ->
    bool
```

可用性与安全性

AVAILABLE / SAFETY_CHECK: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
low_state	LowState	必填	传给该接口的 low 状态参数, Python 类型为 LowState。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。
limit_temp	float	85.0	传给该接口的 limit temp 参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1`
- 签名: `motor_casing_overheat(const unitree_hg::msg::dds_::LowState_ &, float)`
- 绑定策略: `DIRECT`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = motor_casing_overheat(low_state=low_state, limit_temp=limit_temp)
```

unitree_sdk2_cpp.robot.g1.low_battery

对应 C++ SDK 操作 `low_battery(const unitree_hg::msg::dds_::BmsState_ &, float)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def low_battery(bms_state: BmsState, limit_soc: float = 20.0) -> bool
```

可用性与安全性

AVAILABLE / SAFETY_CHECK: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
bms_state	BmsState	必填	传给该接口的 bms 状态参数, Python 类型为 BmsState。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。
limit_soc	float	20.0	传给该接口的 limit soc 参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1`
- 签名: `low_battery(const unitree_hg::msg::dds_::BmsState_ &, float)`
- 绑定策略: `DIRECT`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = low_battery(bms_state=bms_state, limit_soc=limit_soc)
```

unitree_sdk2_cpp.robot.g1.lost_connection

对应 C++ SDK 操作 `lost_connection(unitree::robot::ChannelSubscriberPtr<unitree_hg::msg::dds_::LowState_> &, int64_t)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def lost_connection(subscriber: ChannelSubscriber[LowState], timeout_ms: int
    = 1000) -> bool
```

可用性与安全性

AVAILABLE / SAFETY_CHECK: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
subscriber	<code>ChannelSubscriber[LowState]</code>	必填	传给该接口的 <code>subscriber</code> 参数, Python 类型为 <code>ChannelSubscriber[LowState]</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。
timeout_ms	<code>int</code>	1000	传给该接口的超时 <code>ms</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。

返回值

位置	类型	含义
返回值	bool	返回 bool。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1`
- 签名: `lost_connection(unitree::robot::ChannelSubscriberPtr<unitree_hg::msg::dds_::LowState_> &, int64_t)`
- 绑定策略: `TYPE_ERASED_ADAPTER`
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供; `manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = lost_connection(subscriber=subscriber, timeout_ms=timeout_ms)
```

unitree_sdk2_cpp.robot.g1.InternalFsmMode

SDK 整数枚举。枚举成员和值以目标版本头文件为准。

导入

```
from unitree_sdk2_cpp.robot.g1 import InternalFsmMode
```

基类: `enum.IntEnum`

枚举成员

名称	Stub 值
LAST	...
PASSIVE	...
WALKRUN	...

unitree_sdk2_cpp.robot.g1.AgvClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.g1 import AgvClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>move</code>	<code>def move(self, vx: float, vy: float, vyaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>height_adjust</code>	<code>def height_adjust(self, vz: float) -> int</code>	AVAILABLE	MOTION_COMMAND

unitree_sdk2_cpp.robot.g1.AgvClient.__init__

初始化 `AgvClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::AgvClient`
- 签名: `AgvClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/agv/g1_agv_client.hpp:15`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = AgvClient()
```

`unitree_sdk2_cpp.robot.g1.AgvClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::AgvClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/agv/g1_agv_client.hpp:19`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.g1.AgvClient.move`

对应 C++ SDK 操作 `Move(float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def move(self, vx: float, vy: float, vyaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vx: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vy: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vyaw	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vyaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::g1::AgvClient
- 签名: Move(float, float, float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/g1/agv/g1_agv_client.hpp:56

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.AgvClient.height_adjust

对应 C++ SDK 操作 HeightAdjust(float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def height_adjust(self, vz: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vz	float	必填	传给该接口的 vz 参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 vz: float。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::AgvClient`
- 签名: `HeightAdjust(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/agv/g1_agv_client.hpp:93`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.height_adjust(vz=vz)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.AudioClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.g1 import AudioClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>tts_maker</code>	<code>def tts_maker(self, text: str, speaker_id: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>get_volume</code>	<code>def get_volume(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>set_volume</code>	<code>def set_volume(self, volume: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>play_stream</code>	<code>def play_stream(self, app_name: str, stream_id: str, pcm_data: Sequence[int]) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>play_stop</code>	<code>def play_stop(self, app_name: str) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT

方法	Python 签名	状态	安全分类
<code>led_control</code>	<code>def led_control(self, r: int, g: int, b: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT

unitree_sdk2_cpp.robot.g1.AudioClient.__init__

初始化 `AudioClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::AudioClient`
- 签名: `AudioClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_client.hpp:15`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = AudioClient()
```

unitree_sdk2_cpp.robot.g1.AudioClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::AudioClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_client.hpp:19`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.g1.AudioClient.tts_maker`

对应 C++ SDK 操作 `TtsMaker(const std::string &, int32_t)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def tts_maker(self, text: str, speaker_id: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现, 调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>text</code>	<code>str</code>	必填	要处理的文本内容; 编码、长度和语言支持由目标服务决定。对应 C++ 参数 <code>text: const std::string &</code> 。底层为字符串; 长度、编码和允许值由具体协议决定。
<code>speaker_id</code>	<code>int</code>	必填	语音合成说话人 ID。有效编号由机器人音频服务和固件决定。对应 C++ 参数 <code>speaker_id: int32_t</code> 。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::AudioClient`
- 签名: `TtsMaker(const std::string &, int32_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_client.hpp:31`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.tts_maker(text=text, speaker_id=speaker_id)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.AudioClient.get_volume

查询或检查音量。

签名

```
def get_volume(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] volume	int	传给该接口的音量参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>volume: uint8_t &</code> 。取值范围为 0 到 255。

对应 C++

- 类: `unitree::robot::g1::AudioClient`
- 签名: `GetVolume(uint8_t &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_client.hpp:43`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, volume = obj.get_volume()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.g1.AudioClient.set_volume

设置音量。具体副作用和安全边界见下方状态。

签名

```
def set_volume(self, volume: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
volume	int	必填	传给该接口的音量参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>volume: uint8_t</code> 。取值范围为 0 到 255。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::AudioClient`
- 签名: `SetVolume(uint8_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_client.hpp:57`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set_volume(volume=volume)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.g1.AudioClient.play_stream`

对应 C++ SDK 操作 `PlayStream(std::string, std::string, std::vector<uint8_t>)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def play_stream(self, app_name: str, stream_id: str, pcm_data: Sequence[int])
    -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>app_name</code>	<code>str</code>	必填	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。对应 C++ 参数 <code>app_name: std::string</code> 。底层为字符串；长度、编码和允许值由具体协议决定。
<code>stream_id</code>	<code>str</code>	必填	音频流标识符，用于区分同一应用下的播放流。对应 C++ 参数 <code>stream_id: std::string</code> 。底层为字符串；长度、编码和允许值由具体协议决定。
<code>pcm_data</code>	<code>Sequence[int]</code>	必填	传给该接口的 pcm 数据参数，Python 类型为 <code>Sequence[int]</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>pcm_data: std::vector<uint8_t></code> 。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::AudioClient`
- 签名: `PlayStream(std::string, std::string, std::vector<uint8_t>)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_client.hpp:68`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.play_stream(app_name=app_name, stream_id=stream_id,
                        pcm_data=pcm_data)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.g1.AudioClient.play_stop`

对应 C++ SDK 操作 `PlayStop(std::string)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def play_stop(self, app_name: str) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现, 调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>app_name</code>	<code>str</code>	必填	应用名称, 用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。对应 C++ 参数 <code>app_name: std::string</code> 。底层为字符串; 长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::AudioClient`
- 签名: `PlayStop(std::string)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_client.hpp:80`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.play_stop(app_name=app_name)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.AudioClient.led_control

对应 C++ SDK 操作 `LedControl(uint8_t, uint8_t, uint8_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def led_control(self, r: int, g: int, b: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
r	int	必填	传给该接口的 r 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 R: <code>uint8_t</code> 。取值范围为 0 到 255。
g	int	必填	传给该接口的 g 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 G: <code>uint8_t</code> 。取值范围为 0 到 255。
b	int	必填	传给该接口的 b 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 B: <code>uint8_t</code> 。取值范围为 0 到 255。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::AudioClient`
- 签名: `LedControl(uint8_t, uint8_t, uint8_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_client.hpp:91`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.led_control(r=r, g=g, b=b)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.G1ArmActionClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.g1 import G1ArmActionClient
```

基类: `Client`

公开属性

名称	Python 类型	含义	用法
action_map	dict[str, int]	传给该接口的 action map 参数, Python 类型为 dict[str, int]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.action_map / obj.action_map = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>execute_action</code> (重载 1/2)	<code>def execute_action(self, action_id: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>execute_action</code> (重载 2/2)	<code>def execute_action(self, action_name: str) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stop_custom_action</code>	<code>def stop_custom_action(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>get_action_list</code>	<code>def get_action_list(self) -> tuple[int, str]</code>	AVAILABLE	READ_ONLY

unitree_sdk2_cpp.robot.g1.G1ArmActionClient.__init__

初始化 G1ArmActionClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::G1ArmActionClient`
- 签名: `G1ArmActionClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/arm/g1_arm_action_client.hpp:26`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = G1ArmActionClient()
```

unitree_sdk2_cpp.robot.g1.G1ArmActionClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::G1ArmActionClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/arm/g1_arm_action_client.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

unitree_sdk2_cpp.robot.g1.G1ArmActionClient.execute_action (重载 1/2)

对应 C++ SDK 操作 `ExecuteAction(int32_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def execute_action(self, action_id: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>action_id</code>	<code>int</code>	必填	传给该接口的 action ID 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>action_id: int32_t</code> 。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::G1ArmActionClient`
- 签名: `ExecuteAction(int32_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/arm/g1_arm_action_client.hpp:51`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.execute_action(action_id=action_id)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.g1.G1ArmActionClient.execute_action` (重载 2/2)

对应 C++ SDK 操作 `ExecuteAction(const std::string &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def execute_action(self, action_name: str) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>action_name</code>	<code>str</code>	必填	传给该接口的 <code>action</code> 名称参数，Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>action_name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::G1ArmActionClient`
- 签名: `ExecuteAction(const std::string &)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/arm/g1_arm_action_client.hpp:67`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.execute_action(action_name=action_name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.G1ArmActionClient.stop_custom_action

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_custom_action(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::G1ArmActionClient`
- 签名: `StopCustomAction()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/arm/g1_arm_action_client.hpp:74`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.stop_custom_action()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.G1ArmActionClient.get_action_list

查询或检查 action list。

签名

```
def get_action_list(self) -> tuple[int, str]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩

展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] data	str	传给该接口的数据参数，Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>data: std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

对应 C++

- 类: `unitree::robot::g1::G1ArmActionClient`
- 签名: `GetActionList(std::string &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/arm/g1_arm_action_client.hpp:79`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, data = obj.get_action_list()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

`unitree_sdk2_cpp.robot.g1.JsonizeDataVecFloat`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.g1 import JsonizeDataVecFloat
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
data	<code>list[float]</code>	传给该接口的数据参数，Python 类型为 <code>list[float]</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.data / obj.data = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED

方法	Python 签名	状态	安全分类
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.g1.JsonizeDataVecFloat.__init__

初始化 `JsonizeDataVecFloat` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::JsonizeDataVecFloat`
- 签名: `JsonizeDataVecFloat()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_api.hpp:45`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeDataVecFloat()
```

unitree_sdk2_cpp.robot.g1.JsonizeDataVecFloat.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::JsonizeDataVecFloat`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_api.hpp:48`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.g1.JsonizeDataVecFloat.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1::JsonizeDataVecFloat`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_api.hpp:50`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.g1.JsonizeVelocityCommand`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.g1 import JsonizeVelocityCommand
```


基类: object

公开属性

名称	Python 类型	含义	用法
velocity	list[float]	传给该接口的速度参数, Python 类型为 list[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.velocity / obj.velocity = value
duration	float	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。	value = obj.duration / obj.duration = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.g1.JsonizeVelocityCommand.__init__

初始化 JsonizeVelocityCommand 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::JsonizeVelocityCommand`
- 签名: `JsonizeVelocityCommand()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_api.hpp:59`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeVelocityCommand()
```

unitree_sdk2_cpp.robot.g1.JsonizeVelocityCommand.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::g1::JsonizeVelocityCommand
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/g1/loco/g1_loco_api.hpp:62

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.g1.JsonizeVelocityCommand.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1::JsonizeVelocityCommand`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_api.hpp:67`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

`unitree_sdk2_cpp.robot.g1.LedControlParameter`

SDK 请求参数值类型; 当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.g1 import LedControlParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>r</code>	<code>int</code>	传给该接口的 <code>r</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.r / obj.r = value</code>
<code>g</code>	<code>int</code>	传给该接口的 <code>g</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.g / obj.g = value</code>
<code>b</code>	<code>int</code>	传给该接口的 <code>b</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.b / obj.b = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.g1.LedControlParameter.__init__`

初始化 `LedControlParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::LedControlParameter`
- 签名: `LedControlParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:75`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LedControlParameter()
```

`unitree_sdk2_cpp.robot.g1.LedControlParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::LedControlParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:78`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.g1.LedControlParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1::LedControlParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:80`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.g1.LocoClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.g1 import LocoClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>get_fsm_id</code>	<code>def get_fsm_id(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>get_fsm_mode</code>	<code>def get_fsm_mode(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY

方法	Python 签名	状态	安全分类
<code>get_balance_mode</code>	<code>def get_balance_mode(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>get_swing_height</code>	<code>def get_swing_height(self) -> tuple[int, float]</code>	AVAILABLE	READ_ONLY
<code>get_stand_height</code>	<code>def get_stand_height(self) -> tuple[int, float]</code>	AVAILABLE	READ_ONLY
<code>get_phase</code>	<code>def get_phase(self) -> tuple[int, list[float]]</code>	AVAILABLE	READ_ONLY
<code>set_fsm_id</code>	<code>def set_fsm_id(self, fsm_id: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_balance_mode</code>	<code>def set_balance_mode(self, balance_mode: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_swing_height</code>	<code>def set_swing_height(self, swing_height: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_stand_height</code>	<code>def set_stand_height(self, stand_height: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_velocity</code>	<code>def set_velocity(self, vx: float, vy: float, omega: float, duration: float = 1.0) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_task_id</code>	<code>def set_task_id(self, task_id: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_to_user_ctrl</code>	<code>def switch_to_user_ctrl(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_to_internal_ctrl</code>	<code>def switch_to_internal_ctrl(self, mode: InternalFsmMode) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>damp</code>	<code>def damp(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>start</code>	<code>def start(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>squat</code>	<code>def squat(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>sit</code>	<code>def sit(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stand_up</code>	<code>def stand_up(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>zero_torque</code>	<code>def zero_torque(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stop_move</code>	<code>def stop_move(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>high_stand</code>	<code>def high_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>low_stand</code>	<code>def low_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move (重载 1/2)</code>	<code>def move(self, vx: float, vy: float, vyaw: float, continous_move: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move (重载 2/2)</code>	<code>def move(self, vx: float, vy: float, vyaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>balance_stand</code>	<code>def balance_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>continuous_gait</code>	<code>def continuous_gait(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_move_mode</code>	<code>def switch_move_mode(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND

方法	Python 签名	状态	安全分类
<code>wave_hand</code>	<code>def wave_hand(self, turn_ flag: bool = False) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>shake_hand</code>	<code>def shake_hand(self, stage: int = -1) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_speed_mode</code>	<code>def set_speed_mode(self, speed_mode: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>get_mimic_motion</code>	<code>def get_mimic_motion(self) -> tuple[int, str]</code>	AVAILABLE	READ_ONLY
<code>_fsm_api</code>	<code>def _fsm_api(self, parameter: str) -> tuple[int, str]</code>	AVAILABLE	MOTION_COMMAND

unitree_sdk2_cpp.robot.g1.LocoClient.__init__

初始化 LocoClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `LocoClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:14`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LocoClient()
```

unitree_sdk2_cpp.robot.g1.LocoClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.g1.LocoClient.get_fsm_id`

查询或检查 FSM ID。

签名

```
def get_fsm_id(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] fsm_id	int	传给该接口的 FSM ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>fsm_id: int &</code> 。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `GetFsmId(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:40`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法


```
status, fsm_id = obj.get_fsm_id()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.g1.LocoClient.get_fsm_mode

查询或检查 FSM 模式。

签名

```
def get_fsm_mode(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] fsm_mode	int	传给该接口的 FSM 模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>fsm_mode: int &</code> 。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `GetFsmMode(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:54`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, fsm_mode = obj.get_fsm_mode()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.g1.LocoClient.get_balance_mode

查询或检查平衡模式。

签名

```
def get_balance_mode(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] balance_mode	int	传给该接口的平衡模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 balance_mode: int &。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `GetBalanceMode(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:68`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, balance_mode = obj.get_balance_mode()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.g1.LocoClient.get_swing_height

查询或检查 swing 高度。

签名

```
def get_swing_height(self) -> tuple[int, float]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] swing_height	float	传给该接口的 swing 高度参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 swing_height: float &。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `GetSwingHeight(float &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:82`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, swing_height = obj.get_swing_height()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.g1.LocoClient.get_stand_height

查询或检查 stand 高度。

签名

```
def get_stand_height(self) -> tuple[int, float]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] stand_height	float	传给该接口的 stand 高度参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>stand_height: float &</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `GetStandHeight(float &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:96`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, stand_height = obj.get_stand_height()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.g1.LocoClient.get_phase

查询或检查相位。

签名

```
def get_phase(self) -> tuple[int, list[float]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] phase	list[float]	传给该接口的相位参数，Python 类型为 list[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 phase: std::vector<float> &。底层是可变长度 vector；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `GetPhase(std::vector<float> &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:110`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, phase = obj.get_phase()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.g1.LocoClient.set_fsm_id

设置 FSM ID。具体副作用和安全边界见下方状态。

签名

```
def set_fsm_id(self, fsm_id: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
fsm_id	int	必填	传给该接口的 FSM ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 fsm_id: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `SetFsmId(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:124`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_fsm_id(fsm_id=fsm_id)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.set_balance_mode

设置平衡模式。具体副作用和安全边界见下方状态。

签名

```
def set_balance_mode(self, balance_mode: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
balance_mode	int	必填	传给该接口的平衡模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 balance_mode: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::g1::LocoClient
- 签名: SetBalanceMode(int)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/g1/loco/g1_loco_client.hpp:134

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_balance_mode(balance_mode=balance_mode)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.set_swing_height

设置 swing 高度。具体副作用和安全边界见下方状态。

签名

```
def set_swing_height(self, swing_height: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
swing_height	float	必填	传给该接口的 swing 高度参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 swing_height: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::g1::LocoClient
- 签名: SetSwingHeight(float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/g1/loco/g1_loco_client.hpp:144

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_swing_height(swing_height=swing_height)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.set_stand_height

设置 stand 高度。具体副作用和安全边界见下方状态。

签名

```
def set_stand_height(self, stand_height: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
stand_height	float	必填	传给该接口的 stand 高度参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 stand_height: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `SetStandHeight(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:154`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_stand_height(stand_height=stand_height)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.g1.LocoClient.set_velocity`

设置速度。具体副作用和安全边界见下方状态。

签名

```
def set_velocity(self, vx: float, vy: float, omega: float, duration: float = 1.0) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>vx</code>	<code>float</code>	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vx: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>vy</code>	<code>float</code>	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vy: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>omega</code>	<code>float</code>	必填	角速度参数。旋转轴、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>omega: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>duration</code>	<code>float</code>	<code>1.0</code>	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。对应 C++ 参数 <code>duration: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `SetVelocity(float, float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:164`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_velocity(vx=vx, vy=vy, omega=omega, duration=duration)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.set_task_id

设置任务 ID。具体副作用和安全边界见下方状态。

签名

```
def set_task_id(self, task_id: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>task_id</code>	<code>int</code>	必填	传给该接口的任务 ID 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>task_id: int</code> 。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `SetTaskId(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:176`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_task_id(task_id=task_id)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.switch_to_user_ctrl

对应 C++ SDK 操作 `SwitchToUserCtrl()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_to_user_ctrl(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `SwitchToUserCtrl()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:186`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_to_user_ctrl()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.switch_to_internal_ctrl

对应 C++ SDK 操作 `SwitchToInternalCtrl(unitree::robot::g1::InternalFsmMode)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_to_internal_ctrl(self, mode: InternalFsmMode) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
mode	InternalFsmMode	必填	传给该接口的模式参数，Python 类型为 InternalFsmMode。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 mode: unitree::robot::g1::InternalFsmMode。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::g1::LocoClient
- 签名: SwitchToInternalCtrl(unitree::robot::g1::InternalFsmMode)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/g1/loco/g1_loco_client.hpp:194

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.switch_to_internal_ctrl(mode=mode)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.damp

对应 C++ SDK 操作 Damp()。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def damp(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::g1::LocoClient
- 签名: Damp()
- 绑定策略: DIRECT

- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:214`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.damp()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.start

启动对应 SDK 操作。具体副作用和安全边界见下方状态。

签名

```
def start(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `Start()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:216`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.start()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.squat

对应 C++ SDK 操作 `Squat()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def squat(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `Squat()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:218`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.squat()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.g1.LocoClient.sit`

对应 C++ SDK 操作 `sit()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def sit(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `Sit()`
- 绑定策略: `DIRECT`

- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:220`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.sit()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.stand_up

对应 C++ SDK 操作 `StandUp()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def stand_up(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `StandUp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:222`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_up()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.zero_torque

对应 C++ SDK 操作 `ZeroTorque()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def zero_torque(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `ZeroTorque()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:224`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.zero_torque()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.g1.LocoClient.stop_move`

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_move(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `StopMove()`
- 绑定策略: `DIRECT`

- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:226`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stop_move()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.high_stand

对应 C++ SDK 操作 `HighStand()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def high_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `HighStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:228`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.high_stand()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.low_stand

对应 C++ SDK 操作 `LowStand()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def low_stand(self) -> int
```


可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `LowStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:230`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.low_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.move (重载 1/2)

对应 C++ SDK 操作 `Move(float, float, float, bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def move(self, vx: float, vy: float, vyaw: float, continous_move: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>vx</code>	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vx: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>vy</code>	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vy: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>vyaw</code>	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vyaw: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

名称	Python 类型	默认值	含义
continous_move	bool	必填	传给该接口的 continous move 参数，Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 continous_move: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `Move(float, float, float, bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:232`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw, continous_move=continous_move)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.move (重载 2/2)

对应 C++ SDK 操作 `Move(float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def move(self, vx: float, vy: float, vyaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vx: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vy: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vyaw	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vyaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `Move(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:236`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.g1.LocoClient.balance_stand`

对应 C++ SDK 操作 `BalanceStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def balance_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `BalanceStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:238`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.balance_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.continuous_gait

对应 C++ SDK 操作 ContinuousGait(bool)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def continuous_gait(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::g1::LocoClient
- 签名: ContinuousGait(bool)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/g1/loco/g1_loco_client.hpp:240

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.continuous_gait(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.switch_move_mode

对应 C++ SDK 操作 SwitchMoveMode(bool)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_move_mode(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `SwitchMoveMode(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:242`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_move_mode(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.wave_hand

对应 C++ SDK 操作 `WaveHand(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def wave_hand(self, turn_flag: bool = False) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
turn_flag	bool	False	传给该接口的 turn flag 参数，Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 turn_flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `WaveHand(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:247`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.wave_hand(turn_flag=turn_flag)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.shake_hand

对应 C++ SDK 操作 `ShakeHand(int)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def shake_hand(self, stage: int = -1) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
stage	int	-1	传给该接口的 stage 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>stage: int</code> 。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `ShakeHand(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:249`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.shake_hand(stage=stage)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.set_speed_mode

设置速度模式。具体副作用和安全边界见下方状态。

签名

```
def set_speed_mode(self, speed_mode: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
speed_mode	int	必填	传给该接口的速度模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 speed_mode: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::g1::LocoClient
- 签名: SetSpeedMode(int)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/g1/loco/g1_loco_client.hpp:265

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_speed_mode(speed_mode=speed_mode)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.LocoClient.get_mimic_motion

查询或检查 mimic motion。

签名

```
def get_mimic_motion(self) -> tuple[int, str]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] data	str	传给该接口的数据参数，Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>data: std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `GetMimicMotion(std::string &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:298`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, data = obj.get_mimic_motion()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.g1.LocoClient._fsm_api

对应 C++ SDK 操作 `_fsm_api(std::string, std::string &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def _fsm_api(self, parameter: str) -> tuple[int, str]
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
parameter	str	必填	传给该接口的 <code>parameter</code> 参数，Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>parameter: std::string</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] data	str	传给该接口的数据参数，Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>data: std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

对应 C++

- 类: `unitree::robot::g1::LocoClient`
- 签名: `_fsm_api(std::string, std::string &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/g1/loco/g1_loco_client.hpp:303`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
status, data = obj._fsm_api(parameter=parameter)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.g1.MoveParameter

SDK 请求参数值类型; 当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.g1 import MoveParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>vx</code>	<code>float</code>	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vx / obj.vx = value</code>
<code>vy</code>	<code>float</code>	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vy / obj.vy = value</code>
<code>vyaw</code>	<code>float</code>	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vyaw / obj.vyaw = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.g1.MoveParameter.__init__

初始化 `MoveParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::MoveParameter`
- 签名: `MoveParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/agv/g1_agv_api.hpp:28`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = MoveParameter()
```

`unitree_sdk2_cpp.robot.g1.MoveParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: unitree::common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::MoveParameter`
- 签名: `fromJson(unitree::common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/g1/agv/g1_agv_api.hpp:36`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.g1.MoveParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1::MoveParameter`
- 签名: `toJson(unitree::common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/g1/agv/g1_agv_api.hpp:43`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.g1.PlayStopParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.g1 import PlayStopParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>app_name</code>	<code>str</code>	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。	<code>value = obj.app_name / obj.app_name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.g1.PlayStopParameter.__init__

初始化 PlayStopParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::PlayStopParameter`
- 签名: `PlayStopParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:61`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PlayStopParameter()
```

unitree_sdk2_cpp.robot.g1.PlayStopParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::PlayStopParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:64`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.fromJson(value=value)
```

`unitree_sdk2_cpp.robot.g1.PlayStopParameter.toJson`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def toJson(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1::PlayStopParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:66`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

unitree_sdk2_cpp.robot.g1.PlayStreamParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.g1 import PlayStreamParameter
```

基类：object

公开属性

名称	Python 类型	含义	用法
app_name	str	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。	value = obj.app_name / obj.app_name = value
stream_id	str	音频流标识符，用于区分同一应用下的播放流。	value = obj.stream_id / obj.stream_id = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.g1.PlayStreamParameter.__init__

初始化 PlayStreamParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类：unitree::robot::g1::PlayStreamParameter
- 签名：PlayStreamParameter()
- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/g1/audio/g1_audio_api.hpp:45

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PlayStreamParameter()
```

`unitree_sdk2_cpp.robot.g1.PlayStreamParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::PlayStreamParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:48`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.g1.PlayStreamParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1::PlayStreamParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:50`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.g1.TtsMakerParameter`

SDK 请求参数值类型; 当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.g1 import TtsMakerParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>index</code>	<code>int</code>	传给该接口的 <code>index</code> 参数, <code>value = obj.index / obj.index = value</code> Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	
<code>speaker_id</code>	<code>int</code>	语音合成说话人 ID。有效编号由机器人音频服务和固件决定。	<code>value = obj.speaker_id / obj.speaker_id = value</code>
<code>text</code>	<code>str</code>	要处理的文本内容; 编码、长度和语言支持由目标服务决定。	<code>value = obj.text / obj.text = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.g1.TtsMakerParameter.__init__`

初始化 `TtsMakerParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::g1::TtsMakerParameter`
- 签名: `TtsMakerParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = TtsMakerParameter()
```

`unitree_sdk2_cpp.robot.g1.TtsMakerParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::g1::TtsMakerParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`

- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:30`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.g1.TtsMakerParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::g1::TtsMakerParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/g1/audio/g1_audio_api.hpp:32`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

Go2 Robot API

模块: `unitree_sdk2_cpp.robot.go2`

类索引

类	公开函数签名	属性
<code>ConfigClient</code>	8	0
<code>ConfigDelParameter</code>	3	0
<code>ConfigGetData</code>	3	0
<code>ConfigGetParameter</code>	3	0
<code>ConfigMeta</code>	1	0
<code>ConfigMetaData</code>	3	0
<code>ConfigMetaParameter</code>	3	0

类	公开函数签名	属性
ConfigSetParameter	3	0
JsonizeCommObjInt	3	0
JsonizeConfigMeta	3	0
JsonizeDataBool	3	0
JsonizeDataDouble	3	0
JsonizeDataFloat	3	0
JsonizeDataInt	3	0
JsonizeDataString	3	0
JsonizeFlagBool	3	0
JsonizePathPoint	3	0
JsonizeQuat	3	0
JsonizeVec3	3	0
ObstaclesAvoid	8	0
Client		
ObstaclesAvoidMove	3	0
Parameter		
ObstaclesAvoid	3	0
RemoteCommandSource		
ObstaclesAvoid	3	0
SwitchGetData		
ObstaclesAvoid	3	0
SwitchSetParameter		
RobotStateClient	5	0
ServiceState	1	0
ServiceStateData	3	0
ServiceSwitchData	3	0
ServiceSwitch	3	0
Parameter		
SetReportFreq	3	0
Parameter		
SportClient	41	0
UtrackClient	5	0
UtrackSwitchGetData	3	0
UtrackSwitchSet	3	0
Parameter		
VideoClient	3	0
VuiClient	8	0
stPathPoint	1	0

unitree_sdk2_cpp.robot.go2.ConfigClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.go2 import ConfigClient
```

基类: Client

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>set</code>	<code>def set(self, name: str, content: str) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>get</code>	<code>def get(self, name: str) -> tuple[int, str]</code>	AVAILABLE	READ_ONLY

方法	Python 签名	状态	安全分类
<code>del_</code>	<code>def del_(self, name: str) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>meta_config_meta</code>	<code>def meta_config_meta(self, name: str) -> tuple[int, ConfigMeta]</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>meta_string</code>	<code>def meta_string(self, name: str) -> tuple[int, str]</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>subscribe_change_status</code>	<code>def subscribe_change_status(self, name: str, callback: Callable[[str], None]) -> None</code>	AVAILABLE	HARDWARE_SIDE_EFFECT

`unitree_sdk2_cpp.robot.go2.ConfigClient.__init__`

初始化 `ConfigClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ConfigClient`
- 签名: `ConfigClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/config/config_client.hpp:41`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigClient()
```

`unitree_sdk2_cpp.robot.go2.ConfigClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ConfigClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/config/config_client.hpp:44`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.go2.ConfigClient.set`

对应 C++ SDK 操作 `Set(const std::string &, const std::string &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def set(self, name: str, content: str) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>name</code>	<code>str</code>	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。
<code>content</code>	<code>str</code>	必填	配置、请求或序列化内容。具体格式由对应服务协议定义。对应 C++ 参数 <code>content: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::ConfigClient`
- 签名: `Set(const std::string &, const std::string &)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/config/config_client.hpp:46`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set(name=name, content=content)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.ConfigClient.get

对应 C++ SDK 操作 `Get(const std::string &, std::string &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def get(self, name: str) -> tuple[int, str]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] content	str	配置、请求或序列化内容。具体格式由对应服务协议定义。对应 C++ 参数 <code>content: std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

对应 C++

- 类: `unitree::robot::go2::ConfigClient`
- 签名: `Get(const std::string &, std::string &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/config/config_client.hpp:47`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, content = obj.get(name=name)
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.go2.ConfigClient.del_

对应 C++ SDK 操作 `Del(const std::string &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def del_(self, name: str) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 <code>name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::ConfigClient`
- 签名: `Del(const std::string &)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/config/config_client.hpp:48`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.del_(name=name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.ConfigClient.meta_config_meta

对应 C++ SDK 操作 `Meta(const std::string &, unitree::robot::go2::ConfigMeta &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def meta_config_meta(self, name: str) -> tuple[int, ConfigMeta]
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] meta	ConfigMeta	传给该接口的 meta 参数，Python 类型为 ConfigMeta。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 meta: unitree::robot::go2::ConfigMeta &。

对应 C++

- 类: unitree::robot::go2::ConfigClient
- 签名: Meta(const std::string &, unitree::robot::go2::ConfigMeta &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/go2/config/config_client.hpp:49

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, meta = obj.meta_config_meta(name=name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.ConfigClient.meta_string

对应 C++ SDK 操作 Meta(const std::string &, std::string &)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def meta_string(self, name: str) -> tuple[int, str]
```

可用性与安全性

AVAILABLE / **HARDWARE_SIDE_EFFECT**: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] meta	str	传给该接口的 meta 参数，Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 meta: std::string &。底层为字符串；长度、编码和允许值由具体协议决定。

对应 C++

- 类: `unitree::robot::go2::ConfigClient`
- 签名: `Meta(const std::string &, std::string &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/config/config_client.hpp:50`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, meta = obj.meta_string(name=name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.ConfigClient.subscribe_change_status

对应 C++ SDK 操作 `SubscribeChangeStatus(const std::string &, const unitree::robot::go2::ConfigChangeStatusCallback &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def subscribe_change_status(self, name: str, callback: Callable[[str, str], None]) -> None
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。
callback	Callable[[str, str], None]	必填	收到数据或状态变化时调用的 Python 回调。回调签名必须与类型提示一致，并应快速返回。对应 C++ 参数 callback: const unitree::robot::go2::ConfigChangeStatusCallback &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ConfigClient`
- 签名: `SubscribeChangeStatus(const std::string &, const unitree::robot::go2::ConfigChangeStatusCallback &)`
- 绑定策略: `CALLBACK_MANUAL`
- 声明位置: `include/unitree/robot/go2/config/config_client.hpp:52`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 回调可能由 SDK 工作线程触发; 回调应快速返回、捕获异常, 并通过线程安全队列移交耗时工作。

用法

```
obj.subscribe_change_status(name=name, callback=callback)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.ConfigDelParameter

SDK 请求参数值类型; 当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.go2 import ConfigDelParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.ConfigDelParameter.__init__

初始化 `ConfigDelParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ConfigDelParameter`
- 签名: `ConfigDelParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:140`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigDelParameter()
```

`unitree_sdk2_cpp.robot.go2.ConfigDelParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ConfigDelParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:146`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.ConfigDelParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ConfigDelParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:151`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.ConfigGetData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import ConfigGetData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>content</code>	<code>str</code>	配置、请求或序列化内容。具体格式由对应服务协议定义。	<code>value = obj.content / obj.content = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED

方法	Python 签名	状态	安全分类
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.ConfigGetData.__init__`

初始化 `ConfigGetData` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ConfigGetData`
- 签名: `ConfigGetData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:117`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigGetData()
```

`unitree_sdk2_cpp.robot.go2.ConfigGetData.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common: :JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ConfigGetData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:123`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.go2.ConfigGetData.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ConfigGetData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:128`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.go2.ConfigGetParameter`

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.go2 import ConfigGetParameter
```

基类: object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.ConfigGetParameter.__init__`

初始化 `ConfigGetParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ConfigGetParameter`
- 签名: `ConfigGetParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:94`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigGetParameter()
```

`unitree_sdk2_cpp.robot.go2.ConfigGetParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::go2::ConfigGetParameter
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/go2/config/config_api.hpp:100

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.ConfigGetParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::go2::ConfigGetParameter
- 签名: toJson(common::JsonMap &) const
- 绑定策略: JSON_DICT_OUTPUT

· 声明位置: `include/unitree/robot/go2/config/config_api.hpp:105`

说明

· 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.go2.ConfigMeta`

SDK 数据值类型; 公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import ConfigMeta
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>
<code>last_modified</code>	<code>str</code>	传给该接口的 <code>last modified</code> 参数, Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.last_modified / obj.last_modified = value</code>
<code>size</code>	<code>int</code>	传给该接口的 <code>size</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.size / obj.size = value</code>
<code>epoch</code>	<code>int</code>	传给该接口的 <code>epoch</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.epoch / obj.epoch = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

`unitree_sdk2_cpp.robot.go2.ConfigMeta.__init__`

初始化 `ConfigMeta` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ConfigMeta`
- 签名: `ConfigMeta()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/config/config_client.hpp:20`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigMeta()
```

`unitree_sdk2_cpp.robot.go2.ConfigMetaData`

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import ConfigMetaData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>meta</code>	<code>JsonizeConfigMeta</code>	传给该接口的 <code>meta</code> 参数， Python 类型为 <code>JsonizeConfigMeta</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.meta / obj.meta = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.ConfigMetaData.__init__`

初始化 `ConfigMetaData` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ConfigMetaData`
- 签名: `ConfigMetaData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:186`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigMetaData()
```

`unitree_sdk2_cpp.robot.go2.ConfigMetaData.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ConfigMetaData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:192`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.ConfigMetaData.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ConfigMetaData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:197`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.ConfigMetaParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.go2 import ConfigMetaParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.ConfigMetaParameter.__init__`

初始化 `ConfigMetaParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ConfigMetaParameter`
- 签名: `ConfigMetaParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:163`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigMetaParameter()
```

`unitree_sdk2_cpp.robot.go2.ConfigMetaParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ConfigMetaParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:169`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.go2.ConfigMetaParameter.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ConfigMetaParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:174`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.ConfigSetParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.go2 import ConfigSetParameter
```

基类：object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	value = obj.name / obj.name = value
content	str	配置、请求或序列化内容。具体格式由对应服务协议定义。	value = obj.content / obj.content = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.ConfigSetParameter.__init__

初始化 ConfigSetParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类：unitree::robot::go2::ConfigSetParameter
- 签名：ConfigSetParameter()
- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/go2/config/config_api.hpp:68

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ConfigSetParameter()
```

unitree_sdk2_cpp.robot.go2.ConfigSetParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::go2::ConfigSetParameter
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/go2/config/config_api.hpp:74

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.ConfigSetParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ConfigSetParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:80`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

unitree_sdk2_cpp.robot.go2.JsonizeCommObjInt

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizeCommObjInt
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>value</code>	<code>int</code>	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。	<code>value = obj.value / obj.value = value</code>
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.JsonizeCommObjInt.__init__

初始化 `JsonizeCommObjInt` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizeCommObjInt`
- 签名: `JsonizeCommObjInt()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:287`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeCommObjInt()
```

`unitree_sdk2_cpp.robot.go2.JsonizeCommObjInt.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::JsonizeCommObjInt`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:293`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.JsonizeCommObjInt.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::go2::JsonizeCommObjInt
- 签名: toJson(common::JsonMap &) const
- 绑定策略: JSON_DICT_OUTPUT
- 声明位置: include/unitree/robot/go2/public/jsonize_type.hpp:298

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.JsonizeConfigMeta

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizeConfigMeta
```

基类: object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	value = obj.name / obj.name = value
last_modified	str	传给该接口的 last modified 参数，Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.last_modified / obj.last_modified = value

名称	Python 类型	含义	用法
size	int	传给该接口的 size 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.size / obj.size = value</code>
epoch	int	传给该接口的 epoch 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.epoch / obj.epoch = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.JsonizeConfigMeta.__init__

初始化 JsonizeConfigMeta 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizeConfigMeta`
- 签名: `JsonizeConfigMeta()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:36`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeConfigMeta()
```

unitree_sdk2_cpp.robot.go2.JsonizeConfigMeta.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::go2::JsonizeConfigMeta
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/go2/config/config_api.hpp:42

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.JsonizeConfigMeta.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::JsonizeConfigMeta`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/config/config_api.hpp:50`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

unitree_sdk2_cpp.robot.go2.JsonizeDataBool

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizeDataBool
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>data</code>	<code>bool</code>	传给该接口的数据参数, Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.data / obj.data = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.JsonizeDataBool.__init__

初始化 `JsonizeDataBool` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataBool`
- 签名: `JsonizeDataBool()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:44`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeDataBool()
```

`unitree_sdk2_cpp.robot.go2.JsonizeDataBool.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataBool`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:50`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.JsonizeDataBool.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataBool`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:55`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.JsonizeDataDouble

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizeDataDouble
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>data</code>	<code>float</code>	传给该接口的数据参数，Python 类型为 <code>float</code> 。 精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.data / obj.data = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

方法	Python 签名	状态	安全分类
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.JsonizeDataDouble.__init__`

初始化 `JsonizeDataDouble` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataDouble`
- 签名: `JsonizeDataDouble()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:122`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeDataDouble()
```

`unitree_sdk2_cpp.robot.go2.JsonizeDataDouble.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataDouble`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:128`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.go2.JsonizeDataDouble.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataDouble`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:133`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.go2.JsonizeDataFloat`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizeDataFloat
```

基类: object

公开属性

名称	Python 类型	含义	用法
data	float	传给该接口的数据参数，Python 类型为 float。 精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.data / obj.data = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.JsonizeDataFloat.__init__

初始化 JsonizeDataFloat 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataFloat`
- 签名: `JsonizeDataFloat()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:96`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeDataFloat()
```

unitree_sdk2_cpp.robot.go2.JsonizeDataFloat.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::go2::JsonizeDataFloat
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/go2/public/jsonize_type.hpp:102

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.JsonizeDataFloat.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataFloat`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:107`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

unitree_sdk2_cpp.robot.go2.JsonizeDataInt

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizeDataInt
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>data</code>	<code>int</code>	传给该接口的数据参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.data / obj.data = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.JsonizeDataInt.__init__

初始化 `JsonizeDataInt` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataInt`
- 签名: `JsonizeDataInt()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:70`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeDataInt()
```

`unitree_sdk2_cpp.robot.go2.JsonizeDataInt.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataInt`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:76`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.JsonizeDataInt.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataInt`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:81`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.JsonizeDataString

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizeDataString
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>data</code>	<code>str</code>	传给该接口的数据参数，Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.data / obj.data = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

方法	Python 签名	状态	安全分类
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.JsonizeDataString.__init__

初始化 `JsonizeDataString` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataString`
- 签名: `JsonizeDataString()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:148`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeDataString()
```

unitree_sdk2_cpp.robot.go2.JsonizeDataString.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common: JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataString`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:154`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.go2.JsonizeDataString.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::JsonizeDataString`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:159`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.go2.JsonizeFlagBool`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入


```
from unitree_sdk2_cpp.robot.go2 import JsonizeFlagBool
```

基类： object

公开属性

名称	Python 类型	含义	用法
flag	bool	布尔开关。True 和 False 的具体业务效果由当前方法定义。	value = obj.flag / obj.flag = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.JsonizeFlagBool.__init__

初始化 JsonizeFlagBool 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizeFlagBool`
- 签名: `JsonizeFlagBool()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeFlagBool()
```

unitree_sdk2_cpp.robot.go2.JsonizeFlagBool.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::go2::JsonizeFlagBool
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/go2/public/jsonize_type.hpp:24

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.JsonizeFlagBool.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::JsonizeFlagBool`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:29`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

unitree_sdk2_cpp.robot.go2.JsonizePathPoint

SDK 数据值类型; 公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizePathPoint
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>time_from_start</code>	<code>float</code>	传给该接口的 <code>time from start</code> 参数, Python 类型为 <code>float</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.time_from_start / obj.time_from_start = value</code>
<code>x</code>	<code>float</code>	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.x / obj.x = value</code>
<code>y</code>	<code>float</code>	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.y / obj.y = value</code>
<code>yaw</code>	<code>float</code>	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。	<code>value = obj.yaw / obj.yaw = value</code>
<code>vx</code>	<code>float</code>	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vx / obj.vx = value</code>
<code>vy</code>	<code>float</code>	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vy / obj.vy = value</code>
<code>vyaw</code>	<code>float</code>	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vyaw / obj.vyaw = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED

方法	Python 签名	状态	安全分类
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.JsonizePathPoint.__init__`

初始化 `JsonizePathPoint` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizePathPoint`
- 签名: `JsonizePathPoint()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:241`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizePathPoint()
```

`unitree_sdk2_cpp.robot.go2.JsonizePathPoint.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common: :JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::JsonizePathPoint`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:249`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.go2.JsonizePathPoint.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::JsonizePathPoint`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:260`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.go2.JsonizeQuat`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizeQuat
```

基类：object

公开属性

名称	Python 类型	含义	用法
x	float	X 方向位置或分量参数。 参考系、单位和范围必须 查当前方法及目标型号协 议。	<code>value = obj.x / obj.x = value</code>
y	float	Y 方向位置或分量参数。 参考系、单位和范围必须 查当前方法及目标型号协 议。	<code>value = obj.y / obj.y = value</code>
z	float	Z 方向位置或分量参数。 参考系、单位和范围必须 查当前方法及目标型号协 议。	<code>value = obj.z / obj.z = value</code>
w	float	传给该接口的 w 参数， Python 类型为 float。 精确含义、单位、范围和 枚举值需查目标型号对应 头文件与协议。	<code>value = obj.w / obj.w = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.JsonizeQuat.__init__

初始化 JsonizeQuat 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类：unitree::robot::go2::JsonizeQuat
- 签名：JsonizeQuat()
- 绑定策略：未单独标注

- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:206`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeQuat()
```

`unitree_sdk2_cpp.robot.go2.JsonizeQuat.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::JsonizeQuat`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:212`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.go2.JsonizeQuat.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::JsonizeQuat`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:220`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.JsonizeVec3

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.go2 import JsonizeVec3
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>x</code>	<code>float</code>	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.x / obj.x = value</code>
<code>y</code>	<code>float</code>	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.y / obj.y = value</code>
<code>z</code>	<code>float</code>	Z 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.z / obj.z = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.JsonizeVec3.__init__

初始化 `JsonizeVec3` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::JsonizeVec3`
- 签名: `JsonizeVec3()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/public/jsonize_type.hpp:174`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeVec3()
```

`unitree_sdk2_cpp.robot.go2.JsonizeVec3.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::JsonizeVec3`
- 签名: `fromJson(common::JsonMap &)`

- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/go2/public/jsonize_type.hpp:180

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.JsonizeVec3.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::go2::JsonizeVec3
- 签名: toJson(common::JsonMap &) const
- 绑定策略: JSON_DICT_OUTPUT
- 声明位置: include/unitree/robot/go2/public/jsonize_type.hpp:187

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.go2 import ObstaclesAvoidClient
```

基类: Client

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>init</code>	def init(self) -> None	AVAILABLE	INITIALIZATION

方法	Python 签名	状态	安全分类
<code>switch_set</code>	<code>def switch_set(self, enable: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_get</code>	<code>def switch_get(self) -> tuple[int, bool]</code>	AVAILABLE	MOTION_COMMAND
<code>move</code>	<code>def move(self, x: float, y: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>use_remote_command_from_api</code>	<code>def use_remote_command_from_api(self, is_remote_commands_from_api: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move_to_absolute_position</code>	<code>def move_to_absolute_position(self, x: float, y: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move_to_increment_position</code>	<code>def move_to_increment_position(self, x: float, y: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient.__init__

初始化 ObstaclesAvoidClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidClient`
- 签名: `ObstaclesAvoidClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_client.hpp:15`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ObstaclesAvoidClient()
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient.switch_set`

对应 C++ SDK 操作 `SwitchSet(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_set(self, enable: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>enable</code>	<code>bool</code>	必填	传给该接口的 <code>enable</code> 参数，Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>enable: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidClient`
- 签名: `SwitchSet(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_client.hpp:20`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_set(enable=enable)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient.switch_get`

对应 C++ SDK 操作 `SwitchGet(bool &)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def switch_get(self) -> tuple[int, bool]
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务错误码解释。
[1] enable	bool	传给该接口的 <code>enable</code> 参数, Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>enable: bool &</code> 。只接受布尔语义。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidClient`
- 签名: `SwitchGet(bool &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_client.hpp:21`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
status, enable = obj.switch_get()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient.move

对应 C++ SDK 操作 `Move(float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def move(self, x: float, y: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
x	float	必填	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 <code>x: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
y	float	必填	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 <code>y: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 <code>yaw: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidClient`
- 签名: `Move(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_client.hpp:23`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(x=x, y=y, yaw=yaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient.use_remote_command_from_api

对应 C++ SDK 操作 `UseRemoteCommandFromApi(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def use_remote_command_from_api(self, is_remote_commands_from_api: bool) ->
    int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
is_remote_commands_from_api	bool	必填	传给该接口的 is remote commands from API 参数，Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 is RemoteCommandsFromApi: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidClient`
- 签名: `UseRemoteCommandFromApi(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_client.hpp:24`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.use_remote_command_from_api(is_remote_commands_from_api=is_
    remote_commands_from_api)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient.move_to_absolute_position`

对应 C++ SDK 操作 `MoveToAbsolutePosition(float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def move_to_absolute_position(self, x: float, y: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
x	float	必填	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 x: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
y	float	必填	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 y: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 yaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidClient`
- 签名: `MoveToAbsolutePosition(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_client.hpp:26`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move_to_absolute_position(x=x, y=y, yaw=yaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.ObstaclesAvoidClient.move_to_increment_position`

对应 C++ SDK 操作 `MoveToIncrementPosition(float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def move_to_increment_position(self, x: float, y: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
x	float	必填	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 x: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

名称	Python 类型	默认值	含义
y	float	必填	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 y: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 yaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidClient`
- 签名: `MoveToIncrementPosition(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_client.hpp:27`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move_to_increment_position(x=x, y=y, yaw=yaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidMoveParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.go2 import ObstaclesAvoidMoveParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
m_x	float	传给该接口的 m x 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.m_x / obj.m_x = value</code>
m_y	float	传给该接口的 m y 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.m_y / obj.m_y = value</code>

名称	Python 类型	含义	用法
m_yaw	float	传给该接口的 m_yaw 参数, value = obj.m_yaw / obj.m_yaw = value Python 类型为 float。 精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	
m_mode	int	传给该接口的 m 模式参数, value = obj.m_mode / obj.m_mode = value Python 类型为 int。精 确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.ObstaclesAvoidMoveParameter.__init__`

初始化 `ObstaclesAvoidMoveParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidMoveParameter`
- 签名: `ObstaclesAvoidMoveParameter()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ObstaclesAvoidMoveParameter()
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidMoveParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::go2::ObstaclesAvoidMoveParameter
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_api.hpp:55

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidMoveParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidMoveParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_api.hpp:63`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidRemoteCommandSource

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.go2 import ObstaclesAvoidRemoteCommandSource
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>m_is_remote_commands_from_api</code>	<code>bool</code>	传给该接口的 <code>m_is_remote_commands_from</code> API 参数, Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.m_is_remote_commands_from_api / obj.m_is_remote_commands_from_api = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidRemoteCommandSource.__init__

初始化 `ObstaclesAvoidRemoteCommandSource` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidRemoteCommandSource`
- 签名: `ObstaclesAvoidRemoteCommandSource()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ObstaclesAvoidRemoteCommandSource()
```

`unitree_sdk2_cpp.robot.go2.ObstaclesAvoidRemoteCommandSource.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidRemoteCommandSource`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_api.hpp:87`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidRemoteCommandSource.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidRemoteCommandSource`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_api.hpp:92`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchGetData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import ObstaclesAvoidSwitchGetData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>m_enable</code>	<code>bool</code>	传给该接口的 <code>m_enable</code> 参数, Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.m_enable / obj.m_enable = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

方法	Python 签名	状态	安全分类
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchGetData.__init__

初始化 ObstaclesAvoidSwitchGetData 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidSwitchGetData`
- 签名: `ObstaclesAvoidSwitchGetData()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ObstaclesAvoidSwitchGetData()
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchGetData.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common: JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidSwitchGetData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_api.hpp:39`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchGetData.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidSwitchGetData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_api.hpp:44`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchSetParameter`

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入


```
from unitree_sdk2_cpp.robot.go2 import ObstaclesAvoidSwitchSetParameter
```

基类: object

公开属性

名称	Python 类型	含义	用法
m_enable	bool	传给该接口的 m enable 参数, Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.m_enable / obj.m_enable = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchSetParameter.__init__

初始化 ObstaclesAvoidSwitchSetParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidSwitchSetParameter`
- 签名: `ObstaclesAvoidSwitchSetParameter()`
- 绑定策略: 未单独标注
- 声明位置: 由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ObstaclesAvoidSwitchSetParameter()
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchSetParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidSwitchSetParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_api.hpp:23`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.ObstaclesAvoidSwitchSetParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ObstaclesAvoidSwitchSetParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/obstacles_avoid/obstacles_avoid_api.hpp:28`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

unitree_sdk2_cpp.robot.go2.RobotStateClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.go2 import RobotStateClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>service_list</code>	<code>def service_list(self) -> tuple[int, list[Service State]]</code>	AVAILABLE	READ_ONLY
<code>service_switch</code>	<code>def service_switch(self, name: str, swit: int) -> tuple[int, int]</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>set_report_freq</code>	<code>def set_report_freq(self, interval: int, duration: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT

unitree_sdk2_cpp.robot.go2.RobotStateClient.__init__

初始化 `RobotStateClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::RobotStateClient`

- 签名: `RobotStateClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_client.hpp:32`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = RobotStateClient()
```

unitree_sdk2_cpp.robot.go2.RobotStateClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::RobotStateClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_client.hpp:35`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

unitree_sdk2_cpp.robot.go2.RobotStateClient.service_list

对应 C++ SDK 操作 `ServiceList(std::vector<ServiceState> &)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def service_list(self) -> tuple[int, list[ServiceState]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务; 纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] serviceStateList	list[ServiceState]	传给该接口的 serviceStateList 参数，Python 类型为 list[ServiceState]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 serviceStateList: std::vector<ServiceState> &。底层是可变长度 vector

对应 C++

- 类: unitree::robot::go2::RobotStateClient
- 签名: ServiceList(std::vector<ServiceState> &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/go2/robot_state/robot_state_client.hpp:37

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, serviceStateList = obj.service_list()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.go2.RobotStateClient.service_switch

对应 C++ SDK 操作 ServiceSwitch(const std::string &, int32_t, int32_t &)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def service_switch(self, name: str, swit: int) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
name	str	必填	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。对应 C++ 参数 name: const std::string &。底层为字符串；长度、编码和允许值由具体协议决定。
swit	int	必填	传给该接口的 swit 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 swit: int32_t。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。

对应 C++

- 类: `unitree::robot::go2::RobotStateClient`
- 签名: `ServiceSwitch(const std::string &, int32_t, int32_t &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_client.hpp:38`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, status = obj.service_switch(name=name, swit=swit)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.RobotStateClient.set_report_freq`

设置 `report_freq`。具体副作用和安全边界见下方状态。

签名

```
def set_report_freq(self, interval: int, duration: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现, 调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>interval</code>	<code>int</code>	必填	传给该接口的 <code>interval</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>interval: int32_t</code> 。取值范围为 -2147483648 到 2147483647。
<code>duration</code>	<code>int</code>	必填	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。对应 C++ 参数 <code>duration: int32_t</code> 。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::RobotStateClient`
- 签名: `SetReportFreq(int32_t, int32_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_client.hpp:39`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set_report_freq(interval=interval, duration=duration)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.ServiceState

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import ServiceState
```

基类： object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>
status	int	传给该接口的状态参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.status / obj.status = value</code>
protect	int	传给该接口的 protect 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.protect / obj.protect = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

unitree_sdk2_cpp.robot.go2.ServiceState.__init__

初始化 ServiceState 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION： 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ServiceState`
- 签名: `ServiceState()`

- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/go2/robot_state/robot_state_client.hpp:18

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ServiceState()
```

unitree_sdk2_cpp.robot.go2.ServiceStateData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import ServiceStateData
```

基类：object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	value = obj.name / obj.name = value
status	int	传给该接口的状态参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.status / obj.status = value
protect	int	传给该接口的 protect 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.protect / obj.protect = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.ServiceStateData.__init__

初始化 ServiceStateData 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ServiceStateData`
- 签名: `ServiceStateData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:122`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ServiceStateData()
```

`unitree_sdk2_cpp.robot.go2.ServiceStateData.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ServiceStateData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:128`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.ServiceStateData.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ServiceStateData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:135`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.ServiceSwitchData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import ServiceSwitchData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>name</code>	<code>str</code>	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>
<code>status</code>	<code>int</code>	传给该接口的状态参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.status / obj.status = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.ServiceSwitchData.__init__`

初始化 `ServiceSwitchData` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::ServiceSwitchData`
- 签名: `ServiceSwitchData()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:64`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ServiceSwitchData()
```

`unitree_sdk2_cpp.robot.go2.ServiceSwitchData.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::ServiceSwitchData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:70`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.go2.ServiceSwitchData.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ServiceSwitchData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:76`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.ServiceSwitchParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.go2 import ServiceSwitchParameter
```

基类：object

公开属性

名称	Python 类型	含义	用法
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	value = obj.name / obj.name = value
swit	int	传给该接口的 swit 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.swit / obj.swit = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.ServiceSwitchParameter.__init__

初始化 ServiceSwitchParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类：unitree::robot::go2::ServiceSwitchParameter
- 签名：ServiceSwitchParameter()
- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/go2/robot_state/robot_state_api.hpp:35

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = ServiceSwitchParameter()
```

unitree_sdk2_cpp.robot.go2.ServiceSwitchParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common: :JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::go2::ServiceSwitchParameter
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/go2/robot_state/robot_state_api.hpp:41

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.ServiceSwitchParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::ServiceSwitchParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:47`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.go2.SetReportFreqParameter`

SDK 请求参数值类型; 当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.go2 import SetReportFreqParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>interval</code>	<code>int</code>	传给该接口的 <code>interval</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.interval / obj.interval = value</code>
<code>duration</code>	<code>int</code>	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。	<code>value = obj.duration / obj.duration = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.SetReportFreqParameter.__init__`

初始化 `SetReportFreqParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::SetReportFreqParameter`
- 签名: `SetReportFreqParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:93`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = SetReportFreqParameter()
```

`unitree_sdk2_cpp.robot.go2.SetReportFreqParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::SetReportFreqParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:99`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.SetReportFreqParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::SetReportFreqParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/robot_state/robot_state_api.hpp:105`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.SportClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.go2 import SportClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self, enable_lease: bool = False) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>damp</code>	<code>def damp(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>balance_stand</code>	<code>def balance_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stop_move</code>	<code>def stop_move(self) -> int</code>	AVAILABLE	MOTION_COMMAND

方法	Python 签名	状态	安全分类
<code>stand_up</code>	<code>def stand_up(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stand_down</code>	<code>def stand_down(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>recovery_stand</code>	<code>def recovery_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>euler</code>	<code>def euler(self, roll: float, pitch: float, yaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move</code>	<code>def move(self, vx: float, vy: float, vyaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>sit</code>	<code>def sit(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>rise_sit</code>	<code>def rise_sit(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>speed_level</code>	<code>def speed_level(self, level: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>hello</code>	<code>def hello(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stretch</code>	<code>def stretch(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_joyystick</code>	<code>def switch_joyystick(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>content</code>	<code>def content(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>heart</code>	<code>def heart(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>pose</code>	<code>def pose(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>scrape</code>	<code>def scrape(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>front_flip</code>	<code>def front_flip(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>front_jump</code>	<code>def front_jump(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>front_pounce</code>	<code>def front_pounce(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>dance1</code>	<code>def dance1(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>dance2</code>	<code>def dance2(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>left_flip</code>	<code>def left_flip(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>back_flip</code>	<code>def back_flip(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>hand_stand</code>	<code>def hand_stand(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>free_walk</code>	<code>def free_walk(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>free_bound</code>	<code>def free_bound(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>free_jump</code>	<code>def free_jump(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>free_avoid</code>	<code>def free_avoid(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>classic_walk</code>	<code>def classic_walk(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>walk_upright</code>	<code>def walk_upright(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>cross_step</code>	<code>def cross_step(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>auto_recover_set</code>	<code>def auto_recover_set(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>auto_recover_get</code>	<code>def auto_recover_get(self) -> tuple[int, bool]</code>	AVAILABLE	MOTION_COMMAND

方法	Python 签名	状态	安全分类
<code>static_walk</code>	<code>def static_walk(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>trot_run</code>	<code>def trot_run(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>economic_gait</code>	<code>def economic_gait(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_avoid_mode</code>	<code>def switch_avoid_mode(self) -> int</code>	AVAILABLE	MOTION_COMMAND

unitree_sdk2_cpp.robot.go2.SportClient.__init__

初始化 SportClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self, enable_lease: bool = False) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>enable_lease</code>	<code>bool</code>	<code>False</code>	是否启用 SDK lease 机制；lease 的获得、续期和释放规则以服务协议为准。对应 C++ 参数 <code>enableLease</code> : <code>bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `SportClient(bool)`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:34`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = SportClient(enable_lease=enable_lease)
```

unitree_sdk2_cpp.robot.go2.SportClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:37`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.go2.SportClient.damp`

对应 C++ SDK 操作 `Damp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def damp(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Damp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:39`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.damp()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.balance_stand

对应 C++ SDK 操作 `BalanceStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def balance_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `BalanceStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:40`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.balance_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.stop_move

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_move(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `StopMove()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:41`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stop_move()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.stand_up`

对应 C++ SDK 操作 `StandUp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_up(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `StandUp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:42`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_up()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.stand_down

对应 C++ SDK 操作 `StandDown()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_down(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `StandDown()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:43`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_down()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.recovery_stand

对应 C++ SDK 操作 `RecoveryStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def recovery_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `RecoveryStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:44`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.recovery_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.euler`

对应 C++ SDK 操作 `Euler(float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def euler(self, roll: float, pitch: float, yaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
roll	float	必填	传给该接口的 roll 参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>roll: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
pitch	float	必填	传给该接口的 pitch 参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>pitch: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 <code>yaw: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Euler(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:45`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.euler(roll=roll, pitch=pitch, yaw=yaw)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.move`

对应 C++ SDK 操作 `Move(float, float, float)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def move(self, vx: float, vy: float, vyaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>vx</code>	<code>float</code>	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vx: float</code> 。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。
<code>vy</code>	<code>float</code>	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vy: float</code> 。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。
<code>vyaw</code>	<code>float</code>	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vyaw: float</code> 。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Move(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:46`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.sit

对应 C++ SDK 操作 `Sit()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def sit(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Sit()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:47`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.sit()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.rise_sit

对应 C++ SDK 操作 `RiseSit()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def rise_sit(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `RiseSit()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:48`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.rise_sit()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.speed_level`

对应 C++ SDK 操作 `SpeedLevel(int)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def speed_level(self, level: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>level</code>	<code>int</code>	必填	传给该接口的 <code>level</code> 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>level: int</code> 。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `SpeedLevel(int)`

- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:49`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.speed_level(level=level)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.hello

对应 C++ SDK 操作 `Hello()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def hello(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 <code>0</code> 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Hello()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:50`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.hello()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.stretch

对应 C++ SDK 操作 `Stretch()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def stretch(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Stretch()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:51`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stretch()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.switch_joystick

对应 C++ SDK 操作 `SwitchJoystick(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_joystick(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `SwitchJoystick(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:52`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_joystick(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.content`

对应 C++ SDK 操作 `Content()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def content(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Content()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:53`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.content()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.heart

对应 C++ SDK 操作 `Heart()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def heart(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Heart()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:54`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.heart()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.pose

对应 C++ SDK 操作 `Pose(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def pose(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Pose(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:55`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.pose(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.scrape`

对应 C++ SDK 操作 `Scrape()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def scrape(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Scrape()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:56`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.scrape()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.front_flip

对应 C++ SDK 操作 FrontFlip()。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def front_flip(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND：当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类：unitree::robot::go2::SportClient
- 签名：FrontFlip()
- 绑定策略：DIRECT
- 声明位置：include/unitree/robot/go2/sport/sport_client.hpp:57

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.front_flip()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.front_jump

对应 C++ SDK 操作 FrontJump()。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def front_jump(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `FrontJump()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:58`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.front_jump()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.front_pounce`

对应 C++ SDK 操作 `FrontPounce()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def front_pounce(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `FrontPounce()`
- 绑定策略: `DIRECT`

- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:59`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.front_pounce()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.dance1

对应 C++ SDK 操作 `Dance1()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def dance1(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Dance1()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:60`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.dance1()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.dance2

对应 C++ SDK 操作 `Dance2()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def dance2(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `Dance2()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:61`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.dance2()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.left_flip

对应 C++ SDK 操作 `LeftFlip()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def left_flip(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `LeftFlip()`

- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:62`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.left_flip()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.back_flip`

对应 C++ SDK 操作 `BackFlip()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def back_flip(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 <code>0</code> 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `BackFlip()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:63`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.back_flip()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.hand_stand`

对应 C++ SDK 操作 `HandStand(bool)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def hand_stand(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `HandStand(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:64`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.hand_stand(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.free_walk`

对应 C++ SDK 操作 `FreeWalk()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def free_walk(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `FreeWalk()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:65`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.free_walk()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.free_bound`

对应 C++ SDK 操作 `FreeBound(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def free_bound(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `FreeBound(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:66`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.free_bound(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.free_jump

对应 C++ SDK 操作 `FreeJump(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def free_jump(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `FreeJump(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:67`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.free_jump(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.free_avoid

对应 C++ SDK 操作 `FreeAvoid(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def free_avoid(self, flag: bool) -> int
```


可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `FreeAvoid(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:68`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.free_avoid(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.classic_walk`

对应 C++ SDK 操作 `ClassicWalk(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def classic_walk(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `ClassicWalk(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:69`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.classic_walk(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.walk_upright`

对应 C++ SDK 操作 `WalkUpright(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def walk_upright(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `WalkUpright(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:70`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.walk_upright(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.cross_step

对应 C++ SDK 操作 `CrossStep(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def cross_step(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND：当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类：unitree::robot::go2::SportClient
- 签名：CrossStep(bool)
- 绑定策略：DIRECT
- 声明位置：include/unitree/robot/go2/sport/sport_client.hpp:71

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.cross_step(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.auto_recover_set

对应 C++ SDK 操作 `AutoRecoverSet(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def auto_recover_set(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `AutoRecoverSet(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:72`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.auto_recover_set(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.auto_recover_get

对应 C++ SDK 操作 `AutoRecoverGet(bool &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def auto_recover_get(self) -> tuple[int, bool]
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] flag	bool	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool &。只接受布尔语义。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `AutoRecoverGet(bool &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:73`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
status, flag = obj.auto_recover_get()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.static_walk

对应 C++ SDK 操作 `StaticWalk()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def static_walk(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `StaticWalk()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:74`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.static_walk()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.trot_run

对应 C++ SDK 操作 `TrotRun()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def trot_run(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `TrotRun()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:75`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.trot_run()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.SportClient.economic_gait

对应 C++ SDK 操作 `EconomicGait()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def economic_gait(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `EconomicGait()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:76`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.economic_gait()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.SportClient.switch_avoid_mode`

对应 C++ SDK 操作 `SwitchAvoidMode()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_avoid_mode(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::SportClient`
- 签名: `SwitchAvoidMode()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/sport/sport_client.hpp:77`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_avoid_mode()
```


[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.UtrackClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.go2 import UtrackClient
```

基类: Client

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>switch_set</code>	<code>def switch_set(self, enable: bool) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>switch_get</code>	<code>def switch_get(self) -> tuple[int, bool]</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>is_tracking</code>	<code>def is_tracking(self) -> tuple[int, bool]</code>	AVAILABLE	READ_ONLY

unitree_sdk2_cpp.robot.go2.UtrackClient.__init__

初始化 UtrackClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::UtrackClient`
- 签名: `UtrackClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/utrack/utrack_client.hpp:15`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = UtrackClient()
```


unitree_sdk2_cpp.robot.go2.UtrackClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::UtrackClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/utrack/utrack_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

unitree_sdk2_cpp.robot.go2.UtrackClient.switch_set

对应 C++ SDK 操作 `SwitchSet(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_set(self, enable: bool) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>enable</code>	<code>bool</code>	必填	传给该接口的 <code>enable</code> 参数，Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>enable: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::UtrackClient`
- 签名: `SwitchSet(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/utrack/utrack_client.hpp:20`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.switch_set(enable=enable)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.UtrackClient.switch_get`

对应 C++ SDK 操作 `SwitchGet(bool &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_get(self) -> tuple[int, bool]
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] enable	bool	传给该接口的 <code>enable</code> 参数，Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>enable: bool &</code> 。只接受布尔语义。

对应 C++

- 类: `unitree::robot::go2::UtrackClient`
- 签名: `SwitchGet(bool &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/utrack/utrack_client.hpp:21`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, enable = obj.switch_get()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.UtrackClient.is_tracking

查询或检查 tracking。

签名

```
def is_tracking(self) -> tuple[int, bool]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] enable	bool	传给该接口的 enable 参数，Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 enable: bool &。只接受布尔语义。

对应 C++

- 类: `unitree::robot::go2::UtrackClient`
- 签名: `IsTracking(bool &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/utrack/utrack_client.hpp:22`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, enable = obj.is_tracking()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.go2.UtrackSwitchGetData

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import UtrackSwitchGetData
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
m_enable	int	传给该接口的 m_enable 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.m_enable / obj.m_enable = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.go2.UtrackSwitchGetData.__init__`

初始化 `UtrackSwitchGetData` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::UtrackSwitchGetData`
- 签名: `UtrackSwitchGetData()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；`manifest` 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = UtrackSwitchGetData()
```

`unitree_sdk2_cpp.robot.go2.UtrackSwitchGetData.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::UtrackSwitchGetData`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/go2/utrack/utrack_api.hpp:38`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.go2.UtrackSwitchGetData.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::UtrackSwitchGetData`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/utrack/utrack_api.hpp:43`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.UtrackSwitchSetParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.go2 import UtrackSwitchSetParameter
```

基类： `object`

公开属性

名称	Python 类型	含义	用法
<code>m_enable</code>	<code>int</code>	传给该接口的 <code>m_enable</code> 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.m_enable / obj.m_enable = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.go2.UtrackSwitchSetParameter.__init__

初始化 `UtrackSwitchSetParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION： 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::UtrackSwitchSetParameter`
- 签名: `UtrackSwitchSetParameter()`

- 绑定策略：未单独标注
- 声明位置：由 pybind11 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = UtrackSwitchSetParameter()
```

unitree_sdk2_cpp.robot.go2.UtrackSwitchSetParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类：unitree::robot::go2::UtrackSwitchSetParameter
- 签名：fromJson(common::JsonMap &)
- 绑定策略：JSON_DICT_INPUT
- 声明位置：include/unitree/robot/go2/utrack/utrack_api.hpp:22

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.go2.UtrackSwitchSetParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::go2::UtrackSwitchSetParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/go2/utrack/utrack_api.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.go2.VideoClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.go2 import VideoClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>get_image_sample</code>	<code>def get_image_sample(self) -> tuple[int, list[int]]</code>	AVAILABLE	READ_ONLY

unitree_sdk2_cpp.robot.go2.VideoClient.__init__

初始化 `VideoClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::VideoClient`
- 签名: `VideoClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/video/video_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = VideoClient()
```

`unitree_sdk2_cpp.robot.go2.VideoClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::VideoClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/video/video_client.hpp:21`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.go2.VideoClient.get_image_sample`

查询或检查图像 `sample`。

签名

```
def get_image_sample(self) -> tuple[int, list[int]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] output_1	list[int]	传给该接口的 output 1 参数，Python 类型为 list[int]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 output_1: std::vector<uint8_t> &。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

对应 C++

- 类: `unitree::robot::go2::VideoClient`
- 签名: `GetImageSample(std::vector<uint8_t> &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/video/video_client.hpp:27`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, output_1 = obj.get_image_sample()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.go2.VuiClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.go2 import VuiClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>set_switch</code>	<code>def set_switch(self, enable: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>get_switch</code>	<code>def get_switch(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>set_volume</code>	<code>def set_volume(self, level: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>get_volume</code>	<code>def get_volume(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>set_brightness</code>	<code>def set_brightness(self, level: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT

方法	Python 签名	状态	安全分类
<code>get_brightness</code>	<code>def get_brightness(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY

`unitree_sdk2_cpp.robot.go2.VuiClient.__init__`

初始化 `VuiClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::VuiClient`
- 签名: `VuiClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/go2/vui/vui_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = VuiClient()
```

`unitree_sdk2_cpp.robot.go2.VuiClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::go2::VuiClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/vui/vui_client.hpp:21`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.go2.VuiClient.set_switch`

设置开关。具体副作用和安全边界见下方状态。

签名

```
def set_switch(self, enable: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现, 调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
<code>enable</code>	<code>int</code>	必填	传给该接口的 <code>enable</code> 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>enable: int</code> 。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::VuiClient`
- 签名: `SetSwitch(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/vui/vui_client.hpp:27`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set_switch(enable=enable)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.VuiClient.get_switch`

查询或检查开关。

签名

```
def get_switch(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] output_1	int	传给该接口的 output 1 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>output_1: int &</code> 。

对应 C++

- 类: `unitree::robot::go2::VuiClient`
- 签名: `GetSwitch(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/vui/vui_client.hpp:33`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, output_1 = obj.get_switch()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.go2.VuiClient.set_volume

设置音量。具体副作用和安全边界见下方状态。

签名

```
def set_volume(self, level: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
level	int	必填	传给该接口的 level 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>level: int</code> 。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::VuiClient`
- 签名: `SetVolume(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/vui/vui_client.hpp:39`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set_volume(level=level)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.go2.VuiClient.get_volume`

查询或检查音量。

签名

```
def get_volume(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] output_1	int	传给该接口的 output 1 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>output_1: int &</code> 。

对应 C++

- 类: `unitree::robot::go2::VuiClient`
- 签名: `GetVolume(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/vui/vui_client.hpp:45`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, output_1 = obj.get_volume()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.go2.VuiClient.set_brightness

设置亮度。具体副作用和安全边界见下方状态。

签名

```
def set_brightness(self, level: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
level	int	必填	传给该接口的 level 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 level: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::go2::VuiClient`
- 签名: `SetBrightness(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/go2/vui/vui_client.hpp:51`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set_brightness(level=level)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.go2.VuiClient.get_brightness

查询或检查亮度。

签名

```
def get_brightness(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] output_1	int	传给该接口的 output 1 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 output_1: int &。

对应 C++

- 类: `unitree::robot::go2::VuiClient`
- 签名: `GetBrightness(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/go2/vui/vui_client.hpp:57`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, output_1 = obj.get_brightness()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.go2.stPathPoint

SDK 数据值类型；公开字段和转换方法见下文。

导入

```
from unitree_sdk2_cpp.robot.go2 import stPathPoint
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
time_from_start	float	传给该接口的 time from start 参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.time_from_start / obj.time_from_start = value</code>
x	float	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.x / obj.x = value</code>
y	float	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.y / obj.y = value</code>
yaw	float	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。	<code>value = obj.yaw / obj.yaw = value</code>
vx	float	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vx / obj.vx = value</code>
vy	float	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vy / obj.vy = value</code>

名称	Python 类型	含义	用法
vyaw	float	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。	<code>value = obj.vyaw / obj.vyaw = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

unitree_sdk2_cpp.robot.go2.stPathPoint.__init__

初始化 `stPathPoint` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::go2::stPathPoint`
- 签名: `stPathPoint()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = stPathPoint()
```

H1 Robot API

模块: `unitree_sdk2_cpp.robot.h1`

类索引

类	公开函数签名	属性
<code>JsonizeDataVecFloat</code>	3	0
<code>JsonizeTargetPos</code>	3	0

类	公开函数签名	属性
JsonizeVelocityCommand	3	0
LocoClient	34	0

unitree_sdk2_cpp.robot.h1.JsonizeDataVecFloat

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.h1 import JsonizeDataVecFloat
```

基类: object

公开属性

名称	Python 类型	含义	用法
data	list[float]	传给该接口的数据参数, Python 类型为 list[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.data / obj.data = value

方法索引

方法	Python 签名	状态	安全分类
__init__	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
from_json	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
to_json	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.h1.JsonizeDataVecFloat.__init__

初始化 JsonizeDataVecFloat 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: unitree::robot::h1::JsonizeDataVecFloat

- 签名: `JsonizeDataVecFloat()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_api.hpp:41`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeDataVecFloat()
```

`unitree_sdk2_cpp.robot.h1.JsonizeDataVecFloat.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值, 必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::h1::JsonizeDataVecFloat`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_api.hpp:44`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.h1.JsonizeDataVecFloat.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::h1::JsonizeDataVecFloat`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_api.hpp:46`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.h1.JsonizeTargetPos

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.h1 import JsonizeTargetPos
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>x</code>	<code>float</code>	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.x / obj.x = value</code>
<code>y</code>	<code>float</code>	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。	<code>value = obj.y / obj.y = value</code>
<code>yaw</code>	<code>float</code>	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。	<code>value = obj.yaw / obj.yaw = value</code>
<code>relative</code>	<code>bool</code>	传给该接口的 <code>relative</code> 参数, Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.relative / obj.relative = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION

方法	Python 签名	状态	安全分类
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.h1.JsonizeTargetPos.__init__

初始化 `JsonizeTargetPos` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h1::JsonizeTargetPos`
- 签名: `JsonizeTargetPos()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_api.hpp:72`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeTargetPos()
```

unitree_sdk2_cpp.robot.h1.JsonizeTargetPos.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common: JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::h1::JsonizeTargetPos`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_api.hpp:75`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.h1.JsonizeTargetPos.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::h1::JsonizeTargetPos`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_api.hpp:82`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.h1.JsonizeVelocityCommand`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.h1 import JsonizeVelocityCommand
```

基类: object

公开属性

名称	Python 类型	含义	用法
velocity	list[float]	传给该接口的速度参数, Python 类型为 list[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.velocity / obj.velocity = value
duration	float	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。	value = obj.duration / obj.duration = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.h1.JsonizeVelocityCommand.__init__

初始化 JsonizeVelocityCommand 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h1::JsonizeVelocityCommand`
- 签名: `JsonizeVelocityCommand()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_api.hpp:53`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeVelocityCommand()
```

unitree_sdk2_cpp.robot.h1.JsonizeVelocityCommand.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::h1::JsonizeVelocityCommand
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/h1/loco/h1_loco_api.hpp:56

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.h1.JsonizeVelocityCommand.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::h1::JsonizeVelocityCommand`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_api.hpp:61`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.h1.LocoClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.h1 import LocoClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>get_fsm_id</code>	<code>def get_fsm_id(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>get_fsm_mode</code>	<code>def get_fsm_mode(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>get_balance_mode</code>	<code>def get_balance_mode(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>get_swing_height</code>	<code>def get_swing_height(self) -> tuple[int, float]</code>	AVAILABLE	READ_ONLY
<code>get_stand_height</code>	<code>def get_stand_height(self) -> tuple[int, float]</code>	AVAILABLE	READ_ONLY
<code>get_phase</code>	<code>def get_phase(self) -> tuple[int, list[float]]</code>	AVAILABLE	READ_ONLY
<code>enable_odom</code>	<code>def enable_odom(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>disable_odom</code>	<code>def disable_odom(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>get_odom</code>	<code>def get_odom(self) -> tuple[int, float, float, float]</code>	AVAILABLE	READ_ONLY
<code>set_fsm_id</code>	<code>def set_fsm_id(self, fsm_id: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_balance_mode</code>	<code>def set_balance_mode(self, balance_mode: int) -> int</code>	AVAILABLE	MOTION_COMMAND

方法	Python 签名	状态	安全分类
set_swing_height	<code>def set_swing_height(self, swing_height: float) -> int</code>	AVAILABLE	MOTION_COMMAND
set_stand_height	<code>def set_stand_height(self, stand_height: float) -> int</code>	AVAILABLE	MOTION_COMMAND
set_velocity	<code>def set_velocity(self, vx: float, vy: float, omega: float, duration: float = 1.0) -> int</code>	AVAILABLE	MOTION_COMMAND
set_phase	<code>def set_phase(self, phase: Sequence[float]) -> int</code>	AVAILABLE	MOTION_COMMAND
set_target_pos	<code>def set_target_pos(self, x: float, y: float, yaw: float, relative: bool = True) -> int</code>	AVAILABLE	MOTION_COMMAND
set_task_id	<code>def set_task_id(self, task_id: int) -> int</code>	AVAILABLE	MOTION_COMMAND
damp	<code>def damp(self) -> int</code>	AVAILABLE	MOTION_COMMAND
start	<code>def start(self) -> int</code>	AVAILABLE	MOTION_COMMAND
stand_up	<code>def stand_up(self) -> int</code>	AVAILABLE	MOTION_COMMAND
zero_torque	<code>def zero_torque(self) -> int</code>	AVAILABLE	MOTION_COMMAND
stop_move	<code>def stop_move(self) -> int</code>	AVAILABLE	MOTION_COMMAND
high_stand	<code>def high_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
low_stand	<code>def low_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
move (重载 1/2)	<code>def move(self, vx: float, vy: float, vyaw: float, continous_move: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
move (重载 2/2)	<code>def move(self, vx: float, vy: float, vyaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
balance_stand	<code>def balance_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
continuous_gait	<code>def continuous_gait(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
switch_move_mode	<code>def switch_move_mode(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
set_next_foot	<code>def set_next_foot(self, foot: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
wave_hand	<code>def wave_hand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
shake_hand	<code>def shake_hand(self, stage: int = -1) -> int</code>	AVAILABLE	MOTION_COMMAND

unitree_sdk2_cpp.robot.h1.LocoClient.__init__

初始化 LocoClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `LocoClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:14`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LocoClient()
```

`unitree_sdk2_cpp.robot.h1.LocoClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:18`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

unitree_sdk2_cpp.robot.h1.LocoClient.get_fsm_id

查询或检查 FSM ID。

签名

```
def get_fsm_id(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] fsm_id	int	传给该接口的 FSM ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>fsm_id: int &</code> 。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `GetFsmId(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:42`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, fsm_id = obj.get_fsm_id()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h1.LocoClient.get_fsm_mode

查询或检查 FSM 模式。

签名

```
def get_fsm_mode(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。

位置	类型	含义
[1] fsm_mode	int	传给该接口的 FSM 模式参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 fsm_mode: int &。

对应 C++

- 类: unitree::robot::h1::LocoClient
- 签名: GetFsmMode(int &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/h1/loco/h1_loco_client.hpp:54

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, fsm_mode = obj.get_fsm_mode()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码, 并处理超时和网络中断。

unitree_sdk2_cpp.robot.h1.LocoClient.get_balance_mode

查询或检查平衡模式。

签名

```
def get_balance_mode(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务; 纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务错误码解释。
[1] balance_mode	int	传给该接口的平衡模式参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 balance_mode: int &。

对应 C++

- 类: unitree::robot::h1::LocoClient
- 签名: GetBalanceMode(int &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/h1/loco/h1_loco_client.hpp:66

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, balance_mode = obj.get_balance_mode()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h1.LocoClient.get_swing_height

查询或检查 swing 高度。

签名

```
def get_swing_height(self) -> tuple[int, float]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] swing_height	float	传给该接口的 swing 高度参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>swing_height: float &</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `GetSwingHeight(float &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:78`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, swing_height = obj.get_swing_height()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h1.LocoClient.get_stand_height

查询或检查 stand 高度。

签名

```
def get_stand_height(self) -> tuple[int, float]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] stand_height	float	传给该接口的 stand 高度参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 stand_height: float &。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `GetStandHeight(float &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:90`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, stand_height = obj.get_stand_height()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h1.LocoClient.get_phase

查询或检查相位。

签名

```
def get_phase(self) -> tuple[int, list[float]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] phase	list[float]	传给该接口的相位参数，Python 类型为 list[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 phase: std::vector<float> &。底层是可变长度 vector；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `GetPhase(std::vector<float> &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:102`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, phase = obj.get_phase()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h1.LocoClient.enable_odom

对应 C++ SDK 操作 `EnableOdom()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def enable_odom(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `EnableOdom()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:114`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.enable_odom()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.disable_odom

对应 C++ SDK 操作 `DisableOdom()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def disable_odom(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `DisableOdom()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:119`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.disable_odom()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h1.LocoClient.get_odom`

查询或检查 `odom`。

签名

```
def get_odom(self) -> tuple[int, float, float, float]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] <code>status</code>	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] <code>x</code>	float	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 <code>x: float &</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
[2] <code>y</code>	float	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 <code>y: float &</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
[3] <code>yaw</code>	float	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 <code>yaw: float &</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `GetOdom(float &, float &, float &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:124`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, x, y, yaw = obj.get_odom()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h1.LocoClient.set_fsm_id

设置 FSM ID。具体副作用和安全边界见下方状态。

签名

```
def set_fsm_id(self, fsm_id: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
fsm_id	int	必填	传给该接口的 FSM ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 fsm_id: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::h1::LocoClient
- 签名: SetFsmId(int)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/h1/loco/h1_loco_client.hpp:138

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_fsm_id(fsm_id=fsm_id)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.set_balance_mode

设置平衡模式。具体副作用和安全边界见下方状态。

签名

```
def set_balance_mode(self, balance_mode: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
balance_mode	int	必填	传给该接口的平衡模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 balance_mode: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `SetBalanceMode(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:148`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_balance_mode(balance_mode=balance_mode)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.set_swing_height

设置 swing 高度。具体副作用和安全边界见下方状态。

签名

```
def set_swing_height(self, swing_height: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
swing_height	float	必填	传给该接口的 swing 高度参数，Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 swing_height: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `SetSwingHeight(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:158`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_swing_height(swing_height=swing_height)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h1.LocoClient.set_stand_height`

设置 `stand` 高度。具体副作用和安全边界见下方状态。

签名

```
def set_stand_height(self, stand_height: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>stand_height</code>	<code>float</code>	必填	传给该接口的 <code>stand</code> 高度参数，Python 类型为 <code>float</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>stand_height: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `SetStandHeight(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:168`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_stand_height(stand_height=stand_height)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.set_velocity

设置速度。具体副作用和安全边界见下方状态。

签名

```
def set_velocity(self, vx: float, vy: float, omega: float, duration: float = 1.0) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>vx</code>	<code>float</code>	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vx: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>vy</code>	<code>float</code>	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vy: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>omega</code>	<code>float</code>	必填	角速度参数。旋转轴、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>omega: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
<code>duration</code>	<code>float</code>	<code>1.0</code>	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。对应 C++ 参数 <code>duration: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 <code>0</code> 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `SetVelocity(float, float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:178`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_velocity(vx=vx, vy=vy, omega=omega, duration=duration)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.set_phase

设置相位。具体副作用和安全边界见下方状态。

签名

```
def set_phase(self, phase: Sequence[float]) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
phase	Sequence[float]	必填	传给该接口的相位参数，Python 类型为 Sequence[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 phase: std::vector<float>。底层是可变长度 vector；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::h1::LocoClient
- 签名: SetPhase(std::vector<float>)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/h1/loco/h1_loco_client.hpp:190

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_phase(phase=phase)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.set_target_pos

设置 target pos。具体副作用和安全边界见下方状态。

签名

```
def set_target_pos(self, x: float, y: float, yaw: float, relative: bool = True) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
x	float	必填	X 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 x: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
y	float	必填	Y 方向位置或分量参数。参考系、单位和范围必须查当前方法及目标型号协议。对应 C++ 参数 y: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
yaw	float	必填	偏航角或偏航目标参数。参考系、单位和范围必须查目标型号协议。对应 C++ 参数 yaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
relative	bool	True	传给该接口的 relative 参数, Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 relative: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `SetTargetPos(float, float, float, bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:200`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_target_pos(x=x, y=y, yaw=yaw, relative=relative)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h1.LocoClient.set_task_id`

设置任务 ID。具体副作用和安全边界见下方状态。

签名

```
def set_task_id(self, task_id: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
task_id	int	必填	传给该接口的任务 ID 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 task_id: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `SetTaskId(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:213`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_task_id(task_id=task_id)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.damp

对应 C++ SDK 操作 `Damp()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def damp(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `Damp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:224`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.damp()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.start

启动对应 SDK 操作。具体副作用和安全边界见下方状态。

签名

```
def start(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `Start()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:226`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.start()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.stand_up

对应 C++ SDK 操作 `StandUp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_up(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `StandUp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:228`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_up()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h1.LocoClient.zero_torque`

对应 C++ SDK 操作 `ZeroTorque()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def zero_torque(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `ZeroTorque()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:230`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.zero_torque()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.stop_move

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_move(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `StopMove()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:232`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stop_move()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.high_stand

对应 C++ SDK 操作 `HighStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def high_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `HighStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:234`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.high_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h1.LocoClient.low_stand`

对应 C++ SDK 操作 `LowStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def low_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `LowStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:236`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.low_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.move (重载 1/2)

对应 C++ SDK 操作 Move(float, float, float, bool)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def move(self, vx: float, vy: float, vyaw: float, continous_move: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vx: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vy: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vyaw	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vyaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
continous_move	bool	必填	传给该接口的 continous move 参数，Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 continous_move: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::h1::LocoClient
- 签名: Move(float, float, float, bool)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/h1/loco/h1_loco_client.hpp:238

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw, continous_move=continous_move)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.move (重载 2/2)

对应 C++ SDK 操作 `Move(float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def move(self, vx: float, vy: float, vyaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vx: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vy: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vyaw	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vyaw: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `Move(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:242`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.balance_stand

对应 C++ SDK 操作 `BalanceStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def balance_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `BalanceStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:244`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.balance_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.continuous_gait

对应 C++ SDK 操作 `ContinuousGait(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def continuous_gait(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `ContinuousGait(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:246`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.continuous_gait(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h1.LocoClient.switch_move_mode`

对应 C++ SDK 操作 `SwitchMoveMode(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_move_mode(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>flag</code>	<code>bool</code>	必填	布尔开关。 <code>True</code> 和 <code>False</code> 的具体业务效果由当前方法定义。对应 C++ 参数 <code>flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `SwitchMoveMode(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:248`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.switch_move_mode(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.set_next_foot

设置 next foot。具体副作用和安全边界见下方状态。

签名

```
def set_next_foot(self, foot: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
foot	bool	必填	传给该接口的 foot 参数，Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 foot: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::h1::LocoClient
- 签名: SetNextFoot(bool)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/h1/loco/h1_loco_client.hpp:253

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_next_foot(foot=foot)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h1.LocoClient.wave_hand

对应 C++ SDK 操作 WaveHand()。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def wave_hand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `WaveHand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:257`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.wave_hand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h1.LocoClient.shake_hand`

对应 C++ SDK 操作 `ShakeHand(int)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def shake_hand(self, stage: int = -1) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
stage	int	-1	传给该接口的 <code>stage</code> 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>stage: int</code> 。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h1::LocoClient`
- 签名: `ShakeHand(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h1/loco/h1_loco_client.hpp:259`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.shake_hand(stage=stage)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

H2 Robot API

模块: `unitree_sdk2_cpp.robot.h2`

类索引

类	公开函数签名	属性
FsmIdInfo	2	0
H2ArmActionClient	6	0
JsonizeArmActionCommand	3	0
JsonizeArmActionName	3	0
JsonizeDataVecFloat	3	0
JsonizeFsmIdList	3	0
JsonizeVelocityCommand	3	0
LocoClient	37	0

`unitree_sdk2_cpp.robot.h2.FsmIdInfo`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.h2 import FsmIdInfo
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
id	int	传给该接口的 ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.id / obj.id = value</code>
name	str	SDK 对象、配置项、服务或动作的名称。允许值由调用它的具体接口定义。	<code>value = obj.name / obj.name = value</code>

方法索引

方法	Python 签名	状态	安全分类
__init__ (重载 1/2)	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
__init__ (重载 2/2)	<code>def __init__(self, i: int, n: str) -> None</code>	AVAILABLE	CONSTRUCTION

unitree_sdk2_cpp.robot.h2.FsmIdInfo.__init__ (重载 1/2)

初始化 `FsmIdInfo` 实例。是否能实际构造取决于下方可用性状态。

签名

```
@overload
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h2::FsmIdInfo`
- 签名: `FsmIdInfo()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_api.hpp:73`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = FsmIdInfo()
```

unitree_sdk2_cpp.robot.h2.FsmIdInfo.__init__ (重载 2/2)

初始化 `FsmIdInfo` 实例。是否能实际构造取决于下方可用性状态。

签名

```
@overload
def __init__(self, i: int, n: str) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
i	int	必填	传给该接口的 i 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 i: int。
n	str	必填	传给该接口的 n 参数, Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 n: const std::string &。底层为字符串; 长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: unitree::robot::h2::FsmIdInfo
- 签名: FsmIdInfo(int, const std::string &)
- 绑定策略: 未单独标注
- 声明位置: include/unitree/robot/h2/loco/h2_loco_api.hpp:74

说明

- 示例是局部调用片段; obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = FsmIdInfo(i=i, n=n)
```

unitree_sdk2_cpp.robot.h2.H2ArmActionClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.h2 import H2ArmActionClient
```

基类: Client

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>execute_action</code> (重载 1/2)	<code>def execute_action(self, action_id: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>execute_action</code> (重载 2/2)	<code>def execute_action(self, action_name: str) -> int</code>	AVAILABLE	MOTION_COMMAND

方法	Python 签名	状态	安全分类
<code>stop_custom_action</code>	<code>def stop_custom_action(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>get_action_list</code>	<code>def get_action_list(self) -> tuple[int, str]</code>	AVAILABLE	READ_ONLY

`unitree_sdk2_cpp.robot.h2.H2ArmActionClient.__init__`

初始化 H2ArmActionClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h2::H2ArmActionClient`
- 签名: `H2ArmActionClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h2/arm/h2_arm_action_client.hpp:24`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = H2ArmActionClient()
```

`unitree_sdk2_cpp.robot.h2.H2ArmActionClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::h2::H2ArmActionClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/arm/h2_arm_action_client.hpp:26`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.h2.H2ArmActionClient.execute_action` (重载 1/2)

对应 C++ SDK 操作 `ExecuteAction(int32_t)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
@overload
def execute_action(self, action_id: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>action_id</code>	<code>int</code>	必填	传给该接口的 action ID 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>action_id: int32_t</code> 。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::H2ArmActionClient`
- 签名: `ExecuteAction(int32_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/arm/h2_arm_action_client.hpp:34`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.execute_action(action_id=action_id)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.H2ArmActionClient.execute_action (重载 2/2)

对应 C++ SDK 操作 `ExecuteAction(const std::string &)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def execute_action(self, action_name: str) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
action_name	str	必填	传给该接口的 action 名称参数，Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>action_name: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::H2ArmActionClient`
- 签名: `ExecuteAction(const std::string &)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/arm/h2_arm_action_client.hpp:43`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.execute_action(action_name=action_name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.H2ArmActionClient.stop_custom_action

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_custom_action(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::H2ArmActionClient`
- 签名: `StopCustomAction()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/arm/h2_arm_action_client.hpp:52`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stop_custom_action()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.H2ArmActionClient.get_action_list

查询或检查 `action list`。

签名

```
def get_action_list(self) -> tuple[int, str]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 `getter` 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] <code>status</code>	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] <code>data</code>	<code>str</code>	传给该接口的数据参数，Python 类型为 <code>str</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>data: std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

对应 C++

- 类: `unitree::robot::h2::H2ArmActionClient`
- 签名: `GetActionList(std::string &)`
- 绑定策略: `OUTPUT_WRAPPER`

· 声明位置: `include/unitree/robot/h2/arm/h2_arm_action_client.hpp:58`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, data = obj.get_action_list()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码, 并处理超时和网络中断。

unitree_sdk2_cpp.robot.h2.JsonizeArmActionCommand

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.h2 import JsonizeArmActionCommand
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>action_id</code>	<code>int</code>	传给该接口的 action ID 参数, Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.action_id / obj.action_id = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.h2.JsonizeArmActionCommand.__init__

初始化 `JsonizeArmActionCommand` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / **CONSTRUCTION**: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h2::JsonizeArmActionCommand`
- 签名: `JsonizeArmActionCommand()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeArmActionCommand()
```

`unitree_sdk2_cpp.robot.h2.JsonizeArmActionCommand.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::h2::JsonizeArmActionCommand`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/h2/arm/h2_arm_action_api.hpp:22`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.h2.JsonizeArmActionCommand.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::h2::JsonizeArmActionCommand
- 签名: toJson(common::JsonMap &) const
- 绑定策略: JSON_DICT_OUTPUT
- 声明位置: include/unitree/robot/h2/arm/h2_arm_action_api.hpp:26

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.h2.JsonizeArmActionName

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.h2 import JsonizeArmActionName
```

基类: object

公开属性

名称	Python 类型	含义	用法
action_name	str	传给该接口的 action 名称参数，Python 类型为 str。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.action_name / obj.action_name = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION

方法	Python 签名	状态	安全分类
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.h2.JsonizeArmActionName.__init__

初始化 `JsonizeArmActionName` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h2::JsonizeArmActionName`
- 签名: `JsonizeArmActionName()`
- 绑定策略: 未单独标注
- 声明位置: 由 `pybind11` 手工绑定或生成注册表提供；manifest 未记录独立头文件行号。

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeArmActionName()
```

unitree_sdk2_cpp.robot.h2.JsonizeArmActionName.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common: JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::h2::JsonizeArmActionName`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/h2/arm/h2_arm_action_api.hpp:35`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.h2.JsonizeArmActionName.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::h2::JsonizeArmActionName`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/h2/arm/h2_arm_action_api.hpp:39`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

`unitree_sdk2_cpp.robot.h2.JsonizeDataVecFloat`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.h2 import JsonizeDataVecFloat
```

基类：object

公开属性

名称	Python 类型	含义	用法
data	list[float]	传给该接口的数据参数，Python 类型为 list[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.data / obj.data = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.h2.JsonizeDataVecFloat.__init__

初始化 JsonizeDataVecFloat 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h2::JsonizeDataVecFloat`
- 签名: `JsonizeDataVecFloat()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_api.hpp:40`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeDataVecFloat()
```

unitree_sdk2_cpp.robot.h2.JsonizeDataVecFloat.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: unitree::robot::h2::JsonizeDataVecFloat
- 签名: fromJson(common::JsonMap &)
- 绑定策略: JSON_DICT_INPUT
- 声明位置: include/unitree/robot/h2/loco/h2_loco_api.hpp:43

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.h2.JsonizeDataVecFloat.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::h2::JsonizeDataVecFloat`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_api.hpp:45`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

`unitree_sdk2_cpp.robot.h2.JsonizeFsmIdList`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.h2 import JsonizeFsmIdList
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>fsm_ids</code>	<code>list[FsmIdInfo]</code>	传给该接口的 FSM ID 列表参数, Python 类型为 <code>list[FsmIdInfo]</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.fsm_ids / obj.fsm_ids = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.h2.JsonizeFsmIdList.__init__`

初始化 `JsonizeFsmIdList` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	__init__ 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h2::JsonizeFsmIdList`
- 签名: `JsonizeFsmIdList()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_api.hpp:79`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeFsmIdList()
```

`unitree_sdk2_cpp.robot.h2.JsonizeFsmIdList.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::h2::JsonizeFsmIdList`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_api.hpp:82`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.h2.JsonizeFsmIdList.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: unitree::robot::h2::JsonizeFsmIdList
- 签名: toJson(common::JsonMap &) const
- 绑定策略: JSON_DICT_OUTPUT
- 声明位置: include/unitree/robot/h2/loco/h2_loco_api.hpp:95

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.h2.JsonizeVelocityCommand

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.h2 import JsonizeVelocityCommand
```

基类: object

公开属性

名称	Python 类型	含义	用法
velocity	list[float]	传给该接口的速度参数，Python 类型为 list[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.velocity / obj.velocity = value
duration	float	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。	value = obj.duration / obj.duration = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.h2.JsonizeVelocityCommand.__init__

初始化 `JsonizeVelocityCommand` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h2::JsonizeVelocityCommand`
- 签名: `JsonizeVelocityCommand()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_api.hpp:52`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeVelocityCommand()
```

unitree_sdk2_cpp.robot.h2.JsonizeVelocityCommand.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::h2::JsonizeVelocityCommand`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_api.hpp:55`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.h2.JsonizeVelocityCommand.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::h2::JsonizeVelocityCommand`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_api.hpp:60`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.h2.LocoClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.h2 import LocoClient
```

基类: Client

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>get_fsm_id</code>	<code>def get_fsm_id(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>get_fsm_mode</code>	<code>def get_fsm_mode(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>get_balance_mode</code>	<code>def get_balance_mode(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>get_swing_height</code>	<code>def get_swing_height(self) -> tuple[int, float]</code>	AVAILABLE	READ_ONLY
<code>get_stand_height</code>	<code>def get_stand_height(self) -> tuple[int, float]</code>	AVAILABLE	READ_ONLY
<code>get_phase</code>	<code>def get_phase(self) -> tuple[int, list[float]]</code>	AVAILABLE	READ_ONLY
<code>get_arm_sdk_status</code>	<code>def get_arm_sdk_status(self) -> tuple[int, bool]</code>	AVAILABLE	READ_ONLY
<code>get_available_fsm_ids</code>	<code>def get_available_fsm_ids(self) -> tuple[int, list[int], list[str]]</code>	AVAILABLE	READ_ONLY
<code>set_fsm_id</code>	<code>def set_fsm_id(self, fsm_id: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_balance_mode</code>	<code>def set_balance_mode(self, balance_mode: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_punch_api</code>	<code>def set_punch_api(self, punch_api: Sequence[float]) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_swing_height</code>	<code>def set_swing_height(self, swing_height: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_stand_height</code>	<code>def set_stand_height(self, stand_height: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_velocity</code>	<code>def set_velocity(self, vx: float, vy: float, omega: float, duration: float = 1.0) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_task_id</code>	<code>def set_task_id(self, task_id: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_arm_sdk_status</code>	<code>def set_arm_sdk_status(self, arm_sdk_status: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>damp</code>	<code>def damp(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>start</code>	<code>def start(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>squat</code>	<code>def squat(self) -> int</code>	AVAILABLE	MOTION_COMMAND

方法	Python 签名	状态	安全分类
sit	<code>def sit(self) -> int</code>	AVAILABLE	MOTION_COMMAND
stand_up	<code>def stand_up(self) -> int</code>	AVAILABLE	MOTION_COMMAND
zero_torque	<code>def zero_torque(self) -> int</code>	AVAILABLE	MOTION_COMMAND
stop_move	<code>def stop_move(self) -> int</code>	AVAILABLE	MOTION_COMMAND
high_stand	<code>def high_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
low_stand	<code>def low_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
move (重载 1/2)	<code>def move(self, vx: float, vy: float, vyaw: float, continous_move: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
move (重载 2/2)	<code>def move(self, vx: float, vy: float, vyaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
balance_stand	<code>def balance_stand(self) -> int</code>	AVAILABLE	MOTION_COMMAND
continuous_gait	<code>def continuous_gait(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
switch_move_mode	<code>def switch_move_mode(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
wave_hand	<code>def wave_hand(self, turn_flag: bool = False) -> int</code>	AVAILABLE	MOTION_COMMAND
shake_hand	<code>def shake_hand(self, stage: int = -1) -> int</code>	AVAILABLE	MOTION_COMMAND
set_speed_mode	<code>def set_speed_mode(self, speed_mode: int) -> int</code>	AVAILABLE	MOTION_COMMAND
enable_arm_sdk	<code>def enable_arm_sdk(self) -> int</code>	AVAILABLE	MOTION_COMMAND
disable_arm_sdk	<code>def disable_arm_sdk(self) -> int</code>	AVAILABLE	MOTION_COMMAND

unitree_sdk2_cpp.robot.h2.LocoClient.__init__

初始化 LocoClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台 and 对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::h2::LocoClient`

- 签名: `LocoClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:14`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LocoClient()
```

unitree_sdk2_cpp.robot.h2.LocoClient.init

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值; 失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:18`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

unitree_sdk2_cpp.robot.h2.LocoClient.get_fsm_id

查询或检查 FSM ID。

签名

```
def get_fsm_id(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务; 纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] fsm_id	int	传给该接口的 FSM ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 fsm_id: int &。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `GetFsmId(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:41`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, fsm_id = obj.get_fsm_id()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

`unitree_sdk2_cpp.robot.h2.LocoClient.get_fsm_mode`

查询或检查 FSM 模式。

签名

```
def get_fsm_mode(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] fsm_mode	int	传给该接口的 FSM 模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 fsm_mode: int &。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `GetFsmMode(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:55`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, fsm_mode = obj.get_fsm_mode()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h2.LocoClient.get_balance_mode

查询或检查平衡模式。

签名

```
def get_balance_mode(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] balance_mode	int	传给该接口的平衡模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>balance_mode: int &</code> 。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `GetBalanceMode(int &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:69`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, balance_mode = obj.get_balance_mode()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h2.LocoClient.get_swing_height

查询或检查 swing 高度。

签名

```
def get_swing_height(self) -> tuple[int, float]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务错误码解释。
[1] swing_height	float	传给该接口的 swing 高度参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 swing_height: float &。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `GetSwingHeight(float &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:83`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, swing_height = obj.get_swing_height()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码, 并处理超时和网络中断。

`unitree_sdk2_cpp.robot.h2.LocoClient.get_stand_height`

查询或检查 stand 高度。

签名

```
def get_stand_height(self) -> tuple[int, float]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务; 纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务错误码解释。
[1] stand_height	float	传给该接口的 stand 高度参数, Python 类型为 float。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 stand_height: float &。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `GetStandHeight(float &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:97`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现, 而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, stand_height = obj.get_stand_height()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h2.LocoClient.get_phase

查询或检查相位。

签名

```
def get_phase(self) -> tuple[int, list[float]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] phase	list[float]	传给该接口的相位参数，Python 类型为 list[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>phase: std::vector<float> &</code> 。底层是可变长度 vector；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `GetPhase(std::vector<float> &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:111`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, phase = obj.get_phase()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h2.LocoClient.get_arm_sdk_status

查询或检查机械臂 SDK 状态。

签名

```
def get_arm_sdk_status(self) -> tuple[int, bool]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] arm_sdk_status	bool	传给该接口的机械臂 SDK 状态参数，Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 arm_sdk_status: bool &。只接受布尔语义。

对应 C++

- 类: unitree::robot::h2::LocoClient
- 签名: GetArmSdkStatus(bool &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/h2/loco/h2_loco_client.hpp:125

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, arm_sdk_status = obj.get_arm_sdk_status()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h2.LocoClient.get_available_fsm_ids

查询或检查 available FSM ID 列表。

签名

```
def get_available_fsm_ids(self) -> tuple[int, list[int], list[str]]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] ids	list[int]	传给该接口的 ID 列表参数，Python 类型为 list[int]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 ids: std::vector<int> &。底层是可变长度 vector
[2] names	list[str]	传给该接口的名称列表参数，Python 类型为 list[str]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 names: std::vector<std::string> &。底层是可变长度 vector；元素约束：底层为字符串；长度、编码和允许值由具体协议决定。

对应 C++

- 类: unitree::robot::h2::LocoClient
- 签名: GetAvailableFsmIds(std::vector<int> &, std::vector<std::string> &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/h2/loco/h2_loco_client.hpp:139

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, ids, names = obj.get_available_fsm_ids()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.h2.LocoClient.set_fsm_id

设置 FSM ID。具体副作用和安全边界见下方状态。

签名

```
def set_fsm_id(self, fsm_id: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
fsm_id	int	必填	传给该接口的 FSM ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 fsm_id: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `SetFsmId(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:158`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_fsm_id(fsm_id=fsm_id)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.set_balance_mode

设置平衡模式。具体副作用和安全边界见下方状态。

签名

```
def set_balance_mode(self, balance_mode: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
balance_mode	int	必填	传给该接口的平衡模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 balance_mode: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `SetBalanceMode(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:168`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_balance_mode(balance_mode=balance_mode)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.set_punch_api

设置 punch API。具体副作用和安全边界见下方状态。

签名

```
def set_punch_api(self, punch_api: Sequence[float]) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
punch_api	Sequence[float]	必填	传给该接口的 punch API 参数，Python 类型为 Sequence[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 punch_api: std::vector<float> &。底层是可变长度 vector；元素约束：底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `SetPunchApi(std::vector<float> &)`
- 绑定策略: `MUTABLE_INPUT_COPY`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:178`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_punch_api(punch_api=punch_api)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h2.LocoClient.set_swing_height`

设置 `swing` 高度。具体副作用和安全边界见下方状态。

签名

```
def set_swing_height(self, swing_height: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>swing_height</code>	<code>float</code>	必填	传给该接口的 <code>swing</code> 高度参数，Python 类型为 <code>float</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>swing_height: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `SetSwingHeight(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:189`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_swing_height(swing_height=swing_height)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.set_stand_height

设置 `stand` 高度。具体副作用和安全边界见下方状态。

签名

```
def set_stand_height(self, stand_height: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>stand_height</code>	<code>float</code>	必填	传给该接口的 <code>stand</code> 高度参数，Python 类型为 <code>float</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>stand_height: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `SetStandHeight(float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:199`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_stand_height(stand_height=stand_height)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.set_velocity

设置速度。具体副作用和安全边界见下方状态。

签名

```
def set_velocity(self, vx: float, vy: float, omega: float, duration: float = 1.0) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vx: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vy: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
omega	float	必填	角速度参数。旋转轴、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 omega: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
duration	float	1.0	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。对应 C++ 参数 duration: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::h2::LocoClient
- 签名: SetVelocity(float, float, float, float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/h2/loco/h2_loco_client.hpp:209

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_velocity(vx=vx, vy=vy, omega=omega, duration=duration)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.set_task_id

设置任务 ID。具体副作用和安全边界见下方状态。

签名

```
def set_task_id(self, task_id: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
task_id	int	必填	传给该接口的任务 ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 task_id：int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类：unitree::robot::h2::LocoClient
- 签名：SetTaskId(int)
- 绑定策略：DIRECT
- 声明位置：include/unitree/robot/h2/loco/h2_loco_client.hpp:221

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_task_id(task_id=task_id)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.set_arm_sdk_status

设置机械臂 SDK 状态。具体副作用和安全边界见下方状态。

签名

```
def set_arm_sdk_status(self, arm_sdk_status: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND：当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
arm_sdk_status	bool	必填	传给该接口的机械臂 SDK 状态参数，Python 类型为 bool。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 arm_sdk_status：bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `SetArmSdkStatus(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:231`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_arm_sdk_status(arm_sdk_status=arm_sdk_status)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h2.LocoClient.damp`

对应 C++ SDK 操作 `Damp()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def damp(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `Damp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:242`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.damp()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h2.LocoClient.start`

启动对应 SDK 操作。具体副作用和安全边界见下方状态。

签名

```
def start(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `Start()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:244`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.start()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.squat

对应 C++ SDK 操作 `Squat()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def squat(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`

- 签名: `Squat()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:246`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.squat()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.sit

对应 C++ SDK 操作 `sit()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def sit(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 <code>0</code> 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `Sit()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:248`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.sit()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.stand_up

对应 C++ SDK 操作 `StandUp()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def stand_up(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `StandUp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:250`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_up()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.zero_torque

对应 C++ SDK 操作 `ZeroTorque()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def zero_torque(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`

- 签名: `ZeroTorque()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:252`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.zero_torque()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h2.LocoClient.stop_move`

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_move(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `StopMove()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:254`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stop_move()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h2.LocoClient.high_stand`

对应 C++ SDK 操作 `HighStand()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名


```
def high_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `HighStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:256`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.high_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h2.LocoClient.low_stand`

对应 C++ SDK 操作 `LowStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def low_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`

- 签名: `LowStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:258`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.low_stand()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.move (重载 1/2)

对应 C++ SDK 操作 `Move(float, float, float, bool)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
@overload
def move(self, vx: float, vy: float, vyaw: float, continous_move: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>vx</code>	<code>float</code>	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vx: float</code> 。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。
<code>vy</code>	<code>float</code>	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vy: float</code> 。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。
<code>vyaw</code>	<code>float</code>	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vyaw: float</code> 。底层为浮点数; 类型本身不说明单位、坐标系或业务安全范围。
<code>continous_move</code>	<code>bool</code>	必填	传给该接口的 <code>continous move</code> 参数, Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>continous_move: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `Move(float, float, float, bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:260`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw, continuous_move=continuous_move)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.move (重载 2/2)

对应 C++ SDK 操作 Move(float, float, float)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def move(self, vx: float, vy: float, vyaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vx: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vy: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vyaw	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vyaw: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::h2::LocoClient
- 签名: Move(float, float, float)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/h2/loco/h2_loco_client.hpp:264

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.balance_stand

对应 C++ SDK 操作 `BalanceStand()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def balance_stand(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `BalanceStand()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:266`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.balance_stand()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.continuous_gait

对应 C++ SDK 操作 `ContinuousGait(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def continuous_gait(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `ContinuousGait(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:268`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.continuous_gait(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.switch_move_mode

对应 C++ SDK 操作 `SwitchMoveMode(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_move_mode(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`

- 签名: `SwitchMoveMode(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:270`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_move_mode(flag=flag)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.wave_hand

对应 C++ SDK 操作 `WaveHand(bool)`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def wave_hand(self, turn_flag: bool = False) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
<code>turn_flag</code>	<code>bool</code>	<code>False</code>	传给该接口的 <code>turn flag</code> 参数, Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>turn_flag: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `WaveHand(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:275`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.wave_hand(turn_flag=turn_flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.shake_hand

对应 C++ SDK 操作 `ShakeHand(int)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def shake_hand(self, stage: int = -1) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
stage	int	-1	传给该接口的 stage 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 stage: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `ShakeHand(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:277`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.shake_hand(stage=stage)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.set_speed_mode

设置速度模式。具体副作用和安全边界见下方状态。

签名

```
def set_speed_mode(self, speed_mode: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
speed_mode	int	必填	传给该接口的速度模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。 对应 C++ 参数 speed_mode: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::h2::LocoClient
- 签名: SetSpeedMode(int)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/h2/loco/h2_loco_client.hpp:293

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 stop、damp 或 zero 也不代表它是独立物理急停。

用法

```
result = obj.set_speed_mode(speed_mode=speed_mode)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.h2.LocoClient.enable_arm_sdk

对应 C++ SDK 操作 EnableArmSDK()。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def enable_arm_sdk(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::h2::LocoClient
- 签名: EnableArmSDK()

- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:303`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.enable_arm_sdk()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.h2.LocoClient.disable_arm_sdk`

对应 C++ SDK 操作 `DisableArmSDK()`。上游头文件没有可直接生成的业务说明时, 本参考只保证签名映射, 精确语义需查目标型号协议。

签名

```
def disable_arm_sdk(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现, 调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用; 不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>int</code>	SDK 状态码; Unitree 示例通常以 <code>0</code> 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::h2::LocoClient`
- 签名: `DisableArmSDK()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/h2/loco/h2_loco_client.hpp:305`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.disable_arm_sdk()
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

R1 Robot API

模块: `unitree_sdk2_cpp.robot.r1`

类索引

类	公开函数签名	属性
AudioClient	8	0
JsonizeDataVecFloat	3	0
JsonizeVelocity	3	0
Command		
LedControlParameter	3	0
LocoClient	15	0
PlayStopParameter	3	0
PlayStreamParameter	3	0
TtsMakerParameter	3	0

unitree_sdk2_cpp.robot.r1.AudioClient

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.r1 import AudioClient
```

基类： `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>tts_maker</code>	<code>def tts_maker(self, text: str, speaker_id: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>get_volume</code>	<code>def get_volume(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>set_volume</code>	<code>def set_volume(self, volume: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>play_stream</code>	<code>def play_stream(self, app_name: str, stream_id: str, pcm_data: Sequence[int]) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>play_stop</code>	<code>def play_stop(self, app_name: str) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT
<code>led_control</code>	<code>def led_control(self, r: int, g: int, b: int) -> int</code>	AVAILABLE	HARDWARE_SIDE_EFFECT

unitree_sdk2_cpp.robot.r1.AudioClient.__init__

初始化 `AudioClient` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::r1::AudioClient`
- 签名: `AudioClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/r1/audio/audio_client.hpp:15`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = AudioClient()
```

`unitree_sdk2_cpp.robot.r1.AudioClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::r1::AudioClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/audio/audio_client.hpp:19`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.r1.AudioClient.tts_maker`

对应 C++ SDK 操作 `TtsMaker(const std::string &, int32_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def tts_maker(self, text: str, speaker_id: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
text	str	必填	要处理的文本内容；编码、长度和语言支持由目标服务决定。对应 C++ 参数 <code>text: const std::string &</code> 。底层为字符串；长度、编码和允许值由具体协议决定。
speaker_id	int	必填	语音合成说话人 ID。有效编号由机器人音频服务和固件决定。对应 C++ 参数 <code>speaker_id: int32_t</code> 。取值范围为 -2147483648 到 2147483647。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::AudioClient`
- 签名: `TtsMaker(const std::string &, int32_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/audio/audio_client.hpp:31`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.tts_maker(text=text, speaker_id=speaker_id)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.r1.AudioClient.get_volume`

查询或检查音量。

签名

```
def get_volume(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。

位置	类型	含义
[1] volume	int	传给该接口的音量参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 volume: uint8_t &。取值范围为 0 到 255。

对应 C++

- 类: `unitree::robot::r1::AudioClient`
- 签名: `GetVolume(uint8_t &)`
- 绑定策略: `OUTPUT_WRAPPER`
- 声明位置: `include/unitree/robot/r1/audio/audio_client.hpp:43`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, volume = obj.get_volume()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.r1.AudioClient.set_volume

设置音量。具体副作用和安全边界见下方状态。

签名

```
def set_volume(self, volume: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
volume	int	必填	传给该接口的音量参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 volume: uint8_t。取值范围为 0 到 255。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::AudioClient`
- 签名: `SetVolume(uint8_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/audio/audio_client.hpp:57`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.set_volume(volume=volume)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.AudioClient.play_stream

对应 C++ SDK 操作 PlayStream(std::string, std::string, std::vector<uint8_t>)。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def play_stream(self, app_name: str, stream_id: str, pcm_data: Sequence[int])
    -> int
```

可用性与安全性

AVAILABLE / **HARDWARE_SIDE_EFFECT**: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
app_name	str	必填	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。对应 C++ 参数 app_name: std::string。底层为字符串；长度、编码和允许值由具体协议决定。
stream_id	str	必填	音频流标识符，用于区分同一应用下的播放流。对应 C++ 参数 stream_id: std::string。底层为字符串；长度、编码和允许值由具体协议决定。
pcm_data	Sequence[int]	必填	传给该接口的 pcm 数据参数，Python 类型为 Sequence[int]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 pcm_data: std::vector<uint8_t>。底层是可变长度 vector；元素约束：取值范围为 0 到 255。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: unitree::robot::r1::AudioClient
- 签名: PlayStream(std::string, std::string, std::vector<uint8_t>)
- 绑定策略: DIRECT
- 声明位置: include/unitree/robot/r1/audio/audio_client.hpp:68

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.play_stream(app_name=app_name, stream_id=stream_id,
    pcm_data=pcm_data)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.AudioClient.play_stop

对应 C++ SDK 操作 `PlayStop(std::string)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def play_stop(self, app_name: str) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
app_name	str	必填	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。对应 C++ 参数 <code>app_name: std::string</code> 。底层为字符串；长度、编码和允许值由具体协议决定。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::AudioClient`
- 签名: `PlayStop(std::string)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/audio/audio_client.hpp:80`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.play_stop(app_name=app_name)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.AudioClient.led_control

对应 C++ SDK 操作 `LedControl(uint8_t, uint8_t, uint8_t)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def led_control(self, r: int, g: int, b: int) -> int
```

可用性与安全性

AVAILABLE / HARDWARE_SIDE_EFFECT: 当前绑定源码已实现，调用可能修改机器人或服务状态、启动订阅或产生网络副作用。必须显式检查目标设备和返回状态。

参数

名称	Python 类型	默认值	含义
r	int	必填	传给该接口的 r 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 R: uint8_t。取值范围为 0 到 255。
g	int	必填	传给该接口的 g 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 G: uint8_t。取值范围为 0 到 255。
b	int	必填	传给该接口的 b 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 B: uint8_t。取值范围为 0 到 255。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::AudioClient`
- 签名: `LedControl(uint8_t, uint8_t, uint8_t)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/audio/audio_client.hpp:91`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.led_control(r=r, g=g, b=b)
```

[!CAUTION] 此示例展示签名, 不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查; 运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.r1.JsonizeDataVecFloat`

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.r1 import JsonizeDataVecFloat
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
data	<code>list[float]</code>	传给该接口的数据参数, Python 类型为 <code>list[float]</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.data / obj.data = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.r1.JsonizeDataVecFloat.__init__

初始化 `JsonizeDataVecFloat` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::r1::JsonizeDataVecFloat`
- 签名: `JsonizeDataVecFloat()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_api.hpp:27`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeDataVecFloat()
```

unitree_sdk2_cpp.robot.r1.JsonizeDataVecFloat.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::r1::JsonizeDataVecFloat`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_api.hpp:30`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.r1.JsonizeDataVecFloat.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::r1::JsonizeDataVecFloat`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_api.hpp:34`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.r1.JsonizeVelocityCommand

SDK 类型签名预览。请逐项查看构造函数和方法的可用性。

导入

```
from unitree_sdk2_cpp.robot.r1 import JsonizeVelocityCommand
```

基类: object

公开属性

名称	Python 类型	含义	用法
velocity	list[float]	传给该接口的速度参数, Python 类型为 list[float]。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	value = obj.velocity / obj.velocity = value
duration	float	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。	value = obj.duration / obj.duration = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.r1.JsonizeVelocityCommand.__init__

初始化 JsonizeVelocityCommand 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值; 调用类对象时, 在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::r1::JsonizeVelocityCommand`
- 签名: `JsonizeVelocityCommand()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_api.hpp:43`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = JsonizeVelocityCommand()
```

`unitree_sdk2_cpp.robot.r1.JsonizeVelocityCommand.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::r1::JsonizeVelocityCommand`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_api.hpp:46`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

`unitree_sdk2_cpp.robot.r1.JsonizeVelocityCommand.to_json`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::r1::JsonizeVelocityCommand`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_api.hpp:51`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.r1.LedControlParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.r1 import LedControlParameter
```

基类： `object`

公开属性

名称	Python 类型	含义	用法
<code>r</code>	<code>int</code>	传给该接口的 <code>r</code> 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.r / obj.r = value</code>
<code>g</code>	<code>int</code>	传给该接口的 <code>g</code> 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.g / obj.g = value</code>
<code>b</code>	<code>int</code>	传给该接口的 <code>b</code> 参数，Python 类型为 <code>int</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	<code>value = obj.b / obj.b = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.r1.LedControlParameter.__init__

初始化 LedControlParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::r1::LedControlParameter`
- 签名: `LedControlParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:75`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LedControlParameter()
```

unitree_sdk2_cpp.robot.r1.LedControlParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common: JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::r1::LedControlParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:78`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.fromJson(value=value)
```

`unitree_sdk2_cpp.robot.r1.LedControlParameter.toJson`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def toJson(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现; 实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::r1::LedControlParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:80`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

`unitree_sdk2_cpp.robot.r1.LocoClient`

Robot 服务客户端。构造、初始化和具体方法的可用性必须分别检查。

导入

```
from unitree_sdk2_cpp.robot.r1 import LocoClient
```

基类: `Client`

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>init</code>	<code>def init(self) -> None</code>	AVAILABLE	INITIALIZATION
<code>get_fsm_id</code>	<code>def get_fsm_id(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>get_fsm_mode</code>	<code>def get_fsm_mode(self) -> tuple[int, int]</code>	AVAILABLE	READ_ONLY
<code>set_fsm_id</code>	<code>def set_fsm_id(self, fsm_id: int) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_velocity</code>	<code>def set_velocity(self, vx: float, vy: float, omega: float, duration: float = 1.0) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>damp</code>	<code>def damp(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>start</code>	<code>def start(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stand_up</code>	<code>def stand_up(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>zero_torque</code>	<code>def zero_torque(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>stop_move</code>	<code>def stop_move(self) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move (重载 1/2)</code>	<code>def move(self, vx: float, vy: float, vyaw: float, continous_move: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>move (重载 2/2)</code>	<code>def move(self, vx: float, vy: float, vyaw: float) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>switch_move_mode</code>	<code>def switch_move_mode(self, flag: bool) -> int</code>	AVAILABLE	MOTION_COMMAND
<code>set_speed_mode</code>	<code>def set_speed_mode(self, speed_mode: int) -> int</code>	AVAILABLE	MOTION_COMMAND

unitree_sdk2_cpp.robot.r1.LocoClient.__init__

初始化 LocoClient 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `LocoClient()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:13`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = LocoClient()
```

`unitree_sdk2_cpp.robot.r1.LocoClient.init`

初始化当前 SDK 对象所需的底层通道或服务资源。

签名

```
def init(self) -> None
```

可用性与安全性

AVAILABLE / INITIALIZATION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	<code>None</code>	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `Init()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:17`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.init()
```

`unitree_sdk2_cpp.robot.r1.LocoClient.get_fsm_id`

查询或检查 FSM ID。

签名

```
def get_fsm_id(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] fsm_id	int	传给该接口的 FSM ID 参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 fsm_id: int &。

对应 C++

- 类: unitree::robot::r1::LocoClient
- 签名: GetFsmId(int &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/r1/loco/r1_loco_client.hpp:28

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, fsm_id = obj.get_fsm_id()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.r1.LocoClient.get_fsm_mode

查询或检查 FSM 模式。

签名

```
def get_fsm_mode(self) -> tuple[int, int]
```

可用性与安全性

AVAILABLE / READ_ONLY: 当前绑定源码已实现只读查询。Robot Client 的服务端查询通常仍需匹配的 Linux 扩展、DDS、网络、机器人和服务；纯本地 getter 则不一定需要全部条件。

参数

无显式参数。实例方法中的 self 由 Python 自动传入。

返回值

位置	类型	含义
[0] status	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务错误码解释。
[1] fsm_mode	int	传给该接口的 FSM 模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 fsm_mode: int &。

对应 C++

- 类: unitree::robot::r1::LocoClient
- 签名: GetFsmMode(int &)
- 绑定策略: OUTPUT_WRAPPER
- 声明位置: include/unitree/robot/r1/loco/r1_loco_client.hpp:43

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- C++ 的可修改输出引用不会作为 Python 入参出现，而是按 C++ 参数顺序追加到返回元组中。

用法

```
status, fsm_mode = obj.get_fsm_mode()
```

[!NOTE] 只读查询仍会访问 DDS/机器人服务。必须检查返回状态码，并处理超时和网络中断。

unitree_sdk2_cpp.robot.r1.LocoClient.set_fsm_id

设置 FSM ID。具体副作用和安全边界见下方状态。

签名

```
def set_fsm_id(self, fsm_id: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
fsm_id	int	必填	传给该接口的 FSM ID 参数, Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 fsm_id: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码; Unitree 示例通常以 0 表示成功, 非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `SetFsmId(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:58`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_fsm_id(fsm_id=fsm_id)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.LocoClient.set_velocity

设置速度。具体副作用和安全边界见下方状态。

签名

```
def set_velocity(self, vx: float, vy: float, omega: float, duration: float = 1.0) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vx: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 vy: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
omega	float	必填	角速度参数。旋转轴、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 omega: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
duration	float	1.0	操作持续时间参数。精确单位、范围和默认行为以目标型号接口定义为准。对应 C++ 参数 duration: float。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `SetVelocity(float, float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:69`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_velocity(vx=vx, vy=vy, omega=omega, duration=duration)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.r1.LocoClient.damp`

对应 C++ SDK 操作 `Damp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def damp(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `Damp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:82`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.damp()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.r1.LocoClient.start`

启动对应 SDK 操作。具体副作用和安全边界见下方状态。

签名

```
def start(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `Start()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:84`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.start()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.LocoClient.stand_up

对应 C++ SDK 操作 `StandUp()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def stand_up(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `StandUp()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:90`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stand_up()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.LocoClient.zero_torque

对应 C++ SDK 操作 `ZeroTorque()`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def zero_torque(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `ZeroTorque()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:92`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.zero_torque()
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.r1.LocoClient.stop_move`

请求停止对应 SDK 操作。该名称不等同于经过验证的物理急停。

签名

```
def stop_move(self) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `StopMove()`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:94`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.stop_move()
```


[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.LocoClient.move (重载 1/2)

对应 C++ SDK 操作 `Move(float, float, float, bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def move(self, vx: float, vy: float, vyaw: float, continous_move: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vx: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vy: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vyaw	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vyaw: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
continous_move	bool	必填	传给该接口的 <code>continous move</code> 参数，Python 类型为 <code>bool</code> 。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 <code>continous_move: bool</code> 。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `Move(float, float, float, bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:96`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw, continous_move=continous_move)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.LocoClient.move (重载 2/2)

对应 C++ SDK 操作 `Move(float, float, float)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
@overload
def move(self, vx: float, vy: float, vyaw: float) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
vx	float	必填	X 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vx: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vy	float	必填	Y 方向速度参数。坐标系、单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vy: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。
vyaw	float	必填	偏航角速度参数。单位、符号和安全范围必须查目标型号运动协议。对应 C++ 参数 <code>vyaw: float</code> 。底层为浮点数；类型本身不说明单位、坐标系或业务安全范围。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `Move(float, float, float)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:100`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.move(vx=vx, vy=vy, vyaw=vyaw)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.LocoClient.switch_move_mode

对应 C++ SDK 操作 `SwitchMoveMode(bool)`。上游头文件没有可直接生成的业务说明时，本参考只保证签名映射，精确语义需查目标型号协议。

签名

```
def switch_move_mode(self, flag: bool) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
flag	bool	必填	布尔开关。True 和 False 的具体业务效果由当前方法定义。对应 C++ 参数 flag: bool。只接受布尔语义。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `SwitchMoveMode(bool)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:102`

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.switch_move_mode(flag=flag)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

unitree_sdk2_cpp.robot.r1.LocoClient.set_speed_mode

设置速度模式。具体副作用和安全边界见下方状态。

签名

```
def set_speed_mode(self, speed_mode: int) -> int
```

可用性与安全性

AVAILABLE / MOTION_COMMAND: 当前绑定源码已实现，调用会向实体机器人发送运动相关请求。只能在隔离场地、完成目标型号安全检查并准备物理急停后使用；不得从默认测试或未经确认的 Agent 流程调用。

参数

名称	Python 类型	默认值	含义
speed_mode	int	必填	传给该接口的速度模式参数，Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。对应 C++ 参数 speed_mode: int。

返回值

位置	类型	含义
返回值	int	SDK 状态码；Unitree 示例通常以 0 表示成功，非零值需按具体服务定义解释。

对应 C++

- 类: `unitree::robot::r1::LocoClient`
- 签名: `SetSpeedMode(int)`
- 绑定策略: `DIRECT`
- 声明位置: `include/unitree/robot/r1/loco/r1_loco_client.hpp:107`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。
- 该方法属于运动命令。方法名包含 `stop`、`damp` 或 `zero` 也不代表它是独立物理急停。

用法

```
result = obj.set_speed_mode(speed_mode=speed_mode)
```

[!CAUTION] 此示例展示签名，不是可直接执行的安全脚本。调用前必须完成硬件隔离、目标机器人确认和对应型号的安全检查；运动接口还必须准备物理急停。

`unitree_sdk2_cpp.robot.r1.PlayStopParameter`

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.r1 import PlayStopParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>app_name</code>	<code>str</code>	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。	<code>value = obj.app_name / obj.app_name = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

`unitree_sdk2_cpp.robot.r1.PlayStopParameter.__init__`

初始化 `PlayStopParameter` 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::r1::PlayStopParameter`
- 签名: `PlayStopParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:61`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PlayStopParameter()
```

`unitree_sdk2_cpp.robot.r1.PlayStopParameter.from_json`

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
<code>value</code>	<code>Mapping[str, Any]</code>	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 <code>json: common::JsonMap &</code> 。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::r1::PlayStopParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:64`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.r1.PlayStopParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::r1::PlayStopParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:66`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```

unitree_sdk2_cpp.robot.r1.PlayStreamParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.r1 import PlayStreamParameter
```

基类: `object`

公开属性

名称	Python 类型	含义	用法
<code>app_name</code>	<code>str</code>	应用名称，用于标识音频流或播放会话。允许值和命名规则以目标服务协议为准。	<code>value = obj.app_name / obj.app_name = value</code>
<code>stream_id</code>	<code>str</code>	音频流标识符，用于区分同一应用下的播放流。	<code>value = obj.stream_id / obj.stream_id = value</code>

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	<code>def __init__(self) -> None</code>	AVAILABLE	CONSTRUCTION
<code>from_json</code>	<code>def from_json(self, value: Mapping[str, Any]) -> None</code>	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	<code>def to_json(self) -> dict[str, Any]</code>	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.r1.PlayStreamParameter.__init__

初始化 PlayStreamParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::r1::PlayStreamParameter`
- 签名: `PlayStreamParameter()`
- 绑定策略: 未单独标注
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:45`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = PlayStreamParameter()
```

unitree_sdk2_cpp.robot.r1.PlayStreamParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common::JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类: `unitree::robot::r1::PlayStreamParameter`
- 签名: `fromJson(common::JsonMap &)`
- 绑定策略: `JSON_DICT_INPUT`
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:48`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.fromJson(value=value)
```

`unitree_sdk2_cpp.robot.r1.PlayStreamParameter.toJson`

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def toJson(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	dict[str, Any]	返回 dict[str, Any]。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::r1::PlayStreamParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:50`

说明

- 示例是局部调用片段；`obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.toJson()
```

unitree_sdk2_cpp.robot.r1.TtsMakerParameter

SDK 请求参数值类型；当前可能仅提供设计期签名。

导入

```
from unitree_sdk2_cpp.robot.r1 import TtsMakerParameter
```

基类：object

公开属性

名称	Python 类型	含义	用法
index	int	传给该接口的 index 参数, value = obj.index / obj.index = value Python 类型为 int。精确含义、单位、范围和枚举值需查目标型号对应头文件与协议。	
speaker_id	int	语音合成说话人 ID。有效编号由机器人音频服务和固件决定。	value = obj.speaker_id / obj.speaker_id = value
text	str	要处理的文本内容；编码、长度和语言支持由目标服务决定。	value = obj.text / obj.text = value

方法索引

方法	Python 签名	状态	安全分类
<code>__init__</code>	def __init__(self) -> None	AVAILABLE	CONSTRUCTION
<code>from_json</code>	def from_json(self, value: Mapping[str, Any]) -> None	AVAILABLE	UNCLASSIFIED
<code>to_json</code>	def to_json(self) -> dict[str, Any]	AVAILABLE	UNCLASSIFIED

unitree_sdk2_cpp.robot.r1.TtsMakerParameter.__init__

初始化 TtsMakerParameter 实例。是否能实际构造取决于下方可用性状态。

签名

```
def __init__(self) -> None
```

可用性与安全性

AVAILABLE / CONSTRUCTION: 当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
无	None	<code>__init__</code> 本身不返回值；调用类对象时，在构造函数可用的前提下得到该类实例。

对应 C++

- 类: `unitree::robot::r1::TtsMakerParameter`
- 签名: `TtsMakerParameter()`

- 绑定策略：未单独标注
- 声明位置：include/unitree/robot/r1/audio/audio_api.hpp:27

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
value = TtsMakerParameter()
```

unitree_sdk2_cpp.robot.r1.TtsMakerParameter.from_json

计划从 JSON 风格字典读取字段并更新当前 SDK 值对象。

签名

```
def from_json(self, value: Mapping[str, Any]) -> None
```

可用性与安全性

AVAILABLE / UNCLASSIFIED：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

名称	Python 类型	默认值	含义
value	Mapping[str, Any]	必填	要写入或传递的新值，必须符合该参数的 Python 类型以及底层 C++ 范围约束。对应 C++ 参数 json: common: JsonMap &。

返回值

位置	类型	含义
无	None	该方法不返回 Python 值；失败可能表现为异常或底层状态变化。

对应 C++

- 类：unitree::robot::r1::TtsMakerParameter
- 签名：fromJson(common::JsonMap &)
- 绑定策略：JSON_DICT_INPUT
- 声明位置：include/unitree/robot/r1/audio/audio_api.hpp:30

说明

- 示例是局部调用片段；obj 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
obj.from_json(value=value)
```

unitree_sdk2_cpp.robot.r1.TtsMakerParameter.to_json

计划把当前 SDK 值对象写入 JSON 风格字典。

签名

```
def to_json(self) -> dict[str, Any]
```

可用性与安全性

AVAILABLE / UNCLASSIFIED：当前绑定源码已实现；实际执行条件仍取决于平台和对应 SDK 环境。

参数

无显式参数。实例方法中的 `self` 由 Python 自动传入。

返回值

位置	类型	含义
返回值	<code>dict[str, Any]</code>	返回 <code>dict[str, Any]</code> 。更精确的业务含义见该方法用途、C++ 签名和目标型号协议。

对应 C++

- 类: `unitree::robot::r1::TtsMakerParameter`
- 签名: `toJson(common::JsonMap &) const`
- 绑定策略: `JSON_DICT_OUTPUT`
- 声明位置: `include/unitree/robot/r1/audio/audio_api.hpp:32`

说明

- 示例是局部调用片段; `obj` 和参数变量表示已经按上层业务校验并准备好的对象。

用法

```
result = obj.to_json()
```